



Oracle Database 19c: PL/SQL Workshop

Student Guide – Volume I
D108650GC10

Copyright © 2021, Oracle and/or its affiliates.

Disclaimer

This document contains proprietary information and is protected by copyright and other intellectual property laws. You may copy and print this document solely for your own use in an Oracle training course. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

Restricted Rights Notice

If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

Trademark Notice

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

Third-Party Content, Products, and Services Disclaimer

This documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

1009022021

For Instructor Use Only.
This document should not be distributed.

Contents

1 Introduction

Lesson Objectives	1-2
Lesson Agenda	1-3
Course Objectives	1-4
Course Road Map	1-5
Lesson Agenda	1-11
Human Resources (HR) Schema for This Course	1-12
Course Agenda	1-13
Class Account Information	1-16
Appendixes and Practices Used in This Course	1-17
Lesson Agenda	1-18
Oracle Database 19c: Focus Areas	1-19
Oracle Database 19c	1-20
Lesson Agenda	1-22
PL/SQL Development Environments	1-23
Oracle SQL Developer	1-24
Specifications of SQL Developer	1-25
SQL Developer Interface	1-26
Coding PL/SQL in SQL*Plus	1-27
SQL Developer Web	1-28
Lesson Agenda	1-33
Oracle SQL and PL/SQL Documentation	1-34
Additional Resources	1-35
Summary	1-36
Practice 1 Overview: Getting Started	1-37

2 Introduction to PL/SQL

Course Road Map	2-2
Objectives	2-3
Agenda	2-4
Limitations of SQL	2-5
Why PL/SQL?	2-6
Why PL/SQL	2-7
About PL/SQL	2-8
Benefits of PL/SQL	2-9

PL/SQL Runtime Architecture 2-11
PL/SQL Block Structure 2-12
Agenda 2-13
Block Types 2-14
Examining an Anonymous Block 2-16
Executing an Anonymous Block 2-17
Agenda 2-18
Enabling Output of a PL/SQL Block 2-19
Viewing the Output of a PL/SQL Block 2-20
Quiz 2-21
Summary 2-22
Practice 2: Overview 2-23

3 Declaring PL/SQL Variables

Course Road Map 3-2
Objectives 3-3
Agenda 3-4
Variables 3-5
Variables in PL/SQL 3-6
Requirements for Variable Names 3-8
Using Variables in PL/SQL 3-9
Declaring and Initializing PL/SQL Variables 3-10
Agenda 3-11
Declaring and Initializing PL/SQL Variables 3-12
Initializing Variables Through a SELECT Statement 3-13
Types of Variables 3-15
Declaring Variables 3-16
Guidelines for Declaring and Initializing PL/SQL Variables 3-17
Guidelines for Declaring PL/SQL Variables 3-18
Naming Conventions of the PL/SQL Structures Used in This Course 3-19
Data Types for Strings 3-20
Delimiters in String Literals 3-21
Data Types for Numeric values 3-22
Data Types for Date and Time values 3-23
Data Type Conversion 3-25
Agenda 3-28
The %TYPE Attribute 3-29
Declaring Variables with the %TYPE Attribute 3-30
Declaring Boolean Variables 3-31
LOB Data Type Variables 3-32
Composite Data Types: Records and Collections 3-33

Agenda 3-34
Bind Variables 3-35
Bind Variables: Examples 3-36
Using AUTOPRINT with Bind Variables 3-37
Quiz 3-38
Summary 3-39
Practice 3: Overview 3-40

4 Writing Executable Statements

Course Road Map 4-2
Objectives 4-3
Agenda 4-4
Lexical Units in a PL/SQL Block 4-5
PL/SQL Block Syntax and Guidelines 4-7
Commenting Code 4-8
SQL Functions in PL/SQL 4-9
SQL Functions in PL/SQL: Examples 4-10
Using Sequences in PL/SQL blocks 4-11
Using Sequences in PL/SQL Blocks 4-14
Agenda 4-15
Nested blocks 4-16
Nested Blocks: Example 4-17
Variable Scope and Visibility 4-18
Using a Qualifier with Nested Blocks 4-20
Challenge: Determining the Variable Scope 4-21
Agenda 4-23
Operators in PL/SQL 4-24
Operators in PL/SQL: Examples 4-25
Programming Guidelines 4-26
Indenting Code 4-27
Quiz 4-28
Summary 4-29
Practice 4: Overview 4-30

5 Using SQL Statements Within a PL/SQL Block

Course Road Map 5-2
Objectives 5-3
Agenda 5-4
SQL Statements in PL/SQL 5-5
SELECT Statements in PL/SQL 5-7
Retrieving Data in PL/SQL: Example 5-9

Retrieving Data in PL/SQL 5-10
Naming Ambiguities 5-11
Avoiding Naming Ambiguities 5-12
Agenda 5-13
Using PL/SQL to Manipulate Data 5-14
Insert Data: Example 5-15
Update Data: Example 5-16
Delete Data: Example 5-17
Merging Rows 5-18
Agenda 5-20
SQL Cursor 5-21
SQL Cursor Attributes for Implicit Cursors 5-23
Quiz 5-25
Summary 5-26
Practice 5: Overview 5-27

6 Writing Control Structures

Course Road Map 6-2
Objectives 6-3
PL/SQL Control Structures 6-4
Agenda 6-5
IF Statement 6-6
IF-ELSIF Statements 6-7
Simple IF Statement 6-8
IF THEN ELSE Statement 6-9
IF ELSIF ELSE Clause 6-10
NULL Value an in IF Statement 6-11
Agenda 6-12
CASE Expressions 6-13
Searched CASE Expressions 6-14
CASE Statement 6-16
Handling Nulls 6-17
Logic Tables 6-18
Boolean Expression or Logical Expression? 6-19
Agenda 6-20
Iterative Control: LOOP Statements 6-21
Basic Loops 6-22
Basic Loop: Example 6-23
WHILE Loops 6-24
WHILE Loops: Example 6-25
FOR Loops 6-26

FOR Loops: Example 6-28
FOR Loop Rules 6-29
Suggested Use of Loops 6-30
Nested Loops and Labels 6-31
Nested Loops and Labels: Example 6-32
PL/SQL CONTINUE Statement 6-33
PL/SQL CONTINUE Statement: Example 1 6-34
PL/SQL CONTINUE Statement: Example 2 6-35
Quiz 6-36
Summary 6-37
Practice 6: Overview 6-38

7 Working with Composite Data Types

Course Road Map 7-2
Objectives 7-3
Agenda 7-4
Composite Data Types 7-5
PL/SQL Records Versus Collections 7-7
Agenda 7-8
PL/SQL Records 7-9
Creating a PL/SQL Record 7-10
Creating a PL/SQL Record: Example 7-11
PL/SQL Record Structure 7-12
%ROWTYPE Attribute 7-14
Creating a PL/SQL Record: Example 7-16
Advantages of Using the %ROWTYPE Attribute 7-17
Another %ROWTYPE Attribute: Example 7-18
Inserting a Record by Using %ROWTYPE 7-19
Updating a Row in a Table by Using a Record 7-20
Agenda 7-21
Associative Arrays (INDEX BY Tables) 7-22
Associative Array Structure 7-24
Steps to Create an Associative Array 7-25
Creating and Accessing Associative Arrays 7-26
Associative Arrays with Record values 7-27
Using Collection Methods 7-28
Using Collection Methods with Associative Arrays 7-29
Nested Tables 7-30
Nested Tables: Syntax and Usage 7-31
Variable-Sized Arrays (Varrays) 7-33
VARRAYs: Syntax and Usage 7-34

Summarizing Collections 7-35

Quiz 7-36

Summary 7-37

Practice 7: Overview 7-38

8 Using Explicit Cursors

Course Road Map 8-2

Objectives 8-3

Agenda 8-4

Cursors 8-5

Implicit Cursors 8-6

Explicit Cursor 8-7

Controlling Explicit Cursors 8-8

Agenda 8-9

Declaring the Cursor 8-10

Opening the Cursor 8-12

Fetching Data from the Cursor 8-13

Closing the Cursor 8-15

Cursors and Records 8-16

Cursor FOR Loops 8-17

Explicit Cursor Attributes 8-19

%ISOPEN Attribute 8-20

%ROWCOUNT and %NOTFOUND: Example 8-21

Cursor FOR Loops Using Subqueries 8-22

Agenda 8-23

Cursors with Parameters 8-24

Agenda 8-26

FOR UPDATE Clause 8-27

WHERE CURRENT OF Clause 8-29

WHERE CURRENT OF Clause: Example 8-30

Quiz 8-32

Summary 8-33

Practice 8: Overview 8-34

9 Handling Exceptions

Course Road Map 9-2

Objectives 9-3

Agenda 9-4

What Is an Exception? 9-5

Handling an Exception: Example 9-6

Understanding Exceptions with PL/SQL 9-7

Handling Exceptions 9-8
Exception Types 9-9
Agenda 9-10
Syntax to Trap Exceptions 9-11
Guidelines for Trapping Exceptions 9-13
Trapping Internally Predefined Exceptions 9-14
Internally Defined Exception Trapping: Example 9-15
Trapping Predefined Exceptions 9-16
Functions for Trapping Exceptions 9-19
Trapping User-Defined Exceptions 9-21
RAISE Statement 9-22
Trapping User-Defined Exceptions 9-23
Propagating Exceptions in a Sub-Block 9-24
The RAISE_APPLICATION_ERROR Procedure 9-25
Quiz 9-28
Summary 9-29
Practice 9: Overview 9-30

10 Introducing Stored Procedures and Functions

Course Road Map 10-2
Objectives 10-3
Agenda 10-4
What Are PL/SQL Subprograms? 10-5
Differences Between Anonymous Blocks and Subprograms 10-7
Agenda 10-8
Procedure: Syntax 10-9
Creating a Procedure 10-10
Invoking a Procedure 10-12
Agenda 10-13
Function: Syntax 10-14
Creating a Function 10-15
Invoking a Function 10-16
Passing a Parameter to the Function 10-17
Invoking the Function with a Parameter 10-18
Quiz 10-19
Summary 10-20
Practice 10: Overview 10-21

11 Creating Procedures

Course Road Map 11-2
Objectives 11-3

Lesson Agenda 11-4
Modularized Program Design 11-5
Modularizing Code with PL/SQL 11-6
Benefits of Modularization 11-7
What Are PL/SQL Subprograms? 11-8
Lesson Agenda 11-9
Procedures 11-10
What Are Procedures? 11-11
Creating Procedures: Overview 11-12
Creating Procedures 11-13
Creating Procedures Using SQL Developer 11-14
Compiling Procedures 11-15
Calling Procedures 11-16
Calling Procedures Using SQL Developer 11-17
Procedures 11-18
What Are Parameters and Parameter Modes? 11-19
Formal and Actual Parameters 11-20
Procedural Parameter Modes 11-21
Comparing the Parameter Modes 11-22
Using the IN Parameter Mode: Example 11-23
Using the OUT Parameter Mode: Example 11-24
Using the IN OUT Parameter Mode: Example 11-25
Passing Parameters to Procedures 11-26
Passing Actual Parameters: Creating the raise_sal Procedure 11-27
Passing Actual Parameters: Examples 11-28
Using the DEFAULT Option for the Parameters 11-29
Lesson Agenda 11-30
Handled Exceptions 11-31
Handled Exceptions: Example 11-32
Exceptions Not Handled 11-33
Exceptions Not Handled: Example 11-34
Removing Procedures: Using the DROP SQL Statement or SQL Developer 11-35
Viewing Procedure Information Using the Data Dictionary Views 11-36
Viewing Procedures Information Using SQL Developer 11-37
Quiz 11-38
Summary 11-39
Practice 11-1 Overview: Creating a New Database Connection 11-40
Practice 11-2 Overview: Creating, Compiling, and Calling Procedures 11-41

12 Creating Functions

Course Road Map 12-2
Objectives 12-3
Lesson Agenda 12-4
Functions 12-5
Creating Functions syntax 12-6
Tax Calculation 12-8
The Difference Between Procedures and Functions 12-9
Creating Functions: Overview 12-10
Invoking a Stored Function: Example 12-11
Using Different Methods for Executing Functions 12-12
Creating and Compiling Functions Using SQL Developer 12-14
Lesson Agenda 12-15
Using a Function in a SQL Expression: Example 12-16
Calling User-Defined Functions in SQL Statements 12-17
Restrictions When Calling Functions from SQL Expressions 12-18
Side Effects of Function Execution 12-19
Controlling Side Effects 12-20
Guidelines to Control Side Effects 12-21
Lesson Agenda 12-22
Passing Parameters to Functions 12-23
Named and Mixed Notation from SQL: Example 12-24
Viewing Functions Using Data Dictionary Views 12-25
Viewing Functions Information Using SQL Developer 12-26
Lesson Agenda 12-27
Removing Functions: Using the DROP SQL Statement or SQL Developer 12-28
Quiz 12-29
Summary 12-30
Practice 12: Overview 12-31

13 Debugging Subprograms

Course Road Map 13-2
Objectives 13-3
Lesson Agenda 13-4
Before Debugging PL/SQL Subprograms 13-5
Lesson Agenda 13-9
Debugging a Subprogram: Overview 13-10
Lesson Agenda 13-12
The Debugging – Log Tab Toolbar 13-13
Tracking Data and Execution 13-15
Lesson Agenda 13-16

Debugging a Procedure Example: Creating a New emp_list Procedure 13-17
Debugging a Procedure Example: Creating a New get_location Function 13-18
Setting Breakpoints and Compiling emp_list for Debug Mode 13-19
Compiling the get_location Function for Debug Mode 13-20
Debugging emp_list and Entering Values for the PMAXROWS Parameter 13-21
Debugging emp_list: Step Into (F7) the Code 13-22
Viewing the Data 13-23
Modifying the Variables While Debugging the Code 13-24
Debugging emp_list: Step Over Versus Step Into 13-25
Debugging emp_list: Step Out of the Code (Shift + F7) 13-26
Debugging emp_list: Step to End of Method 13-27
Debugging a Subprogram Remotely: Overview 13-28
Summary 13-29
Practice 13 Overview: Introduction to the SQL Developer Debugger 13-30

14 Creating Packages

Course Road Map 14-2
Objectives 14-3
Lesson Agenda 14-4
DBMS_OUTPUT.PUT_LINE 14-5
What Is a Package? 14-6
Advantages of Packages 14-7
How Do You Create PL/SQL Packages? 14-8
Components of a PL/SQL Package 14-9
Application Program Interface 14-10
Lesson Agenda 14-11
Creating the Package Specification: Using the CREATE PACKAGE Statement 14-12
Creating Package Specification: Using SQL Developer 14-13
Creating the Package Body: Using SQL Developer 14-14
Example of a Package Specification: comm_pkg 14-15
Creating the Package Body 14-16
Example of a Package Body: comm_pkg 14-17
Invoking the Package Subprograms: Examples 14-18
Invoking Package Subprograms: Using SQL Developer 14-19
Creating and Using Bodiless Packages 14-20
Viewing Packages by Using the Data Dictionary 14-21
Viewing Packages by Using SQL Developer 14-22
Removing Packages 14-23
Removing Package Bodies 14-24
Guidelines for Writing Packages 14-25

Quiz 14-26
Summary 14-27
Practice 14 Overview: Creating and Using Packages 14-28

15 Working with Packages

Course Road Map 15-2
Objectives 15-3
Lesson Agenda 15-4
Why Overload Subprograms? 15-5
Overloading Subprograms in PL/SQL 15-6
Overloading Procedures Example: Creating the Package Specification 15-8
Overloading Procedures Example: Creating the Package Body 15-9
Restrictions on Overloading 15-10
STANDARD package 15-11
Overloading and the STANDARD Package 15-12
Lesson Agenda 15-13
Package Instantiation and Initialization 15-14
Initializing Packages in Package Body 15-15
Using User-Defined Package Functions in SQL 15-16
User-Defined Package Function in SQL: Example 15-17
Lesson Agenda 15-18
Package State 15-19
Serially Reusable Packages 15-20
Memory Architecture 15-21
Serially Reusable Packages 15-23
Persistent State of Packages 15-24
Persistent State of Package Variables: Example 15-25
Persistent State of a Package Cursor: Example 15-26
Executing the CURS_PKG Package 15-28
Quiz 15-29
Summary 15-30
Practice 15 Overview: Working with Packages 15-31

16 Using Oracle-Supplied Packages in Application Development

Course Road Map 16-2
Objectives 16-3
Lesson Agenda 16-4
Using Oracle-Supplied Packages 16-5
Examples of Some Oracle-Supplied Packages 16-6
Lesson Agenda 16-7
How the DBMS_OUTPUT Package Works 16-8

Using the UTL_FILE Package	16-9
Some of the UTL_FILE Procedures and Functions	16-10
File Processing Using the UTL_FILE Package: Overview	16-11
Using the Available Declared Exceptions in the UTL_FILE Package	16-12
FOPEN and IS_OPEN Functions: Example	16-13
Using UTL_FILE: Example	16-16
What Is the UTL_MAIL Package?	16-18
Setting Up and Using the UTL_MAIL: Overview	16-19
Summary of UTL_MAIL Subprograms	16-20
Installing and Using UTL_MAIL	16-21
The SEND Procedure Syntax	16-22
The SEND_ATTACH_RAW Procedure	16-23
Sending Email with a Binary Attachment: Example	16-24
The SEND_ATTACH_VARCHAR2 Procedure	16-26
Sending Email with a Text Attachment: Example	16-27
Quiz	16-32
Summary	16-33
Practice 16 Overview: Using Oracle-Supplied Packages in Application Development	16-34

17 Using Dynamic SQL

Course Road Map	17-2
Objectives	17-3
Lesson Agenda	17-4
What is Dynamic SQL?	17-5
When do you use Dynamic SQL?	17-6
Using Dynamic SQL	17-7
Execution Flow of SQL Statements	17-8
Dynamic SQL implementation	17-9
Lesson Agenda	17-10
Native Dynamic SQL (NDS)	17-11
Using the EXECUTE IMMEDIATE Statement	17-12
Dynamic SQL with a DDL Statement: Examples	17-13
Dynamic SQL with DML Statements	17-14
Dynamic SQL with a Single-Row Query: Example	17-15
Executing a PL/SQL Anonymous Block Dynamically	17-16
BULK COLLECT INTO clause	17-17
OPEN FOR clause	17-18
Using BULK COLLECT and OPEN FOR clause	17-19
Summarizing Methods for Using Dynamic SQL	17-20
Lesson Agenda	17-22

Using the DBMS_SQL Package 17-25
Using the DBMS_SQL Package Subprograms 17-26
Using DBMS_SQL with a DML Statement: Deleting Rows 17-27
Using DBMS_SQL with a Parameterized DML Statement 17-29
Quiz 17-30
Summary 17-31
Practice 17 Overview: Using Dynamic SQL 17-32

18 Creating Triggers

Course Road Map 18-2
Objectives 18-3
Lesson Agenda 18-4
What are Triggers? 18-6
Defining Triggers 18-7
Why do you use Triggers? 18-8
Trigger Event Types 18-9
Available Trigger Types 18-10
Trigger Event Types and Body 18-11
Lesson Agenda 18-12
Creating DML Triggers by Using the CREATE TRIGGER Statement 18-13
Creating DML Triggers by Using SQL Developer 18-14
Specifying the Trigger Execution Time 18-15
Creating a DML Statement Trigger Example: SECURE_EMP 18-16
Testing Trigger SECURE_EMP 18-17
Using Conditional Predicates 18-18
Multiple Triggers of the Same Type 18-19
CALL Statements in Triggers 18-20
Lesson Agenda 18-21
Statement-Level Triggers Versus Row-Level Triggers 18-22
Creating a DML Row Trigger 18-24
Correlation names and Pseudorecords 18-25
Correlation Names and Pseudorecords 18-26
Using OLD and NEW Qualifiers 18-27
Using OLD and NEW Qualifiers: Example 18-28
Using the WHEN Clause to Fire a Row Trigger Based on a Condition 18-30
Trigger-Firing Sequence: Single-Row Manipulation 18-31
Trigger-Firing Sequence: Multirow Manipulation 18-32
Summary of the Trigger Execution Model 18-33
Lesson Agenda 18-34
INSTEAD OF Triggers 18-35
Creating an INSTEAD OF Trigger: Example 18-36

Creating an INSTEAD OF Trigger to Perform DML on Complex Views 18-37
Lesson Agenda 18-39
The Status of a Trigger 18-40
System Privileges Required to Manage Triggers 18-41
Managing Triggers by Using the ALTER and DROP SQL Statements 18-42
Managing Triggers by Using SQL Developer 18-43
Viewing Trigger Information 18-44
Using USER_TRIGGERS 18-45
Testing Triggers 18-46
Quiz 18-47
Summary 18-48
Practice 18 Overview: Creating Statement and Row Triggers 18-49

19 Creating Compound, DDL, and Event Database Triggers

Course Road Map 19-2
Objectives 19-3
Lesson Agenda 19-4
What is a Compound Trigger? 19-5
Working with Compound Triggers 19-6
Why Compound Triggers? 19-7
Compound Trigger Structure 19-8
Compound Trigger Structure for Views 19-9
Compound Trigger Restrictions 19-10
Lesson Agenda 19-11
Mutating Tables 19-12
Mutating Table: Example 19-13
Using a Compound Trigger to Resolve the Mutating Table Error 19-15
Lesson Agenda 19-18
Creating Triggers on DDL Statements 19-19
Creating Triggers on DDL Statements -Example 19-20
Lesson Agenda 19-21
Creating Database Triggers 19-22
Creating Triggers on System Events 19-23
LOGON and LOGOFF Triggers: Example 19-24
Lesson Agenda 19-25
Guidelines for Designing Triggers 19-26
Quiz 19-27
Summary 19-28
Practice 19 Overview: Creating Compound, DDL, and Event Database Triggers 19-29

20 Design Considerations for the PL/SQL Code

Course Road Map	20-2
Objectives	20-3
Lesson Agenda	20-4
Standardizing Constants and Exceptions	20-5
Standardizing Exceptions	20-6
Standardizing Exception Handling	20-7
Standardizing Constants	20-8
Local Subprograms	20-9
Lesson Agenda	20-10
Definer's and Invoker's Rights	20-11
Specifying Invoker's Rights: Setting AUTHID to CURRENT_USER	20-12
Granting Privileges to Invoker's Rights Unit	20-13
Lesson Agenda	20-14
Autonomous Transactions	20-15
Features of Autonomous Transactions	20-16
Using Autonomous Transactions: Example	20-17
Lesson Agenda	20-19
Using the NOCOPY Hint	20-20
Effects of the NOCOPY Hint	20-21
When Does the PL/SQL Compiler Ignore the NOCOPY Hint?	20-22
Using the PARALLEL_ENABLE Hint	20-23
Using the Cross-Session PL/SQL Function Result Cache	20-24
Declaring and Defining a Result-Cached Function: Example	20-25
Using the DETERMINISTIC Clause with Functions	20-26
Using the RETURNING Clause	20-27
Lesson Agenda	20-28
Using Bulk Binding	20-29
Bulk Binding: Syntax and Keywords	20-30
Bulk Binding FORALL: Example	20-31
Using BULK COLLECT INTO with Queries	20-34
Using BULK COLLECT INTO with Cursors	20-35
Using BULK COLLECT INTO with a RETURNING Clause	20-36
Quiz	20-37
Summary	20-38
Practice 20 Overview: Design Considerations for PL/SQL Code	20-39

21 Tuning the PL/SQL Compiler

Course Road Map	21-2
Objectives	21-3
Lesson Agenda	21-4

Optimizing PL/SQL Compiler Performance	21-5
Initialization Parameters for PL/SQL Compilation	21-6
Using the Initialization Parameters for PL/SQL Compilation	21-8
Displaying the PL/SQL Initialization Parameters	21-9
Displaying and Setting PL/SQL Initialization Parameters	21-10
Changing PL/SQL Initialization Parameters: Example	21-11
Lesson Agenda	21-12
PL/SQL Compile-Time Warnings	21-13
Benefits of Compiler Warnings	21-14
Categories of PL/SQL Compile-Time Warning Messages	21-15
Enabling Warning Messages	21-16
Setting Compiler Warning Levels: Using PLSQL_WARNINGS, Examples	21-17
Enabling Compiler Warnings: Using PLSQL_WARNINGS in SQL Developer	21-18
Viewing the Current Setting of PLSQL_WARNINGS	21-19
Viewing Compiler Warnings	21-20
SQL*Plus Warning Messages: Example	21-21
Defining PLSQL_WARNINGS for Program Units	21-22
Lesson Agenda	21-23
Using the DBMS_WARNINGS Package	21-24
Using the DBMS_WARNING Package Subprograms	21-25
The DBMS_WARNING Procedures: Syntax, Parameters, and Allowed Values	21-26
The DBMS_WARNING Procedures: Example	21-27
The DBMS_WARNING Functions: Syntax, Parameters, and Allowed Values	21-28
The DBMS_WARNING Functions: Example	21-29
Using DBMS_WARNING: Example	21-30
Quiz	21-32
Summary	21-33
Practice 21 Overview: Tuning PL/SQL Compiler	21-34

22 Managing Dependencies

Course Road Map	22-2
Objectives	22-3
Lesson Agenda	22-4
What are Dependencies in a Schema?	22-5
How Dependencies Work?	22-6
Dependent and Referenced Objects	22-8
Querying Object Dependencies: Using the USER_DEPENDENCIES View	22-10
Querying an Object's Status	22-11
Categorizing Dependencies	22-12
Lesson Agenda	22-13

Direct Dependencies	22-14
Indirect Dependencies	22-15
Displaying Direct and Indirect Dependencies	22-16
Lesson Agenda	22-18
Fine-Grained Dependency Management	22-19
Fine-Grained Dependency Management: Example 1	22-21
Fine-Grained Dependency Management: Example 2	22-23
Guidelines for Reducing Invalidation	22-24
Object Revalidation	22-25
Lesson Agenda	22-26
Remote Dependencies	22-27
Managing Remote Procedure Dependencies	22-28
Setting the REMOTE_DEPENDENCIES_MODE Parameter	22-29
Timestamp Checking	22-30
Signature Checking	22-35
Lesson Agenda	22-36
Revalidating PL/SQL Program Units	22-37
Unsuccessful Recompilation	22-38
Successful Recompilation	22-39
Recompiling Procedures	22-40
Lesson Agenda	22-41
Packages and Dependencies: Subprogram References the Package	22-42
Packages and Dependencies: Package Subprogram References Procedure	22-43
Quiz	22-44
Summary	22-45
Practice 22 Overview: Managing Dependencies in Your Schema	22-46

A Using SQL Developer

Objectives	A-2
What Is Oracle SQL Developer?	A-3
Specifications of SQL Developer	A-4
SQL Developer Interface	A-5
Creating a Database Connection	A-7
Browsing Database Objects	A-10
Displaying the Table Structure	A-11
Browsing Files	A-12
Creating a Schema Object	A-13
Creating a New Table: Example	A-14
Using the SQL Worksheet	A-15
Executing SQL Statements	A-19
Saving SQL Scripts	A-20

Executing Saved Script Files: Method 1 A-21
Executing Saved Script Files: Method 2 A-22
Formatting the SQL Code A-23
Using Snippets A-24
Using Snippets: Example A-25
Debugging Procedures and Functions A-26
Database Reporting A-27
Creating a User-Defined Report A-28
Search Engines and External Tools A-29
Setting Preferences A-30
Resetting the SQL Developer Layout A-32
Data Modeler in SQL Developer A-33
Summary A-34

B Using SQL*Plus

Objectives B-2
SQL and SQL*Plus Interaction B-3
SQL Statements Versus SQL*Plus Commands B-4
Overview of SQL*Plus B-5
Logging In to SQL*Plus B-6
Displaying the Table Structure B-7
SQL*Plus Editing Commands B-9
Using LIST, n, and APPEND B-11
Using the CHANGE Command B-12
SQL*Plus File Commands B-13
Using the SAVE, START Commands B-14
SERVEROUTPUT Command B-15
Using the SQL*Plus SPOOL Command B-16
Using the AUTOTRACE Command B-17
Summary B-18

C Commonly Used SQL Commands

Objectives C-2
Basic SELECT Statement C-3
SELECT Statement C-4
WHERE Clause C-5
ORDER BY Clause C-6
GROUP BY Clause C-7
Data Definition Language C-8
CREATE TABLE Statement C-9
ALTER TABLE Statement C-10

DROP TABLE Statement C-11
GRANT Statement C-12
Privilege Types C-13
REVOKE Statement C-14
TRUNCATE TABLE Statement C-15
Data Manipulation Language C-16
INSERT Statement C-17
UPDATE Statement Syntax C-18
DELETE Statement C-19
Transaction Control Statements C-20
COMMIT Statement C-21
ROLLBACK Statement C-22
SAVEPOINT Statement C-23
Joins C-24
Types of Joins C-25
Qualifying Ambiguous Column Names C-26
Natural Join C-27
Equijoins C-28
Retrieving Records with Equijoins C-29
Additional Search Conditions Using the AND and WHERE Operators C-30
Retrieving Records with Nonequijoins C-31
Retrieving Records by Using the USING Clause C-32
Retrieving Records by Using the ON Clause C-33
Left Outer Join C-34
Right Outer Join C-35
Full Outer Join C-36
Self-Join: Example C-37
Cross Join C-38
Summary C-39

D REF Cursors

Cursor Variables D-2
Using Cursor Variables D-3
Defining REF CURSOR Types D-4
Using the OPEN-FOR, FETCH, and CLOSE Statements D-7
Example of Fetching D-10

E Managing PL/SQL Code

Objectives E-2
Agenda E-3
Conditional Compilation E-4

How Does Conditional Compilation Work?	E-5
Using Selection Directives	E-6
Using Predefined and User-Defined Inquiry Directives	E-7
The PLSQL_CCFLAGS Parameter and the Inquiry Directive	E-8
Displaying the PLSQL_CCFLAGS Initialization Parameter Setting	E-9
The PLSQL_CCFLAGS Parameter and the Inquiry Directive: Example	E-10
Using Conditional Compilation Error Directives to Raise User-Defined Errors	E-11
Using Static Expressions with Conditional Compilation	E-12
DBMS_DB_VERSION Package: Boolean Constants	E-13
DBMS_DB_VERSION Package Constants	E-14
Using Conditional Compilation with Database Versions: Example	E-15
Using DBMS_PREPROCESSOR Procedures to Print or Retrieve Source Text	E-17
Agenda	E-18
Obfuscation	E-19
Benefits of Obfuscating	E-20
What's New in Dynamic Obfuscating Since Oracle 10g?	E-21
Nonobfuscated PL/SQL Code: Example	E-22
Obfuscated PL/SQL Code: Example	E-23
Dynamic Obfuscation: Example	E-24
PL/SQL Wrapper Utility	E-25
Running the PL/SQL Wrapper Utility	E-26
Results of Wrapping	E-27
Guidelines for Wrapping	E-28
DBMS_DDL Package Versus wrap Utility	E-29
Summary	E-30

F Implementing Triggers

Objectives	F-2
Controlling Security Within the Server	F-3
Controlling Security with a Database Trigger	F-4
Enforcing Data Integrity Within the Server	F-5
Protecting Data Integrity with a Trigger	F-6
Enforcing Referential Integrity Within the Server	F-7
Protecting Referential Integrity with a Trigger	F-8
Replicating a Table Within the Server	F-9
Replicating a Table with a Trigger	F-10
Computing Derived Data Within the Server	F-11
Computing Derived Values with a Trigger	F-12
Logging Events with a Trigger	F-13
Summary	F-15

G Using the DBMS_SCHEDULER and HTP Packages

Objectives G-2

Generating Webpages with the HTP Package G-3

Using the HTP Package Procedures G-4

Creating an HTML File with SQL*Plus G-5

The DBMS_SCHEDULER Package G-6

Creating a Job G-8

Creating a Job with Inline Parameters G-9

Creating a Job Using a Program G-10

Creating a Job for a Program with Arguments G-11

Creating a Job Using a Schedule G-12

Setting the Repeat Interval for a Job G-13

Creating a Job Using a Named Program and Schedule G-14

Managing Jobs G-15

Data Dictionary Views G-16

Summary G-17

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Introduction

Lesson Objectives

After completing this lesson, you should be able to do the following:

- Discuss the goals of the course
- Describe the HR database schema that is used in this course
- Identify the available user interface environments that can be used in this course
- Reference the available appendixes, documentation, and other resources



2

This lesson gives you a high-level overview of the course and its flow. You learn about the database schema and the tables that the course uses. The course discusses the fundamentals of PL/SQL. You learn how to write PL/SQL code blocks, and are introduced to the concepts of procedures and functions.

You are also introduced to a programming tool, SQL Developer 19.1.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Course objectives and course agenda
- The schema and appendixes used in this course
- Overview of Oracle Database 19c and related products
- Available PL/SQL development environments
- Oracle documentation and additional resources



For Instructor Use Only.
This document should not be distributed.

Course Objectives

After completing this course, you should be able to do the following:

- Identify the programming extensions that PL/SQL provides to SQL
- Write PL/SQL code to interface with the database
- Design PL/SQL anonymous blocks that execute efficiently
- Use PL/SQL programming constructs and conditional control statements
- Handle runtime errors
- Describe stored procedures and functions
- Create Packages
- Create Triggers



4

This course presents the basics of PL/SQL. You learn about PL/SQL syntax, blocks, and programming constructs, and also about the advantages of integrating SQL with those constructs. You learn how to write PL/SQL program units and execute them efficiently. In addition, you learn how to use SQL Developer as a development environment for PL/SQL. You also learn how to design reusable program units such as procedures, functions, packages and triggers.

For Instructor Use Only.
This document should not be distributed.

Course Road Map



5

The slide provides a graphical representation of the overall course structure.

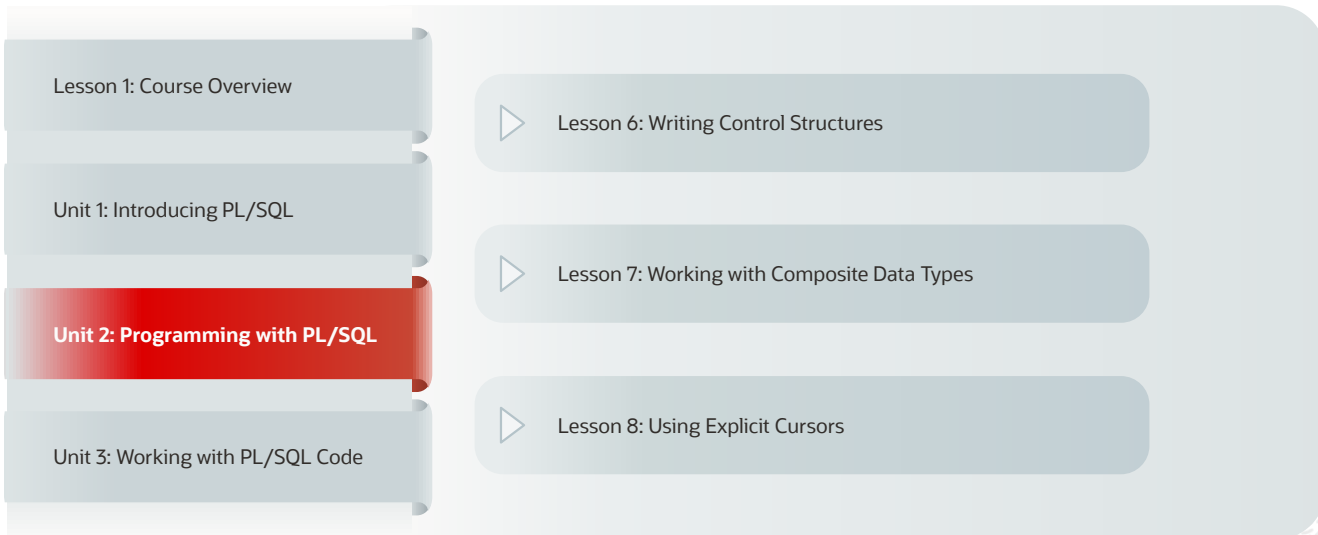
The course is organized into three units: Introducing PL/SQL, Programming with PL/SQL, and Working with PL/SQL Code.

In the first unit, we have four lessons:

1. Introduction to PL/SQL
2. Declaring PL/SQL Variables
3. Writing Anonymous PL/SQL Blocks
4. Using SQL Statements in PL/SQL Blocks.

You learn to write code in PL/SQL and define some anonymous PL/SQL blocks in this unit.

Course Road Map



6

In Unit 2, you learn to program with PL/SQL by using control structures, composite data types, and explicit cursors.

0
For Instructor Use Only.
This document should not be distributed.

Course Road Map

Lesson 1: Course Overview

Unit 1: Introducing PL/SQL

Unit 2: Programming with PL/SQL

Unit 3: Working with PL/SQL Code



Lesson 9: Handling Exceptions



Lesson 10: Creating Procedures and Functions

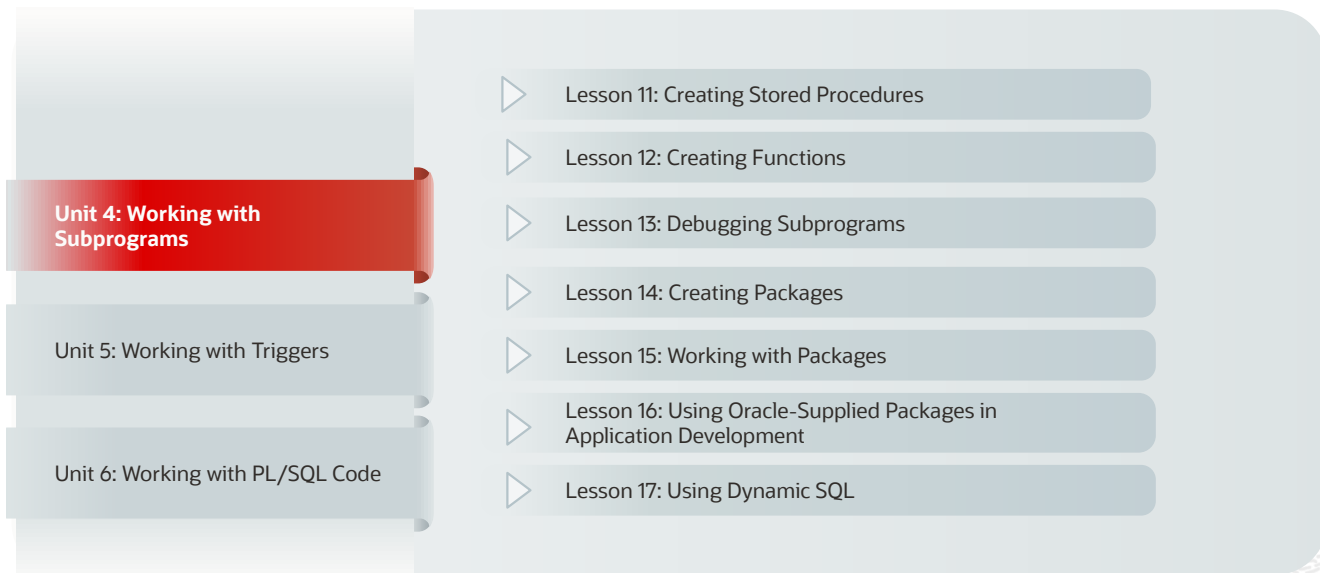
7

In Unit 3, you learn how to handle exceptions that might occur during the execution of PL/SQL blocks. You are also introduced to the concept of procedures and functions in PL/SQL.

0

For Instructor Use Only.
This document should not be distributed.

Course Road Map

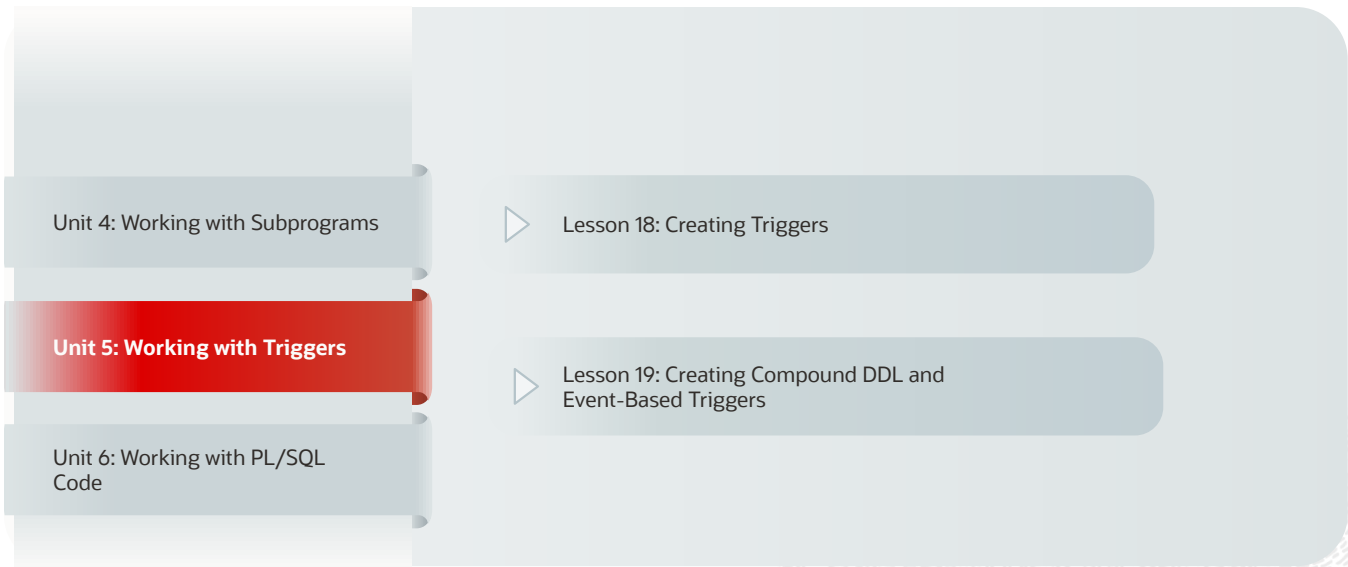


8

In Unit 4, you learn to write code in stored PL/SQL objects, including Procedures, Functions, and Packages. You also learn about Oracle predefined packages and how to work with Dynamic SQL.

For Instructor Use Only.
This document should not be distributed.

Course Road Map

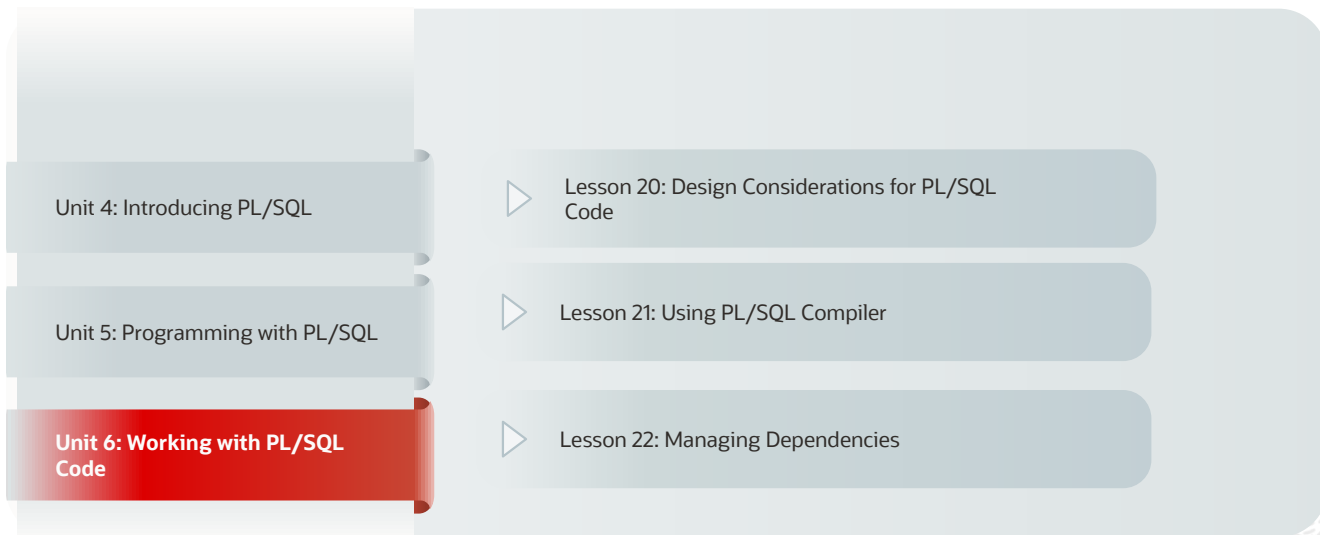


9

In Unit 5, you learn to create and use Triggers.

0
For Instructor Use Only.
This document should not be distributed.

Course Road Map



10

In Unit 6, you learn how to improve performance, compile and recompile your code, and discover dependencies between objects.

0
For Instructor Use Only.
This document should not be distributed.

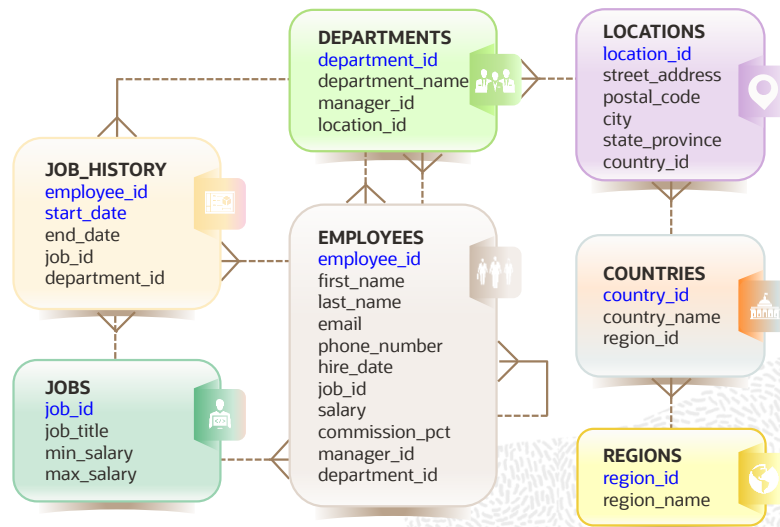
Lesson Agenda

- Course objectives and course agenda
- The schema and appendixes used in this course
- Overview of Oracle Database 19c and related products
- Available PL/SQL development environments
- Oracle documentation and additional resources



For Instructor Use Only.
This document should not be distributed.

Human Resources (HR) Schema for This Course



12

The Human Resources (HR) schema is part of the Oracle Sample Schemas that can be installed in an Oracle database. You will use the HR schema in all the practice lessons of the course. The fields marked in blue are the primary key attributes for the respective entities.

A description of each of the tables in the HR schema is as follows:

- The **REGIONS** table contains rows that represent a region such as the Americas or Asia.
- The **COUNTRIES** table contains rows for countries, each of which is associated with a region.
- The **LOCATIONS** table contains the specific address of a specific office, warehouse, or production site of a company in a particular country.
- The **DEPARTMENTS** table shows details of the departments in which employees work. Each department may have a relationship that represents the department manager in the **EMPLOYEES** table.
- The **EMPLOYEES** table contains details of each employee working in a department. Some employees may not be assigned to any department.
- The **JOBS** table contains the job types that can be held by each employee.
- The **JOB_HISTORY** table contains the job history of employees. If an employee changes departments within a job or changes jobs within a department, a new row is inserted into this table with the old job information of the employee.

Course Agenda

- Day 1:
 - Introduction
 - Introduction to PL/SQL
 - Declaring PL/SQL Variables
 - Writing Executable Statements
 - Using SQL Statements within a PL/SQL Block
 - Writing Control Structures
- Day 2:
 - Working with Composite Data Types
 - Using Explicit Cursors
 - Handling Exceptions
 - Introducing Stored Procedures and Functions

13



For Instructor Use Only.
This document should not be distributed.

Course Agenda

- Day 3:
 - Creating Procedures
 - Creating Functions
 - Debugging Subprograms
 - Creating Packages
- Day 4:
 - Working with Packages
 - Using Oracle-Supplied Packages in Application Development
 - Using Dynamic SQL
 - Creating Triggers



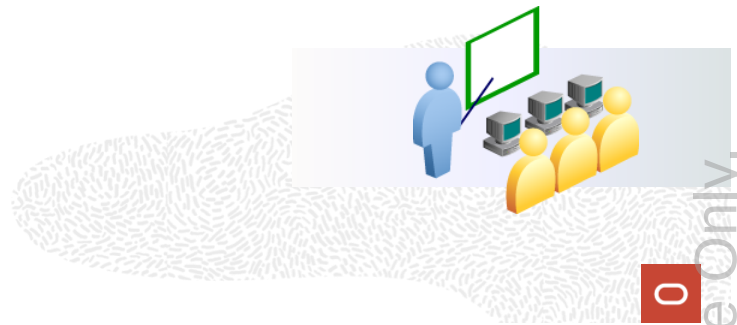
For Instructor Use Only.
This document should not be distributed.

Course Agenda

- Day 5:
 - Creating Compound DDL and Event Database Triggers
 - Design Considerations for PL/SQL Code
 - Tuning PL/SQL Compiler
 - Managing Dependencies

Class Account Information

- A cloned HR account ID is set up for you.
- Your account IDs are ora41, ora61, and ora62.
- The password is provided in the Activity Guide.
- Each machine has its own complete environment and is assigned the same accounts.



For Instructor Use Only.

This document should not be distributed.

Appendixes and Practices Used in This Course

- Appendix A: Table Descriptions and Data
- Appendix B: Using SQL Developer
- Appendix C: Using SQL*Plus
- Appendix D: Commonly Used SQL Commands
- Appendix E: Managing PL/SQL Code
- Appendix F: Implementing Triggers
- Appendix G: Using DBMS_SCHEDULER and HTP Packages
- Activity Guide: Practices and Solutions

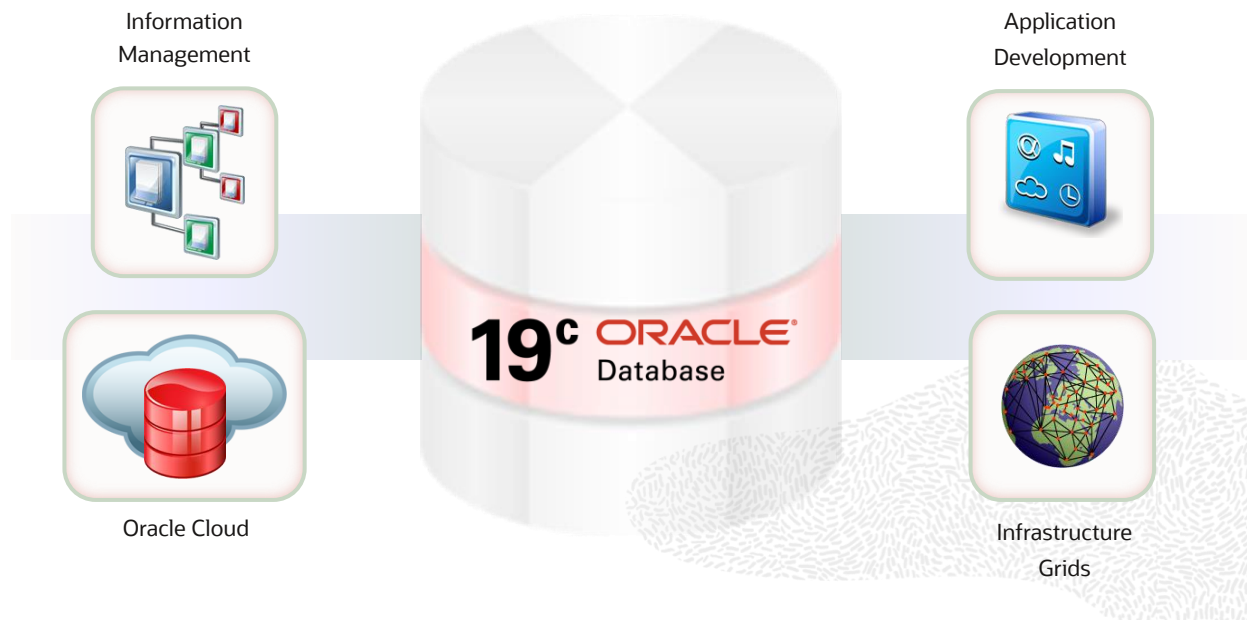
Lesson Agenda

- Course objectives and course agenda
- The schema and appendixes used in this course
- Overview of Oracle Database 19c and related products
- Available PL/SQL development environments
- Oracle documentation and additional resources



For Instructor Use Only.
This document should not be distributed.

Oracle Database 19c: Focus Areas



19

By using Oracle Database 19c, you can utilize the following features across focus areas:

- With the **Infrastructure Grid** technology of Oracle, you can pool low-cost servers and storage to form systems that deliver the highest quality of service in terms of manageability, high availability, and performance. Oracle Database 19c also helps you to consolidate and extend the benefits of grid computing and manage changes in a controlled and cost-effective manner.
- Oracle Database 19c enables **Information Management** by providing capabilities in content management, information integration, and information lifecycle management areas. You can manage content of advanced data types such as Extensible Markup Language (XML), text, spatial, multimedia, medical imaging, and semantic technologies using the features provided by Oracle.
- With Oracle Database 19c, you can manage all the major **Application Development** environments such as PL/SQL, Java/JDBC, .NET, Windows, PHP, SQL Developer, and Application Express.
- You can now plug into **Oracle Cloud** with Oracle Database 19c. This will help you to standardize, consolidate, and automate database services on the cloud.

Oracle Database 19c



20

Imagine you have an organization that needs to support multiple terabytes of information for users who demand fast and secure access to business applications round the clock. The database systems must be reliable and must be able to recover quickly in the event of any kind of failure. Oracle Database 19c is designed to help organizations manage infrastructure grids easily and deliver high-quality service:

- **Manageability:** By using some of the change assurance, management automation, and fault diagnostics features, the database administrators (DBAs) can increase their productivity, reduce costs, minimize errors, and maximize quality of service. Some of the useful features that promote better management are the Database Replay facility, the SQL Performance Analyzer, the Automatic SQL Tuning facility, and Real-Time Database Operations Monitoring.

Oracle EM Express is a web-based tool for managing Oracle databases. It greatly simplifies database performance diagnostics by consolidating the relevant database performance screens into a view called Database Performance Hub. DBAs get a single, consolidated view of the current real-time and historical view of the database performance across multiple dimensions such as database load, monitored SQL and PL/SQL, and Active Session History (ASH) on a single page for the selected time period.

- **High availability:** By using the high availability features, you can reduce the risk of down time and data loss. These features improve online operations and enable faster database upgrades.
- **Performance:** By using capabilities such as SecureFiles, Result Caches, and so on, you can greatly improve the performance of your database. Oracle Database 19c enables organizations to manage large, scalable, transactional, and data warehousing systems that deliver fast data access using low-cost modular storage.
- **Security:** Oracle Database 19c helps in protecting your information with unique secure configurations, data encryption and masking, and sophisticated auditing capabilities.
- **Information integration:** You can utilize Oracle Database 19c features to integrate data throughout the enterprise in a better way. You can also manage the changing data in your database by using Oracle Database 19c's advanced information lifecycle management capabilities.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Course objectives and course agenda
- The schema and appendixes used in this course
- Overview of Oracle Database 19c and related products
- Available PL/SQL development environments
- Oracle documentation and additional resources



PL/SQL Development Environments

The course setup provides the following tools for developing PL/SQL code:

- Oracle SQL Developer (used in this course)
- Oracle SQL*Plus
- Oracle SQL Developer Web

23

You can use various tools provided by Oracle to write PL/SQL code. In this course, we primarily focus on SQL Developer. We also look at the usage of SQL*Plus.

- **Oracle SQL Developer** is an integrated development environment, which allows database users and administrators to perform their database tasks in fewer clicks and keystrokes. For developers, it provides powerful editors for working with SQL, PL/SQL, stored Java procedures, and XML. You can run queries, generate execution plans, export data to different formats such as XML and HTML, and so on.
- **Oracle SQL*Plus:** SQL*Plus is an interactive and batch query tool that is installed with every installation of Oracle Database. It has a command-line user interface, a Windows Graphical User Interface (GUI), and the *i*SQL*Plus web-based user interface. SQL*Plus has its own commands and environment, and it provides access to Oracle Database.

You can use SQL*Plus to generate reports interactively or as batch processes, and to output the results to a text file, a screen, or to an HTML file for browsing on the Internet. You can generate reports dynamically by using the HTML output facility of SQL*Plus, or by using the dynamic reporting capability of *i*SQL*Plus to run a script from a web page.

- **Oracle SQL Developer Web** is a browser-based application that uses ORDS (Oracle REST Data Services) to provide many of the database development and administration features of desktop-based Oracle SQL Developer

Note: The code and screen examples presented in the course notes were generated from the output in the SQL Developer environment.

Oracle SQL Developer

- Oracle SQL Developer is a free graphical tool that enhances productivity and simplifies database development tasks.
- You can connect to any target Oracle database schema by using standard Oracle database authentication.
- You use SQL Developer in this course.



SQL Developer

24

What Is Oracle SQL Developer?

Oracle SQL Developer is a free graphical tool that is designed to improve productivity and simplify the performance of everyday database tasks. The user interface of SQL Developer allows you to browse, create, and manage database objects.

Using SQL Developer, you can easily create and maintain stored procedures, test SQL statements, and view optimizer plans, with just a few clicks.

SQL Developer, the visual tool for database development, simplifies the following tasks:

- Browsing and managing database objects
- Executing SQL statements and scripts
- Editing and debugging PL/SQL statements
- Creating reports

You can connect to any target Oracle database schema by using standard Oracle database authentication. When you are connected, you can perform operations on the objects in the database.

For Instructor Use Only.
This document should not be distributed.

Specifications of SQL Developer

- Is developed in Java
- Supports the Windows, Linux, and Mac OS X platforms
- Enables default connectivity by using the JDBC Thin driver
- Connects to Oracle Database version 9.2.0.1 and later
- Connects to Oracle Database on Cloud also



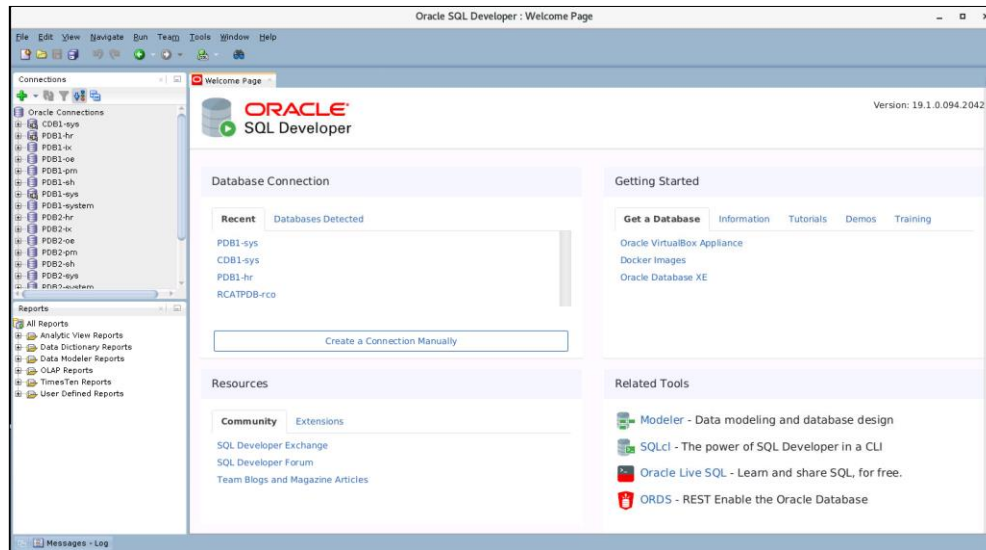
25

Oracle SQL Developer is developed in Java, leveraging the Oracle JDeveloper integrated development environment (IDE). Therefore, it is a cross-platform tool. The tool runs on the Windows, Linux, and Mac operating system (OS) X platforms.

Default connectivity to the database is through the Java Database Connectivity (JDBC) Thin driver; therefore, no Oracle Home is required. SQL Developer does not require an installer. All you need to do is simply unzip the downloaded file. With SQL Developer, users can connect to Oracle Databases 9.2.0.1 and later, and all Oracle database editions, including Express Edition. SQL Developer allows you to connect to Oracle Database Service on cloud also.

For Instructor Use Only.
This document should not be distributed.

SQL Developer Interface



26

The SQL Developer interface contains three main navigation tabs, from left to right:

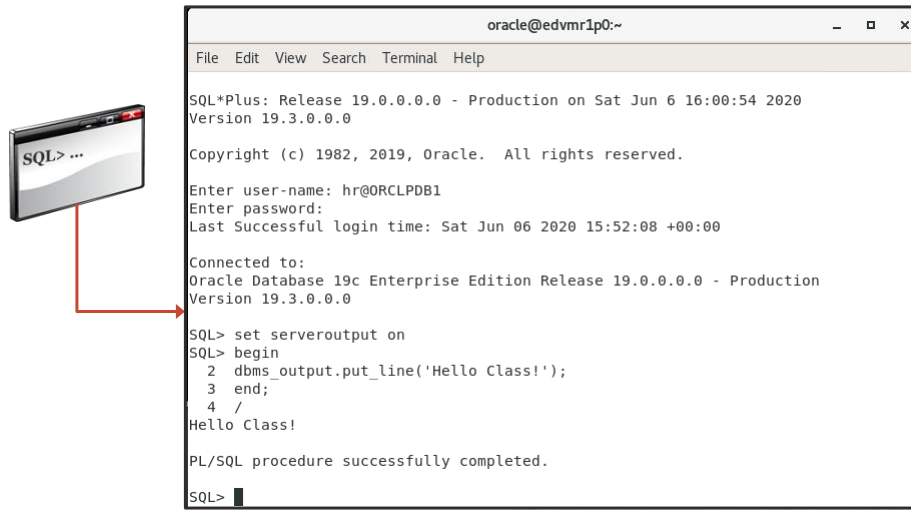
- **The Connections tab:** By using this tab, you can browse database objects and users to which you have access.
- **The Reports tab:** Identified by the Reports icon, this tab enables you to run predefined reports or create and add your own reports.
- **The Start page:** This page gives you some links that are helpful when you use SQL Developer to create applications.

General Navigation and Use

SQL Developer uses the left pane for navigation to find and select objects. You can customize many aspect of the appearance and behavior of SQL Developer by setting preferences.

Note: You need to define at least one connection to be able to connect to a database schema and issue SQL queries or run procedures and functions. You can start the connection creation wizard by clicking “+” on the Connections tab.

Coding PL/SQL in SQL*Plus



27

Oracle SQL*Plus is a command-line interface that enables you to submit SQL statements and PL/SQL blocks for execution and receive results in an application or a command window.

SQL*Plus is:

- Shipped with the database
- Installed on a client and on the database server system
- Accessed by using an icon or the command line

When you code PL/SQL subprograms by using SQL*Plus, remember the following:

- You create subprograms by using the `CREATE SQL` statement.
- You execute subprograms by using either an anonymous PL/SQL block or the `EXECUTE` command.
- If you use the `DBMS_OUTPUT` package procedures to print text to the screen, you must first execute the `SET SERVEROUTPUT ON` command in your session.

SQL Developer Web

Oracle SQL Developer Web is a browser-based application that uses Oracle REST Data Services (ORDS) to provide many of the database development and administration features of the desktop-based Oracle SQL Developer.

The SQL Developer Web user interface has three components:

- The header at the top
- The page body, whose content varies depending on which page you are viewing
- The status bar at the bottom

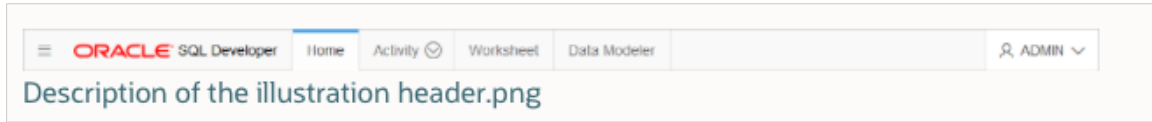
28

The main features of SQL Developer Web include running SQL statements and scripts in the worksheet, exporting data, creating Data Modeler diagrams for existing schemas, and enabling database administrators to monitor the database.

SQL Developer Web

The SQL Developer Web user interface has three components:

- The header at the top



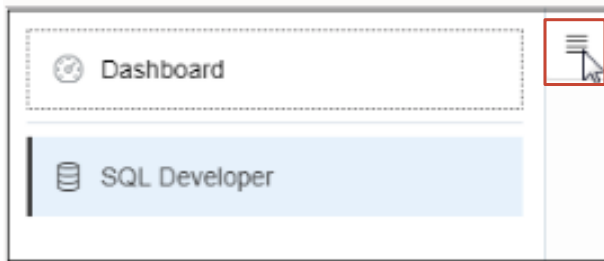
29

The header contains the selector icon, the main menu, and the schema drop-down list.

SQL Developer Web

The SQL Developer Web user interface has three components:

- The page body, whose content varies depending on which page you are viewing
- Use the selector icon to control what you are viewing.



30

Main Menu

The main menu lists the pages available. The **Dashboard** is a single page and so it has no main menu. The **SQL Developer** header's main menu has the following tabs:

- **Home:** Displays widgets that provide a general overview of the activity and status in SQL Developer Web
- **Activity:** Provides several pages to view the performance and other characteristics of your database
- **Worksheet:** Enables you to enter and execute SQL and PL/SQL commands
- **Data Modeler:** Enables you to create diagrams from existing database schemas, generate DDL statements, and create reports

Schema Drop-Down List

The schema drop-down list shows the database user you are signed in as, and provides the following items when you open it:

- **Preferences:** Opens a dialog box that lets you set your preferred time zone
- **Log:** Opens a dialog box that shows the list of HTTP calls made during your SQL Developer Web session
- **Help:** Opens online help for the page you are viewing in a new browser tab
- **About:** Opens a dialog box providing version information for the database and other components as well as copyright and licensing information
- **Sign Out:** Signs you out of your database session

SQL Developer Web

The SQL Developer Web user interface has three components:

- The status bar at the bottom



31

The status bar contains icons that link to log files. The three icons (Errors, Warnings, Processes) are filters that have been applied to the log file.

- **Errors, Warnings:** Displays an Errors or Warnings dialog box, which lists log entries from unsuccessful REST calls or from any other problem in the application
- **Processes:** Displays a Processes dialog, which logs REST calls that are either finished or ongoing
- **Log Notification Link:** Displays a Log dialog box, containing log entries of the following types: Errors, Warnings, Processes, SQL History, and SQL Result

SQL Developer Web

Accessing Oracle SQL Developer Web

Oracle SQL Developer Web runs in Oracle REST Data Services and access to it is provided through schema-based authentication. To use Oracle SQL Developer Web, you must sign in as a database user whose schema has been enabled for SQL Developer Web.

In Oracle Autonomous Database databases, the `ADMIN` user is pre-enabled for SQL Developer Web. To enable another database user's schema.

On the SQL Developer Web login page, enter the username and password of the database user for the enabled schema. Then, click **Sign In**. The Worksheet page is displayed.

32

The main features include running SQL statements and scripts in the worksheet, exporting data, creating Data Modeler diagrams for existing schemas, and enabling database administrators to monitor the database.

Lesson Agenda

- Course objectives and course agenda
- The schema and appendixes used in this course
- Overview of Oracle Database 19c and related products
- Available PL/SQL development environments
- Oracle documentation and additional resources



Oracle SQL and PL/SQL Documentation

- *Oracle Database PL/SQL Language Reference*
- *Oracle Database Reference*
- *Oracle Database SQL Language Reference*
- *Oracle Database Concepts*
- *Oracle Database PL/SQL Packages and Types Reference*
- *Oracle Database SQL Developer User's Guide*
- *Oracle Database 2-Day Developer's Guide*
- *Getting Started with Oracle Cloud*



Additional Resources

For additional information about the new Oracle SQL and PL/SQL features, refer to:

- Oracle Database: New Features self study
- What's New in PL/SQL in Oracle Database on the Oracle Technology Network (OTN):
 - <https://www.oracle.com/database/technologies/appdev/plsql.html>
- The online SQL Developer home page and the SQL Developer tutorial available at:
 - http://www.oracle.com/technology/products/database/sql_developer/index.html
 - <http://download.oracle.com/oll/tutorials/SQLDeveloper/index.htm>

Summary

In this lesson, you should have learned how to:

- Discuss the goals of the course
- Describe the HR database schema that is used in the course
- Identify the available user interface environments that can be used in this course
- Reference the available appendixes, documentation, and other resources



For Instructor Use Only.
This document should not be distributed.

Practice 1 Overview: Getting Started

This practice covers the following topics:

- Starting SQL Developer
- Creating a new database connection
- Browsing the HR schema tables
- Setting a SQL Developer preference



0

37

In this practice, you use SQL Developer to execute SQL statements to examine data in the `HR` schema. You also create a simple anonymous block.

Note: All written practices use SQL Developer as the development environment. Although it is recommended that you use SQL Developer, you can also use the SQL*Plus environment that is available in this course.

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Introduction to PL/SQL

Course Road Map



2

You are in lesson 2, which is part of Unit 1: Introducing PL/SQL.

0
For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to do the following:

- Explain the need for PL/SQL
- Explain the benefits of PL/SQL
- Identify the different types of PL/SQL blocks
- Output messages in PL/SQL



3

This lesson introduces PL/SQL and the PL/SQL programming constructs. You also learn about the benefits of PL/SQL.

For Instructor Use Only.
This document should not be distributed.

Agenda

- Understanding the benefits and structure of PL/SQL
- Understanding PL/SQL blocks
- Generating output messages in PL/SQL



Limitations of SQL

- Performs one operation at a time on the database
- Lacks the capability of logically grouping multiple database operations



5

Structured Query Language (SQL) provides a standard interface to access relational databases. All SQL statements are instructions to relational databases.

A SQL statement can query and manipulate a database, one operation at a time. A single SQL statement can create a table in a database, query data from the database, modify data in the database, or delete data in the database; however each SQL statement can perform any one of these operation only at a time.

Let's look at a scenario in which you transfer funds by using a Bank application. A typical funds transfer will have the following steps to be performed on the database.

1. Deduct funds from the sender's account (an update operation on the corresponding row).
2. Add funds to the receiver's account (an update operation on the receiver's account).

This scenario may also have conditional checks such as:

1. You may want to check whether there are sufficient funds in the sender's account.
2. You may also want to check whether the current transaction will take the balance in the sender's account below minimum balance.

SQL provides for all these conditional checks and updates as independent individual operations. PL/SQL enables you to group such operations which together make a logical unit in the application context.

Why PL/SQL?

- Enables modularization in the application
- Provides better security
- Enables maintainability
- Provides exception handling



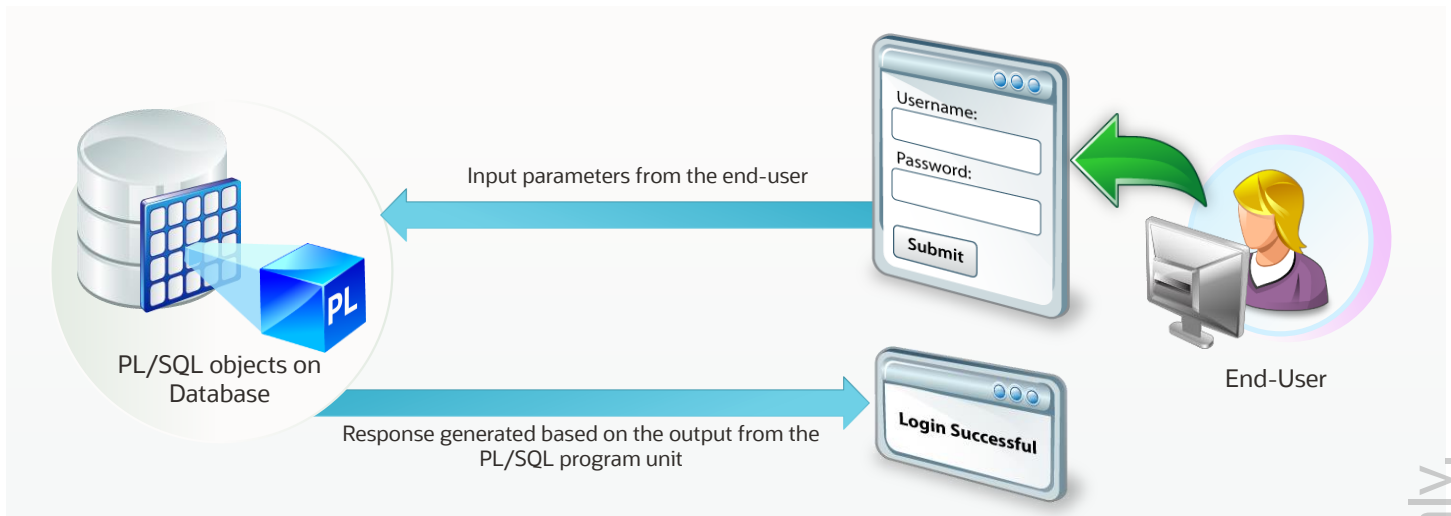
6

PL/SQL provides the following features, which enable efficient application development:

- **Modularization:** Large software systems are developed as modules first, which are later integrated together. The modules communicate through interfaces and work together as a single application.
- **Security:** The implementation of the module is invisible to the end user. That is, the end user knows only the input and output of the module, which is the interface of the module. Security is provided by hiding the implementation details.
- **Maintainability:** When you want to upgrade the performance of a module, you can do it independent of the other modules in the application.
- **Exception handling:** PL/SQL enables you to handle exceptions efficiently. You can define separate blocks for dealing with exceptions. You learn more about exception handling in the lesson titled “Handling Exceptions.”

For Instructor Use Only.
This document should not be distributed.

Why PL/SQL



7

Consider a scenario where the HR manager of a company has to make some changes to the database. The HR manager cannot directly access the database. The system will first have to authenticate the HR manager. He/she must provide security credentials such as a username and password. These credentials are then verified against the credentials in the user database to authenticate the HR manager.

The HR application would, therefore, provide a user interface that would accept the required credentials from the end user.

These credentials are provided as parameters to a PL/SQL object on the database.

Note: Practical implementation of this scenario would include other intermediary infrastructure components. For simplicity, we have considered only the user interface and the database components.

The PL/SQL program units reside on the database and hide the implementation from end users. A PL/SQL unit that is defined for user authentication would accept credentials as parameters and perform the authentication process on the database.

In this case, the PL/SQL program unit would accept a username and password from the end user, and verify them against those that exist in the database. The authentication process would require execution of multiple SQL statements. A PL/SQL program unit groups these SQL statements into a single program unit, which can be explicitly invoked.

About PL/SQL



- Stands for “Procedural Language Extension to SQL”
- Integrates procedural constructs with SQL
- Provides a block structure for executable units of code
- Provides procedural constructs such as:
 - Variables, constants, and data types
 - Control structures: Loops, conditional statements
- Enables writing reusable program units

8

PL/SQL is a Procedural Language extension to SQL. PL/SQL integrates SQL statements into procedural language structures such as conditional statements and loop constructs.

Consider a scenario where you want to run an operation such as “Giving 10% bonus to all the employees in the company” on the database. If you execute this operation through SQL, you may have to execute an `UPDATE` statement for each row in the `Employees` table.

With PL/SQL, you can execute the `UPDATE` statement (SQL statement) on each row of the `Employees` table by using a loop construct (procedural construct). Thus, PL/SQL makes things simpler.

Maintaining and debugging code is also made easier with such a structure because you can easily understand the flow and execution of the program unit.

For Instructor Use Only.
This document should not be distributed.

Benefits of PL/SQL

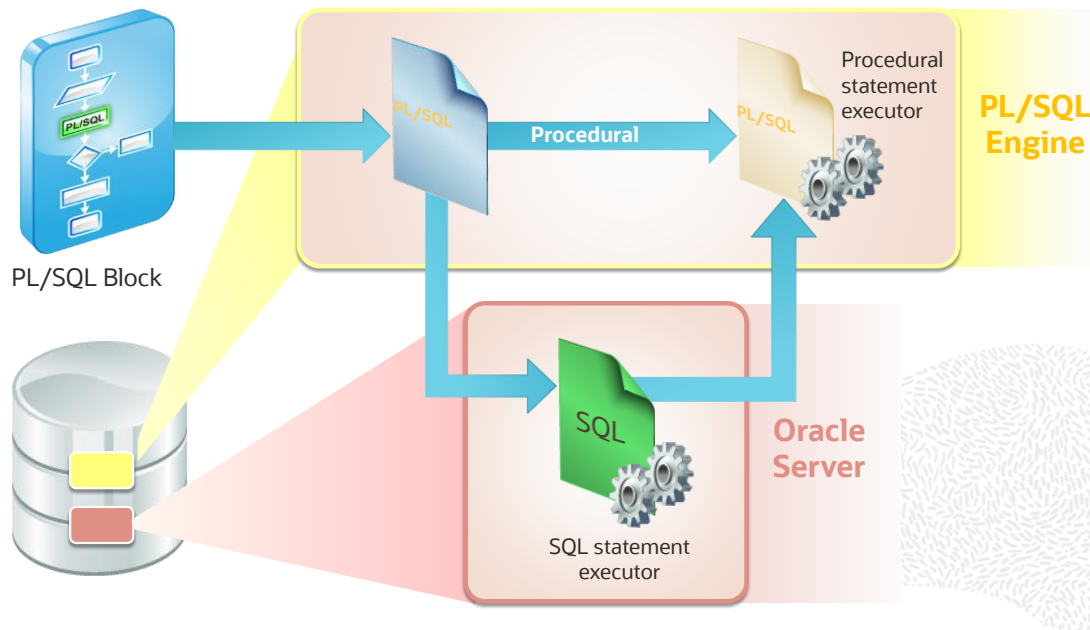
- Integration of procedural constructs with SQL
- Improved performance
- Modularized program development
- Integration with Oracle tools
- Portability
- Exception handling
- Support for Object Oriented Programming

9

- **Integration of procedural constructs with SQL:** Being a declarative language, SQL does not have procedural constructs such as conditional statements and loop statements. With PL/SQL, you can combine the data manipulating capability of SQL with the processing power of procedural languages.
- **Improved performance:** You can create a program unit in PL/SQL, which can execute multiple SQL statements. When a SQL statement is executed, there is data exchange between the client and the database server. In the case of a PL/SQL program unit, which contains multiple SQL statements, there is exchange of data only once as a single PL/SQL program unit. This, in turn, improves application performance because of reduced data exchange.
- **Modularized program development:** The basic unit in all PL/SQL programs is the block. Blocks can be in a sequence or they can be nested in other blocks. Modularized program development has the following advantages:
 - You can group logically related statements within blocks.
 - You can nest blocks inside larger blocks to build powerful programs.
 - You can break down your application into smaller modules. If you are designing a complex application, PL/SQL allows you to break down the application into smaller, manageable, and logically related modules.
 - You can easily maintain and debug code.

- **Integration with tools:** The PL/SQL engine is integrated in Oracle tools such as Oracle Forms and Oracle Reports. When you use these tools, the locally available PL/SQL engine processes the procedural statements; only the SQL statements are passed to the database.
- **Portability:** PL/SQL programs can run anywhere that an Oracle Server runs, irrespective of the operating system and platform. You do not need to customize them for each new environment. You can write portable program packages and create libraries that can be reused in different environments.
- **Exception handling:** PL/SQL enables you to handle exceptions efficiently. You can define separate blocks for dealing with exceptions. You learn more about exception handling in the lesson titled “Handling Exceptions.”

PL/SQL Runtime Architecture



11

The execution of a PL/SQL block has two parts:

1. Execution of the procedural statements
2. Execution of the SQL statements

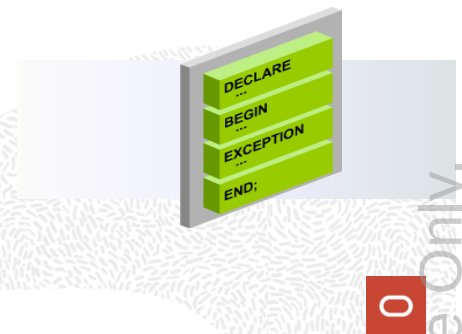
The procedural logic is executed on a PL/SQL engine and the SQL logic is executed on the Oracle Server.

The PL/SQL engine is a virtual machine that resides in memory and processes the PL/SQL m-code instructions. You can install the PL/SQL engine in the database or in an application development tool such as Oracle Forms. It processes all the procedural statements and passes the SQL statements to the SQL Engine on the Oracle Server.

If the PL/SQL unit does not have any SQL statements, the entire PL/SQL unit is processed by the PL/SQL engine. The SQL engine may invoke a PL/SQL statement executor to execute any function calls that are present in the SQL statements.

PL/SQL Block Structure

- **DECLARE (optional)**
 - Variables, cursors, user-defined exceptions
- **BEGIN (mandatory)**
 - SQL statements
 - PL/SQL statements
- **EXCEPTION (optional)**
 - Actions to perform when exceptions occur
- **END (mandatory)**



12

The slide shows a basic PL/SQL block. A PL/SQL block consists of four sections:

- **DECLARE (optional):** The declarative section begins with the keyword `DECLARE` and ends when the executable section starts. You may or may not have a declaration section, because you may not always declare variables or constants for the PL/SQL block. You can declare variables, constants, cursors, and user-defined exceptions here.
- **BEGIN (required):** The executable section begins with the keyword `BEGIN`. This is the section where you write the procedural or SQL statements that must be executed. You should have at least one statement. The executable section of a PL/SQL block can include any number of PL/SQL blocks.

Note: Every `BEGIN` statement should have a corresponding `END` statement to demarcate the PL/SQL block.

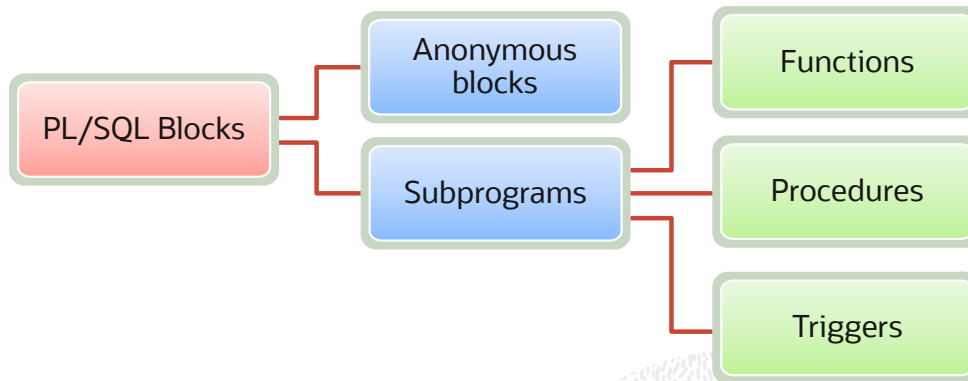
- **EXCEPTION (optional):** The exception section is nested within the executable section. This section begins with the keyword `EXCEPTION`.
- **END (required):** All PL/SQL blocks must conclude with an `END` statement. Observe that `END` is terminated with a semicolon.

Agenda

- Understanding the benefits and structure of PL/SQL
- Understanding PL/SQL blocks
- Generating output messages in PL/SQL



Block Types



14

A PL/SQL program comprises one or more blocks.

There are two types of PL/SQL blocks:

- Anonymous blocks
- Named PL/SQL blocks, which are also termed as subprograms

Anonymous blocks: Anonymous blocks are unnamed blocks. They are declared inline at the point in an application where they are to be executed, and are compiled each time the application is executed. These blocks are not stored in the database. They are passed to the PL/SQL engine for execution at run time. If you want to execute the same block again, you may have to rewrite the block.

Subprograms: Subprograms are named PL/SQL blocks that are stored in the database. They are reusable blocks, and can be invoked by the application as per requirement. Subprograms are stored on the database server, and can also be invoked by other subprograms.

Procedures: Procedures are named PL/SQL blocks that can be explicitly executed. Procedures help in modularizing application logic.

Functions: Functions, like procedures, are named PL/SQL blocks that are explicitly invoked, which are capable of returning a value.

Both procedures and functions can accept some input parameters and process based on these parameters.

Triggers: Triggers are PL/SQL blocks that are conditionally invoked based on an event or operation that occurs in the database context.

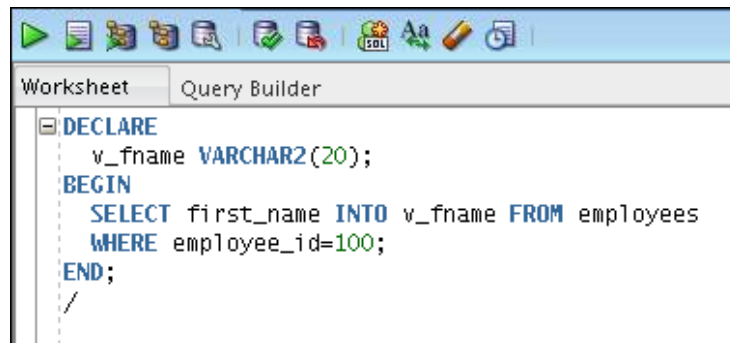
For instance, if you want to write all the operations that occur in the database to a log file, you can define a trigger, which would do just that. You can define the trigger such that each time an insert, a delete, or an update is performed on the database, an entry is made to the log.

We discuss Procedures, Functions, and Triggers later in this course.

For Instructor Use Only.
This document should not be distributed.

Examining an Anonymous Block

An anonymous block in the SQL Developer workspace:

A screenshot of the SQL Developer Query Builder workspace. The window has a toolbar at the top with icons for running, saving, and other actions. Below the toolbar, there are two tabs: 'Worksheet' and 'Query Builder'. The 'Query Builder' tab is active, and it contains a text area with the following PL/SQL code:

```
DECLARE
  v_fname VARCHAR2(20);
BEGIN
  SELECT first_name INTO v_fname FROM employees
  WHERE employee_id=100;
END;
```

16

To create an anonymous block by using SQL Developer, enter the block in the workspace (as shown in the slide).

Example

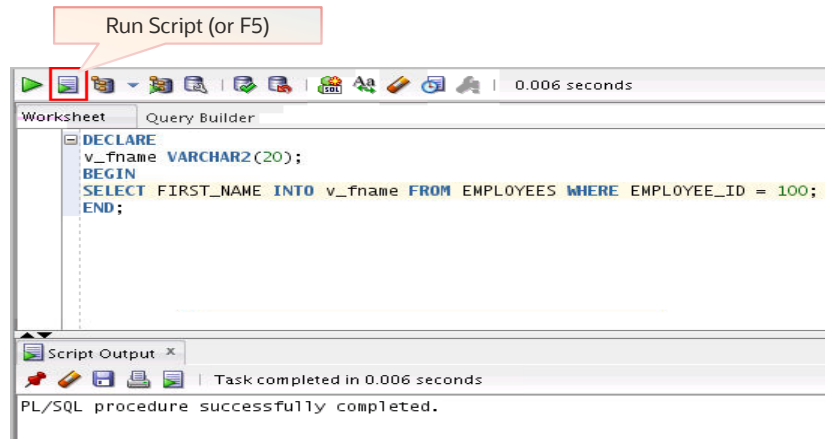
The example block has a declarative section and an executable section. You need not pay attention to the syntax of the statements in the block at this point in time.

The anonymous block gets the `first_name` of the employee whose `employee_id` is 100, and stores it in a variable called `v_fname`.

Note: You can also write PL/SQL code through SQL*Plus, which is a command-line tool.

Executing an Anonymous Block

Click the Run Script icon to execute the anonymous block:



17

To execute an anonymous block, click the Run Script icon (or press F5).

Note: The message “PL/SQL procedures successfully completed” is displayed in the Script Output window after the block is executed.

Agenda

- Understanding the benefits and structure of PL/SQL
- Understanding PL/SQL blocks
- Generating output messages in PL/SQL



Enabling Output of a PL/SQL Block

1. To enable output in SQL Developer, execute the following command before running the PL/SQL block:

```
SET SERVEROUTPUT ON
```

2. Use a predefined Oracle package and its procedure in the anonymous block:

```
-- DBMS_OUTPUT.PUT_LINE
```

```
DBMS_OUTPUT.PUT_LINE('The First Name of the Employee is ' ||  
v_fname);
```

```
...
```

19

PL/SQL does not have built-in input or output functionality. Therefore, you need to use predefined Oracle packages for input and output. To generate output, perform the following steps:

1. Execute the following command:

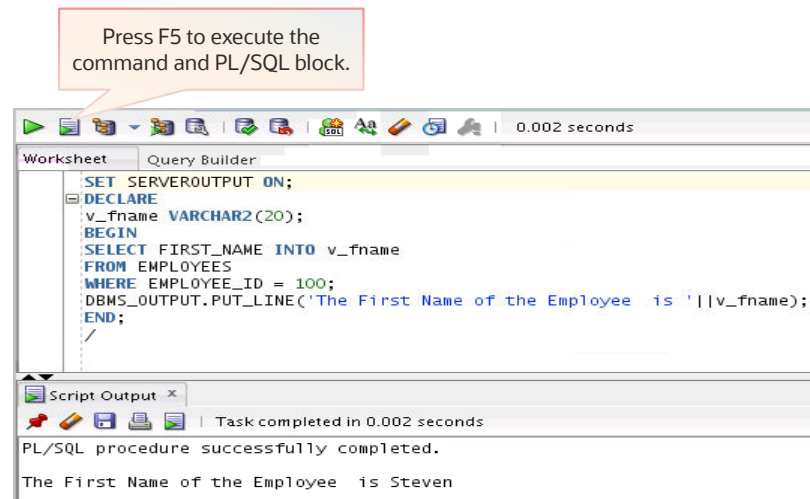
```
SET SERVEROUTPUT ON
```

Note: To enable output in SQL*Plus, you must explicitly issue the `SET SERVEROUTPUT ON` command.

2. In the PL/SQL block, use the `PUT_LINE` procedure of the `DBMS_OUTPUT` package to display the output. Pass the value that must be printed as an argument to this procedure (as shown in the slide). The procedure then outputs the argument.

Note: We discuss Packages later in this course. A package is a logical grouping of different PL/SQL program blocks and other database objects. It may have procedures, functions, triggers, sequences, and other database objects grouped according to context.

Viewing the Output of a PL/SQL Block



20

Press F5 (or click the Run Script icon) to view the output for the PL/SQL block. This action:

1. Executes the `SET SERVEROUTPUT ON` command
2. Runs the anonymous PL/SQL block

The output appears on the Script Output tab.

Quiz



A PL/SQL block *must* consist of the following three sections:

- A Declarative section, which begins with the keyword `DECLARE` and ends when the executable section starts
 - An Executable section, which begins with the keyword `BEGIN` and ends with `END`
 - An Exception handling section, which begins with the keyword `EXCEPTION` and is nested within the executable section
- a. True
b. False



21

Answer: b

A PL/SQL block consists of three sections:

- **Declarative (optional):** The optional declarative section begins with the keyword `DECLARE`, and ends when the executable section starts.
- **Executable (required):** The required executable section begins with the keyword `BEGIN` and ends with `END`. This section essentially needs to have at least one statement. Observe that `END` is terminated with a semicolon. The executable section of a PL/SQL block can, in turn, include any number of PL/SQL blocks.
- **Exception handling (optional):** The optional exception section is nested within the executable section. This section begins with the keyword `EXCEPTION`.

Summary

In this lesson, you should have learned how to:

- Explain the need for PL/SQL
- Explain the benefits of PL/SQL
- Identify the different types of PL/SQL blocks
- Output messages in PL/SQL



22

PL/SQL is a language that has programming features that serve as extensions to SQL. SQL, which is a non-procedural language, is made procedural with the PL/SQL programming constructs. PL/SQL applications can run on any platform or operating system on which an Oracle Server runs. In this lesson, you learned how to build basic PL/SQL blocks.

For Instructor Use Only.
This document should not be distributed.

Practice 2: Overview

This practice covers the following topics:

- Identifying PL/SQL blocks that execute successfully
- Creating and executing a simple PL/SQL block



0

23

This practice reinforces the basics of PL/SQL covered in this lesson.

- Exercise 1 is a paper-based exercise in which you identify PL/SQL blocks that execute successfully.
- Exercise 2 involves creating and executing a simple PL/SQL block.

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Declaring PL/SQL Variables

Course Road Map



2

You are in lesson 3, which is part of Unit 1: Introducing PL/SQL.

0
For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to do the following:

- Recognize valid and invalid identifiers
- List the uses of variables
- Declare and initialize variables
- List and describe various data types
- Identify the benefits of using the `%TYPE` attribute
- Declare, use, and print bind variables



0

3

You have already learned about basic PL/SQL blocks and their sections. In this lesson, you learn about valid and invalid identifiers. You learn how to declare and initialize variables in the declarative section of a PL/SQL block. The lesson describes the various data types. You also learn about the `%TYPE` attribute and its benefits.

For Instructor Use Only.
This document should not be distributed.

Agenda

- Introducing variables
- Handling variables of different types
- Using the %TYPE attribute and composite data types
- Using bind variables



Variables

- Are labeled storage locations
- Are used to store and process data in a PL/SQL block
- Can hold different types of data
- Should declare variables before using them in the PL/SQL block



5

Why do we need variables?

We talk about variables in every programming language. PL/SQL is no exception. Variables are storage locations to store data that is processed in a given PL/SQL block.

These variables serve as placeholders in the machine's memory. You should declare these variables in the `DECLARE` section of the PL/SQL block before using them in the code.

Variables can hold different types of data such as numbers, characters, character strings, date, time, and so on.

For Instructor Use Only.
This document should not be distributed.

Variables in PL/SQL

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY
1	100	Steven	King	SKING	515.123.4567	17-JUN-11	AD_PRES	24000
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-SEP-09	AD_VP	17000
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13-JAN-09	AD_VP	17000
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	03-JAN-14	IT_PROG	9000

```
DECLARE
  v_sal NUMBER(8,2);
BEGIN
  SELECT salary INTO v_sal
  FROM employees
  WHERE employee_id = 100;
  UPDATE employees
  SET salary = v_sal+100
  WHERE employee_id = 100;
END;
```

Script Output x
Task completed in 0.002 seconds
PL/SQL procedure successfully completed.

6

Let us now see the usage of variables in PL/SQL.

Consider a scenario where you are required to update the salary of an employee with employee ID 100. The update will be an increment by \$100.

To perform this operation, you should first retrieve the salary of the employee into a variable `emp_sal`.

```
SELECT salary INTO v_sal FROM employees WHERE employee_id = 100;
```

Then you run an `UPDATE` command on it.

```
UPDATE employees SET salary = v_sal+100 WHERE employee_id = 100;
```

Variables in PL/SQL

Columns Data Constraints Grants Statistics Triggers Flashback Dependencies Details Partitions Indexes SQL								
Sort... Filter:								
	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY
1	100	Steven	King	SKING	515.123.4567	17-JUN-11	AD_PRES	24100
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-SEP-09	AD_VP	17000
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13-JAN-09	AD_VP	17000
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	03-JAN-14	IT_PROG	9000

7

The slide displays data in the employees table after executing the UPDATE operation. The salary of the employee with employee_id 100 is updated to \$24,100 from the initial value of \$24,000.

For Instructor Use Only.
This document should not be distributed.

Requirements for Variable Names

A variable name:

- Must start with a letter
- Can include letters or numbers
- Can include special characters (\$, _, #)
- Must contain no more than 30 characters
- Must not include reserved words

8

Each variable is associated with a name, which is also termed an identifier. When you declare a variable, you specify the identifier name and the type of data that the variable will have during the execution of the PL/SQL block. (You learn about the data types that are supported in PL/SQL later in this lesson).

Follow the rules shown in the slide while defining variable names in a PL/SQL block.

Examples of some valid identifiers are as follows:

- X
- t2
- phone#
- credit_limit
- LastName
- oracle\$number
- money\$\$\$
- try_again_

Using Variables in PL/SQL

Variables are:

- Declared and (optionally) initialized in the declaration section
- Used and assigned new values in the executable section
- Passed as parameters to PL/SQL subprograms
- Used to hold the output of a PL/SQL subprogram

9

You can use variables in the following ways:

- **Declare and initialize them in the declaration section:** You can declare variables in the `DECLARE` section of any PL/SQL block, subprogram, or package. Declarations allocate storage space for a value, specify its data type, and name the storage location so that you can reference it. Declarations can also assign an initial value and impose the `NOT NULL` constraint on the variable. You must declare a variable before referencing it in other statements, including other declarative statements.
- **Use variables and assign new values to them in the executable section:** In the executable section, the existing value of the variable can be replaced with a new value.
- **Pass them as parameters to PL/SQL subprograms:** Subprograms can take parameters. You can pass variables as parameters to subprograms.
- **Use them to hold the output of a PL/SQL subprogram:** Variables can be used to hold the value that is returned by a function.

Declaring and Initializing PL/SQL Variables

Syntax:

```
identifier [CONSTANT] datatype [NOT NULL]
[:= | DEFAULT expr];
```

Examples:

```
DECLARE
  v_hiredate      DATE;
  v_location      VARCHAR2(13) := 'Atlanta';
  v_deptno        NUMBER(2) NOT NULL := 10;
  c_comm          CONSTANT NUMBER := 1400;
```

10

In the syntax shown in the slide:

<i>identifier</i>	Is the name of the variable
<i>data type</i>	Is a scalar, composite, reference, or <code>LOB</code> data type (This course covers only scalar, composite, and <code>LOB</code> data types.)
<code>CONSTANT</code>	Constrains the variable so that its value cannot change (Constants must be initialized.)
<code>NOT NULL</code>	Constrains the variable so that it contains a value (<code>NOT NULL</code> variables must be initialized.)
<i>expr</i>	Is any PL/SQL expression that can be a literal expression, another variable, or an expression involving operators and functions
<code>DEFAULT</code>	Is the predefined default value that every data type has. You can initialize the variable of a data type to a default value by using <code>DEFAULT</code> during declaration.

Note: In addition to variables, you can also declare cursors and exceptions in the `DECLARE` section. You learn about declaring cursors in the lesson titled “**Using Explicit Cursors**” and about exceptions in the lesson titled “**Handling Exceptions**.”

Agenda

- Introducing variables
- Handling variables of different types
- Using the %TYPE attribute and composite data types
- Using bind variables



Declaring and Initializing PL/SQL Variables

1

```
DECLARE
    v_myName VARCHAR2(20);

BEGIN
    DBMS_OUTPUT.PUT_LINE('My name is: '||v_myName );
    v_myName := 'John';
    DBMS_OUTPUT.PUT_LINE('My name is: '||v_myName );
END;
/
```

2

```
DECLARE
    v_myName VARCHAR2(20) := 'John';

BEGIN
    v_myName := 'Steven';
    DBMS_OUTPUT.PUT_LINE('My name is: '|| v_myName);
END;
/
```

12

Initializing is assigning an initial value to the variable. In the slide, we demonstrate the difference between initializing and declaring.

1. In the first block, the `v_myName` variable is declared but not initialized. A value `John` is assigned to the variable in the executable section.
 - String literals must be enclosed in single quotation marks while initializing.
 - Variables are assigned values with the assignment operator “:=”.
 - The `PUT_LINE` procedure is invoked by passing the `v_myName` variable. The value of the variable is concatenated with the string `'My name is: '`.
 - The output of this anonymous block is:

```
PL/SQL procedure successfully completed.

My Name is:
My Name is: John
```

2. In the second block, the `v_myName` variable is declared and initialized in the `DECLARE` section. `v_myName` holds the value `John` after initialization. This value is manipulated in the executable section of the block. The output of this anonymous block is:

```
PL/SQL procedure successfully completed.

My Name is: Steven
```

Initializing Variables Through a SELECT Statement

Retrieve data from the database with a `SELECT` statement.

Syntax:

```
SELECT  select_list
INTO    {variable_name[, variable_name]...}
FROM    table
[WHERE  condition];
```

13

You can initialize the variables in a PL/SQL block through a `SELECT` statement by using the `INTO` clause. The variables following the `INTO` clause should be declared in the `DECLARE` section.

<i>select_list</i>	Specifies a list of at least one column; can include SQL expressions, row functions, or group functions
<i>variable_name</i>	Is the scalar variable that holds the retrieved value
<i>record_name</i>	Is the PL/SQL record that holds the retrieved values
<i>table</i>	Specifies the database table name
<i>condition</i>	Is composed of column names, expressions, constants, and comparison operators, including PL/SQL variables and constants

Guidelines for Retrieving Data Through a `SELECT` Statement:

- Terminate each SQL statement with a semicolon (;).
- Every value retrieved must be stored in a variable by using the `INTO` clause.
- When you use the `INTO` clause, you should fetch only one row. You can use the `WHERE` clause such that the number of values retrieved must match the number of variables following the `INTO` clause.
- Specify the same number of variables in the `INTO` clause as the number of database columns in the `SELECT` clause. Be sure that they correspond positionally and that their data types are compatible.
- You can use group functions, such as `SUM`, in a SQL statement, and fetch that value into a variable. Group functions apply to groups of rows in a table and result in a single value.

Types of Variables

- PL/SQL variables:
 - Scalar
 - Reference
 - Large object (LOB)
 - Composite
- Non-PL/SQL variables: Bind variables

15

Each PL/SQL variable has a data type, which specifies a storage format, constraints, and a valid range of values. PL/SQL supports several data type categories, including scalar, reference, large object (LOB), and composite. The variable type is defined by the data type it is declared for. There are four categories of data types:

- **Scalar data types:** Scalar data types hold a single value. The value depends on the data type of the variable. For example, `VARCHAR2` is a scalar data type. The `v_myName` variable in the example in the section titled “Declaring and Initializing PL/SQL Variables” (in this lesson) is of type `VARCHAR2`. Therefore, `v_myName` can hold a string value.
- **Reference data types:** Reference data types hold values, called *pointers*, which point to a storage location.
- **LOB data types:** LOB data types hold values, called *locators*, which specify the location of large objects (such as graphic images) that are stored outside the table.
- **Composite data types:** Composite data types are available by using PL/SQL *collection* and *record* variables. PL/SQL collections and records contain internal elements that you can treat as individual variables.

In this lesson, we look at the Scalar data types.

Non-PL/SQL variables include host language variables or data from the screen fields in an application user interface. You learn more about bind variables later in this lesson.

For more information about LOBs, see the *PL/SQL User's Guide and Reference*.

Declaring Variables

Examples:

```
DECLARE
  v_emp_job          VARCHAR2(9);
  v_count_loop       BINARY_INTEGER := 0;
  v_dept_total_sal    NUMBER(9,2) := 0;
  v_orderdate         DATE := SYSDATE + 7;
  c_tax_rate          CONSTANT NUMBER(3,2) := 8.25;
  v_valid             BOOLEAN NOT NULL := TRUE;
  ...
```

16

The example variable declarations shown in the slide are defined as follows:

- **v_emp_job:** Variable to store an employee job title; the string is a variable length string with an upper limit of 9 characters
- **v_count_loop:** Variable to count the iterations of a loop; initialized to 0
- **v_dept_total_sal:** Variable to accumulate the total salary for a department; initialized to 0. The numeric value can accommodate decimal values with a precision of two digits.
- **v_orderdate:** Variable to store the ship date of an order; initialized to one week from today
- **c_tax_rate:** Constant variable for the tax rate (which never changes throughout the PL/SQL block); set to 8.25
- **v_valid:** Flag to indicate whether a piece of data is valid or invalid; initialized to TRUE

Guidelines for Declaring and Initializing PL/SQL Variables

- Follow consistent naming conventions.
- Use meaningful identifiers for variables.
- Initialize variables that are designated as `NOT NULL` and `CONSTANT`.
- Initialize variables with the assignment operator (`:=`) or the `DEFAULT` keyword:

```
v_myName VARCHAR2(20) := 'John';
```

```
v_myName VARCHAR2(20) DEFAULT 'John';
```

- Declare one identifier per line for better readability and code maintenance.

17

0

Some guidelines to follow when you declare PL/SQL variables are as follows:

- Follow consistent naming conventions—for example, you might use `name` to represent a variable and `c_name` to represent a constant. Similarly, to name a variable, you can use `v_fname`. The key is to apply your naming convention consistently for easier identification.
- Use meaningful and appropriate identifiers for variables. For example, consider using `salary` and `sal_with_commission` instead of `salary1` and `salary2`, where we understand the semantics of the column name according to the application context.
- If you use the `NOT NULL` constraint, you must assign a value when you declare the variable.
- In constant declarations, the `CONSTANT` keyword must precede the data type name. The following declaration names a constant of the `NUMBER` type and assigns the value of 50,000 to the constant. A constant must be initialized in its declaration; otherwise, you get a compilation error. After initializing a constant, you cannot change its value.

```
c_sal CONSTANT NUMBER := 50000.00;
```

Guidelines for Declaring PL/SQL Variables

- Avoid using column names as identifiers.

```
DECLARE
  employee_id  NUMBER(6);
BEGIN
  SELECT  employee_id
  INTO    employee_id
  FROM    employees
  WHERE   last_name = 'Kochhar';
END;
/
```



- Use the `NOT NULL` constraint when the variable must hold a value.

- Initialize the variable to an expression with the assignment operator (`:=`) or with the `DEFAULT` reserved word. If you do not assign an initial value, the new variable contains `NULL` by default until you assign a value. To assign or reassign a value to a variable, you write a PL/SQL assignment statement. However, it is a good programming practice to initialize all variables.
- Two objects can have the same name only if they are defined in different blocks. Where they coexist, you can qualify them with labels and use them.
- Avoid using column names as identifiers. If PL/SQL variables occur in SQL statements and have the same name as a column, the Oracle Server assumes that it is the column that is being referenced. Although the code example in the slide works, code that is written using the same name for a database table and a variable is not easy to read or maintain.
- Impose the `NOT NULL` constraint when the variable must contain a value. You cannot assign nulls to a variable that is defined as `NOT NULL`. The `NOT NULL` constraint must be followed by an initialization clause.

```
v_pincode VARCHAR2(15) NOT NULL := 'Oxford';
```

Naming Conventions of the PL/SQL Structures Used in This Course

PL/SQL Structure	Convention	Example
Variable	v_variable_name	v_rate
Constant	c_constant_name	c_rate
Subprogram parameter	p_parameter_name	p_id
Bind (host) variable	b_bind_name	b_salary
Cursor	cur_cursor_name	cur_emp
Record	rec_record_name	rec_emp
Type	type_name_type	ename_table_type
Exception	e_exception_name	e_products_invalid
File handle	f_file_handle_name	f_file

19

The table in the slide displays some examples of the naming conventions for the PL/SQL structures that are used in this course.

Data Types for Strings

- A string is a sequence of symbols that are part of a character set. To declare a string variable, PL/SQL offers:
 - CHAR
 - NCHAR
 - VARCHAR
 - NVARCHAR
 - CLOB
 - NCLOB

20

The data types that are prefixed with “N” are “national character set” data types, which means that they are used to store Unicode character data.

- **CHAR [(maximum_length)]**

Base type for fixed-length character data up to 32,767 bytes. If you do not specify a maximum length, the default length is set to 1 byte. NCHAR is a national character set variant of CHAR.

- **VARCHAR2 (maximum_length)**

Base type for variable-length character data up to 32,767 bytes. There is no default size for VARCHAR2 variables and constants. However, the database will raise a compiler error if the size of the string is not specified.

- **CLOB**

Used to store a variable length string that can store data up to 128 terabytes. You can use a CLOB data type if you expect the character string length to be greater than 32,767 bytes.

Delimiters in String Literals

```
DECLARE
    v_event VARCHAR2(15);
BEGIN
    v_event := q'!Father's day!';
    DBMS_OUTPUT.PUT_LINE('3rd Sunday in June is :
    '|| v_event );
    v_event := q'[Mother's day]';
    DBMS_OUTPUT.PUT_LINE('2nd Sunday in May is :
    '|| v_event );
END;
/
```

Resulting
Output

PL/SQL procedure successfully completed.

3rd day in June is :Father's day
2nd Sunday in May is :Mother's day

21

If your string contains an apostrophe (identical to a single quotation mark), you must double the quotation mark, as in the following example:

```
v_event VARCHAR2(15) := 'Father''s day';
```

The first quotation mark acts as the escape character. This makes your string complicated, especially if you have SQL statements as a string. The slide shows how to use the `q'` notation to specify delimiters. You can specify any character that is not present in the string as a delimiter. The example uses `!` and `[` as delimiters. Consider the following example:

```
v_event := q'!Father's day!';
```

You can compare this with the first example on this page. You start the string with `q'` if you want to use a delimiter. The character following the notation is the delimiter used. Enter your string after specifying the delimiter, close the delimiter, and close the notation with a single quotation mark. The following example shows how to use `[` as a delimiter:

```
v_event := q'[Mother's day]';
```


Data Types for Numeric values

- PL/SQL offers different numeric data types to suit different purposes:

- NUMBER
- PLS_INTEGER
- SIMPLE_INTEGER
- BINARY_INTEGER
- BINARY_FLOAT
- BINARY_DOUBLE
- BOOLEAN (TRUE, FALSE or NULL)



BOOLEAN is a non-numeric data type, which can assume one of the three values – TRUE, FALSE or NULL

22

- **NUMBER [(precision, scale)]**

Number having precision p and scale s . The precision p can range from 1 through 38. The scale s can range from -84 through 127.

- **PLS_INTEGER**

Integer data type, which conforms to the integer representation of your underlying hardware. It is also referred as `BINARY_INTEGER`, and can accommodate integers in the range $-2,147,483,647$ through $2,147,483,647$.

- **SIMPLE_INTEGER**

Numeric data type, which results in shorter execution time for natively compiled code. `SIMPLE_INTEGER` and `PLS_INTEGER` differ in overflow semantics. `PLS_INTEGER` provides better performance according to the underlying hardware architecture. Declare a variable as a `SIMPLE_INTEGER` if you know that the value is never going to be NULL and the variable does not need overflow checking.

- **BINARY_INTEGER** is used to store signed integers.

- **BINARY_FLOAT**

Floating-point number in IEEE 754 format. It requires 5 bytes to store a value.

- **BINARY_DOUBLE**

Floating-point number in IEEE 754 format. It requires 9 bytes to store a value.

BINARY data types are useful to improve the performance of computation-intensive operations.

- **BOOLEAN**

Base type that stores one of the three possible values used for logical calculations: TRUE, FALSE, or NULL

Data Types for Date and Time values

- The Oracle database provides three data types to work with dates and times:

- `DATE`
- `TIMESTAMP WITH TIME ZONE`
- `TIMESTAMP WITH LOCAL TIME ZONE`
- `INTERVAL YEAR TO MONTH`
- `INTERVAL DAY TO SECOND`



- **DATE:** This is the base type for dates and times. `DATE` values include the time of day in seconds since midnight. The range for dates is between 4712 B.C. and A.D. 9999.
- **TIMESTAMP:** The `TIMESTAMP` data type, which extends the `DATE` data type, stores the year, month, day, hour, minute, second, and fraction of second.

The syntax is `TIMESTAMP[(precision)]`, where the optional parameter `precision` specifies the number of digits in the fractional part of the seconds field. To specify the precision, you must use an integer in the range 0–9. The default is 6.

- **TIMESTAMP WITH TIME ZONE:** The `TIMESTAMP WITH TIME ZONE` data type, which extends the `TIMESTAMP` data type, includes a time-zone displacement. Time-zone displacement is the difference (in hours and minutes) between local time and Coordinated Universal Time (UTC), formerly known as Greenwich Mean Time. The syntax is `TIMESTAMP[(precision)] WITH TIME ZONE`, where the optional parameter `precision` specifies the number of digits in the fractional part of the seconds field. To specify the precision, you must use an integer in the range 0–9. The default is 6.

- **TIMESTAMP WITH LOCAL TIME ZONE:** The `TIMESTAMP WITH LOCAL TIME ZONE` data type, which extends the `TIMESTAMP` data type, includes a time-zone displacement. The syntax is `TIMESTAMP[ (precision) ] WITH LOCAL TIME ZONE`, where the optional parameter `precision` specifies the number of digits in the fractional part of the seconds field. You cannot use a symbolic constant or variable to specify the precision; you must use an integer literal in the range 0–9. The default is 6. This data type does not store time zone information internally, but you can see the local time zone information in the SQL output. In the case of `TIMESTAMP WITH TIME ZONE`, the time zone information is stored locally.
- **INTERVAL YEAR TO MONTH:** You use the `INTERVAL YEAR TO MONTH` data type to store and manipulate intervals of years and months. The syntax is `INTERVAL YEAR[ (precision) ] TO MONTH`, where `precision` specifies the number of digits in the years field. You cannot use a symbolic constant or variable to specify the precision; you must use an integer literal in the range 0–4. The default is 2.
- **INTERVAL DAY TO SECOND:** You use the `INTERVAL DAY TO SECOND` data type to store and manipulate intervals of days, hours, minutes, and seconds. The syntax is `INTERVAL DAY[ (precision1) ] TO SECOND[ (precision2) ]`, where `precision1` and `precision2` specify the number of digits in the days field and seconds field, respectively. In both cases, you cannot use a symbolic constant or variable to specify the precision; you must use an integer literal in the range 0–9. The defaults are 2 and 6, respectively.

Choosing the right data type

Following are some guidelines that will enable you to choose the right data type for a date variable:

- Use `TIMESTAMP` only when you have to keep track of values down to fractions of seconds.
- Use `TIMESTAMP WITH TIMEZONE` when you have to keep track of the session time zone when the data was entered.
- Use `TIMESTAMP WITH LOCAL TIMEZONE` when you have to convert the value between the database time and the session time very often.
- Use the `DATE` type when your current application has to be compatible with an older application when the `TIMESTAMP` data type was not available in databases.

Data Type Conversion

- Converts data to comparable data types
- Is of two types:
 - Implicit conversion
 - Explicit conversion
- Includes functions:
 - TO_CHAR
 - TO_DATE
 - TO_NUMBER
 - TO_TIMESTAMP



25

In any programming language, converting one data type to another is a common requirement. PL/SQL can handle such conversions with scalar data types. Data type conversions can be of two types:

- **Implicit conversions:** PL/SQL attempts to convert data types dynamically if they are mixed in a statement. Consider the following example:

```
DECLARE

    v_salary NUMBER(6) := 6000;
    v_sal_hike VARCHAR2(5) := '1000';
    v_total_salary v_salary%TYPE;

BEGIN
    v_total_salary := v_salary + v_sal_hike;
    DBMS_OUTPUT.PUT_LINE(v_total_salary);
END;
/
```

In this example, the `sal_hike` variable is of the `VARCHAR2` type. When calculating the total salary, PL/SQL first converts `sal_hike` to `NUMBER`, and then performs the operation. The result is of the `NUMBER` type.

Implicit conversions can be between:

- Characters and numbers
- Characters and dates

Explicit conversions: To convert values from one data type to another, use built-in functions. For example, to convert a `CHAR` value to a `DATE` or `NUMBER` value, use `TO_DATE` or `TO_NUMBER`, respectively.

For Instructor Use Only.
This document should not be distributed.

Data Type Conversion

1

```
-- implicit data type conversion  
v_date_of_joining DATE:= '02-Feb-2000';
```

2

```
-- error in data type conversion  
v_date_of_joining DATE:= 'February 02,2000';
```

3

```
-- explicit data type conversion  
v_date_of_joining DATE:= TO_DATE('February 02,2000','Month DD, YYYY');
```

27

0

Note the three examples of implicit and explicit conversions of the `DATE` data type in the slide:

1. Because the string literal that is being assigned to `date_of_joining` is in certain format, this example performs implicit conversion, and assigns the specified date to `date_of_joining`.
2. PL/SQL returns an error because the date that is being assigned is not in the default format.
3. The `TO_DATE` function is used to explicitly convert the given date in a particular format and assign it to the `DATE` data type variable `date_of_joining`.

Agenda

- Introducing variables
- Handling variables of different types
- Using the %TYPE attribute and composite data types
- Using bind variables



The %TYPE Attribute

- Is used to declare a variable according to:
 - A database column definition
 - Another declared variable
- Is prefixed with:
 - The database table and column name
 - The name of the declared variable

29

PL/SQL variables are usually declared to hold and manipulate the data stored in a database. When you declare PL/SQL variables to hold column values, you must ensure that the variable is of the correct data type and precision. If it is not, a PL/SQL error occurs during execution. If you have to design large subprograms, this can be time-consuming and error prone.

Rather than hard-coding the data type and precision of a variable, you can use the %TYPE attribute to declare a variable according to another previously declared variable or database column. The %TYPE attribute is most often used when the value stored in the variable is derived from a table in the database. When you use the %TYPE attribute to declare a variable, you should prefix it with the database table and column names. If you refer to a previously declared variable, prefix the variable name of the previously declared variable to the variable that is being declared.

Advantages of the %TYPE Attribute

- You can avoid errors caused by data type mismatch or wrong precision.
- You can avoid hard coding the data type of a variable.
- You need not change the variable declaration if the column definition changes. If you have already declared some variables for a particular table without using the %TYPE attribute, the PL/SQL block may throw errors if the column for which the variable is declared is altered. When you use the %TYPE attribute, PL/SQL determines the data type and size of the variable when the block is compiled. This ensures that such a variable is always compatible with the column that is used to populate it.

Declaring Variables with the %TYPE Attribute

Syntax:

```
identifier      table.column_name%TYPE;
```

Examples:

```
...  
v_emp_lname     employees.last_name%TYPE;  
...
```

```
...  
v_balance       NUMBER(7,2);  
v_min_balance   v_balance%TYPE := 1000;  
...
```

30

Declare variables to store the last name of an employee.

The `v_emp_lname` variable is defined to be of the same data type as the `last_name` column in the `employees` table. The `%TYPE` attribute provides the data type of a database column.

Declare variables to store the balance of a bank account, as well as the minimum balance, which is 1,000. The `v_min_balance` variable is defined to be of the same data type as the `v_balance` variable. The `%TYPE` attribute provides the data type of a variable.

A `NOT NULL` database column constraint does not apply to variables that are declared by using `%TYPE`. Therefore, if you declare a variable by using the `%TYPE` attribute that uses a database column that is defined as `NOT NULL`, you can assign a `NULL` value to the variable.

Declaring Boolean Variables

- Only `TRUE`, `FALSE`, and `NULL` values can be assigned to a Boolean variable.
- Conditional expressions use the logical operators `AND` and `OR`, and the unary operator `NOT` to check the variable values.
- The variables always yield `TRUE`, `FALSE`, or `NULL`.
- Arithmetic, character, and date expressions can be used to return a Boolean value.

31

With PL/SQL, you can compare variables in both SQL and procedural statements. These comparisons, called Boolean expressions, consist of simple or complex expressions that are separated by relational operators. In a SQL statement, you can use Boolean expressions to specify the rows in a table that are affected by the statement. In a procedural statement, Boolean expressions are the basis for conditional control. `NULL` stands for a missing, an inapplicable, or an unknown value.

Examples

```
emp_sal1 := 50000;  
emp_sal2 := 60000;
```

The following expression yields `TRUE`:

```
emp_sal1 < emp_sal2
```

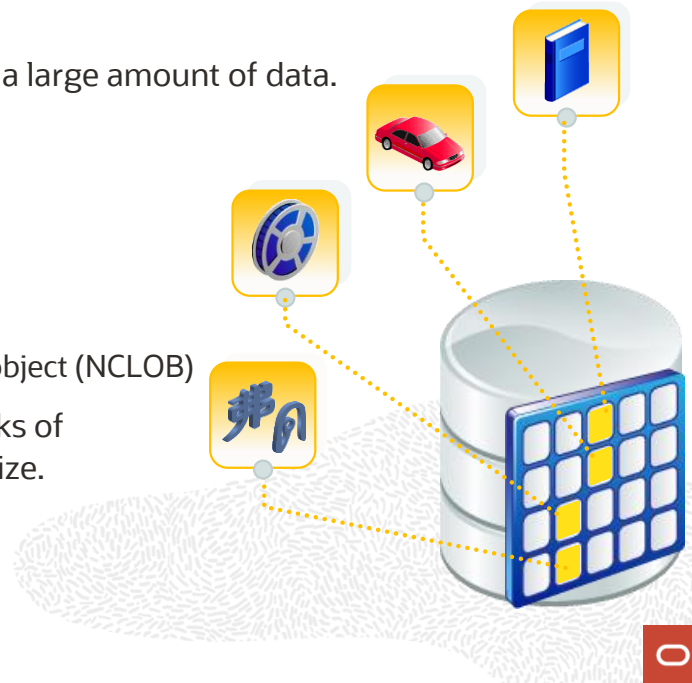
Declare and initialize a Boolean variable:

```
DECLARE  
  flag BOOLEAN := FALSE;  
BEGIN  
  flag := TRUE;  
END;  
/
```

You can initialize a Boolean variable to one value among `TRUE`, `FALSE`, and `NULL`.

LOB Data Type Variables

- Large objects (LOBs) are meant to store a large amount of data.
- LOB data types:
 - Character large object (CLOB)
 - Binary large object (BLOB)
 - Binary file (BFILE)
 - National language character large object (NCLOB)
- LOB data types enable you to store blocks of unstructured data up to 4 gigabytes in size.



32

Large objects (LOBs) are meant to store a large amount of data. A database column can be of the LOB category. With the LOB category of data types (BLOB, CLOB, and so on), you can store blocks of unstructured data (such as text, graphic images, video clips, and sound wave forms) of up to 128 terabytes depending on the database block size. LOB data types allow efficient, random, piecewise access to data and can be attributes of an object type.

- The character large object (CLOB) data type is used to store large blocks of character data in the database.
- The binary large object (BLOB) data type is used to store large unstructured or structured binary objects in the database. When you insert such data into or retrieve such data from the database, the database does not interpret the data. The external applications that use this data must interpret the data.
- The binary file (BFILE) data type is used to store large binary files. Unlike other LOBs, BFILES are stored outside the database and not in the database. They could be operating system files. Only a pointer to the BFILE is stored in the database.
- The national language character large object (NCLOB) data type is used to store large blocks of single-byte or fixed-width multibyte NCHAR Unicode data in the database.

Composite Data Types: Records and Collections

PL/SQL Record:

123	23-Dec-98	ATLANTA	
-----	-----------	---------	---

PL/SQL Collections:

1	SMITH
2	JONES
3	NANCY
4	TIM

↑ ↑
PLS_INTEGER index VARCHAR2

33

As mentioned previously, a scalar data type holds a single value and has no internal components. Composite data types—called PL/SQL records and PL/SQL collections—have internal components that you can treat as individual variables.

- In a PL/SQL record, the internal components can be of different data types, and are called fields. You access each field with this syntax: `record_name.field_name`. A record variable can hold a table row, or some columns from a table row. Each record field corresponds to a table column.
- In a PL/SQL collection, the internal components are always of the same data type, and are called elements. You access each element by its unique subscript. Lists and arrays are classic examples of collections. There are three types of PL/SQL collections: Associative Arrays, Nested Tables, and `VARRAY` types.

Note

- PL/SQL records and associative arrays are covered in the lesson titled “Working with Composite Data Types.”
- `NESTED TABLE` and `VARRAY` data types are covered in the *Advanced PL/SQL* course.

Agenda

- Introducing variables
- Handling variables of different types
- Using variable data types and the %TYPE attribute
- Using bind variables



Bind Variables

- Bind variables are:
 - Created in the host environment
 - Also called host variables
 - Created with the `VARIABLE` keyword in PL/SQL
 - Used in SQL statements and PL/SQL blocks
 - Accessed even after the PL/SQL block is executed
 - Referenced with a preceding colon
- Values can be output by using the `PRINT` command.
- Bind variables are required when using SQL*Plus and SQL Developer.

35

Bind variables are variables that you create in a host environment; therefore, they are also known as host variables.

Uses of Bind Variables

Bind variables are created in the application environment and not in the `DECLARE` section of a PL/SQL block. Therefore, bind variables are accessible even after the block is executed and, can be accessed by other PL/SQL blocks. These variables can be passed as runtime values into or out of PL/SQL subprograms.

Note: A bind variable is an environment variable, but is not a global variable.

Creating Bind Variables

To create a bind variable in SQL Developer, use the `VARIABLE` command. For example, you declare a variable of type `NUMBER` and `VARCHAR2` as follows:

```
VARIABLE return_code NUMBER
VARIABLE return_msg VARCHAR2(30)
```

Viewing Values in Bind Variables

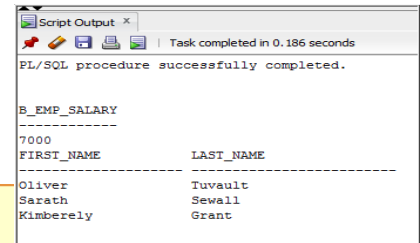
You can reference the bind variable by using SQL Developer and view its value by using the `PRINT` command.

Bind Variables: Examples

```
VARIABLE b_result NUMBER
BEGIN
  SELECT (SALARY*12) + NVL(COMMISSION_PCT,0) INTO :b_result
  FROM employees WHERE employee_id = 144;
END;
/
PRINT b_result
```

B_RESULT
30000

```
VARIABLE b_emp_salary NUMBER
BEGIN
  SELECT salary INTO :b_emp_salary
  FROM employees WHERE employee_id = 178;
END;
/
PRINT b_emp_salary
SELECT first_name, last_name
FROM employees
WHERE salary=:b_emp_salary;
```



Script Output x

Task completed in 0.186 seconds

PL/SQL procedure successfully completed.

B_EMP_SALARY	
FIRST_NAME	LAST_NAME
Oliver	Tuvault
Sarah	Sewall
Kimberely	Grant

36

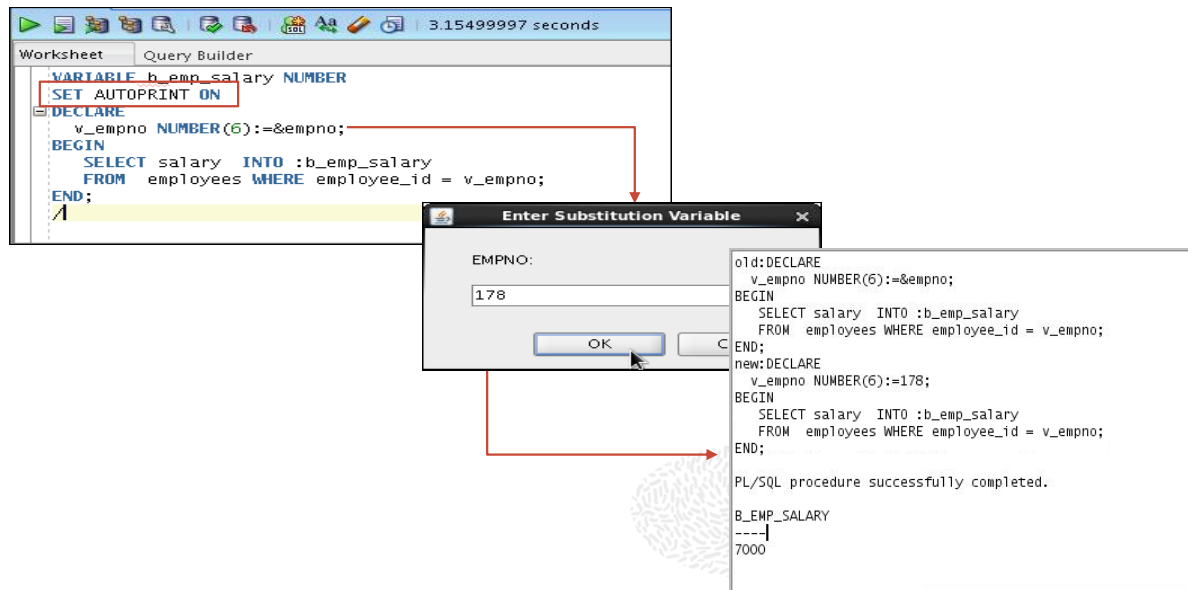
The first PL/SQL block in the slide creates and uses the bind variable `b_result`. The output resulting from the `PRINT` command is shown below the code in the slide.

As stated previously, after you create a bind variable, you can reference that variable in any other SQL statement or PL/SQL program.

In the second example in the slide, `b_emp_salary` is created as a bind variable. It is then used in the `SELECT` statement.

In an application, you can use bind variables to hold the values received from the application user interface. These variables can later be processed by the PL/SQL blocks on the database according to the application's requirement.

Using AUTOPRINT with Bind Variables



37

Use the `SET AUTOPRINT ON` command to automatically display the bind variables that are used in a successful PL/SQL block.

Example

In the code example in the slide:

- A bind variable named `b_emp_salary` is created and `AUTOPRINT` is turned on
- A variable named `v_empno` is declared, and a substitution variable is used to receive user input
- Finally, the bind variable and temporary variables are used in the executable section of the PL/SQL block

When a valid employee number is entered—in this case 178—the output of the bind variable is automatically printed. The bind variable contains the salary for the employee number that is provided by the user.

Quiz



The `%TYPE` attribute:

- a. Is used to declare a variable according to a database column definition
- b. Is used to declare a variable according to a collection of columns in a database table or view
- c. Is used to declare a variable according to the definition of another declared variable
- d. Is prefixed with the database table and column names or the name of the declared variable



38

Answer: a, c, d

The `%TYPE` Attribute

PL/SQL variables are usually declared to hold and manipulate the data stored in a database. When you declare PL/SQL variables to hold column values, you must ensure that the variable is of the correct data type and precision. If it is not, a PL/SQL error occurs during execution. If you have to design large subprograms, this can be time-consuming and error-prone.

Rather than hard-coding the data type and precision of a variable, you can use the `%TYPE` attribute to declare a variable according to another previously declared variable or database column. The `%TYPE` attribute is most often used when the value stored in the variable is derived from a table in the database. When you use the `%TYPE` attribute to declare a variable, you should prefix it with the database table and column names. If you refer to a previously declared variable, prefix the variable name of the previously-declared variable to the variable that is being declared. The benefit of `%TYPE` is that you do not have to change the variable if the column is altered. Also, if the variable is used in any calculations, you need not worry about its precision.

Summary

In this lesson, you should have learned how to:

- Recognize valid and invalid identifiers
- Declare variables in the declarative section of a PL/SQL block
- Initialize variables and use them in the executable section
- Differentiate between scalar and composite data types
- Use the `%TYPE` attribute
- Use bind variables



39

An anonymous PL/SQL block is a basic, unnamed unit of a PL/SQL program. It consists of a set of SQL or PL/SQL statements to perform a logical function. The declarative part is the first part of a PL/SQL block, which is used for declaring objects such as variables, constants, cursors, and definitions of error situations called *exceptions*.

In this lesson, you learned how to declare variables in the declarative section. You saw some of the guidelines for declaring variables. You also learned how to initialize variables when you declare them.

The executable part of a PL/SQL block is the mandatory part, and contains SQL and PL/SQL statements for querying and manipulating data. You learned how to initialize variables in the executable section and also how to use them and manipulate the values of variables.

Practice 3: Overview

This practice covers the following topics:

- Determining valid identifiers
- Determining valid variable declarations
- Declaring variables within an anonymous block
- Using the `%TYPE` attribute to declare variables
- Declaring and printing a bind variable
- Executing a PL/SQL block



0

40

Exercises 1, 2, and 3 are paper based.

For Instructor Use Only.
This document should not be distributed.

Writing Executable Statements

Course Road Map



2

0
For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to do the following:

- Identify the lexical units in a PL/SQL block
- Use built-in SQL functions in PL/SQL
- Describe when implicit conversions take place and when explicit conversions have to be dealt with
- Write nested blocks and qualify variables with labels
- Write readable code with appropriate indentation
- Use sequences in PL/SQL expressions



3

You learned how to declare variables and write executable statements in a PL/SQL block. In this lesson, you learn how lexical units make up a PL/SQL block. You learn to write nested blocks. You also learn about the scope and visibility of variables in nested blocks and about qualifying variables with labels.

For Instructor Use Only.
This document should not be distributed.

Agenda

- Writing executable statements in a PL/SQL block
- Writing nested blocks
- Using operators and developing readable code



Lexical Units in a PL/SQL Block

Lexical units:

- Are building blocks of any PL/SQL block
- Are sequences of characters, including letters, numerals, tabs, spaces, returns, and symbols
- Can be classified as:
 - Identifiers: `v_fname`, `c_percent`
 - Delimiters: `;`, `,`, `+`, `-`
 - Literals: `John`, `428`, `True`
 - Comments: `--`, `/* */`

5

Lexical units include letters, numerals, special characters, tabs, spaces, returns, and symbols.

- **Identifiers:** Identifiers are the names given to PL/SQL objects. You learned to identify valid and invalid identifiers. Recall that keywords cannot be used as identifiers.

Quoted identifiers:

- Make identifiers case-sensitive.
- Include characters such as spaces.
- Use reserved words.

Examples:

```
"begin date" DATE;  
"end date"    DATE;  
"exception thrown" BOOLEAN DEFAULT TRUE;
```

All subsequent usage of these variables should have double quotation marks. However, use of quoted identifiers is not recommended.

- **Delimiters:** Delimiters are symbols that have special meaning. You already learned that the semicolon (`;`) is used to terminate a SQL or PL/SQL statement. Therefore, `;` is an example of a delimiter.

For more information, refer to the *PL/SQL User's Guide and Reference*.

Delimiters are simple or compound symbols that have special meaning in PL/SQL.

Simple symbols

Symbol	Meaning
+	Addition operator
-	Subtraction/negation operator
*	Multiplication operator
/	Division operator
=	Equality operator
@	Remote access indicator
;	Statement terminator

Compound symbols

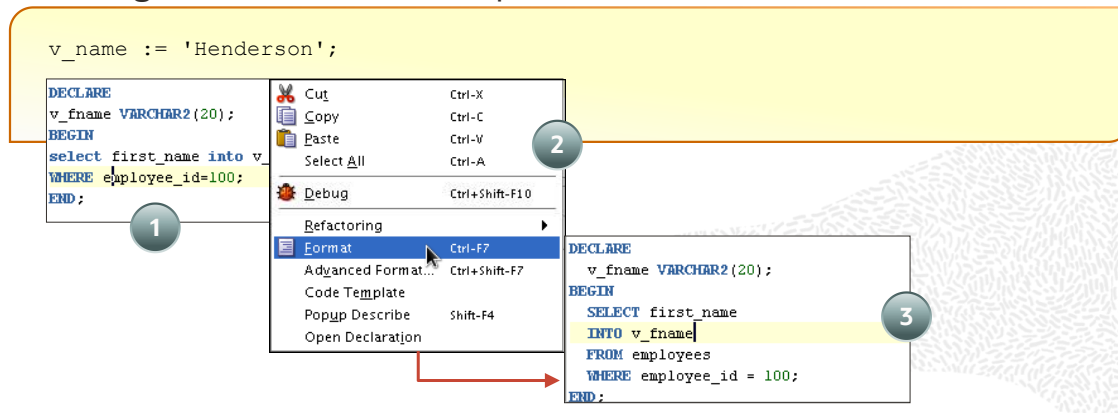
Symbol	Meaning
<>	Inequality operator
!=	Inequality operator
	Concatenation operator
--	Single-line comment indicator
/*	Beginning comment delimiter
*/	Ending comment delimiter
:=	Assignment operator

Note: This is only a subset and not a complete list of delimiters.

- **Literals:** Any value that is assigned to a variable is a literal. Any character, numeral, Boolean, or date value that is not an identifier is a literal. Literals are classified as:
 - **Character literals:** All string literals have the data type `CHAR` or `VARCHAR2`, and are therefore called character literals (for example, `John`, and `19c`).
 - **Numeric literals:** A numeric literal represents an integer or a real value (for example, `428` and `1.276`).
 - **Boolean literals:** Values that are assigned to Boolean variables are Boolean literals. `TRUE`, `FALSE`, and `NULL` are Boolean literals or keywords.
- **Comments:** It is a good programming practice to explain what a piece of code is trying to achieve. However, when you include the explanation in a PL/SQL block, the compiler cannot interpret these instructions. Therefore, there should be a way in which you can indicate that these instructions need not be compiled. Comments are mainly used for this purpose. Any instruction that is commented is not interpreted by the compiler.
 - Two hyphens (`--`) are used to comment a single line.
 - The beginning and ending comment delimiters (`/*` and `*/`) are used to comment multiple lines.

PL/SQL Block Syntax and Guidelines

- Using Literals
 - Character and date literals must be enclosed in single quotation marks.
 - Numbers can be simple values or in scientific notation.
- Formatting Code: Statements can span several lines.



7

Using Literals

A literal is an explicit numeric, character string, date, or Boolean value that is not represented by an identifier.

- Character literals include all printable characters in the PL/SQL character set: letters, numerals, spaces, and special symbols.
- Numeric literals can be represented either by a simple value (for example, -32.5) or in scientific notation (for example, $2E5$ means $2 * 10^5 = 200,000$).

Formatting Code

In a PL/SQL block, a SQL statement can span several lines (as shown in example 3 in the slide).

You can format an unformatted SQL statement (as shown in example 1 in the slide) by using the SQL Worksheet shortcut menu. Right-click the active SQL Worksheet, and in the shortcut menu that appears, select the Format option (as shown in example 2).

Note: You can also use the shortcut key combination of Ctrl + F7 to format your code.

For Instructor Use Only.
This document should not be distributed.

Commenting Code

- Prefix single-line comments with two hyphens (--).
- Place a block comment between the symbols /* and */.

Example:

```
DECLARE
...
v_annual_sal NUMBER (9,2);
BEGIN
/* Compute the annual salary based on the
   monthly salary input from the user */
v_annual_sal := monthly_sal * 12;
--The following line displays the annual salary DBMS_OUTPUT.PUT_LINE(v_annual_sal);
END;
/
```

8

You should comment code to document each phase and to assist in debugging. In PL/SQL code:

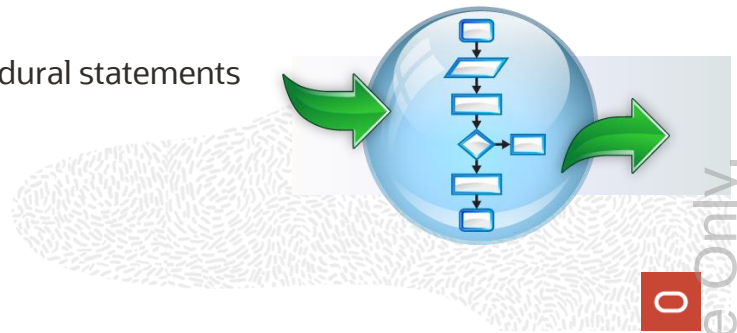
- A single-line comment is commonly prefixed with two hyphens (--)
- You can also enclose a comment between the symbols /* and */

Note: For multiline comments, you can either precede each comment line with two hyphens, or use the block comment format.

Comments are strictly informational and do not enforce any conditions or behavior on the logic or data. Well-placed comments are extremely valuable for code readability and future code maintenance.

SQL Functions in PL/SQL

- Predefined functions that are used in SQL can also be used in PL/SQL.
- Functions that are available in procedural statements are:
 - Single-row functions
 - Built-in functions with Strings
 - Built-in functions with Numbers
 - Built-in functions with Dates
- Functions that are not available in procedural statements are:
 - `DECODE`
 - Group functions



9

SQL provides several predefined functions that can be used in SQL statements. Most of these functions (such as single-row, number, character, data type conversion, and date and time-stamp functions) are valid in PL/SQL expressions as well.

The following functions are not available in procedural statements:

- `DECODE`
- **Group functions:** `AVG`, `MIN`, `MAX`, `COUNT`, `SUM`, `STDDEV`, and `VARIANCE`
Group functions apply to groups of rows in a table and are therefore, available only in SQL statements in a PL/SQL block. The functions mentioned here form only a subset of the complete list.

SQL Functions in PL/SQL: Examples

- Get the length of a string:

```
v_desc_size INTEGER(5);  
v_prod_description VARCHAR2(70):='You can use this product with your radios for higher frequency';  
  
-- get the length of the string in prod_description  
v_desc_size:= LENGTH(v_prod_description);
```

- Get the number of months an employee has worked:

```
v_tenure:= MONTHS_BETWEEN (CURRENT_DATE,  
v_hiredate);
```

10

You can use SQL functions to manipulate data. These functions are grouped into the following categories:

- Number
- Character
- Conversion
- Date
- Miscellaneous

Using Sequences in PL/SQL blocks



11

Consider a scenario where a manager needs to assign an `employee_id` to each new employee joining his or her department. When you want to define an employee ID, you may have the following requirements:

1. You do not want the `employee_id` to be 0.
2. No two employees in the organization should have the same employee ID.
3. You cannot remember the employee ID assigned to the employee who last joined the organization so that you can increment it by 1 and assign it to the new employee.
4. Sometimes, you may not want the employee ID to start with a single digit, depending on the context requirement.

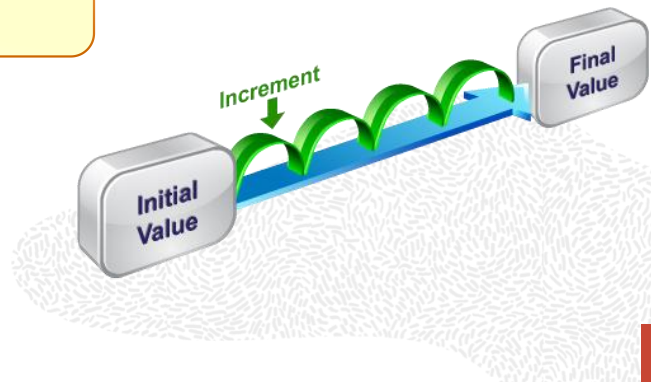
In such cases, using a sequence object that can be shared by the managers of multiple departments is the solution.

A sequence is an incrementing numeric value. Developers can define its initial value, the increment with which the next value can be generated, and the maximum value or the upper limit of the sequence. This object on the database can be shared by multiple users who can eventually access the next value in the sequence.

Using Sequences in PL/SQL blocks

- Sequences are database objects that can be used by multiple users to generate sequential numbers.
- Sequences can be created through the `CREATE SEQUENCE` statement.

```
CREATE SEQUENCE emp_sequence  
INCREMENT BY 1  
START WITH 1  
NOMAXVALUE;
```



12

Sequences

Sequences are database objects like views, constraints, and so on. You can use sequences to generate a sequence of numbers. You can define the increment size, starting value, and maximum value while creating a sequence. The numeric value can have 28 or fewer digits.

A single sequence object can be used by multiple users who use the database.

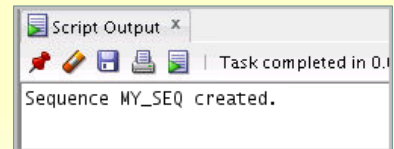
The following table shows the properties associated with a sequence that can be used to define its behavior:

INCREMENT BY	You can specify the value of each increment in the sequence. In the given example, you see that the sequence is incremented by 1. You can change it to any other value.
START WITH	You define the starting value of the sequence.
MAXVALUE	You define the upper limit of the sequence through MAXVALUE.

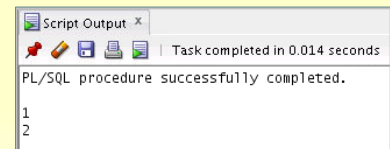
NOMAXVALUE	When you do not want to define an upper limit, you use NOMAXVALUE during sequence creation.
MINVALUE	You can specify the minimum value of a sequence.
NOMINVALUE	NOMINVALUE in CREATE SEQUENCE sets the minimum value of a sequence to 1.
CYCLE	This indicates that the sequence will restart from the minimum value after it reaches the maximum value.
NOCYCLE	This indicates that the sequence will not restart from the minimum after it reaches the maximum value.

Using Sequences in PL/SQL Blocks

```
CREATE SEQUENCE my_seq  
INCREMENT BY 1  
START WITH 1  
NOMAXVALUE;
```



```
DECLARE  
  v_new_id NUMBER;  
BEGIN  
  v_new_id := my_seq.NEXTVAL;  
  DBMS_OUTPUT.PUT_LINE(v_new_id);  
  DBMS_OUTPUT.PUT_LINE(my_seq.NEXTVAL);  
END;
```



14

The usage of sequences in PL/SQL expressions is similar to that in SQL statements.

You can create a sequence once and use it multiple times in an application. You create a sequence by executing the **CREATE SEQUENCE** statement as shown in the slide.

After creating the sequence object, you can access the value of the sequence through the **NEXTVAL** and **CURRVAL** pseudocolumns.

In the PL/SQL block shown in the slide, we use **NEXTVAL** with the `my_seq` sequence twice. First, we assign it to the variable `v_new_id` and output the value. Next, we use it within the `DBMS_OUTPUT.PUT_LINE` function call. Observe that you see a new incremented value the second time.

Agenda

- Writing executable statements in a PL/SQL block
- Writing nested blocks
- Using operators and developing readable code



Nested blocks

PL/SQL blocks can be nested.

- An executable section (`BEGIN ... END`) can contain nested blocks.
- An exception section can contain nested blocks.

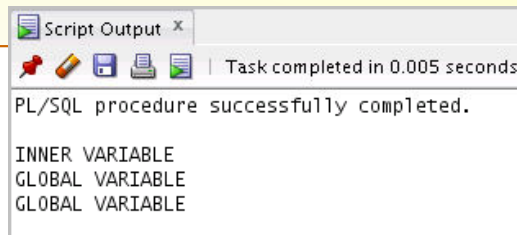


16

Being procedural gives PL/SQL the ability to nest statements. You can nest blocks wherever an executable statement is allowed, thus making the nested block a statement. If your executable section has code for many logically related functionality to support multiple business requirements, you can divide the executable section into smaller blocks. The exception section can also contain nested blocks.

Nested Blocks: Example

```
DECLARE
  v_outer_variable VARCHAR2(20):='GLOBAL VARIABLE';
BEGIN
  DECLARE
    v_inner_variable VARCHAR2(20):='INNER VARIABLE';
  BEGIN
    DBMS_OUTPUT.PUT_LINE(v_inner_variable);
    DBMS_OUTPUT.PUT_LINE(v_outer_variable);
  END;
  DBMS_OUTPUT.PUT_LINE(v_outer_variable);
END;
```



17

The example shown in the slide has an outer (parent) block and a nested (child) block. The `v_outer_variable` variable is declared in the outer block and the `v_inner_variable` variable is declared in the inner block.

The `v_outer_variable` variable is local to the outer block but global to the inner block. When you access this variable in the inner block, PL/SQL first looks for a local variable in the inner block with that name. When there is no variable with the same name in the inner block, the variable in the outer block is accessed. Therefore, the `v_outer_variable` variable is considered to be the global variable for all the enclosed blocks.

The `v_inner_variable` variable is local to the inner block and is not global because the inner block does not have any nested blocks. This variable can be accessed only within the inner block. If PL/SQL does not find the variable declared locally, it looks up in the `DECLARE` section of the parent blocks. PL/SQL does not look downward in the child blocks.

Variable Scope and Visibility

```
DECLARE
  v_father_name VARCHAR2(20):='Patrick';
  v_date_of_birth DATE:='20-Apr-1972';
BEGIN
  DECLARE
    v_child_name VARCHAR2(20):='Mike';
    v_date_of_birth DATE:='12-Dec-2002';
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Father's Name: '||v_father_name);
    DBMS_OUTPUT.PUT_LINE('Date of Birth: '||v_date_of_birth);
    DBMS_OUTPUT.PUT_LINE('Child's Name: '||v_child_name);
  END;
  DBMS_OUTPUT.PUT_LINE('Date of Birth: '||v_date_of_birth);
END;
/
```

18

The output of the block shown in the slide is as follows:

```
PL/SQL procedure successfully completed.

Father's Name: Patrick
Date of Birth: 12-DEC-02
Child's Name: Mike
Date of Birth: 20-APR-72
```

Examine the date of birth that is printed for father and child. The output does not provide the correct information, because the scope and visibility of the variables are not applied correctly.

- The *scope* of a variable is the portion of the program in which the variable is declared and is accessible.
- The *visibility* of a variable is the portion of the program where the variable can be accessed without using a qualifier.

Scope

- The `v_father_name` variable and the first occurrence of the `v_date_of_birth` variable are declared in the outer block. These variables have the scope of the block in which they are declared. Therefore, the scope of these variables is limited to the outer block.

Scope

- The `v_child_name` and `v_date_of_birth` variables are declared in the inner block or the nested block. These variables are accessible only within the nested block and are not accessible in the outer block. When a variable is out of scope, PL/SQL frees the memory used to store the variable; therefore, these variables cannot be referenced.

Visibility

- The `v_date_of_birth` variable that is declared in the outer block has scope even in the inner block. However, this variable is not visible in the inner block because the inner block has a local variable with the same name.
 1. Examine the code in the executable section of the PL/SQL block. You can print the father's name, the child's name, and the date of birth. Only the child's date of birth can be printed here because the father's date of birth is not visible.
 2. The father's date of birth is visible in the outer block and, therefore, can be printed.

Note: You cannot have variables with the same name in a block. However, as shown in this example, you can declare variables with the same name in two different blocks (nested blocks). The two items represented by the identifiers are distinct; changes in one do not affect the other.

Using a Qualifier with Nested Blocks

```
BEGIN <<outer>>
DECLARE
  v_father_name VARCHAR2(20):='Patrick';
  v_date_of_birth DATE:='20-Apr-1972';
BEGIN
  DECLARE
    v_child_name VARCHAR2(20):='Mike';
    v_date_of_birth DATE:='12-Dec-2002';
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Father''s Name: '||v_father_name);
    DBMS_OUTPUT.PUT_LINE('Date of Birth: '
                          ||outer.v_date_of_birth);
    DBMS_OUTPUT.PUT_LINE('Child''s Name: '||v_child_name);
    DBMS_OUTPUT.PUT_LINE('Date of Birth: '||v_date_of_birth);
  END;
END;
END outer;
```

20

A *qualifier* is a label given to a block. You can use a qualifier to access the variables that have scope but are not visible.

Example

In the code example in the slide:

- The outer block is labeled `outer`
- Within the inner block, the `outer` qualifier is used to access the `v_date_of_birth` variable that is declared in the outer block. Therefore, the father's date of birth and the child's date of birth can both be printed from within the inner block.
- The output of the code in the slide shows the correct information:

```
PL/SQL procedure successfully completed.
Father's name: Patrick
Date of Birth: 20-APR-72
Child's name: Mike
Date of Birth: 12-DEC-02
```

Note: Labeling is not limited to the outer block. You can label any block.

Challenge: Determining the Variable Scope

```
BEGIN <<outer>>
DECLARE
  v_sal      NUMBER(7,2) := 60000;
  v_comm     NUMBER(7,2) := v_sal * 0.20;
  v_message  VARCHAR2(255) := ' eligible for commission';
BEGIN
  DECLARE
    v_sal     NUMBER(7,2) := 50000;
    v_comm    NUMBER(7,2) := 0;
    v_total_comp NUMBER(7,2) := v_sal + v_comm;
  BEGIN
    1 → v_message := 'CLERK not' || v_message;
      outer.v_comm := v_sal * 0.30;
    END;
    2 → v_message := 'SALESMAN' || v_message;
  END;
END outer;
/
```

21

0

Evaluate the PL/SQL block in the slide. Determine each of the following values according to the rules of scoping:

1. Value of `v_message` at position 1
2. Value of `v_total_comp` at position 2
3. Value of `v_comm` at position 1
4. Value of `outer.v_comm` at position 1
5. Value of `v_comm` at position 2
6. Value of `v_message` at position 2

For Instructor Use Only.
This document should not be distributed.

Answers: Determining the Variable Scope

Answers to the questions of scope are as follows:

1. Value of `v_message` at position 1: **CLERK not eligible for commission**
2. Value of `v_total_comp` at position 2: **Error. `v_total_comp` is not visible here because it is defined within the inner block.**
3. Value of `v_comm` at position 1: **0**
4. Value of `outer.v_comm` at position 1: **12000**
5. Value of `v_comm` at position 2: **15000**
6. Value of `v_message` at position 2: **SALESMANCLERK not eligible for commission**

For Instructor Use Only.
This document should not be distributed.

Agenda

- Writing executable statements in a PL/SQL block
- Writing nested blocks
- Using operators and developing readable code



Operators in PL/SQL

- Logical
- Arithmetic
- Concatenation
- Parentheses to control order of operations

Same as in SQL

- Exponential operator (**)

24

The operations in an expression are performed in a particular order depending on their precedence (priority). The following table shows the default order of operations from high priority to low priority:

Operator	Operation
**	Exponentiation
+, -	Identity, negation
*, /	Multiplication, division
+, -,	Addition, subtraction, concatenation
=, <, >, <=, >=, <>, !=, ~=, ^=, IS NULL, LIKE, BETWEEN, IN	Comparison
NOT	Logical negation
AND	Conjunction
OR	Inclusion

Operators in PL/SQL: Examples

- Increment the counter for a loop.

```
loop_count := loop_count + 1;
```

- Set the value of a Boolean flag.

```
good_sal := sal BETWEEN 50000 AND 150000;
```

- Validate whether an employee number contains a value.

```
valid := (empno IS NOT NULL);
```

25

When you are working with nulls, you can avoid some common mistakes by keeping in mind the following rules:

- Comparisons involving nulls always yield `NULL`.
- Applying the logical operator `NOT` to a null yields `NULL`.
- In conditional control statements, if the condition yields `NULL`, its associated sequence of statements is not executed.

Programming Guidelines

Make code maintenance easier by:

- Documenting the code with comments
- Developing a case convention for the code
- Developing naming conventions for identifiers and other objects
- Enhancing readability by indenting



26

Follow the programming guidelines shown in the slide when developing a PL/SQL block to produce clear code and to ease maintenance of the code.

Code Conventions

The following table provides guidelines for writing code in uppercase or lowercase characters to help distinguish keywords from named objects.

Category	Case Convention	Examples
SQL statements	Uppercase	SELECT, INSERT
PL/SQL keywords	Uppercase	DECLARE, BEGIN, IF
Data types	Uppercase	VARCHAR2, BOOLEAN
Identifiers and parameters	Lowercase	v_sal, emp_cursor, g_sal, p_empno
Database tables	Lowercase, plural	employees, departments
Database columns	Lowercase, singular	employee_id, department_id

Indenting Code

For clarity, indent each level of code.

```
BEGIN
  IF x=0 THEN
    y:=1;
  END IF;
END;
/
```

```
DECLARE
  v_deptno      NUMBER(4);
  v_location_id NUMBER(4);
BEGIN
  SELECT department_id,
         location_id
  INTO    v_deptno,
         v_location_id
  FROM    departments
  WHERE   department_name
         = 'Sales';

  ...
END;
/
```

27

For clarity and enhanced readability, indent each level of code. To show structure, you can divide lines by using carriage returns and you can indent lines by using spaces and tabs. Compare the following IF statements for readability:

```
IF x>y THEN max:=x;ELSE max:=y;END IF;
```

```
IF x > y THEN
  max := x;
ELSE
  max := y;
END IF;
```

Quiz



You can use most single-row SQL functions such as number, character, conversion, and date in PL/SQL expressions.

- a. True
- b. False



28

Answer: a

SQL Functions in PL/SQL

SQL provides several predefined functions that can be used in SQL statements. Most of these functions (such as single-row number and character functions, data type conversion functions, and date and time-stamp functions) are valid in PL/SQL expressions.

The following functions are not available in procedural statements:

- `DECODE`
- **Group functions:** `AVG`, `MIN`, `MAX`, `COUNT`, `SUM`, `STDDEV`, and `VARIANCE`
Group functions apply to groups of rows in a table and are therefore, available only in SQL statements in a PL/SQL block. The functions mentioned here form only a subset of the complete list.

Summary

In this lesson, you should have learned how to:

- Identify the lexical units in a PL/SQL block
- Use built-in SQL functions in PL/SQL
- Write nested blocks to break logically related functionalities
- Decide when to perform explicit conversions
- Qualify variables in nested blocks
- Use sequences in PL/SQL expressions



29

The general syntax rules that apply to SQL also apply to PL/SQL. A block can have any number of nested blocks defined within its executable part.

- Blocks defined within a block are called sub blocks.
- You can nest blocks only in the executable part of a block.
- Because the exception section is also a part of the executable section, it can also contain nested blocks.
- Ensure correct scope and visibility of the variables when you have nested blocks.
- Avoid using the same identifiers in the parent and child blocks.
- Most of the functions available in SQL are also valid in PL/SQL expressions.
- Conversion functions convert a value from one data type to another.
- Comparison operators compare one expression with another. The result is always `TRUE`, `FALSE`, or `NULL`. Typically, you use comparison operators in conditional control statements and in the `WHERE` clause of SQL data manipulation statements.
- The relational operators enable you to compare arbitrarily complex expressions.

Practice 4: Overview

This practice covers the following topics:

- Reviewing scoping and nesting rules
- Writing and testing PL/SQL blocks

30

Exercises 1 and 2 are paper based.

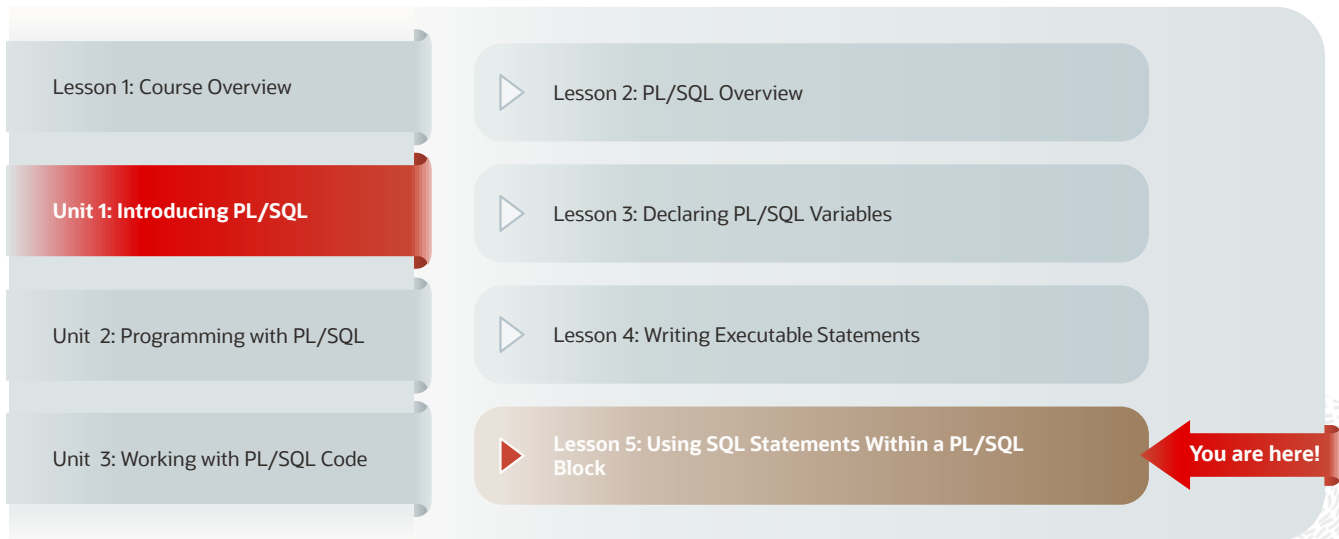


0

For Instructor Use Only.
This document should not be distributed.

Using SQL Statements Within a PL/SQL Block

Course Road Map



2

You are in lesson 5, which is part of Unit 1: Introducing PL/SQL.

0
For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to do the following:

- Determine the SQL statements that can be directly included in a PL/SQL executable block
- Make use of the `INTO` clause to hold the values returned by a SQL statement
- Manipulate data with DML statements in PL/SQL
- Use transaction control statements in PL/SQL
- Differentiate between implicit cursors and explicit cursors
- Use SQL cursor attributes



0

3

In this lesson, you learn to embed standard SQL `SELECT`, `INSERT`, `UPDATE`, `DELETE`, and `MERGE` statements in PL/SQL blocks. You learn how to include data manipulation language (DML) and transaction control statements in PL/SQL. You understand the need for cursors and differentiate between the two types of cursors. This lesson also presents the various SQL cursor attributes that can be used with implicit cursors.

For Instructor Use Only.
This document should not be distributed.

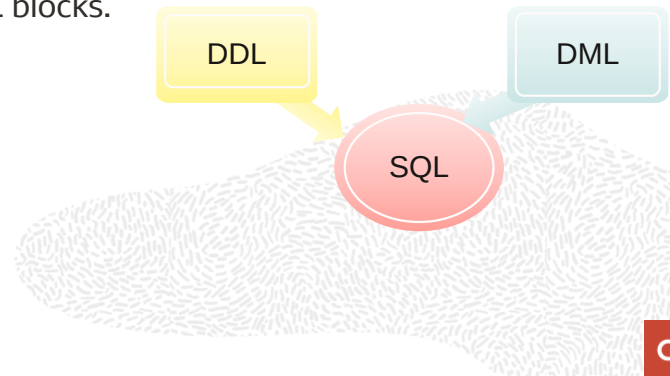
Agenda

- SQL statements in PL/SQL blocks
- Manipulating data with PL/SQL
- Introducing SQL cursors



SQL Statements in PL/SQL

- SQL consists of three sub-languages:
 - DDL
 - DML
- DDL statements are generally not included in PL/SQL blocks.
- DML statements are included in PL/SQL blocks.



5

SQL has three sub-languages:

- Data Definition Language (DDL)
- Data Manipulation Language (DML)
- Data Control Language (DCL)

DDL is used to define the database schema, define the structure of tables, and modify the structure of the tables.

DML is used to add data to, retrieve data from, and modify data in tables based on the structure of the database defined by the DDL statements.

DDL statements are executed once for every database to define the structure. After the structure is defined, operations are performed through DML statements.

DCL statements COMMIT and ROLLBACK are used to control transactions on a database and might be included in PL/SQL blocks. GRANT and REVOKE statements are used by the administrators and are generally not included in the PL/SQL block

When you define PL/SQL blocks, you generally write reusable code in those blocks. PL/SQL does not support DDL statements directly. You do not need reusable code for creating the database structure because it is created only once during the database life cycle.

DDL and DCL statements are supported through dynamic SQL.

Dynamic SQL statements are built as character strings at run time, and can contain placeholders for parameters. The details of working with dynamic SQL are covered later in this course.

SQL Statements in PL/SQL

Use:

- The `SELECT` command to retrieve data into the PL/SQL block
- DML commands to make changes to the database
- The `COMMIT`, `ROLLBACK`, or `SAVEPOINT` commands for transaction control



6

In a PL/SQL block, you use SQL statements to retrieve data from and modify data in a database table. PL/SQL supports data manipulation language (DML) and transaction control commands. However, remember the following points while using DML statements and transaction control commands in PL/SQL blocks:

- The `END` keyword signals the end of a PL/SQL block, not the end of a transaction. Just as a block can span multiple transactions, a transaction can span multiple blocks.
- PL/SQL does not directly support data definition language (DDL) statements such as `CREATE TABLE`, `ALTER TABLE`, or `DROP TABLE`. PL/SQL supports early binding, which cannot happen if applications have to create database objects at run time by passing values. DDL statements are executed through dynamic SQL statements.
- PL/SQL does not directly support data control language (DCL) statements such as `GRANT` or `REVOKE`. You can use dynamic SQL to execute them. The DCL statements that can be executed in PL/SQL are `COMMIT`, `ROLLBACK`, and `SAVEPOINT`.

SELECT Statements in PL/SQL

- The `INTO` clause is required.
- Queries must return only one row.

```
DECLARE
  v_fname VARCHAR2(25);
BEGIN
  SELECT first_name INTO v_fname
  FROM employees WHERE employee_id=200;
  DBMS_OUTPUT.PUT_LINE(' First Name is : '||v_fname);
END;
/
```

```
PL/SQL procedure successfully completed.

First name is :Jennifer
```

7

The INTO Clause

You use the **INTO** clause to retrieve data from a database through a **SELECT** statement into the variables in the PL/SQL block. The `INTO` clause is mandatory, and occurs between the `SELECT` and `FROM` clauses. You must specify one variable for each item retrieved from the database. The order and type of the variables must correspond with the items selected.

You generally use the `INTO` clause to populate either PL/SQL variables or host variables.

Queries Must Return Only One Row

The `SELECT` statements within a PL/SQL block fall into the ANSI classification of embedded SQL, for which the following rule applies: Queries must return only one row. A query that returns more than one row or no row generates an error.

PL/SQL manages these errors by raising standard exceptions, which you can handle in the exception section of the block, with the `NO_DATA_FOUND` and `TOO_MANY_ROWS` exceptions. Include a `WHERE` condition in the SQL statement so that the statement returns a single row. You learn about exception handling in the lesson titled “Handling Exceptions.”

Note: In all cases where `DBMS_OUTPUT.PUT_LINE` is used in the code examples, the `SET SERVEROUTPUT ON` statement should precede the block. It is sufficient to execute `SET SERVEROUTPUT ON` once for every session.

How to Retrieve Multiple Rows from a Table and Operate on the Data

A `SELECT` statement with the `INTO` clause can retrieve only one row at a time from the result set into the variables in the PL/SQL block. If you have to retrieve multiple rows and operate on the data, you have to make use of explicit cursors. Cursors hold the result set of a `SELECT` statement and allow the developer to operate on each row of the result set by iterating through it in a loop.

You are introduced to cursors later in this lesson and learn about explicit cursors in the lesson titled “Using Explicit Cursors.”

For Instructor Use Only.
This document should not be distributed.

Retrieving Data in PL/SQL: Example

Retrieve `hire_date` and `salary` for the specified employee.

```
DECLARE
  v_emp_hiredate   employees.hire_date%TYPE;
  v_emp_salary     employees.salary%TYPE;
BEGIN
  SELECT   hire_date, salary
  INTO     v_emp_hiredate, v_emp_salary
  FROM     employees
  WHERE    employee_id = 100;
  DBMS_OUTPUT.PUT_LINE ('Hire date is :'|| v_emp_hiredate);
  DBMS_OUTPUT.PUT_LINE ('Salary is :'|| v_emp_salary);
END;
/
```

```
PL/SQL procedure successfully completed.

Hire date is :17-JUN-11
Salary is :24000
```

9

In the code in slide, the `v_emp_hiredate` and `v_emp_salary` variables are declared in the `DECLARE` section of the PL/SQL block. In the executable section, the values of the `hire_date` and `salary` columns for the employee with `employee_id` 100 are retrieved from the `employees` table and stored in the `v_emp_hiredate` and `v_emp_salary` variables, respectively. Observe how the `INTO` clause, along with the `SELECT` statement, retrieves the database column values and stores them in the PL/SQL variables.

The PL/SQL engine will retrieve the data in the order specified in the `SELECT` statement and place them in the variables as they are retrieved. The `hire_date` value will be placed in `v_emp_hiredate` and the salary value is placed in `v_emp_salary`. If you reverse the variable names, the engine will try to place `hire_date` in `v_emp_salary`, realize that they are of incompatible types, and give an error.

Retrieving Data in PL/SQL

Return the sum of salaries for all the employees in the specified department.

Example:

```
DECLARE
    v_sum_sal    NUMBER(10,2);
    v_deptno     NUMBER NOT NULL := 60;
BEGIN
    SELECT SUM(salary) -- group function
    INTO v_sum_sal FROM employees
    WHERE department_id = v_deptno;
    DBMS_OUTPUT.PUT_LINE ('The sum of salary is ' || v_sum_sal);
END;
```

```
PL/SQL procedure successfully completed.
The sum of salary is :28800
```

10

In the example in the slide, the `v_sum_sal` and `v_deptno` variables are declared in the declarative section of the PL/SQL block. In the executable section, the total salary for the employees in the department with `department_id` 60 is computed by using the SQL aggregate function `SUM`. The calculated total salary is assigned to the `v_sum_sal` variable.

Note: Group functions cannot be used in PL/SQL syntax. They must be used in SQL statements within a PL/SQL block as shown in the example in the slide.

For instance, you *cannot* use group functions by using the following syntax:

```
v_sum_sal := SUM(employees.salary);
```

Naming Ambiguities

```
DECLARE
  hire_date      employees.hire_date%TYPE;
  sysdate        hire_date%TYPE;
  employee_id     employees.employee_id%TYPE := 176;
BEGIN
  SELECT          hire_date, sysdate
  INTO            hire_date, sysdate
  FROM            employees
  WHERE           employee_id = employee_id;
END;
/
```

```
Error report -
ORA-01422: exact fetch returns more than requested number of rows
ORA-06512: at line 6
01422. 00000 - "exact fetch returns more than requested number of rows"
*Cause:      The number specified in exact fetch is less than the rows returned.
*Action:     Rewrite the query or change number of rows requested
```

11

In potentially ambiguous SQL statements, the names of database columns take precedence over the names of local variables.

The example shown in the slide is defined as follows: Retrieve the hire date and today's date from the `employees` table for `employee_id` 176. This example raises an unhandled runtime exception because, in the `WHERE` clause, the PL/SQL variable names are the same as the database column names in the `employees` table.

The following `DELETE` statement removes all employees from the `employees` table, where the last name is not null (not just "King"), because the Oracle Server assumes that both occurrences of `last_name` in the `WHERE` clause refer to the database column:

```
DECLARE
  last_name VARCHAR2(25) := 'King';
BEGIN
  DELETE FROM employees WHERE last_name = last_name;
  . . .
```

Avoiding Naming Ambiguities

- Use a naming convention to avoid ambiguity in the `WHERE` clause.
- Avoid using database column names as identifiers.
- The names of local variables and formal parameters take precedence over the names of database tables.
- The names of database table columns take precedence over the names of local variables.
- The names of variables take precedence over the function names.

12

Avoid ambiguity in the `WHERE` clause by adhering to a naming convention that distinguishes database column names from PL/SQL variable names.

- Database columns and identifiers should have distinct names.
Syntax errors can arise when you have identifier names that are the same as database column names because PL/SQL checks the database first for a column in the table.
- Keep a watch on the following precedence rules while defining local variables and formal parameters in the PL/SQL blocks.
 - Local variables and formal parameters have higher precedence than database table names.
 - Database table column names have higher precedence than the local variables in a PL/SQL block.
 - Variable names have higher precedence than function names; therefore, keep a watch if you are giving the same identifier names to local variables and function blocks.

Agenda

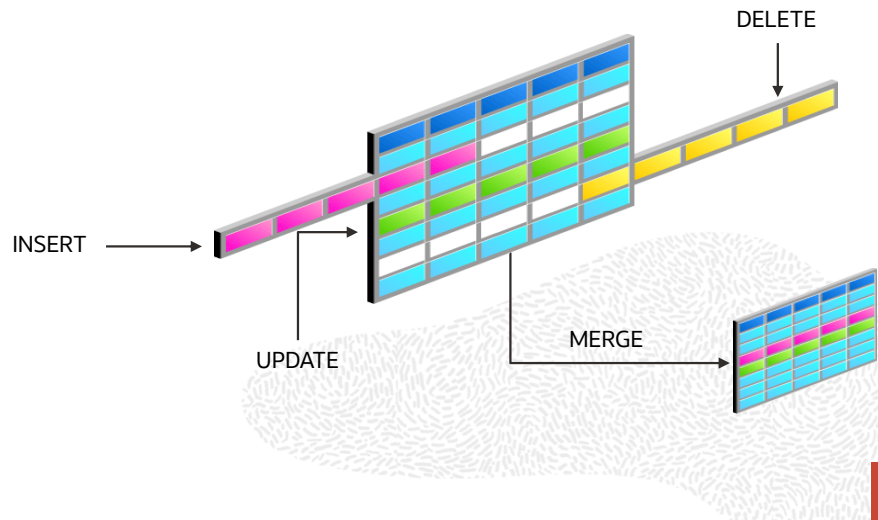
- SQL statements in PL/SQL blocks
- Manipulating data with PL/SQL
- Introducing SQL cursors



Using PL/SQL to Manipulate Data

Make changes to database tables by using DML commands:

- INSERT
- UPDATE
- DELETE
- MERGE



14

You manipulate data in a database by using DML commands. You can issue DML commands such as `INSERT`, `UPDATE`, `DELETE`, and `MERGE` without restriction in PL/SQL. Row locks (and table locks) are released by including the `COMMIT` or `ROLLBACK` statements in the PL/SQL code.

- The `INSERT` statement adds new rows to the table.
- The `UPDATE` statement modifies existing rows in the table.
- The `DELETE` statement removes rows from the table.
- The `MERGE` statement selects rows from one table to update or insert into another table. The decision whether to update or insert into the target table is based on a condition in the `ON` clause.

Note: `MERGE` is a deterministic statement. That is, you cannot update the same row of the target table multiple times in the same `MERGE` statement. You must have `INSERT` and `UPDATE` object privileges on the target table and `SELECT` privilege on the source table.

Insert Data: Example

Add new employee information to the EMPLOYEES table.

```
BEGIN
  INSERT INTO employees
    (employee_id, first_name, last_name, email,
     hire_date, job_id, salary)
  VALUES (employees_seq.NEXTVAL, 'Ruth', 'Cores',
    'RCORES', CURRENT_DATE, 'AD_ASST', 4000);
END;
/
```

15

In the example in the slide, an `INSERT` statement is used within a PL/SQL block to insert a record into the `employees` table. While using the `INSERT` command in a PL/SQL block, you can:

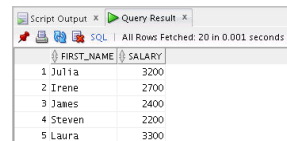
- Use SQL functions such as `USER` and `CURRENT_DATE`
- Generate primary key values by using existing database sequences
- Derive values in the PL/SQL block

Note: The data in the `employees` table needs to remain unchanged. Even though the `employees` table is not read-only, inserting, updating, and deleting are not allowed on this table to ensure consistency of output. Therefore, the command `rollback` is used as shown in the code for slide 15_sa in `code_ex_05.sql`.

Update Data: Example

Increase the salary of all employees who are stock clerks.

```
SELECT first_name,salary
FROM employees
WHERE job_id = 'ST_CLERK';
```

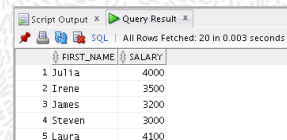


	FIRST_NAME	SALARY
1	Julia	3200
2	Irene	2700
3	Janes	2400
4	Steven	2200
5	Laura	3300

```
DECLARE
  v_sal_increase employees.salary%TYPE := 800;
BEGIN
  UPDATE employees
  SET salary = salary + v_sal_increase
  WHERE job_id = 'ST_CLERK';
END;
/
```

PL/SQL procedure successfully completed.

```
SELECT first_name,salary
FROM employees
WHERE job_id = 'ST_CLERK';
```



	FIRST_NAME	SALARY
1	Julia	4000
2	Irene	3500
3	Janes	3200
4	Steven	3000
5	Laura	4100

16

Let us first see the current state of the table before we run the update in the PL/SQL block.

For simplicity, we will see only 5 rows of the result set. You can see the actual result when you execute the code in your work station.

The `SELECT` statement retrieves the `first_name` and `salary` of employees whose `job_id` is `ST_CLERK`.

Execute the PL/SQL block with the `UPDATE` statement. If the block executes without errors, you see the response `PL/SQL Procedure successfully completed` on the Script Output tab.

Execute the same `SELECT` statement again to see the update in the Employees table. You see the new values in the Query result as a result of the `UPDATE` operation.

Note: Remember to run a rollback after executing the operation to maintain data consistency for further demonstrations.

Delete Data: Example

Delete rows that belong to department 10 from the `employees` table.

```
DECLARE
  v_emp employees.employee_id%TYPE := 176;
BEGIN
  DELETE FROM employees
  WHERE employee_id = v_emp;
END;
/
```

17

The `DELETE` statement removes unwanted rows from a table. If the `WHERE` clause is not used, all the rows in a table can be removed if there are no integrity constraints.

The code shown in the slide deletes the row from `employees` table whose `employee_id = 176`

Note : A `DELETE` operation will not complete if it is violating a constraint.

Merging Rows

Insert or update rows in the `copy_emp` table to match the `employees` table.

```
BEGIN
MERGE INTO copy_emp c
  USING employees e
  ON (e.employee_id = c.empno)
  WHEN MATCHED THEN
    UPDATE SET
      c.first_name      = e.first_name,
      c.last_name       = e.last_name,
      c.email           = e.email,
      . . .
  WHEN NOT MATCHED THEN
    INSERT VALUES(e.employee_id, e.first_name, e.last_name,
      . . ., e.department_id);
END;
/
```

18

The `MERGE` statement inserts or updates rows in one table by using data from another table. Each row is inserted or updated in the target table depending on an equijoin condition.

The example in the slide matches the `emp_no` column in the `copy_emp` table to the `employee_id` column in the `employees` table. If a match is found, the row is updated to match the row in the `employees` table. If the row is not found, it is inserted into the `copy_emp` table.

Before executing the `MERGE` statement, you should have the `copy_emp` table created. You can create the `copy_emp` table with the following `CREATE` statement:

```
CREATE TABLE copy_emp (emp_no NUMBER(6,0) PRIMARY
  KEY,
  first_name VARCHAR2(20), last_name VARCHAR2(25), email
  VARCHAR2(25), phone_nmber VARCHAR2(20), hire_date
  DATE,
  job_id VARCHAR2(10), salary NUMBER(8,2),
  commission_pct NUMBER(2,2), manager_id NUMBER(6,0),
  department_id NUMBER(4,0));
```

The complete example of using `MERGE` in a PL/SQL block is shown in the next slide.

```

BEGIN
MERGE INTO copy_emp c
    USING employees e
    ON (e.employee_id = c.empno)
WHEN MATCHED THEN
    UPDATE SET
        c.first_name      = e.first_name,
        c.last_name       = e.last_name,
        c.email           = e.email,
        c.phone_number    = e.phone_number,
        c.hire_date       = e.hire_date,
        c.job_id          = e.job_id,
        c.salary           = e.salary,
        c.commission_pct  = e.commission_pct,
        c.manager_id      = e.manager_id,
        c.department_id   = e.department_id
WHEN NOT MATCHED THEN
    INSERT VALUES (e.employee_id, e.first_name, e.last_name,
        e.email, e.phone_number, e.hire_date, e.job_id,
        e.salary, e.commission_pct, e.manager_id,
        e.department_id);
END;
/

```

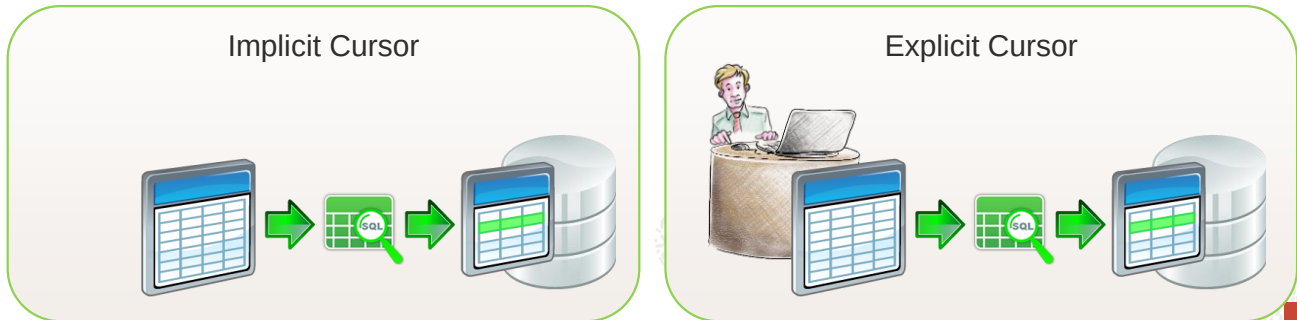
Agenda

- SQL statements in PL/SQL blocks
- Manipulating data with PL/SQL
- Introducing SQL cursors



SQL Cursor

- A cursor is a pointer to the private memory area that stores information about processing a specific `SELECT` or `DML` statement.
- There are two types of cursors:
 - **Implicit:** Created and managed by PL/SQL
 - **Explicit:** Created and managed explicitly by the programmer



21

Where Does the Oracle Server Process SQL Statements?

The Oracle Server allocates a private memory area called the *context area* for processing SQL statements. The SQL statement is parsed and processed in this area. The information that is required for processing and the information that is retrieved after processing are all stored in this area. You have no control over this area because it is internally managed by the Oracle Server.

A cursor is a pointer to the context area. However, this cursor is an implicit cursor, and is automatically managed by the Oracle Server. When the executable block issues a SQL statement, PL/SQL creates an implicit cursor. The result set of the SQL statement is stored in the context area for further processing.

Types of Cursors

There are two types of cursors:

- **Implicit:** An *implicit cursor* is created and managed by the Oracle Server. You do not have access to it. The Oracle Server creates such a cursor when it has to execute a SQL statement.
- **Explicit:** As a programmer, you may want to retrieve multiple rows from a database table, have a pointer to each row that is retrieved, and work on the rows one at a time. In such cases, you can declare cursors explicitly. A cursor that is declared by programmers is called an *explicit cursor*. You declare such a cursor in the `DECLARE` section of a PL/SQL block.

PL/SQL blocks execute on the database server. Cursors declared in the PL/SQL block point to the memory area on the server while executing. When a client initiates a PL/SQL block, a corresponding cursor is created for the `SELECT` statements in the PL/SQL block and the result set is retrieved into the memory area pointed by the cursor.

For Instructor Use Only.
This document should not be distributed.

SQL Cursor Attributes for Implicit Cursors

Using SQL cursor attributes, you can test the outcome of your SQL statements.

SQL%FOUND	Boolean attribute that evaluates to TRUE if the most recent SQL statement affected at least one row
SQL%NOTFOUND	Boolean attribute that evaluates to TRUE if the most recent SQL statement did not affect even one row
SQL%ROWCOUNT	An integer value that represents the number of rows affected by the most recent SQL statement

23

SQL cursor attributes enable you to evaluate what happened when an implicit cursor was last used. You can use these attributes in PL/SQL statements but not in SQL statements.

You can test the `SQL%ROWCOUNT`, `SQL%FOUND`, and `SQL%NOTFOUND` attributes in the executable section of a block to gather information after the execution of the appropriate DML command. PL/SQL does not return an error if a DML statement does not affect rows in the underlying table. However, if a `SELECT` statement does not retrieve any rows, PL/SQL returns an exception.

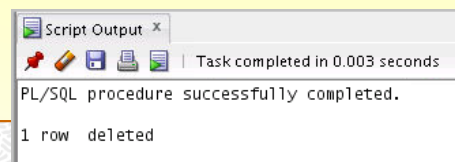
Observe that the attributes are prefixed with `SQL`. These cursor attributes are used with implicit cursors that are automatically created by PL/SQL and for which you do not know the names. Therefore, you use `SQL` instead of the cursor name.

SQL Cursor Attributes for Implicit Cursors

Delete rows that have the specified employee ID from the `employees` table. Print the number of rows deleted.

Example:

```
DECLARE
    v_rows_deleted VARCHAR2(30);
    v_empno employees.employee_id%TYPE := 165;
BEGIN
    DELETE FROM employees
    WHERE employee_id = v_empno;
    v_rows_deleted := (SQL%ROWCOUNT ||
                      ' row deleted. ');
    DBMS_OUTPUT.PUT_LINE (v_rows_deleted);
END;
```



24

The example in the slide deletes a row with `employee_id` 165 from the `employees` table. Using the `SQL%ROWCOUNT` attribute, you can print the number of rows deleted.

`SQL%ROWCOUNT` will return the number of rows in the table that satisfy the given condition. In this case, `SQL%ROWCOUNT` will return 1 because there is only one row in the table whose `employee_id` value is 165.

Quiz



When using the `SELECT` statement in PL/SQL, the `INTO` clause is required and queries can return one or more rows.

- a. True
- b. False



0

Answer: b

The `INTO` Clause

The `INTO` clause is mandatory, and occurs between the `SELECT` and `FROM` clauses. It is used to specify the names of variables that hold the values that SQL returns from the `SELECT` statement. You must specify one variable for each item selected, and the order of the variables must correspond with the items selected.

Use the `INTO` clause to populate either PL/SQL variables or host variables.

Queries Must Return Only One Row

The `SELECT` statements within a PL/SQL block fall into the ANSI classification of embedded SQL, for which the following rule applies: Queries must return only one row. A query that returns more than one row or no row generates an error.

PL/SQL manages these errors by raising standard exceptions, which you can handle in the exception section of the block with the `NO_DATA_FOUND` and `TOO_MANY_ROWS` exceptions. Include a `WHERE` condition in the SQL statement so that the statement returns a single row.

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Embed DML statements, transaction control statements, and DDL statements in PL/SQL
- Use the `INTO` clause, which is mandatory for all `SELECT` statements in PL/SQL
- Differentiate between implicit cursors and explicit cursors
- Use SQL cursor attributes to determine the outcome of SQL statements



26

DML commands and transaction control statements can be used in PL/SQL programs without restriction. However, the DDL commands cannot be used directly.

A `SELECT` statement in a PL/SQL block can return only one row. It is mandatory to use the `INTO` clause to hold the values retrieved by the `SELECT` statement.

A cursor is a pointer to the memory area. There are two types of cursors. Implicit cursors are created and managed internally by the Oracle Server to execute SQL statements. You can use SQL cursor attributes with these cursors to determine the outcome of the SQL statement. Explicit cursors are declared by programmers.

Practice 5: Overview

This practice covers the following topics:

- Selecting data from a table
- Inserting data into a table
- Updating data in a table
- Deleting a record from a table

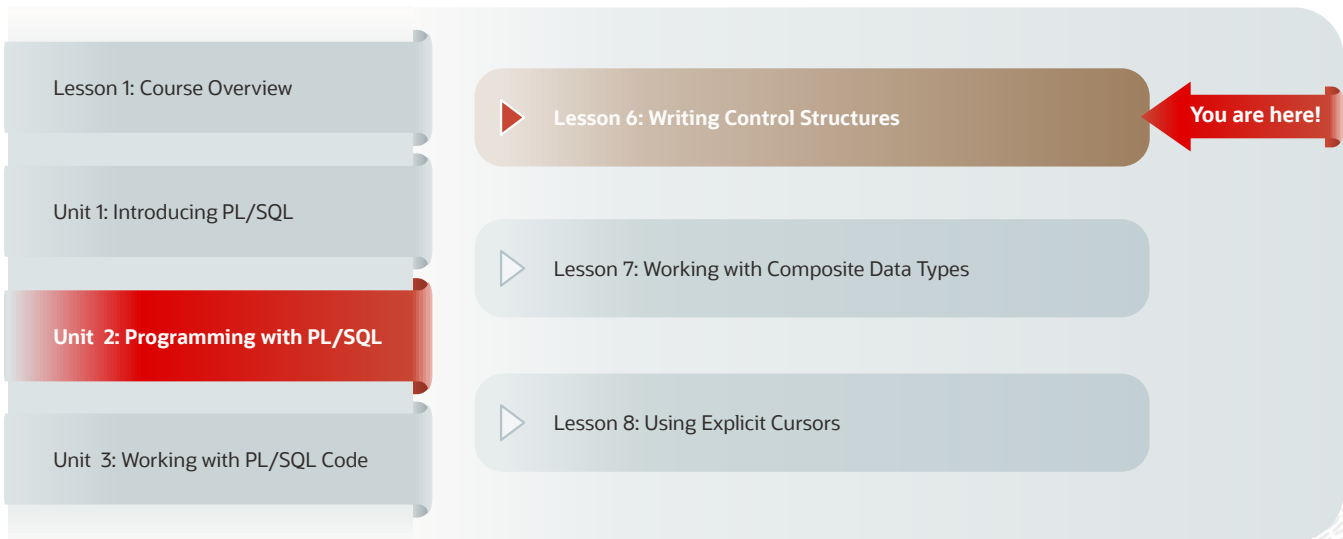
For Instructor Use Only.
This document should not be distributed.

Writing Control Structures

For Instructor Use Only.

This document should not be distributed.

Course Road Map



2

You are in lesson 6, which is part of Unit 2: Programming with PL/SQL.

0
For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to do the following:

- Identify the uses and types of control structures
- Construct an `IF` statement
- Use `CASE` statements and `CASE` expressions
- Construct and identify loop statements
- Use guidelines when using conditional control structures



0

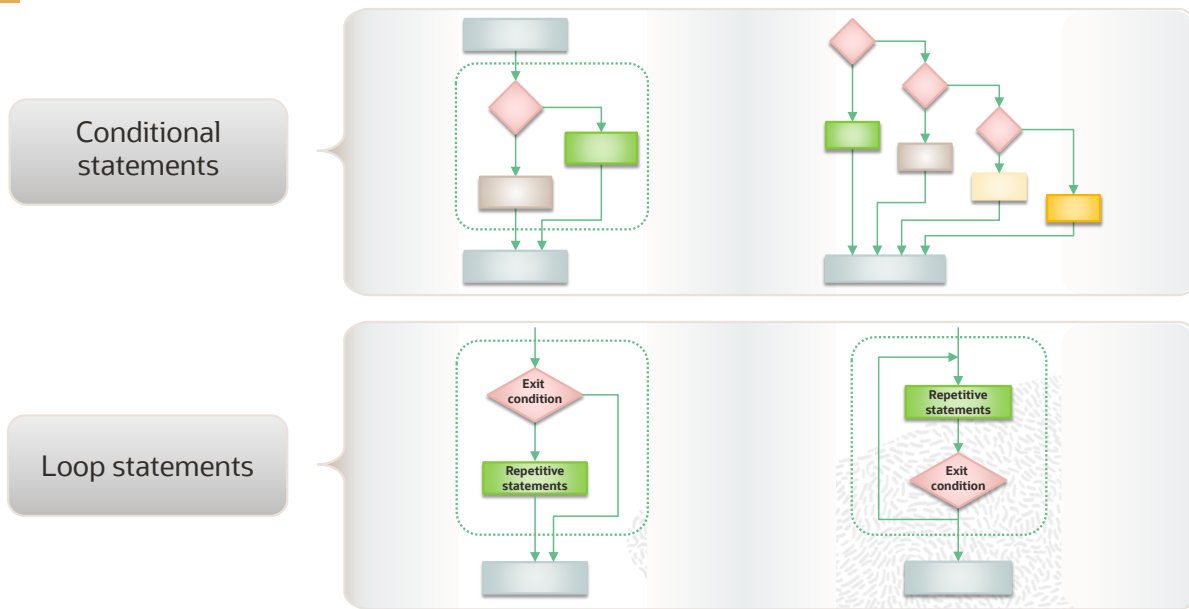
3

You have learned to write PL/SQL blocks containing declarative and executable sections. You have also learned to include expressions and SQL statements in the executable block.

In this lesson, you learn how to use control structures such as `IF` statements, `CASE` expressions, and `LOOP` structures in a PL/SQL block.

For Instructor Use Only.
This document should not be distributed.

PL/SQL Control Structures



4

You can change the logical flow of statements within a PL/SQL block with a number of control structures. This lesson addresses four types of PL/SQL control structures: conditional constructs with the `IF` statement, `CASE` statement, `LOOP` control structures.

For Instructor Use Only.
This document should not be distributed.

Agenda

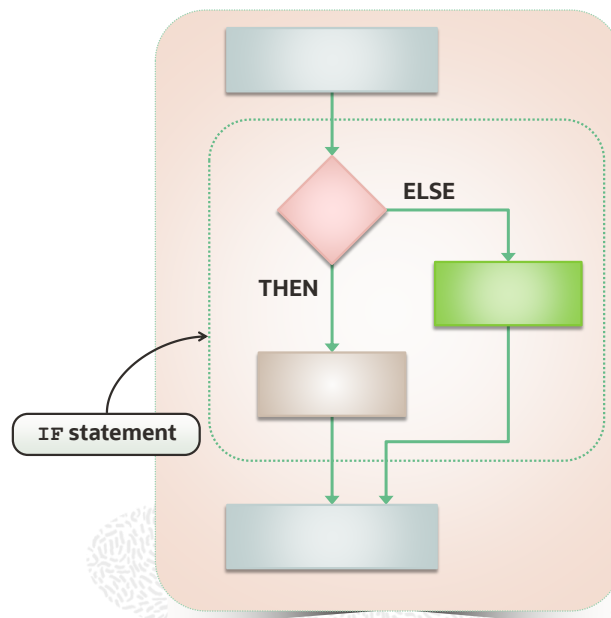
- Using `IF` statements
- Using `CASE` statements and `CASE` expressions
- Constructing and identifying loop statements



IF Statement

Syntax:

```
IF condition THEN
    statements;
[ELSE
    statements;]
END IF;
```



6

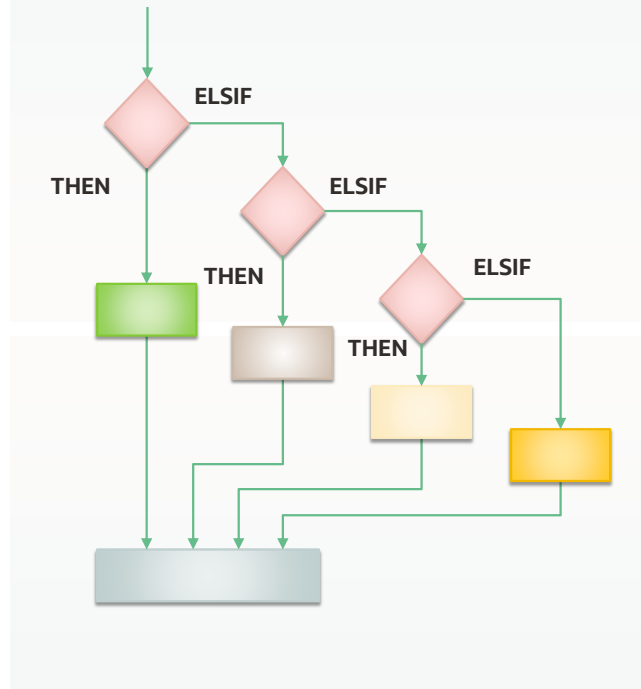
The structure of a PL/SQL `IF` statement is similar to the structure of `IF` statements in other procedural languages. It allows PL/SQL to perform actions selectively based on conditions.

In the syntax:

- | | |
|--------------------------|--|
| <i>condition</i> | Is a Boolean variable or expression that returns <code>TRUE</code> , <code>FALSE</code> , or <code>NULL</code> |
| <i>THEN</i> | Introduces a clause that associates the Boolean expression with the sequence of statements that follows it |
| <i>statements</i> | Can be one or more PL/SQL or SQL statements. (They may include additional <code>IF</code> statements containing several nested <code>IF</code> , <code>ELSE</code> , and <code>ELSIF</code> statements.) The statements in the <code>THEN</code> clause are executed only if the condition in the associated <code>IF</code> clause evaluates to <code>TRUE</code> . |

IF-ELSIF Statements

```
IF condition THEN
    statements;
[ELSIF condition THEN
    statements;]
[ELSE
    statements;]
END IF;
```



7

In the syntax:

- ELSIF** Is a keyword that introduces a Boolean expression (If the first condition yields FALSE or NULL, the ELSIF keyword introduces additional conditions.)
- ELSE** Introduces the default clause that is executed only if none of the earlier predicates (introduced by IF and ELSIF) are TRUE. The tests are executed in sequence so that a later predicate that might be true is pre-empted by an earlier predicate that is true.
- END IF** Marks the end of an IF statement

Note: ELSIF and ELSE are optional in an IF statement. You can have any number of ELSIF keywords but only one ELSE keyword is allowed in your IF statement. END IF marks the end of an IF statement and must be terminated by a semicolon.

Simple IF Statement

```
DECLARE
  v_myage  NUMBER := 10;
BEGIN
  IF v_myage < 11
  THEN
    DBMS_OUTPUT.PUT_LINE(' I am a child ');
  END IF;
END;
/
```

```
PL/SQL procedure successfully completed.
I am a child
```

8

In the slide, you see an example of a simple IF statement with the THEN clause.

- The v_myage variable is initialized to 10.
- The condition for the IF statement returns TRUE because v_myage is less than 11.
- Therefore, the control reaches the THEN clause.

Adding Conditional Expressions

You can relate multiple conditional expressions in an IF statement through logical operators such as AND, OR, and NOT.

For example:

```
IF (myfirstname='Christopher' AND v_myage <11)
...
```

The condition uses the AND operator and, therefore, evaluates to TRUE only if both conditions are evaluated as TRUE.

You can have any number of conditional expressions for a single IF statement. However, these statements must be related with appropriate logical operators.

IF THEN ELSE Statement

```
DECLARE
  v_myage  number:=31;
BEGIN
  IF
    v_myage < 11 THEN
    DBMS_OUTPUT.PUT_LINE(' I am a child ');
  ELSE
    DBMS_OUTPUT.PUT_LINE(' I am not a child ');
  END IF;
END;
/
```

```
PL/SQL procedure successfully completed.

I am not a child
```

9

An **ELSE** clause is added to the code from the previous slide. The condition has not changed, but the value of the variable has changed from 10 to 31. The condition now evaluates to **FALSE**. Recall that statements in the **THEN** clause are executed only if the condition returns **TRUE**. In this case, the condition returns **FALSE** and the control moves to the **ELSE** statement.

The output of the block is shown below the code.

IF ELSIF ELSE Clause

```
DECLARE
  v_myage number:=31;
BEGIN
  IF v_myage < 11 THEN
    DBMS_OUTPUT.PUT_LINE(' I am a child ');
  ELSIF v_myage < 20 THEN
    DBMS_OUTPUT.PUT_LINE(' I am young ');
  ELSIF v_myage < 30 THEN
    DBMS_OUTPUT.PUT_LINE(' I am in my twenties');
  ELSIF v_myage < 40 THEN
    DBMS_OUTPUT.PUT_LINE(' I am in my thirties');
  ELSE
    DBMS_OUTPUT.PUT_LINE(' I am always young ');
  END IF;
END;
/
```

```
PL/SQL procedure successfully completed.
I am in my thirties
```

10

You can have multiple `ELSIF` clauses in an `IF` statement and a final `ELSE` clause. The example in the slide illustrates the following characteristics of these clauses:

- The `ELSIF` clauses can have conditions, unlike the `ELSE` clause. You can check additional conditions through the `ELSIF` clauses if the condition in the `IF` clause evaluates to `FALSE` or `NULL`.
- The condition for `ELSIF` should be followed by the `THEN` clause, which is executed if the condition for `ELSIF` returns `TRUE`.
- When you have multiple `ELSIF` clauses, if the first condition is `FALSE` or `NULL`, the control shifts to the next `ELSIF` clause.
- Conditions are evaluated one by one from the top.
- If all conditions are `FALSE` or `NULL`, the statements in the `ELSE` clause are executed.
- The final `ELSE` clause is optional.

In the example, the output of the block is shown below the code.

NULL Value an in IF Statement

```
DECLARE
  v_myage  number;
BEGIN
  IF v_myage < 11 THEN
    DBMS_OUTPUT.PUT_LINE(' I am a child ');
  ELSE
    DBMS_OUTPUT.PUT_LINE(' I am not a child ');
  END IF;
END;
/
```

```
PL/SQL procedure successfully completed.

I am not a child
```

11

In the example in the slide, the variable `v_myage` is declared but not initialized. The condition in the `IF` statement returns `NULL` rather than `TRUE` or `FALSE`. In such a case, the control goes to the `ELSE` statement.

Guidelines

- You can perform actions selectively based on conditions that are being met.
- When you write code, remember the spelling of the keywords:
 - `ELSIF` is one word.
 - `END IF` is two words.
- If the controlling Boolean condition is `TRUE`, the associated sequence of statements is executed; if the controlling Boolean condition is `FALSE` or `NULL`, the associated sequence of statements is passed over. Any number of `ELSIF` clauses is permitted.
- Indent the conditionally executed statements for clarity.

Agenda

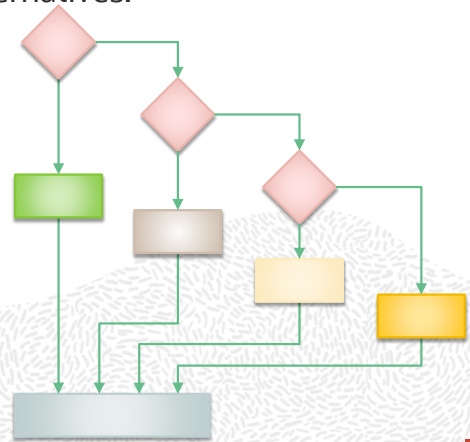
- Using `IF` statements
- Using `CASE` statements and `CASE` expressions
- Constructing and identifying loop statements



CASE Expressions

- A CASE expression selects a result and returns it.
- To select the result, the CASE expression uses expressions. The value returned by these expressions is used to select one of several alternatives.

```
CASE selector
  WHEN expression1 THEN result1
  [WHEN expression2 THEN result2
  ...
  WHEN expressionN THEN resultN]
  [ELSE resultN+1]
END;
```



13

You can use a CASE expression to return a result from many alternatives based on the value of a selector. The selector is followed by one or more WHEN clauses that are checked sequentially. Whenever the value of the selector equals the value of a WHEN expression, the statements in the WHEN clause are executed and that result is returned.

PL/SQL also provides a searched CASE expression, which has the following form:

```
CASE
  WHEN search_condition1 THEN result1
  [WHEN search_condition2 THEN result2
  ...
  WHEN search_conditionN THEN resultN]
  [ELSE resultN+1]
END;
```

A searched CASE expression has no selector. Furthermore, the WHEN clauses in CASE expressions contain search conditions that yield a Boolean value rather than expressions that can yield a value of any type.

Searched CASE Expressions

```
DECLARE
    v_grade CHAR(1) := UPPER('&grade');
    v_appraisal VARCHAR2(20);
BEGIN
    v_appraisal := CASE
        WHEN v_grade = 'A' THEN 'Excellent'
        WHEN v_grade IN ('B','C') THEN 'Good'
        ELSE 'No such grade'
    END;
    DBMS_OUTPUT.PUT_LINE ('Grade: ' || v_grade ||
        ' Appraisal ' || v_appraisal);
END;
/
```

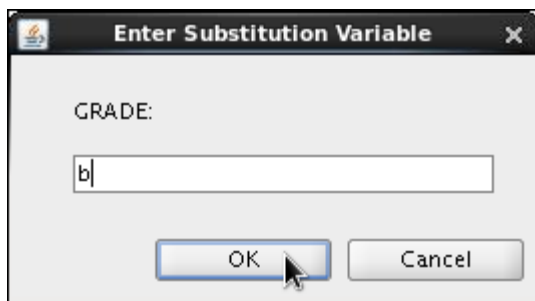
14

In the previous example, you saw a single test expression, the `v_grade` variable. The `WHEN` clause compares a value against this test expression.

In searched `CASE` statements, you do not have a test expression. Instead, the `WHEN` clause contains an expression that results in a Boolean value. The same example is rewritten in this slide to show searched `CASE` statements.

Result

The output of the example is as follows when you enter `b` or `B` for `v_grade`:



```
PL/SQL procedure successfully completed.
Grade: B Appraisal Good
```

Searched CASE Expressions

```
SET SERVEROUTPUT ON

DECLARE
    v_grade CHAR(1) := UPPER('&grade');
    v_appraisal VARCHAR2(20);
BEGIN
    v_appraisal :=
        CASE
            WHEN v_grade = 'A' THEN 'Excellent'
            WHEN v_grade IN ('B','C') THEN 'Good'
            ELSE 'No such grade'
        END;
    DBMS_OUTPUT.PUT_LINE ('Grade: ' || v_grade || ' Appraisal ' ||
        v_appraisal);
END;
/
```

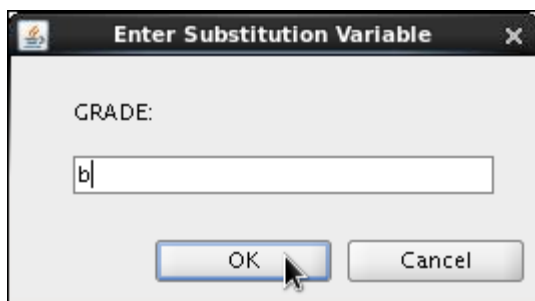
15

In the previous example, you saw a single test expression, the `v_grade` variable. The `WHEN` clause compares a value against this test expression.

In searched `CASE` statements, you do not have a test expression. Instead, the `WHEN` clause contains an expression that results in a Boolean value. The same example is rewritten in this slide to show searched `CASE` statements.

Result

The output of the example is as follows when you enter `b` or `B` for `v_grade`:



```
PL/SQL procedure successfully completed.
Grade: B Appraisal Good
```

CASE Statement

```
DECLARE
  v_deptid NUMBER;
  v_deptname VARCHAR2(20);
  v_emps NUMBER;
  v_mngid NUMBER:= 108;
BEGIN
  CASE v_mngid
    WHEN 108 THEN
      SELECT department_id, department_name
        INTO v_deptid, v_deptname FROM departments
        WHERE manager_id=108;
      SELECT count(*) INTO v_emps FROM employees
        WHERE department_id=v_deptid;
    WHEN 200 THEN
      ...
    END CASE;
    DBMS_OUTPUT.PUT_LINE ('You are working in the '|| v_deptname||
      ' department. There are '||v_emps ||' employees in this
      department');
  END;
/
```

If 108 matches with the value of v_mngid, an action is performed instead of just returning a value.

The action defined can modify the database also.

Uses END CASE instead of END

```
PL/SQL procedure successfully completed.
You are working in the Finance department. There are 6 employees in this department
```

16

Recall the use of the `IF` statement. You may include n number of PL/SQL statements in the `THEN` clause and also in the `ELSE` clause. Similarly, you can include statements in the `CASE` statement, which is more readable compared to multiple `IF` and `ELSIF` statements.

How a CASE Expression Differs from a CASE Statement

A `CASE` expression evaluates a condition and returns a value, whereas a `CASE` statement evaluates a condition and performs an action. A `CASE` statement can be a complete PL/SQL block.

- `CASE` statements end with `END CASE;`
- `CASE` expressions end with `END;`

Note: Whereas an `IF` statement is able to do nothing (the conditions could all be false and the `ELSE` clause is not mandatory), a `CASE` statement must execute some PL/SQL statement.

Handling Nulls

When you are working with `NULL` values, you can avoid some common mistakes by keeping the following rules in mind:

- Simple comparisons involving `NULL`s always yield `NULL`.
- Applying the logical operator `NOT` to a `NULL` yields `NULL`.
- If the condition yields `NULL` in conditional control statements, its associated sequence of statements is not executed.

17

Consider the following example:

```
x := 5;
y := NULL;
...
IF x != y THEN -- yields NULL, not TRUE
    -- sequence_of_statements that are not executed
END IF;
```

You may expect the sequence of statements to execute because `x` and `y` seem unequal. But nulls are indeterminate. Whether or not `x` is equal to `y` is unknown. Therefore, the `IF` condition yields `NULL` and the sequence of statements is bypassed.

```
a := NULL;
b := NULL;
...
IF a = b THEN -- yields NULL, not TRUE
    -- sequence_of_statements that are not executed
END IF;
```

In the second example, you may expect the sequence of statements to execute because `a` and `b` seem equal. But, again, equality is unknown, so the `IF` condition yields `NULL` and the sequence of statements is bypassed.

Logic Tables

TRUE takes precedence in an OR condition.

FALSE takes precedence in an AND condition.

The negation of NULL (NOT NULL) results in a null value because null values are indeterminate.

NOT	
TRUE	FALSE
FALSE	TRUE
NULL	NULL

OR	TRUE	FALSE	NULL
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	NULL
NULL	TRUE	NULL	NULL

AND	TRUE	FALSE	NULL
TRUE	TRUE	FALSE	NULL
FALSE	FALSE	FALSE	FALSE
NULL	NULL	FALSE	NULL

Boolean Expression or Logical Expression?

What is the value of `flag` in each case?

```
flag := reorder_flag AND available_flag;
```

REORDER_FLAG	AVAILABLE_FLAG	FLAG
TRUE	TRUE	? (1)
TRUE	FALSE	? (2)
NULL	TRUE	? (3)
NULL	FALSE	? (4)

19

The **AND** logic table can help you to evaluate possibilities for the Boolean condition in the slide.

Answers

1. TRUE
2. FALSE
3. NULL
4. FALSE

Agenda

- Using `IF` statements
- Using `CASE` statements and `CASE` expressions
- Constructing and identifying loop statements



Iterative Control: LOOP Statements

- Loops repeat a statement (or a sequence of statements) multiple times.
- There are three loop types:
 - Basic loop
 - FOR loop
 - WHILE loop



21

Loops are programming language constructs that are used to execute a set of statements repeatedly until an exit condition is reached. It is mandatory to have an exit condition in a loop; otherwise, the loop executes for infinite number of iterations. PL/SQL provides three types of loop structures:

- Basic loop that performs repetitive actions till the exit condition is met
- FOR loops that perform iterative actions based on a count
- WHILE loops that perform iterative actions based on a condition

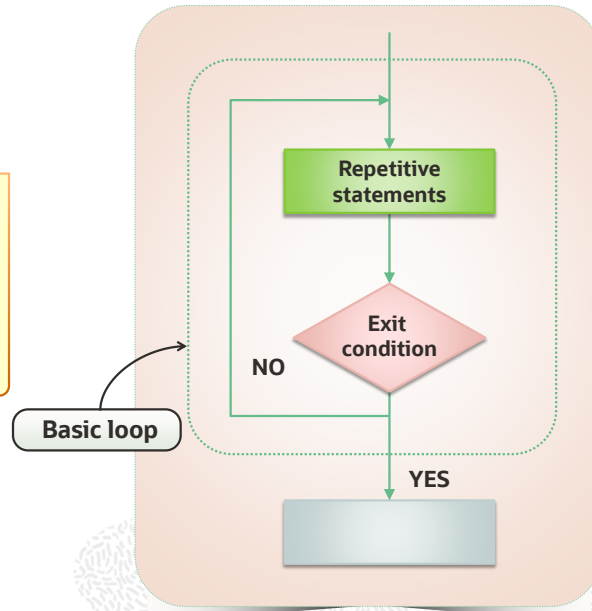
Note: An `EXIT` statement can be used to terminate loops. A basic loop must have an `EXIT`. The cursor `FOR` loop (which is another type of `FOR` loop) is discussed in the lesson titled “Using Explicit Cursors.”

For Instructor Use Only.
This document should not be distributed.

Basic Loops

Syntax:

```
LOOP
  statement1;
  . . .
  EXIT [WHEN condition];
END LOOP;
```



22

The simplest form of a `LOOP` statement is the basic loop, which encloses a sequence of statements between the `LOOP` and `END LOOP` keywords. Each time the flow of execution reaches the `END LOOP` statement, the exit condition is checked. If the exit condition evaluates to `FALSE`, the statements in the loop are repeated. A basic loop allows execution of its statements at least once, even if the `EXIT` condition is already met upon entering the loop.

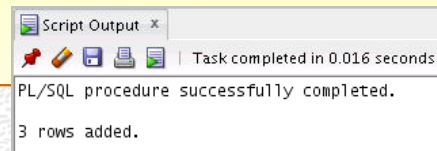
Without the `EXIT` statement, the loop would be infinite.

EXIT Statement

You use the `EXIT` statement to terminate a loop. Control passes to the next statement after the `END LOOP` statement. You can issue `EXIT` either as an action within an `IF` statement or as a stand-alone statement within the loop. The `EXIT` statement must be placed inside a loop. In the latter case, you can attach a `WHEN` clause to enable conditional termination of the loop. When the `EXIT` statement is encountered, the condition in the `WHEN` clause is evaluated. If the condition yields `TRUE`, the loop ends and control passes to the next statement after the loop. A basic loop can contain multiple `EXIT` statements, but it is recommended that you have only one `EXIT` point.

Basic Loop: Example

```
DECLARE
  v_countryid    locations.country_id%TYPE := 'CA';
  v_loc_id       locations.location_id%TYPE;
  v_counter      NUMBER(2) := 1;
  v_new_city     locations.city%TYPE := 'Montreal';
BEGIN
  SELECT MAX(location_id) INTO v_loc_id FROM locations
  WHERE country_id = v_countryid;
  LOOP
    INSERT INTO locations(location_id, city, country_id)
    VALUES((v_loc_id + v_counter), v_new_city, v_countryid);
    v_counter := v_counter + 1;
    EXIT WHEN v_counter > 3;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(v_counter-1||' rows added.');
```



23

The basic loop example shown in the slide is defined as follows: “Insert three new location IDs for the CA country code and the city of Montreal.”

Note

- A basic loop allows execution of its statements until the `EXIT WHEN` condition is met.
- If the condition is placed in the loop such that it is not checked until after the loop statements execute, the loop executes at least once.
- However, if the exit condition is placed at the top of the loop (before any of the other executable statements) and if that condition is true, the loop exits and the statements never execute.

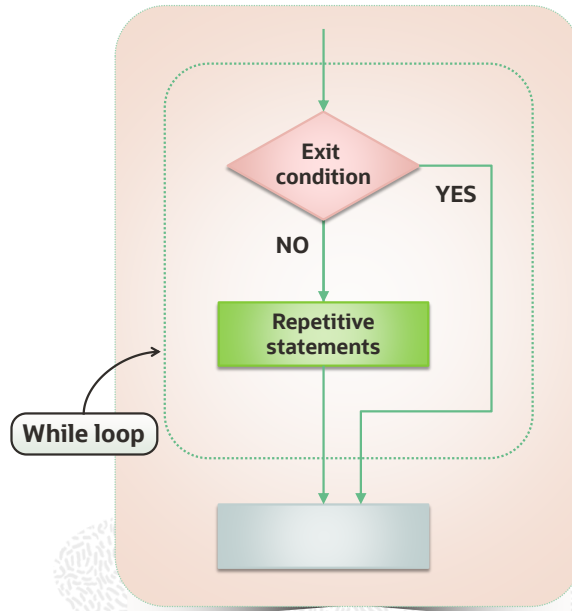
Results

To view the output, run the code example under slide 23_sa in `code_ex_06.sql`.

WHILE Loops

Syntax:

```
WHILE condition LOOP
    statement1;
    statement2;
    . . .
END LOOP;
```



24

You can use the `WHILE` loop to repeat a sequence of statements until the controlling condition is no longer `TRUE`. The condition is evaluated at the start of each iteration. The loop terminates when the condition is `FALSE` or `NULL`. If the condition is `FALSE` or `NULL` at the start of the loop, no further iterations are performed. Thus, it is possible that none of the statements inside the loop are executed.

In the syntax:

condition Is a Boolean variable or expression (`TRUE`, `FALSE`, or `NULL`)

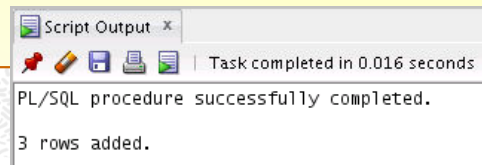
statement Can be one or more PL/SQL or SQL statements

If the variables that are involved in the conditions do not change during the body of the loop, the condition remains `TRUE` and the loop does not terminate. It is an infinite loop.

Note: If the condition yields `NULL`, the loop is bypassed and control passes to the next statement.

WHILE Loops: Example

```
DECLARE
  v_countryid  locations.country_id%TYPE := 'CA';
  v_loc_id     locations.location_id%TYPE;
  v_new_city   locations.city%TYPE := 'Montreal';
  v_counter    NUMBER := 1;
BEGIN
  SELECT MAX(location_id) INTO v_loc_id FROM locations
  WHERE country_id = v_countryid;
  WHILE v_counter <= 3 LOOP
    INSERT INTO locations(location_id, city, country_id)
    VALUES((v_loc_id + v_counter), v_new_city, v_countryid);
    v_counter := v_counter + 1;
  END LOOP;
  DBMS_OUTPUT.PUT_LINE(v_counter-1||' rows added. ');
END;
/
```



25

In the example in the slide, three new location IDs for the CA country code and the city of Montreal are added.

- With each iteration through the `WHILE` loop, a counter (`v_counter`) is incremented.
- If the number of iterations is less than or equal to the number 3, the code within the loop is executed and a row is inserted into the `locations` table.
- After `v_counter` exceeds the number of new locations for this city and country, the condition that controls the loop evaluates to `FALSE` and the loop terminates.

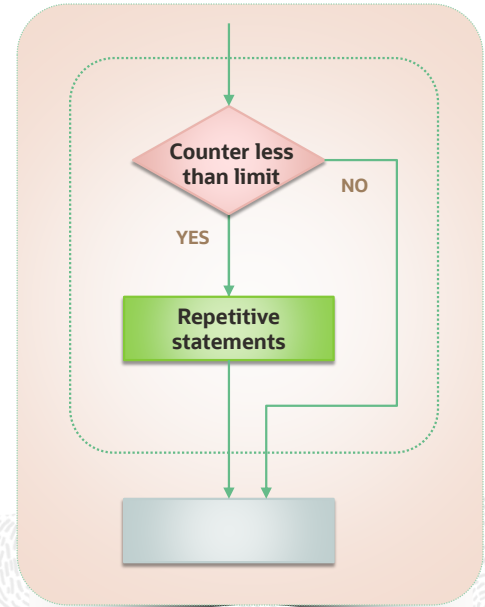
Results

To view the output, run the code example under slide 25_sa in `code_ex_06.sql`.

FOR Loops

- Use a FOR loop when you know the number of iterations.
- Do not declare the counter; it is declared implicitly.

```
FOR counter IN [REVERSE]
    lower_bound..upper_bound LOOP
    statement1;
    statement2;
    . . .
END LOOP;
```



26

FOR loops have the same general structure as the basic loop. You can use a FOR loop when you know the number of iterations that a given set of PL/SQL statements should execute.

In the syntax:

<i>counter</i>	Is an implicitly declared integer whose value automatically increases or decreases (decreases if the <code>REVERSE</code> keyword is used) by 1 on each iteration of the loop until the upper or lower bound is reached
<code>REVERSE</code>	Causes the counter to decrement with each iteration from the upper bound to the lower bound
<i>lower_bound</i>	Note: The lower bound is still referenced first. Specifies the lower bound for the range of counter values
<i>upper_bound</i>	Specifies the upper bound for the range of counter values

Do not declare the counter. It is declared implicitly as an integer.

Note: The sequence of statements is executed each time the counter is incremented, as determined by the two bounds. The lower and upper bounds of the loop range can be literals, variables, or expressions, but they must evaluate to integers. The bounds are rounded to integers; that is, $11/3$ and $8/5$ are valid upper or lower bounds. The lower bound and upper bound are inclusive in the loop range. If the lower bound of the loop range evaluates to a larger integer than the upper bound, the sequence of statements is not executed.

For example, the following statement is executed only once:

```
FOR i IN 3..3
LOOP
    statement1;
END LOOP;
```


FOR Loops: Example

```
DECLARE
  v_countryid  locations.country_id%TYPE := 'CA';
  v_loc_id     locations.location_id%TYPE;
  v_new_city   locations.city%TYPE := 'Montreal';
BEGIN
  SELECT MAX(location_id) INTO v_loc_id
    FROM locations
   WHERE country_id = v_countryid;
  FOR i IN 1..3 LOOP
    INSERT INTO locations(location_id, city, country_id)
      VALUES((v_loc_id + i), v_new_city, v_countryid );
  END LOOP;
END;
/
```

Result in the LOCATIONS table

	LOCATION_ID	STREET_ADDRESS	POSTAL_CODE	CITY	STATE_PROVINCE	COUNTRY_ID
1	1901	(null)	(null)	Montreal	(null)	CA
2	1902	(null)	(null)	Montreal	(null)	CA
3	1903	(null)	(null)	Montreal	(null)	CA

28

You have already learned how to insert three new locations for the CA country code and the city of Montreal by using the basic loop and the WHILE loop. The example in this slide shows how to achieve the same by using the FOR loop.

Results

The slide shows the three new rows that are inserted into the LOCATIONS table based on the logic specified.

FOR Loop Rules

- Reference the counter only within the loop; it is undefined outside the loop.
- Do not reference the counter as the target of an assignment.
- The loop bound should not be `NULL`.

29

The slide lists the guidelines to follow when writing a `FOR` loop.

Note: The lower and upper bounds of a `LOOP` statement do not need to be numeric literals. They can be expressions that convert to numeric values.

Example:

```
DECLARE
  v_lower  NUMBER := 1;
  v_upper  NUMBER := 100;
BEGIN
  FOR i IN v_lower..v_upper LOOP
    ...
  END LOOP;
END;
/
```

Suggested Use of Loops

- Use the basic loop when the statements inside the loop must execute at least once.
- Use the `WHILE` loop if the condition must be evaluated at the start of each iteration.
- Use a `FOR` loop if the number of iterations is known.

30

A basic loop allows the execution of its statement at least once, even if the condition is already met upon entering the loop. Without the `EXIT` statement, the loop would be infinite.

You can use the `WHILE` loop to repeat a sequence of statements until the controlling condition is no longer `TRUE`. The condition is evaluated at the start of each iteration. The loop terminates when the condition is `FALSE`. If the condition is `FALSE` at the start of the loop, no further iterations are performed.

`FOR` loops have a control statement before the `LOOP` keyword to determine the number of iterations that the PL/SQL performs. Use a `FOR` loop if the number of iterations is predetermined.

Nested Loops and Labels

- You can nest loops to multiple levels.
- Use labels to distinguish between blocks and loops.
- Exit the outer loop with the `EXIT` statement that references the label.

31

You can nest the `FOR`, `WHILE`, and basic loops within one another. The termination of a nested loop does not terminate the enclosing loop unless an exception is raised. However, you can label loops and exit the outer loop with the `EXIT` statement.

Label names follow the same rules as the other identifiers. A label is placed before a statement, either on the same line or on a separate line. White space is insignificant in all PL/SQL parsing except inside literals. Label basic loops by placing the label before the word `LOOP` within label delimiters (`<<label>>`). In `FOR` and `WHILE` loops, place the label before `FOR` or `WHILE`.

If the loop is labeled, the label name can be included (optionally) after the `END LOOP` statement for clarity.

Nested Loops and Labels: Example

```
...
BEGIN
  <<Outer_loop>>
  LOOP
    v_counter := v_counter+1;
    EXIT WHEN v_counter>10;
    <<Inner_loop>>
    LOOP
      ...
      EXIT Outer_loop WHEN total_done = 'YES';
      -- Leave both loops
      EXIT WHEN done = 'YES';
      -- Leave inner loop only
      ...
    END LOOP Inner_loop;
    ...
  END LOOP Outer_loop;
END;
/
```

32

In the example in the slide, there are two loops. The outer loop is identified by the label <<Outer_Loop>> and the inner loop is identified by the label <<Inner_Loop>>. The identifiers are placed before the word `LOOP` within label delimiters (<<label>>). The inner loop is nested within the outer loop. The label names are included after the `END LOOP` statements for clarity.

PL/SQL CONTINUE Statement

- Definition
 - Adds functionality to begin the next loop iteration
 - Provides programmers with the ability to transfer control to the next iteration of a loop
- Benefits
 - Eases the programming process
 - May provide a small performance improvement over the previous programming workarounds to simulate the `CONTINUE` statement



33

The `CONTINUE` statement enables you to transfer control within a loop to a new iteration or to leave the loop. Many other programming languages have this functionality. With the Oracle Database 11g release, PL/SQL also offers this functionality. Before the Oracle Database 11g release, you could code a workaround by using Boolean variables and conditional statements to simulate the `CONTINUE` programmatic functionality. In some cases, the workarounds are less efficient.

The `CONTINUE` statement offers you a simplified means to control loop iterations. It may be more efficient than the previous coding workarounds.

The `CONTINUE` statement is commonly used to filter data within a loop body before the main processing begins.

PL/SQL CONTINUE Statement: Example 1

The diagram illustrates the execution of a PL/SQL loop with a `CONTINUE` statement. On the left, a code block contains the following PL/SQL code:

```
DECLARE
  v_total SIMPLE_INTEGER := 0;
BEGIN
  FOR i IN 1..10 LOOP
    1 v_total := v_total + i;
      DBMS_OUTPUT.PUT_LINE
        ('Total is: ' || v_total);
    2 CONTINUE WHEN i > 5;
      v_total := v_total + i;
      DBMS_OUTPUT.PUT_LINE
        ('Out of Loop Total is:
        ' || v_total);
  END LOOP;
END;
/
```

Annotations on the right side of the code block explain the execution flow:

- A dashed box labeled "Executed for each of the 10 iterations of the loop" points to the first assignment (1) `v_total := v_total + i;`.
- A dashed box labeled "Executed only for the first 5 iterations of the loop" points to the second assignment (2) `v_total := v_total + i;`.

On the right, a screenshot of a "Script Output" window shows the results of the execution:

```
Script Output x
Task completed in 0.004 seconds

PL/SQL procedure successfully completed.

Total is : 1
Out of loop total is : 2
Total is : 4
Out of loop total is : 6
Total is : 9
Out of loop total is : 12
Total is : 16
Out of loop total is : 20
Total is : 25
Out of loop total is : 30
Total is : 36
Total is : 43
Total is : 51
Total is : 60
Total is : 70
```

34

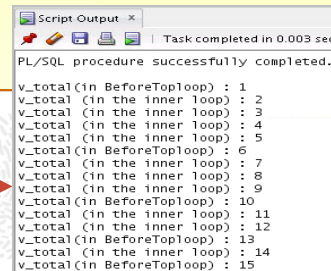
In the example in the slide, there are two assignments that use the `v_total` variable:

1. The first assignment is executed for each of the 10 iterations of the loop.
2. The second assignment is executed only for the first five iterations of the loop. The `CONTINUE` statement transfers control within the loop back to the beginning of the loop for a new iteration, so for the last five iterations of the loop, the second `TOTAL` assignment is not executed.

The end result of the `TOTAL` variable is 70.

PL/SQL CONTINUE Statement: Example 2

```
DECLARE
  v_total NUMBER := 0;
BEGIN
  <<BeforeTopLoop>>
  FOR i IN 1..5 LOOP
    v_total := v_total + 1;
    DBMS_OUTPUT.PUT_LINE
      ('v_total(in BeforeTopLoop): ' || v_total);
    FOR j IN 1..5 LOOP
      CONTINUE BeforeTopLoop WHEN i + j > 5;
      v_total := v_total + 1;
      DBMS_OUTPUT.PUT_LINE('v_total(in the inner loop): ' || v_total);
    END LOOP;
  END LOOP BeforeTopLoop;
END;
```



Script Output - x Task completed in 0.003 sec

PL/SQL procedure successfully completed.

v_total(in BeforeTopLoop) : 1
v_total(in the inner loop) : 2
v_total(in the inner loop) : 3
v_total(in the inner loop) : 4
v_total(in the inner loop) : 5
v_total(in BeforeTopLoop) : 6
v_total(in the inner loop) : 7
v_total(in the inner loop) : 8
v_total(in the inner loop) : 9
v_total(in BeforeTopLoop) : 10
v_total(in the inner loop) : 11
v_total(in the inner loop) : 12
v_total(in BeforeTopLoop) : 13
v_total(in the inner loop) : 14
v_total(in BeforeTopLoop) : 15

35

You can use the `CONTINUE` statement to jump to the next iteration of an outer loop.

To do this, provide the outer loop with a label to identify where the `CONTINUE` statement should go.

The `CONTINUE` statement in the innermost loop terminates that loop whenever the `WHEN` condition is true (just like the `EXIT` keyword). After the innermost loop is terminated by the `CONTINUE` statement, control transfers to the next iteration of the outermost loop, which is labeled `BeforeTopLoop` in this example.

When the pair of loops completes, the value of the `v_total` variable is 15.

You can also use the `CONTINUE` statement within an inner block of code, which does not contain a loop as long as the block is nested inside an appropriate outer loop.

Restrictions

- The `CONTINUE` statement cannot appear outside a loop at all—this generates a compiler error.
- You cannot use the `CONTINUE` statement to pass through a procedure, function, or method boundary—this generates a compiler error.

Quiz



There are three types of loops: basic, `FOR`, and `WHILE`.

- a. True
- b. False



36

Answer: a

Loop Types

PL/SQL provides the following types of loops:

- Basic loops that perform repetitive actions without overall conditions
- `FOR` loops that perform iterative actions based on a count
- `WHILE` loops that perform iterative actions based on a condition

Summary

In this lesson, you should have learned to change the logical flow of statements by using the following control structures:

- Conditional (`IF` statement)
- `CASE` expressions and `CASE` statements
- Loops:
 - Basic loop
 - `FOR` loop
 - `WHILE` loop
- `EXIT` statement
- `CONTINUE` statement



37

A language can be called a programming language only if it provides control structures for the implementation of business logic. These control structures are also used to control the flow of the program. PL/SQL is a programming language that integrates programming constructs with SQL.

A conditional control construct checks for the validity of a condition, and performs an action accordingly. You use the `IF` construct to perform a conditional execution of statements.

An iterative control construct executes a sequence of statements repeatedly, as long as a specified condition holds `TRUE`. You use the various loop constructs to perform iterative operations.

Practice 6: Overview

This practice covers the following topics:

- Performing conditional actions by using `IF` statements
- Performing iterative steps by using `LOOP` structures



0

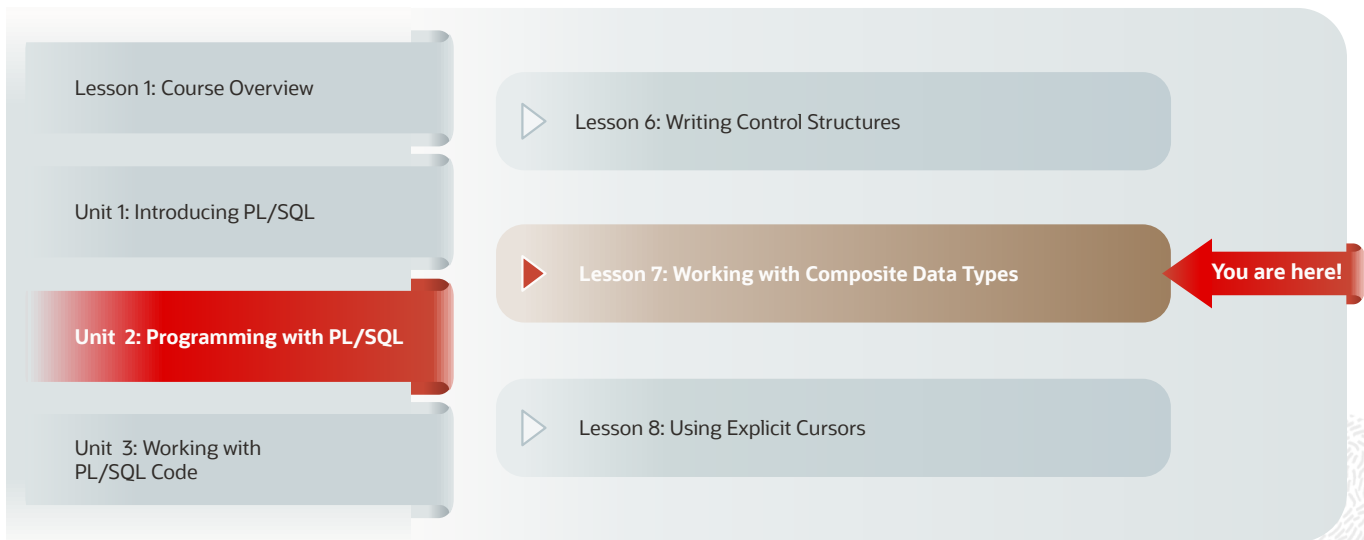
38

In this practice, you create the PL/SQL blocks that incorporate loops and conditional control structures. The exercises test your understanding of writing various `IF` statements and `LOOP` constructs.

For Instructor Use Only.
This document should not be distributed.

Working with Composite Data Types

Course Road Map



2

You are in lesson 7, which is part of Unit 2: Programming with PL/SQL.

0
For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to do the following:

- Describe PL/SQL collections and records
- Create user-defined PL/SQL records
- Create a PL/SQL record with the `%ROWTYPE` attribute
- Create associative arrays
 - `INDEX BY table`
 - `INDEX BY table of records`



3

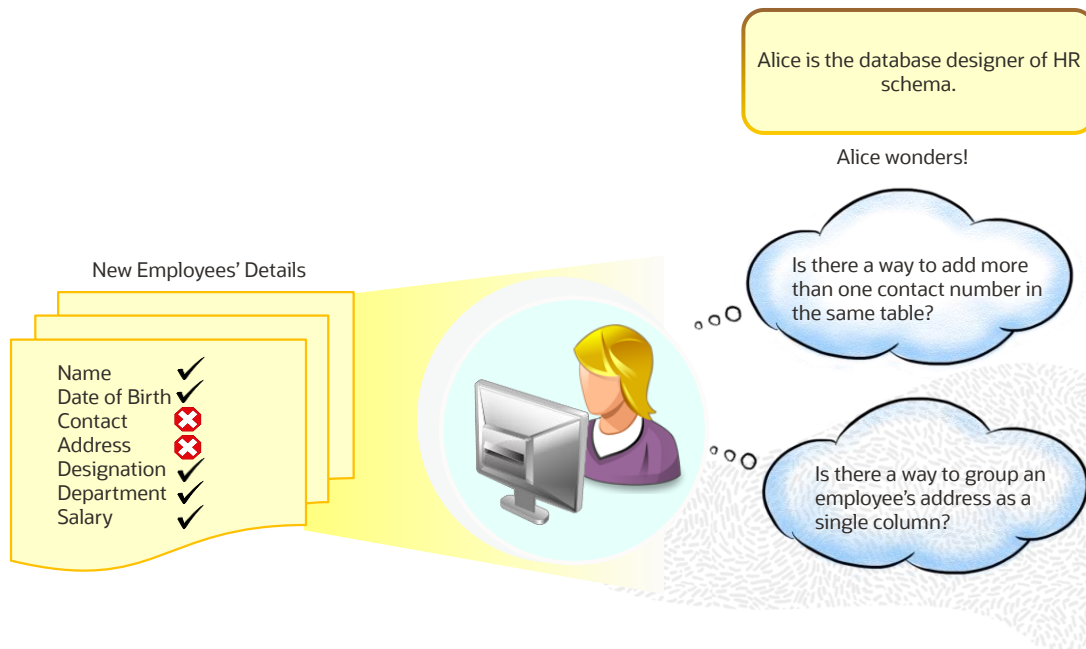
You have already been introduced to composite data types. In this lesson, you learn more about composite data types and their uses.

For Instructor Use Only.
This document should not be distributed.

Agenda

- Understanding composite data types
- Using PL/SQL records
 - Manipulating data with PL/SQL records
 - Advantages of the %ROWTYPE attribute
- Using PL/SQL collections
 - Examining associative arrays
 - Introducing nested tables
 - Introducing VARRAY

Composite Data Types

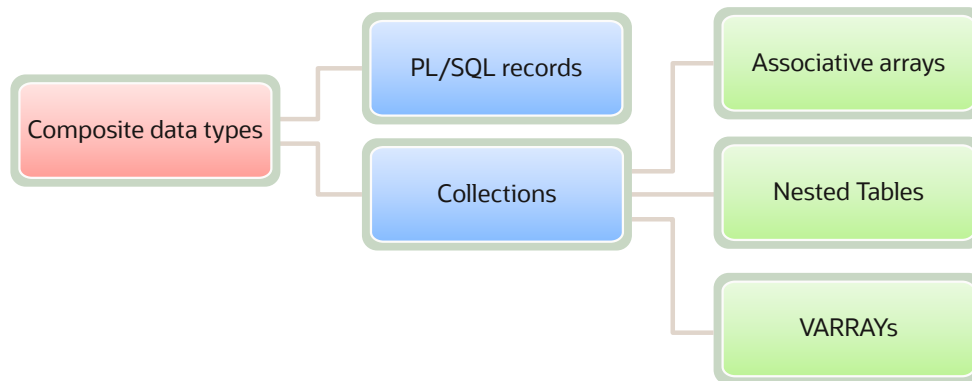


5

Consider a scenario where you must store the address of an employee in a table and you may have to refer to each field of the address occasionally. In such a case, storing the complete address as a single record comprising the street name, door number, and so on would be an effective solution.

There are instances where an employee may have more than one contact number. At times, the employee may have only one. The number of contact numbers is not definite, yet you still want to have all the contact numbers in the database. Storing all the contact numbers as a collection would be an effective solution for this situation.

Composite Data Types



6

Composite data types hold data that may further have logical internal components. The internal components can be manipulated, and can be of the scalar data type or the composite data type. There are two types of composite data types:

- **PL/SQL records:** Records are used to treat related but dissimilar data as a logical unit. A PL/SQL record can have variables of different types. For example, you can define a record to hold employee details. This involves storing an employee number as `NUMBER`, a first name and last name as `VARCHAR2`, and so on. By creating a record to store employee details, you create a logical collective unit.

Each internal component of a record is termed as a field. You can access each component of the record through the following syntax:

```
variable_name.field_name
```

- **PL/SQL collections:** Collections allow you to store multiple rows of data in memory, enabling developers to treat data as a single unit. Collections are of three types:
 - Associative array
 - Nested table
 - VARRAY

Each internal component of a collection is known as an element. You can refer to each element through an index:

```
variable_name(index)
```

PL/SQL Records Versus Collections

PL/SQL Records	Collections
These are used to store related but dissimilar data as a logical unit.	These are used to store data as a single unit.
Use when you want to store values of different data types that are logically related.	Use when you want to store values of the same data type.
Each element can be accessed as: <code>record_name.field_name.</code>	Each element can be accessed by its unique subscript.
Example: A record to hold employee details that are related because they provide information about a particular employee	Example: A collection to hold the emails of all employees. It may store n email IDs; however, email 1 is not related to email 2, and so on.

7

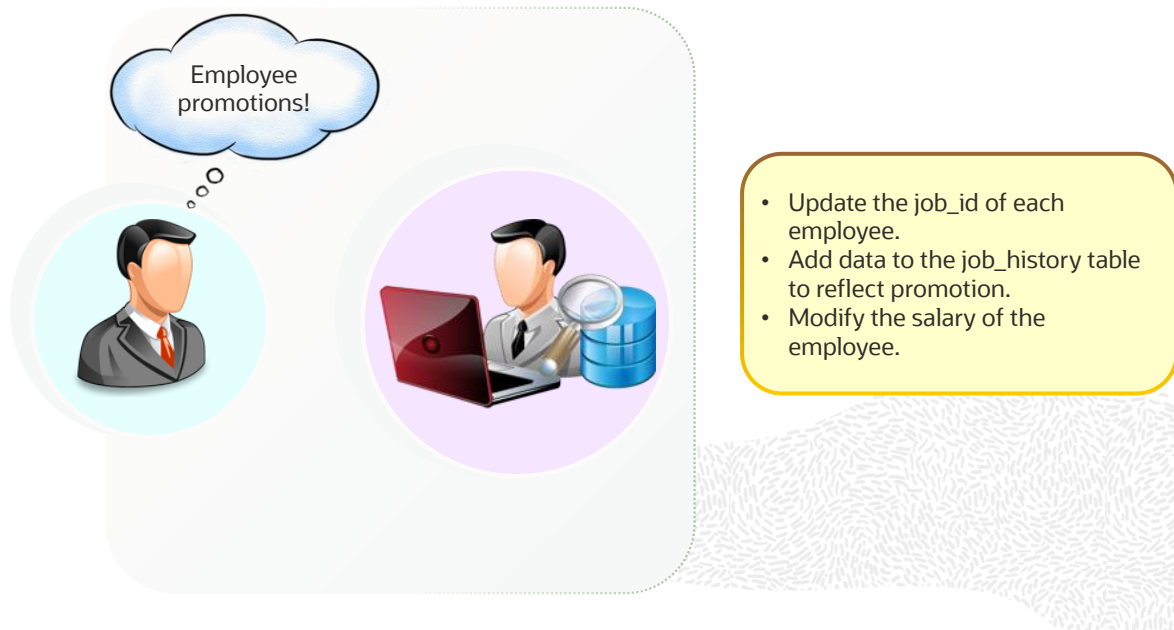
If both PL/SQL records and PL/SQL collections are composite types, how do you choose which one to use?

- Use PL/SQL records when you want to store values of different data types that are logically related. For example, you can create a PL/SQL record to hold employee details and indicate that all the values stored are related because they provide information about a particular employee.
- Use PL/SQL collections when you want to store values of the same data type. Note that this data type can also be of the composite type (such as records). You can define a collection to hold the emails of all employees. You may have stored n emails in the collection; however, email 1 is not related to email 2. The relation between these emails is only that they are employee emails. These collections are similar to arrays in programming languages such as C, C++, and Java.

Agenda

- Understanding composite data types
- Using PL/SQL records
 - Manipulating data with PL/SQL records
 - Advantages of the `%ROWTYPE` attribute
- Using PL/SQL collections
 - Examining associative arrays
 - Introducing nested tables
 - Introducing VARRAY

PL/SQL Records



9

It is appraisal time and the HR manager has the job of updating all the organizational changes in the system. To reflect a promotion in the organization, the following operations are to be carried out in the database:

1. Update the job_id in the employees table according to the promotion.
2. Add the old job details to the job_history table in relation to the employee.
3. Update the salary field in the employees table according to the new job.

When you implement this task in PL/SQL, it would be a better option to retrieve all the details of the employee into the PL/SQL block, update or manipulate them, and then write the details back into the database.

Using a structure that can hold all the data from multiple tables would be an efficient way to execute this task.

A PL/SQL record allows you to hold data that is logically related. In this case, data relevant to an employee such as employee_id, job_id, salary, and so on are of different data types but they are logically connected because they belong to a single employee.

For Instructor Use Only.
This document should not be distributed.

Creating a PL/SQL Record

Syntax:

1

```
TYPE type_name IS RECORD  
    (field_declaration[, field_declaration]...);
```

2

```
identifiertype_name;
```

field_declaration:

```
field_name {field_type | variable%TYPE  
            | table.column%TYPE | table%ROWTYPE}  
[[NOT NULL] {:= | DEFAULT} expr]
```

10

0

PL/SQL records are user-defined composite types. You must create the record type first, and then declare an identifier by using that type. To use them, perform the following steps:

1. Define the record in the declarative section of a PL/SQL block. The syntax for defining the record is shown in the slide.
2. Declare (and optionally initialize) the internal components of this record type.

In the syntax:

<i>type_name</i>	Is the name of the RECORD type (This identifier is used to declare records.)
<i>field_name</i>	Is the name of a field within the record
<i>field_type</i>	Is the data type of the field (It represents any PL/SQL data type except REF CURSOR. You can also use the %TYPE and %ROWTYPE attributes to declare the data types.)
<i>expr</i>	Is the initial value

The NOT NULL constraint prevents assigning of nulls to the specified fields. Be sure to initialize the NOT NULL fields.

The field declarations that are used in defining a record are like variable declarations. Each field has a unique name and a specific data type.

Creating a PL/SQL Record: Example

```
DECLARE
  TYPE t_rec IS RECORD
  (
    v_first_name employees.first_name%type,
    v_sal number(8),
    v_hire_date employees.hire_date%type,
  );
  v_myrec t_rec;
BEGIN
  SELECT first_name,salary, hire_date INTO v_myrec
    FROM employees WHERE employee_id = 100;
  DBMS_OUTPUT.PUT_LINE('First Name:
'||v_myrec.v_first_name ||'Salary:
'||v_myrec.v_sal ||'Hire Date:
'|| v_myrec.v_hire_date);
END;
```

PL/SQL procedure successfully completed.

First Name:Steven
Salary: 24000
Hire Date: 17-JUN-11

11

In the example in the slide, a PL/SQL record is created by using the required two-step process:

1. A record type (`t_rec`) is defined.
2. A record (`v_myrec`) of the `t_rec` type is declared.

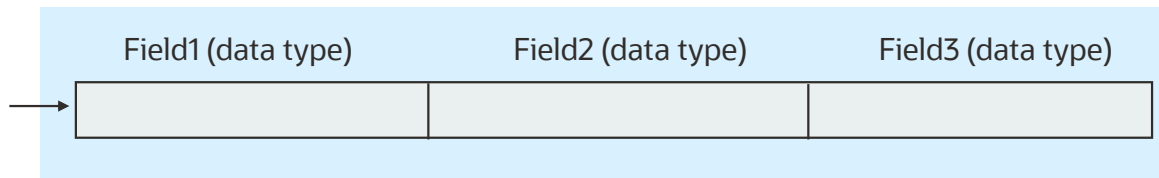
You retrieve values from the employees table through a `SELECT` statement into the record, and then access the fields of the record.

Notes

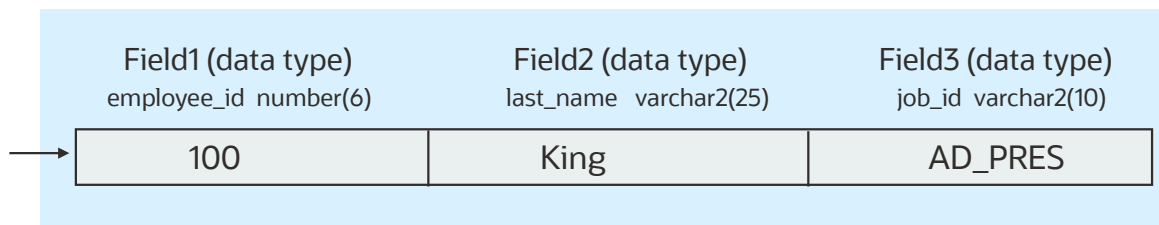
- The record contains three fields: `v_first_name`, `v_sal`, and `v_hire_date`.
- PL/SQL record fields are referenced by using the `<record>.<field>` notation.
- You can add the `NOT NULL` constraint to any field declaration to prevent assigning of nulls to that field. Remember that fields that are declared as `NOT NULL` must be initialized.

PL/SQL Record Structure

Field declarations:



Example:



12

You access the fields in a record with the name of the record. To reference or initialize an individual field, use the dot notation:

```
record_name.field_name
```

For example, you reference the `job_id` field in the `emp_record` record as follows:

```
emp_record.job_id
```

You can then assign a value to the record field:

```
emp_record.job_id := 'ST_CLERK';
```

In a block or subprogram, user-defined records are instantiated when you enter the block or subprogram. They cease to exist when you exit the block or subprogram.

A record is a group of related data items stored in fields, each with its own name and data type.

- Each record defined can have one or more than one fields with different data types.
- You can assign initial values to the fields and also apply a `NOT NULL` constraint.
- Fields without initial values are initialized to `NULL`.
- You can use the `DEFAULT` keyword or `:=` to initialize the fields.
- You can define the `RECORD` types and declare the user-defined records in the declarative part of any block, subprogram, or package.
- You can declare and reference nested records. One record can be the component of another record.

For Instructor Use Only.
This document should not be distributed.

%ROWTYPE Attribute

- Declare a variable according to a collection of columns in a database table or view.
- Fields in the record take their names and data types from the columns of the table or view.

Syntax:

```
DECLARE
  identifier    reference%ROWTYPE;
```

Prefix %ROWTYPE with the database table or view.

Example:

```
DECLARE
  employees     employees%ROWTYPE;
```

14

When you declare a record, you may want to have the record fields to be exactly same as the definition of some table in the database. You can use %ROWTYPE attribute in this scenario.

The %ROWTYPE attribute is to a record what the %TYPE attribute is to a variable type.

The %ROWTYPE attribute is used to declare a record that can hold an entire row of a table or view. The fields in the record take their names and data types from the columns of the table or view. The record can also store an entire row of data fetched from a cursor or cursor variable.

The slide shows the syntax for declaring a record. In the syntax:

- **Identifier:** Is the name chosen for the record as a whole
- **Reference:** Is the name of the table, view, cursor, or cursor variable on which the record is to be based (The table or view must exist for this reference to be valid.)

In the following example, a record is declared by using %ROWTYPE as a data type specifier:

```
DECLARE
  emp_record    employees%ROWTYPE;
  ...
```

The `emp_record` record has a structure consisting of the following fields, each representing a column in the `employees` table.

Note: This is not code, but only the structure of the composite variable.

```
(employee_id      NUMBER(6),
 first_name       VARCHAR2(20),
 last_name        VARCHAR2(20),
 email            VARCHAR2(20),
 phone_number     VARCHAR2(20),
 hire_date        DATE,

 job_id           VARCHAR2(10),
 salary           NUMBER(8,2),
 commission_pct   NUMBER(2,2),
 manager_id       NUMBER(6),
 department_id    NUMBER(4))
```

To reference an individual field, use the dot notation:

```
record_name.field_name
```

For example, you reference the `commission_pct` field in the `emp_record` record as follows:

```
emp_record.commission_pct
```

You can then assign a value to the record field:

```
emp_record.commission_pct:= .35;
```

Assigning Values to Records

You can assign a list of common values to a record by using the `SELECT` or `FETCH` statement. Make sure that the column names appear in the same order as the fields in your record. You can also assign one record to another if both have the same corresponding data types. A record of type `employees%ROWTYPE` and a user-defined record type that has analogous fields of the `employees` table will have the same data type. Therefore, if a user-defined record contains fields similar to the fields of a `%ROWTYPE` record, you can assign that user-defined record to the `%ROWTYPE` record.

Creating a PL/SQL Record: Example

```
DECLARE
  TYPE t_rec IS RECORD
    (v_sal number(8),
     v_minsal number(8) default 1000,
     v_hire_date employees.hire_date%type,
     v_rec1 employees%rowtype);
  v_myrec t_rec;
BEGIN
  v_myrec.v_sal := v_myrec.v_minsal + 500;
  v_myrec.v_hire_date := sysdate;
  SELECT * INTO v_myrec.v_rec1
    FROM employees WHERE employee_id = 100;
  DBMS_OUTPUT.PUT_LINE(v_myrec.v_rec1.last_name || ' ' ||
    v_myrec.v_hire_date || ' ' || v_myrec.v_sal);
END;
```

```
PL/SQL procedure successfully completed.
King 11-JUL-16 1500
```

16

The example in the slide shows the usage of the `ROWTYPE` declaration. The code in the slide has a new field added, `v_rec1`, which is a row from the `employees` table.

The `v_rec1` field in the record declaration is a row from the `employees` table. The `v_myrec` variable is a record variable of type `t_rec`.

Note: A record can have other records as its fields.

In the executable block, we set the field values of the record `v_myrec`:

1. `v_myrec.v_sal` is set to `v_myrec.v_minsal + 500` where the value of `v_minsal` was set to a default of 1,000.
2. The value of `v_hire_date` is set to the current system date.
3. The record variable `v_rec1` is set to the row with `employee_id = 100` in the `employees` table through a `SELECT` statement. In the output, you access the last name of the employee record by identifying with the record variable, `v_myrec`, and then the composite field variable, `v_rec1`, and then the field `last_name`.

Observe the difference in accessing the scalar fields in the record and the values of the composite field in the record.

Advantages of Using the %ROWTYPE Attribute

- The number and data types of the underlying database columns need not be known—and, in fact, might change at run time.
- The %ROWTYPE attribute is useful when you want to retrieve a row with:
 - The `SELECT *` statement
 - The row-level `INSERT` and `UPDATE` statements

17

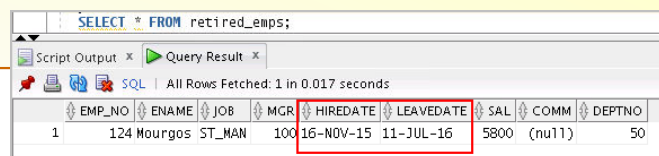
The advantages of using the %ROWTYPE attribute are listed in the slide. Use the %ROWTYPE attribute when you are not sure about the structure of the underlying database table.

Using %ROWTYPE simplifies maintenance, which is its main advantage. Using %ROWTYPE ensures that the data types of the variables declared with this attribute change dynamically when the underlying table is altered. If a DDL statement changes the columns in a table, the PL/SQL program unit is invalidated. When the program is recompiled, it automatically reflects the new table format.

The %ROWTYPE attribute is particularly useful when you want to retrieve an entire row from a table. In the absence of this attribute, you would be forced to declare a variable for each of the columns retrieved by the `SELECT` statement.

Another %ROWTYPE Attribute: Example

```
DECLARE
  v_employee_number number:= 124;
  v_emp_rec employees%ROWTYPE;
BEGIN
  SELECT * INTO v_emp_rec FROM employees
  WHERE employee_id = v_employee_number;
  INSERT INTO retired_emps(empno, ename, job, mgr,
    hiredate, leavedate, sal, comm, deptno)
  VALUES (v_emp_rec.employee_id, v_emp_rec.last_name,
    v_emp_rec.job_id, v_emp_rec.manager_id,
    v_emp_rec.hire_date, SYSDATE,
    v_emp_rec.salary, v_emp_rec.commission_pct,
    v_emp_rec.department_id);
END;
/
```



EMP_NO	ENAME	JOB	MGR	HIREDATE	LEAVEDATE	SAL	COMM	DEPTNO
1	124 Mourgos	ST_MAN	100	16-NOV-15	11-JUL-16	5800	(null)	50

18

Another example of the %ROWTYPE attribute is shown in the slide. If an employee is retiring, information about that employee is added to a table that holds information about retired employees. The user supplies the employee number. The record of the employee specified by the user is retrieved from the employees table and stored in the emp_rec variable, which is declared by using the %ROWTYPE attribute.

The CREATE statement that creates the retired_emps table is:

```
CREATE TABLE retired_emps
(EMPNO          NUMBER(4), ENAME          VARCHAR2(10),
 JOB            VARCHAR2(9), MGR          NUMBER(4),
 HIREDATE       DATE, LEAVEDATE          DATE,
 SAL            NUMBER(7,2), COMM         NUMBER(7,2),
 DEPTNO         NUMBER(2))
```

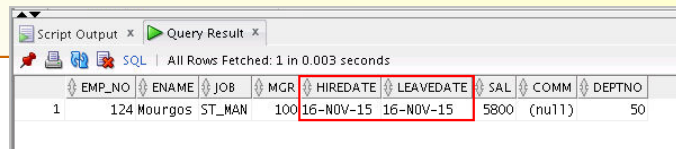
Notes

- The record that is inserted into the retired_emps table is shown in the slide.
- To see the output shown in the slide, execute the SELECT statement shown in the slide in SQL Developer.
- The complete code example is found under slide 14_s-n in code_ex_07.sql.

Inserting a Record by Using %ROWTYPE

```
DECLARE
    v_employee_number number:= 124;
    v_emp_rec    employees%ROWTYPE;
BEGIN
    SELECT * INTO v_emp_rec FROM employees
    WHERE  employee_id = v_employee_number;
    INSERT INTO retired_emps(empno, ename, job, mgr,
        hiredate, leavedate, sal, comm, deptno)
    VALUES (v_emp_rec.employee_id, v_emp_rec.last_name,
        v_emp_rec.job_id, v_emp_rec.manager_id,
        v_emp_rec.hire_date, SYSDATE,
        v_emp_rec.salary, v_emp_rec.commission_pct,
        v_emp_rec.department_id);

END;
/
```



EMP_NO	ENAME	JOB	MGR	HIREDATE	LEAVEDATE	SAL	COMM	DEPTNO
1	124 Mourgos	ST_MAN	100	16-NOV-15	16-NOV-15	5800	(null)	50

19

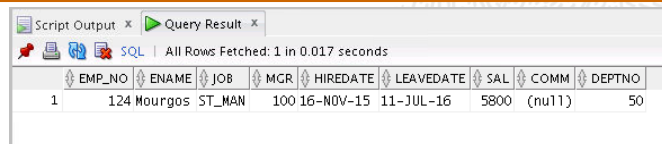
Compare the `INSERT` statement in the previous slide with the `INSERT` statement in this slide. The `emp_rec` record is of type `retired_emps`. The number of fields in the record must be equal to the number of field names in the `INTO` clause. You can use this record to insert values into a table. This makes the code more readable.

Examine the `SELECT` statement in the slide. You select `hire_date` twice and insert the `hire_date` value in the `leavedate` field of `retired_emps`. Practically no employee retires on the hire date. The inserted record is shown in the slide. You will see how to update this in the next slide.

Note: To see the output shown in the slide, execute the `SELECT` statement shown in the slide in SQL Developer.

Updating a Row in a Table by Using a Record

```
DECLARE
  v_employee_number number:= 124;
  v_emp_rec  retired_ems%ROWTYPE;
BEGIN
  SELECT * INTO v_emp_rec FROM retired_ems WHERE
  empno = v_employee_number;
  v_emp_rec.leavedate:= CURRENT_DATE;
  UPDATE retired_ems SET ROW = v_emp_rec WHERE
  empno=v_employee_number;
END;
/
SELECT * FROM retired_ems;
```



The screenshot shows the SQL Developer interface with a 'Query Result' window. It displays a single row from the 'retired_ems' table. The columns are EMP_NO, ENAME, JOB, MGR, HIREDATE, LEAVEDATE, SAL, COMM, and DEPTNO. The row contains the values: 1, 124 Mourgos, ST_MAN, 100, 16-NOV-15, 11-JUL-16, 5800, (null), and 50.

EMP_NO	ENAME	JOB	MGR	HIREDATE	LEAVEDATE	SAL	COMM	DEPTNO
1	124 Mourgos	ST_MAN	100	16-NOV-15	11-JUL-16	5800	(null)	50

20

You learned to insert a row by using a record. This slide shows you how to update a row by using a record.

- The `ROW` keyword is used to represent the entire row.
- The code shown in the slide updates the `leavedate` field of the employee.
- The record is updated as shown in the slide.

Note: To see the output shown in the slide, run the `SELECT` statement shown at the end of the code in the slide, in SQL Developer.

Agenda

- Understanding composite data types
- Using PL/SQL records
 - Manipulating data with PL/SQL records
 - Advantages of the `%ROWTYPE` attribute
- Using PL/SQL collections
 - Examining associative arrays
 - Introducing nested tables
 - Introducing `VARRAY`

21

As stated previously, PL/SQL collections are used when you want to store values of the same data type, such as a single dimensional array.

Therefore, collections are used to treat data as a single unit. Collections are of three types:

- Associative array
- Nested table
- `VARRAY`

Note: Of these three types of collections, the associative array is the focus of this lesson. The nested table and `VARRAY` are introduced only for comparative purposes.

Associative Arrays (INDEX BY Tables)

An associative array is a PL/SQL collection with two columns:

- Primary key of integer or string data type
- Column of scalar or record data type

Key	Value
1	JONES
2	HARDEY
3	MADURO
4	KRAMER

22

An associative array is a type of PL/SQL collection. It is a single dimensional array where each element in the collection is referred to through an index value. You can store data in an associative array by using a primary key value as the index, where the key values are not necessarily sequential.

Associative arrays are sets of key-value pairs. Each key is a unique index, which is used to locate the associated value with the syntax `variable_name(index)`.

Associative arrays are also known as *INDEX BY* tables.

Associative arrays have only two columns, neither of which can be named:

- The first column, of integer or string type, acts as the primary key.
- The second column, of scalar or record data type, holds values.

You use associative arrays in PL/SQL blocks. You cannot store the data from associative arrays onto the database after completing execution of the PL/SQL block.



23

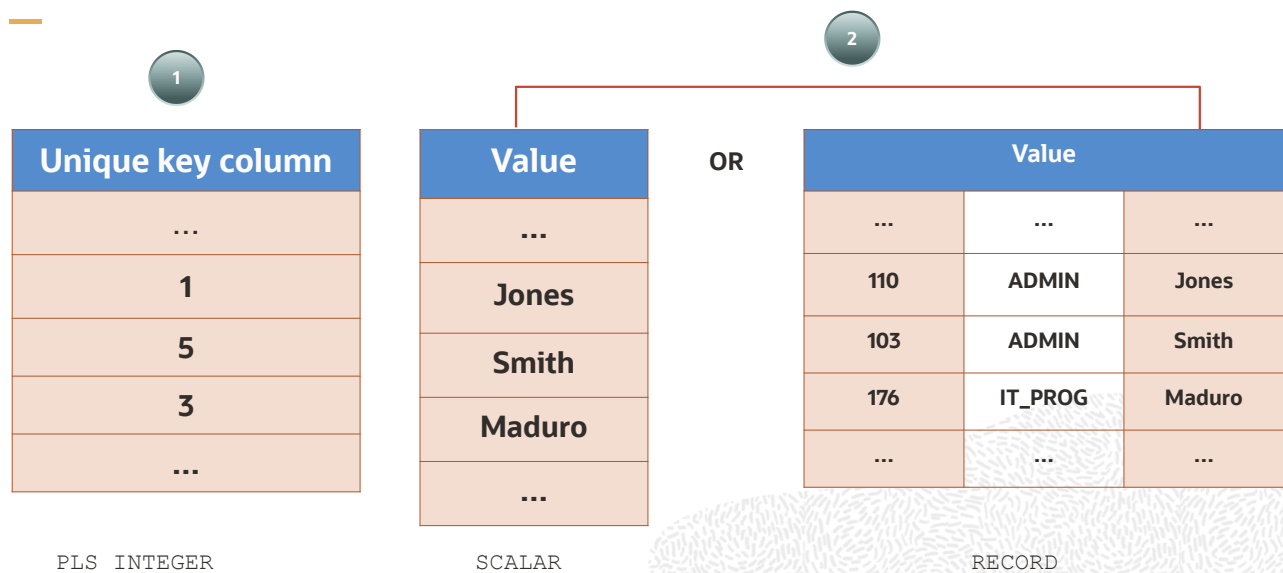
Consider a scenario where the HR Manager must generate a report of expenses per annum for each department to present it in the executive meeting. An expense for a department is the total salary paid to all its employees. While generating the report, the HR manager must extrapolate the expenses by 10% to include the inflation factor for that year. This is just a tentative report; therefore, it need not be retained in the database.

When you execute this task through a PL/SQL block, you may perform the following operations:

1. Execute a select query to find the sum of salaries paid to all employees, department-wise.
2. The `SUM` on each employee's salary would be the department expense. This value can be stored in an associative array that is indexed by the department number.
3. Multiply each entry in the array with 12 to find the expense per annum.
4. Add 10% to each entry in the array.
5. Display the associative array as output.

Using an associative array in this scenario is meaningful because you manipulate the data in the PL/SQL block. Also, the data that is generated in the PL/SQL block need not be retained in the database. You use associative arrays when you have to manipulate or process certain data in the PL/SQL block and need not persist the data in the database.

Associative Array Structure



24

Associative arrays have two columns: a key column and a value column. The first column is a key column, which is used to index the value column. The key column helps you to uniquely access a certain value column. The second column holds a value.

Unique Key Column: The data type of the key column can be:

- Numeric, either `BINARY_INTEGER` or `PLS_INTEGER`. These two numeric data types require less storage than `NUMBER`, and arithmetic operations on these data types are faster than the `NUMBER` arithmetic.
- `VARCHAR2` or one of its subtypes

Value Column: The value column can be either a scalar data type or a record data type. A column with the scalar data type can hold only one value per row, whereas a column with the record data type can hold multiple values per row.

Other Characteristics

- An associative array is not populated at the time of declaration. It contains no keys or values, and you cannot initialize an associative array in its declaration.
- An explicit executable statement is required to populate an associative array.
- Like the size of a database table, the size of an associative array is unconstrained. That is, the number of rows can increase dynamically so that your associative array grows as new rows are added. Note that the keys do not have to be sequential, and can be either positive or negative.

Steps to Create an Associative Array

Syntax:

```
TYPE type_name IS TABLE OF
{ column_type [NOT NULL] | variable%TYPE [NOT NULL]
| table.column%TYPE [NOT NULL]
| table%ROWTYPE }
INDEX BY { PLS_INTEGER | BINARY_INTEGER
| VARCHAR2(<size>) } ;
identifier    type_name;
```

Example:

```
...
TYPE ename_table_type IS TABLE OF
employees.last_name%TYPE
INDEX BY PLS_INTEGER;
...
ename_table ename_table_type;
```

25

Two steps are involved in creating an associative array:

1. Declare a `TABLE` data type by using the `INDEX BY` option.
2. Declare a variable of that data type.

Syntax

<i>type_name</i>	Is the name of the <code>TABLE</code> type (This name is used in the subsequent declaration of the array identifier.)
<i>column_type</i>	Is any scalar or composite data type such as <code>VARCHAR2</code> , <code>DATE</code> , <code>NUMBER</code> , or <code>%TYPE</code> (You can use the <code>%TYPE</code> attribute to provide the column data type.)
<i>identifier</i>	Is the name of the identifier that represents an entire associative array

Note: The `NOT NULL` constraint prevents nulls from being assigned to the associative array.

Example:

In the example in the slide, an associative array with the variable name `ename_table` is declared to store the last names of employees.

Creating and Accessing Associative Arrays

```
...  
DECLARE  
  TYPE email_table IS TABLE OF  
    employees.email%TYPE  
    INDEX BY PLS_INTEGER;  
  email_list      email_table;  
BEGIN  
  email_list(100) = 'SKING';  
  email_list(105) = 'DAUSTIN';  
  email_list(110) = 'JCHEN';  
  DBMS_OUTPUT.PUT_LINE(email_list(100));  
  DBMS_OUTPUT.PUT_LINE(email_list(105));  
  DBMS_OUTPUT.PUT_LINE(email_list(110));  
END;  
/  
...
```

PL/SQL procedure successfully completed.

SKING
DAUSTIN
JCHEN

26

The example in the slide creates an associative array with the identifier, `email_list`. In the executable section, the associative array is initialized with three elements at indexes 100, 105, and 110.

The index of each associative array element is used to access an element in the array, by using the following syntax:

`identifier(index)`

In the `DBMS_OUTPUT.PUT_LINE` statement, you access the elements of the associative array and display them as output.

Note: The magnitude range of a `PLS_INTEGER` is $-2,147,483,647$ through $2,147,483,647$, so the primary key value can be negative. Indexing does not need to start with 1.

Associative Arrays with Record values

Define an associative array to hold an entire row from a table.

```
DECLARE
TYPE dept_table_type
IS
    TABLE OF departments%ROWTYPE INDEX BY VARCHAR2(20);
dept_table dept_table_type;
-- Each element of dept_table is a record
BEGIN
    SELECT * INTO dept_table(1) FROM departments
    WHERE department_id = 10;
    DBMS_OUTPUT.PUT_LINE(dept_table(1).department_id || ' ' ||
    dept_table(1).department_name || ' ' ||
    dept_table(1).manager_id);
END;
/
```

```
PL/SQL procedure successfully completed.
10 Administration 200
```

27

As previously discussed, an associative array that is declared as a table of scalar data type can store the details of only one column in a database table. However, there is often a need to store all the columns retrieved by a query. Associative arrays enable you to hold rows of data from a table in the value column.

Creating and referencing an associative array of rows from a table:

As shown in the associative array example in the slide, you can:

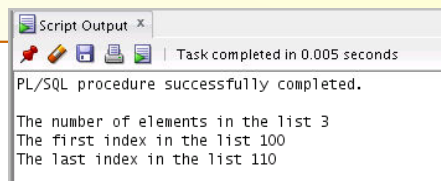
- Use the %ROWTYPE attribute to declare a record that represents a row in a database table
- Refer to fields within the dept_table array because each element of the array is a record

The differences between the %ROWTYPE attribute and the composite data type PL/SQL record are as follows:

- PL/SQL record types can be user-defined, whereas %ROWTYPE implicitly defines the record.
- PL/SQL records enable you to specify the fields and their data types while declaring them. When you use %ROWTYPE, you cannot specify the fields. The %ROWTYPE attribute represents a table row with all the fields based on the definition of that table.
- User-defined records are static, but %ROWTYPE records are dynamic—they are based on a table structure. If the table structure changes, the record structure also picks up the change.

Using Collection Methods

```
DECLARE
  TYPE email_table IS TABLE OF
    employees.email%TYPE
    INDEX BY PLS_INTEGER;
  email_list          email_table;
BEGIN
  email_list(100) = 'SKING';
  email_list(105) = 'DAUSTIN';
  email_list(110) = 'JCHEN';
  DBMS_OUTPUT.PUT_LINE('The number of elements in the list' ||
    email_list.COUNT);
  DBMS_OUTPUT.PUT_LINE('The first index in the list ' || email_list.FIRST);
  DBMS_OUTPUT.PUT_LINE('The last index in the list ' || email_list.LAST);
END;
/
```



Script Output x
Task completed in 0.005 seconds
PL/SQL procedure successfully completed.
The number of elements in the list 3
The first index in the list 100
The last index in the list 110

28

A collection method is a built-in procedure or function that operates on an associative array, and is called by using the dot notation as shown in the slide. Following are some of the commonly used functions:

Method	Description
EXISTS	- This returns TRUE if any elements exist in the collection. EXISTS (n) returns true if an element exists at the given index.
COUNT	This returns the number of elements that a collection currently contains.
FIRST	- This returns the first (smallest) index number in a collection. - This returns NULL if the collection is empty.
LAST	- This returns the last (largest) index number in a collection. - This returns NULL if the collection is empty.
PRIOR (n)	This returns the index number that precedes index <i>n</i> in the collection.
NEXT (n)	This returns the index number that succeeds index <i>n</i>.
DELETE	DELETE removes all the elements from a collection. DELETE (n) removes the index <i>n</i> from a collection. DELETE (m, n) removes all the elements in the range <i>m</i> ... <i>n</i> from a collection.

In the slide we use the **COUNT**, **FIRST**, **LAST**, and **EXISTS** methods for the collection `email_list`.

Note: There are **EXTEND** and **TRIM** procedures, which can be used only with varrays and nested tables.

Using Collection Methods with Associative Arrays

```
DECLARE
  TYPE emp_table_type IS TABLE OF
    employees%ROWTYPE INDEX BY PLS_INTEGER;
  my_emp_table emp_table_type;
  max_count    NUMBER(3) := 104;
BEGIN
  FOR i IN 100..max_count
  LOOP
    SELECT * INTO my_emp_table(i) FROM employees
    WHERE employee_id = i;
  END LOOP;
  FOR i IN my_emp_table.FIRST..my_emp_table.LAST
  LOOP
    DBMS_OUTPUT.PUT_LINE(my_emp_table(i).last_name);
  END LOOP;
END;
/
```

PL/SQL procedure successfully completed.

King
Kochhar
De Haan
Hunold
Ernst

29

The example in the slide declares an associative array, to temporarily store the details of employees whose employee IDs are between 100 and 104. The variable name for the array is `emp_table_type`.

Using a loop, the information of the employees is retrieved from the `EMPLOYEES` table and stored in the array. Another loop is used to print the last names from the array. Note the use of the `first` and `last` methods in the example.

Note: The slide demonstrates one way to work with an associative array. However, you can do the same more efficiently by using cursors. Cursors are explained in the lesson titled “Using Explicit Cursors.”

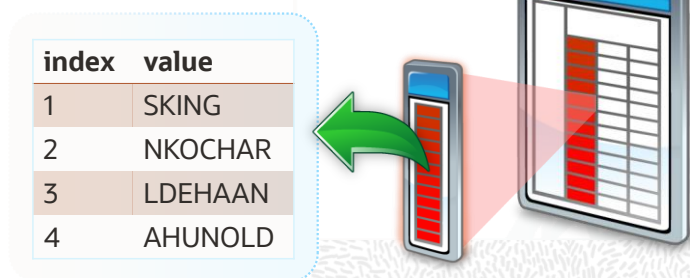
The results of the code example are as follows:

PL/SQL procedure successfully completed.

King
Kochhar
De Haan
Hunold
Ernst

Nested Tables

- Nested tables represent sets of values.
- Nested tables can be used as column types in database tables.
- Each entry in the column of the database table can hold a set of values.



30

Consider a situation where the HR manager has to store all the email IDs of employees working in a department. In this case, using nested tables would be an appropriate choice.

Unlike associative arrays, nested tables can be defined as columns of a table in the database, where each entry in the column can hold a set of values.

Nested tables represent sets of values. You can think of them as one-dimensional arrays with no declared number of elements.

Recall that in the case of associative arrays, the index value can be explicitly provided in the PL/SQL block. When you retrieve a nested table from the database into a PL/SQL variable, the rows are given consecutive subscripts starting at 1. The indexing of elements is done by PL/SQL.

Nested Tables: Syntax and Usage

Syntax:

```
TYPE type_name IS TABLE OF  
{column_type | variable%TYPE  
| table.column%TYPE} [NOT NULL]  
| table.%ROWTYPE
```

PL/SQL procedure successfully completed.

```
SKING  
NKOCHAR  
LDEHAAN  
AHUNOLD
```

Example:

```
DECLARE  
  TYPE dept_mail IS TABLE OF VARCHAR2(20);  
  mails dept_mail := dept_mail('SKING', 'NKOCHAR', 'LDEHAAN', 'AHUNOLD');  
BEGIN  
  FOR i IN mails.FIRST..mails.LAST LOOP  
    DBMS_OUTPUT.PUT_LINE(mails(i));  
  END LOOP;  
END;  
/
```

31

In the slide, you see the syntax for creating nested tables and usage of the nested tables.

In the example, a nested table type, `dept_mail`, is created. The nested table will hold data of type `VARCHAR2`. A variable `mails` of type `dept_mail` is declared in the `DECLARE` section.

The nested table is initialized with values in the `DECLARE` section. In the executable section, you retrieve these values with a `FOR` loop.

The functionality of nested tables is similar to that of associative arrays; however, there are differences in a nested table implementation.

- A nested table is a valid data type in a schema-level table, but an associative array is not. Therefore, unlike associative arrays, nested tables can be stored in the database.
- The size of a nested table can increase dynamically, although the maximum size is 2 GB.
- The “key” cannot be a negative value in a nested table (unlike in an associative array). Although reference is made to the first column as key, there is no key in a nested table. There is a column with numbers.
- Elements can be deleted from anywhere in a nested table, leaving a sparse table with non-sequential “keys.” The rows of a nested table are not in any particular order.
- When you retrieve values from a nested table, the rows are given consecutive subscripts starting from 1.

For Instructor Use Only.
This document should not be distributed.

Variable-Sized Arrays (Varrays)

```
TYPE emp_mail IS VARRAY(5) OF VARCHAR2(20);  
mails emp_mail;
```

Declaration

mails(1)	mails(2)	mails(3)	mails(4)	mails(5)
----------	----------	----------	----------	----------

Subscripts to access each
varray

33

Consider a situation where a company has an upper limit on the number of employees working in each department. In this case, we can use varrays instead of nested tables to hold the email IDs.

A variable-size array (`VARRAY`) is an array with a fixed number of elements. The number of elements that the array would accommodate must be defined during declaration.

- A `VARRAY` is valid in a schema-level table.
- Items of the `VARRAY` type are called varrays.
- `VARRAYS` have a fixed upper bound on the number of elements, and you have to specify the upper bound when you declare them. This is similar to arrays in C language. The maximum size of a `VARRAY` is 2 GB, as in nested tables.
- To access an element of a varray, you use standard subscripting syntax.

The declaration in the slide shows a `VARRAY` type, `emp_mail`, of size 5. The identifier `mails` is a variable of type `emp_mail`. Observe that the varray is uninitialized.

The subscripts to access each element of the varray starts with 1, and increments according to the size of the varray.

VARRAYS: Syntax and Usage

Syntax:

```
TYPE type_name IS VARRAY(SIZE) OF  
{column_type | variable%TYPE  
| table.column%TYPE}[NOT NULL]  
| table.%ROWTYPE
```

PL/SQL procedure successfully completed.

NKOCHAR
LDEHAAN
AHUNOLD

Example:

```
DECLARE  
  TYPE emp_mail IS VARRAY(5) OF VARCHAR2(20);  
  mails emp_mail := emp_mail('NKOCHAR', 'LDEHAAN', 'AHUNOLD');  
BEGIN  
  FOR i IN mails.FIRST..mails.LAST LOOP  
    DBMS_OUTPUT.PUT_LINE(mails(i));  
  END LOOP;  
END;  
/
```

34

0

In the slide, you see the syntax for creating a varray and usage of the varray.

In the example in the slide, a varray type, `emp_mail`, is created. It will hold data of type `VARCHAR2` with a maximum size of 5 defined. A variable `mails` of type `emp_mail` is declared in the `DECLARE` section.

The varray is initialized with values in the `DECLARE` section. In the executable section, you retrieve these values with a `FOR` loop.

You will learn more about nested tables and varrays in the *Advanced PL/SQL* course.

Summarizing Collections

Associative arrays

- Are key-value pairs
- Are used only in PL/SQL blocks; cannot be persisted to the database

Nested tables

- Are sequentially indexed data
- Can be persisted to the database as columns of a table

Varrays

- Are sequentially indexed data with an upper limit
- Can be persisted to the database as columns of a table

35

The slide compares the three types of collections. The following table has more details:

Associative Arrays	Nested Tables	Varrays
These are key-value pairs where each key is a unique index associated with a value.	These store an unspecified number of elements in no particular order.	These store a specified number of elements.
The values in an associative array are indexed according to the key value provided during initialization.	The elements in a nested table are indexed starting from 1.	Varrays are indexed from 1 to the upper limit specified during declaration in the PL/SQL block.
They cannot be persisted to a database, and can be used only in PL/SQL blocks.	They can be a table column and persisted to the database.	They can be a table column and persisted to the database.

Quiz



Identify situations in which you can use the `%ROWTYPE` attribute.

- a. When you are not sure about the structure of the underlying database table
- b. When you want to retrieve an entire row from a table
- c. When you want to declare a variable according to another previously declared variable or database column



0

36

Answer: a, b

Advantages of Using the `%ROWTYPE` Attribute

Use the `%ROWTYPE` attribute when you are not sure about the structure of the underlying database table.

The main advantage of using `%ROWTYPE` is that it simplifies maintenance. Using `%ROWTYPE` ensures that the data types of the variables declared with this attribute change dynamically when the underlying table is altered. If a DDL statement changes the columns in a table, the PL/SQL program unit is invalidated. When the program is recompiled, it automatically reflects the new table format.

The `%ROWTYPE` attribute is particularly useful when you want to retrieve an entire row from a table. In the absence of this attribute, you would be forced to declare a variable for each of the columns retrieved by the `SELECT` statement.

Summary

In this lesson, you should have learned how to:

- Define and reference PL/SQL variables of composite data types
 - PL/SQL record
 - Associative array
 - `INDEX BY table`
 - `INDEX BY table of records`
- Define a PL/SQL record by using the `%ROWTYPE` attribute
- Compare and contrast the three PL/SQL collection types:
 - Associative array
 - Nested table
 - `VARRAY`



37

A PL/SQL record is a collection of individual fields that represent a row in a table. By using records, you can group the data into one structure, and then manipulate this structure as one entity or logical unit. This helps reduce coding, and makes the code easy to maintain and understand.

Like PL/SQL records, a PL/SQL collection is another composite data type. PL/SQL collections include:

- Associative arrays (also known as `INDEX BY` tables). They are objects of the `TABLE` type that look similar to database tables, but come with a slight difference. The so-called `INDEX BY` tables use a primary key to give you array-like access to rows. The size of an associative array is unconstrained.
- Nested tables. The key for nested tables cannot have a negative value, unlike the `INDEX BY` tables. The key must also be in a sequence.
- Variable-size arrays (`VARRAY`). A `VARRAY` is similar to associative arrays, except that a `VARRAY` is constrained in size.

Practice 7: Overview

This practice covers the following topics:

- Declaring associative arrays
- Processing data by using associative arrays
- Declaring a PL/SQL record
- Processing data by using a PL/SQL record



0

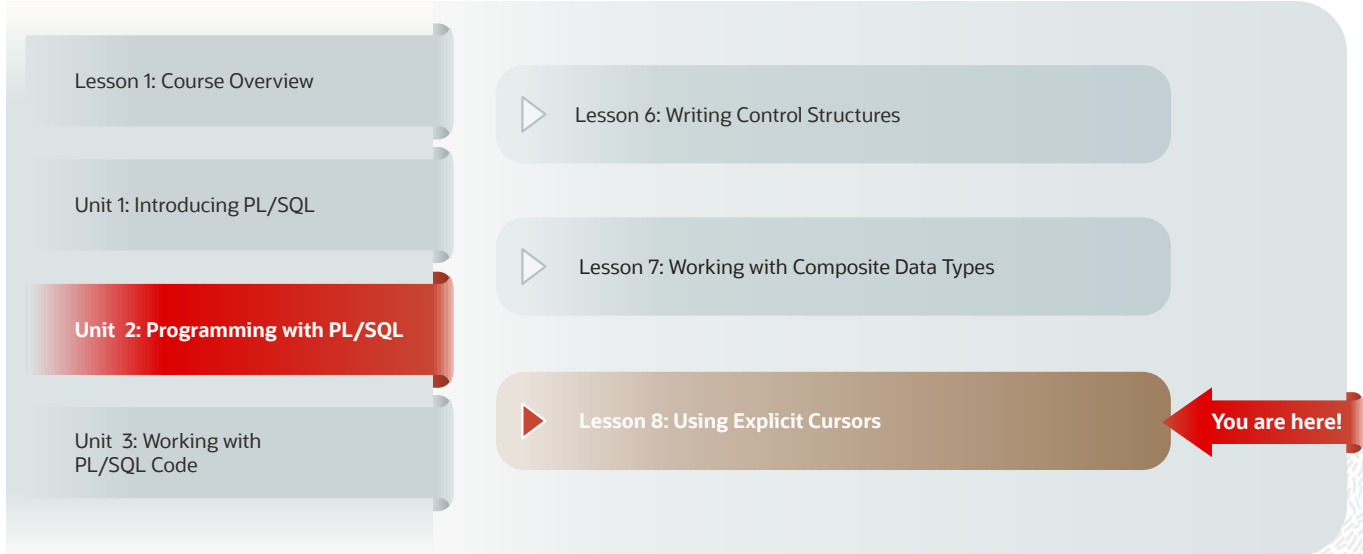
38

In this practice, you define, create, and use associative arrays and PL/SQL records.

For Instructor Use Only.
This document should not be distributed.

Using Explicit Cursors

Course Road Map



2

You are in lesson 8, which is part of Unit 2: Programming with PL/SQL.

0
For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to do the following:

- Distinguish between implicit and explicit cursors
- Discuss the reasons for using explicit cursors
- Declare and control explicit cursors
- Use simple loops and cursor `FOR` loops to fetch data
- Declare and use cursors with parameters
- Lock rows with the `FOR UPDATE` clause
- Reference the current row with the `WHERE CURRENT OF` clause



0

3

You have learned about implicit cursors that are automatically created by PL/SQL when you execute a SQL `SELECT` or DML statement. The `SELECT` statements that are executed with an `INTO` clause are implicit cursors. In this lesson, you learn about explicit cursors. You learn to differentiate between implicit and explicit cursors. You also learn to declare and control simple cursors, as well as cursors with parameters.

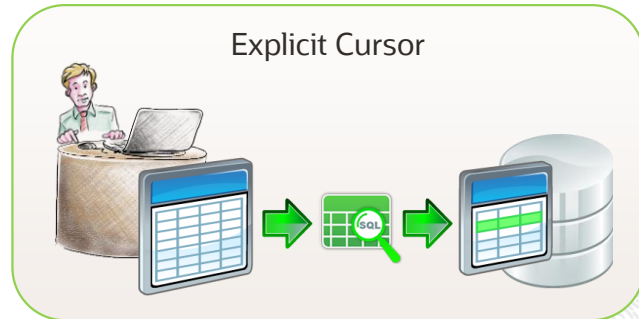
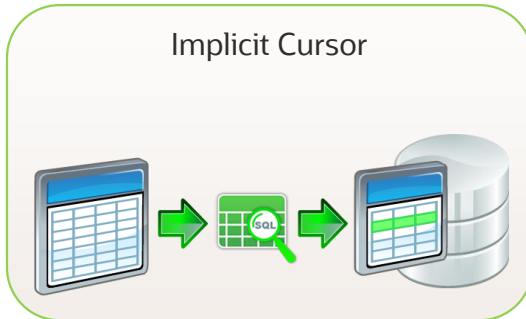
For Instructor Use Only.
This document should not be distributed.

Agenda

- What are explicit cursors?
- Using explicit cursors
- Using cursors with parameters
- Locking rows and referencing the current row

Cursors

- A cursor is a pointer to a private memory area that is used to execute SQL statements in PL/SQL blocks.
- There are two type of cursors:
 - Implicit cursors
 - Explicit cursors



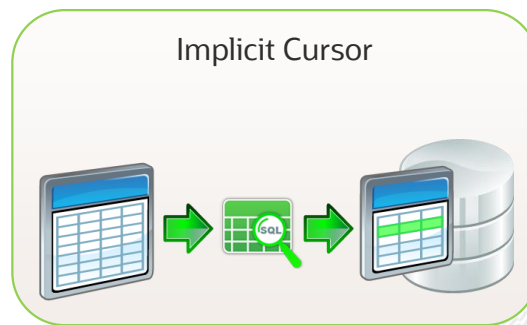
5

The Oracle Server uses work areas (called *private SQL areas*) to execute SQL statements, and to store processing information. Whenever you execute a `SELECT` or DML statement in a PL/SQL block, an implicit cursor is created by the PL/SQL engine. However, you can create and use explicit cursors according to your application requirement.

Cursor Type	Description
Implicit	Implicit cursors are created and managed by PL/SQL. They are also known as session cursors.
Explicit	For queries that return multiple rows, explicit cursors are declared and managed by the programmer.

Implicit Cursors

- Are created and managed by PL/SQL
- Are created for `SELECT` or DML statements
- Closes after the associated statement is run



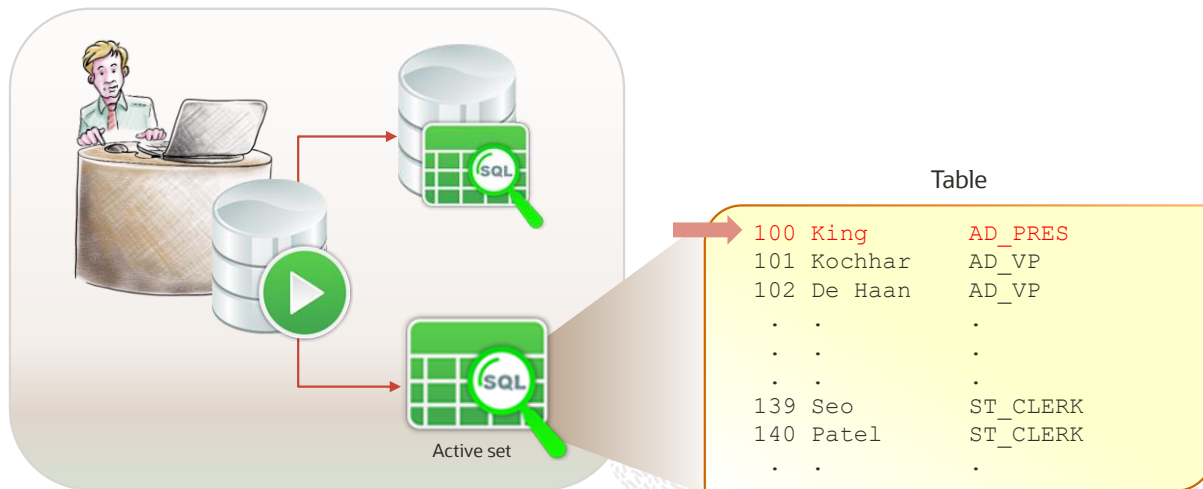
6

Implicit cursors are created by the PL/SQL engine when a `SELECT` statement is executed with the `INTO` clause, or when `INSERT`, `UPDATE`, or `DELETE` statements are executed in the PL/SQL block.

The cursor closes after executing the statement associated with it. You cannot control an implicit cursor but you can get information about the implicit cursor through its attributes.

Explicit Cursor

An explicit cursor is created and managed by a programmer in the PL/SQL block.



7

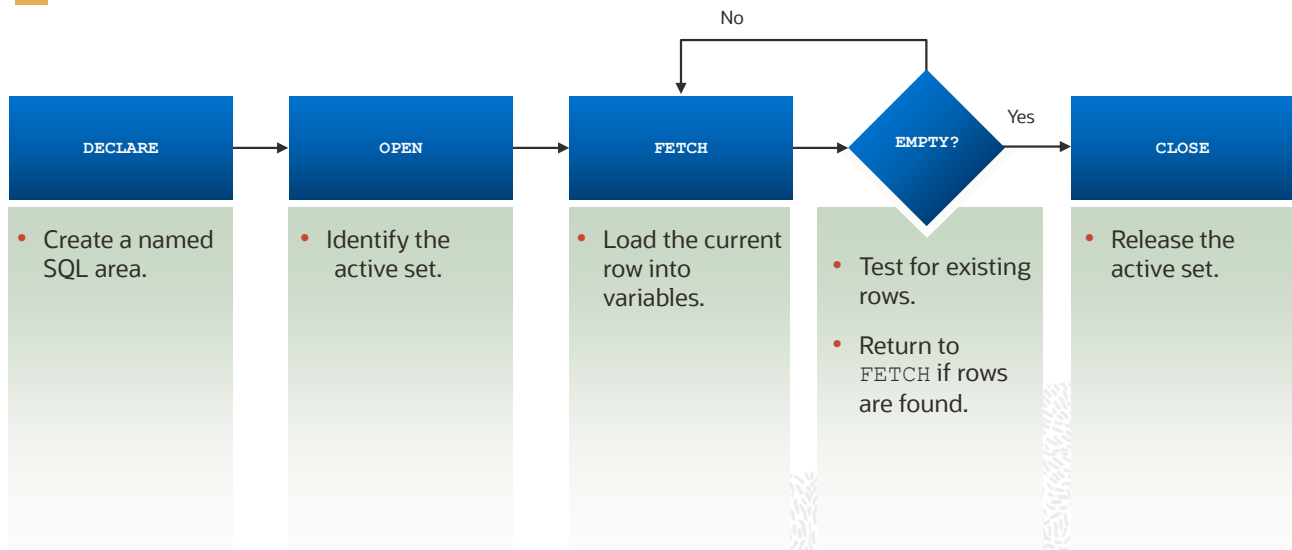
You must declare and define an explicit cursor, giving it a name and associating it with a `SELECT` statement (typically, the query returns multiple rows). You can process each row returned by the `SELECT` statement through a `FOR` loop.

The set of rows returned by a multiple-row query is called the *active set*. Its size is the number of rows that meet your search criteria. The diagram in the slide shows how an explicit cursor “points” to the current row in the active set. This enables your program to process the rows one at a time.

Explicit cursor functions:

- Can perform row-by-row processing beyond the first row returned by a query
- Keep track of the row that is currently being processed
- Enable the programmer to manually control explicit cursors in the PL/SQL block

Controlling Explicit Cursors



8

Now that you have a conceptual understanding of cursors, review the steps to use them.

1. In the declarative section of a PL/SQL block, declare the cursor by naming it and defining the structure of the query to be associated with it.
2. Open the cursor.
The `OPEN` statement executes the query, binds any variables that are referenced, and positions the cursor at the first row. The rows that are identified by the query are called the *active set*, and are now available for fetching.
3. Fetch data from the cursor.
In the flow diagram in the slide, after each fetch, you test the cursor for any existing row. If there are no more rows to process, you must close the cursor.

The `FETCH` statement retrieves the current row, and advances the cursor to the next row until there are no more rows, or a specified condition is met.
4. Close the cursor.
The `CLOSE` statement releases the active set of rows. It is now possible to reopen the cursor to establish a fresh active set.

Agenda

- What are explicit cursors?
- Using explicit cursors
- Using cursors with parameters
- Locking rows and referencing the current row

Declaring the Cursor

Syntax:

```
CURSOR cursor_name IS  
    select_statement;
```

Examples:

```
DECLARE  
    CURSOR c_emp_cursor IS  
        SELECT employee_id, last_name FROM employees  
        WHERE department_id =30;
```

```
DECLARE  
    v_locid NUMBER:= 1700;  
    CURSOR c_dept_cursor IS  
        SELECT * FROM departments  
        WHERE location_id = v_locid;  
    ...
```

10

The syntax to declare a cursor is shown in the slide. In the syntax:

cursor_name
select_statement

Is a PL/SQL identifier

Is a **SELECT** statement without an **INTO** clause

The active set of a cursor is determined by the **SELECT** statement in the cursor declaration. It is mandatory to have an **INTO** clause for a **SELECT** statement in PL/SQL. However, note that the **SELECT** statement in the cursor declaration cannot have an **INTO** clause. This is because you are only defining a cursor in the declarative section, and not retrieving any rows into the cursor.

Notes

- Do not include the **INTO** clause in the cursor declaration because it appears later in the **FETCH** statement.
- If you want the rows to be processed in a specific sequence, use the **ORDER BY** clause in the query.
- The cursor can be any valid **SELECT** statement, including joins, subqueries, and so on.

The `c_emp_cursor` cursor is declared to retrieve the `employee_id` and `last_name` columns for those employees working in the department with `department_id` 30.

The `c_dept_cursor` cursor is declared to retrieve all the details for the department with the `location_id` 1700. Note that a variable is used while declaring the cursor. These variables are considered bind variables, which must be visible when you are declaring the cursor. These variables are examined only once at the time the cursor opens. You have learned that explicit cursors are used when you have to retrieve and operate on multiple rows in PL/SQL. However, this example shows that you can use the explicit cursor even if your `SELECT` statement returns only one row.

Opening the Cursor

```
DECLARE
  CURSOR c_emp_cursor IS
    SELECT employee_id, last_name FROM employees
    WHERE department_id =30;
  ...
BEGIN
  OPEN c_emp_cursor;
```

12

The `OPEN` statement executes the query associated with the cursor, identifies the active set, and positions the cursor pointer at the first row. The `OPEN` statement is included in the executable section of the PL/SQL block.

`OPEN` is an executable statement that performs the following operations:

1. Dynamically allocates memory for a context area
2. Parses the `SELECT` statement
3. Binds the input variables (sets values for the input variables by obtaining their memory addresses)
4. Identifies the active set (the set of rows that satisfy the search criteria). Rows in the active set are not retrieved into variables when the `OPEN` statement is executed. Rather, the `FETCH` statement retrieves the rows from the cursor to the variables.
5. Positions the pointer to the first row in the active set

Note: If a query returns no rows when the cursor is opened, PL/SQL does not raise an exception.

Fetching Data from the Cursor

```
DECLARE
  CURSOR c_emp_cursor IS
    SELECT employee_id, last_name FROM employees
    WHERE department_id =30;
  v_empno employees.employee_id%TYPE;
  v_lname employees.last_name%TYPE;
BEGIN
  OPEN c_emp_cursor;
  FETCH c_emp_cursor INTO v_empno, v_lname;
  DBMS_OUTPUT.PUT_LINE( v_empno || ' ' ||v_lname);
END;
/
```

PL/SQL procedure successfully completed.

114 Raphaely

13

The `FETCH` statement retrieves rows from a cursor, one at a time. After each fetch, the cursor advances to the next row in the active set.

Consider the example in the slide. Two variables, `v_empno` and `v_lname`, are declared to hold the values that are fetched from the cursor. Examine the `FETCH` statement.

You have successfully fetched the values from the cursor to the variables. However, there are six employees in department 30, but only one row was fetched. To fetch all rows, you must use loops. In the next slide, you see how a loop is used to fetch all the rows.

The `FETCH` statement performs the following operations:

1. Reads the data for the current row into the output PL/SQL variables
2. Advances the pointer to the next row in the active set

You should include the same number of variables in the `INTO` clause of the `FETCH` statement as there are columns in the `SELECT` statement. Be sure that the data types are compatible. Match each variable to correspond to the columns positionally. Alternatively, you can also define a record for the cursor and reference the record in the `FETCH INTO` clause. If a fetch acquires no values, there are no rows left to process in the active set and no error is recorded.

Fetching Data from the Cursor

```
DECLARE
  CURSOR c_emp_cursor IS
    SELECT employee_id, last_name FROM employees
    WHERE department_id =30;
  v_empno employees.employee_id%TYPE;
  v_lname employees.last_name%TYPE;
BEGIN
  OPEN c_emp_cursor;
  LOOP
    FETCH c_emp_cursor INTO v_empno, v_lname;
    EXIT WHEN c_emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE( v_empno || ' ' ||v_lname);
  END LOOP;
END;
/
```

14

Observe that a simple `LOOP` is used to fetch all the rows. Also, the cursor attribute, `%NOTFOUND`, is used to test for the exit condition.

The output of the PL/SQL block is:

```
PL/SQL procedure successfully completed.

114  Raphaely
115  Khoo
116  Baida
117  Tobias
118  Himuro
119  Colmenares
```

Apart from `NOTFOUND`, there are other cursor attributes that you can use to check on various aspects of the cursors. We will discuss them in the following slides.

Closing the Cursor

```
...
LOOP
  FETCH c_emp_cursor INTO empno, lname;
  EXIT WHEN c_emp_cursor%NOTFOUND;
  DBMS_OUTPUT.PUT_LINE( v_empno || ' ' || v_lname);
END LOOP;
CLOSE c_emp_cursor;
END;
/
```

15

The `CLOSE` statement disables the cursor, releases the context area, and “undefines” the active set. Close the cursor after completing the processing of the `FETCH` statement. You can reopen the cursor if required. A cursor can be reopened only if it is closed. If you attempt to fetch data from a cursor after it is closed, an `INVALID_CURSOR` exception is raised.

Note: Although it is possible to terminate a PL/SQL block without closing cursors, you should make it a practice to close any cursor that you declare to free resources. The amount of memory used by an application is directly proportional to the number of open cursors, which would also affect the performance of the application.

There is a maximum limit on the number of open cursors per session, which is determined by the `OPEN_CURSORS` parameter in the database parameter file. (`OPEN_CURSORS` = 50 by default.)

Cursors and Records

Process the rows of the active set by fetching values into a PL/SQL record.

```
DECLARE
  CURSOR c_emp_cursor IS
    SELECT employee_id, last_name FROM employees
    WHERE department_id = 30;
  v_emp_record c_emp_cursor%ROWTYPE;
BEGIN
  OPEN c_emp_cursor;
  LOOP
    FETCH c_emp_cursor INTO v_emp_record;
    EXIT WHEN c_emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE( v_emp_record.employee_id
                          || ' ' || v_emp_record.last_name);
  END LOOP;
  CLOSE c_emp_cursor;
END;
```

16

You have already seen that you can define records that have the structure of columns in a table. You can also define a record based on the selected list of columns in an explicit cursor. This is convenient for processing the rows of the active set, because you can simply fetch into the record. Therefore, the values of the rows are loaded directly into the corresponding fields of the record.

PL/SQL procedure successfully completed.

```
114 Raphaely
115 Khoo
116 Baida
117 Tobias
118 Himuro
119 Colmenares
```

Cursor FOR Loops

Syntax:

```
FOR record_name IN cursor_name LOOP
    statement1;
    statement2;
    . . .
END LOOP;
```

- The cursor FOR loop is a shortcut to process explicit cursors.
- Implicit open, fetch, exit, and close occur.
- The record is implicitly declared.

17

You learned to fetch data from cursors by using simple loops. You now learn to use a cursor FOR loop, which processes the rows in an explicit cursor. It is a shortcut because the cursor is opened, a row is fetched once for each iteration in the loop, the loop exits when the last row is processed, and the cursor is closed automatically. The loop itself is terminated automatically at the end of the iteration where the last row is fetched.

In the syntax:

record_name Is the name of the implicitly declared record
cursor_name Is a PL/SQL identifier for the previously declared cursor

Guidelines

- Do not declare the record that controls the loop; it is declared implicitly.
- Test the cursor attributes during the loop if required.
- Supply the parameters for a cursor, if required, in parentheses following the cursor name in the FOR statement.

Cursor FOR Loops

```
DECLARE
  CURSOR c_emp_cursor IS
    SELECT employee_id, last_name FROM employees
    WHERE department_id = 30;
BEGIN
  FOR emp_record IN c_emp_cursor
  LOOP
    DBMS_OUTPUT.PUT_LINE( emp_record.employee_id
      || ' ' || emp_record.last_name);
  END LOOP;
END;
/
```

PL/SQL procedure successfully completed.

```
114 Raphaely
115 Khoo
116 Baida
117 Tobias
118 Himuro
119 Colmenares
```

18

The example that was used to demonstrate the usage of a simple loop to fetch data from cursors is rewritten to use the cursor `FOR` loop.

`emp_record` is the record that is implicitly declared. You can access the fetched data with this implicit record (as shown in the slide). Observe that no variables are declared to hold the fetched data by using the `INTO` clause. The code does not have the `OPEN` and `CLOSE` statements to open and close the cursor, respectively.

Explicit Cursor Attributes

Use explicit cursor attributes to obtain status information about a cursor.

Attribute	Type	Description
%ISOPEN	Boolean	Evaluates to TRUE if the cursor is open
%NOTFOUND	Boolean	Evaluates to TRUE if the most recent fetch does not return a row
%FOUND	Boolean	Evaluates to TRUE if the most recent fetch returns a row; complement of %NOTFOUND
%ROWCOUNT	Number	Evaluates to the number of rows that has been fetched

19

As with implicit cursors, there are four attributes for obtaining status information about a cursor. When appended to the cursor variable name, these attributes return useful information about the execution of a cursor manipulation statement.

Note: You cannot reference cursor attributes directly in a SQL statement.

%ISOPEN Attribute

- You can fetch rows only when the cursor is open.
- Use the %ISOPEN cursor attribute before performing a fetch to test whether the cursor is open.

Example:

```
IF NOT c_emp_cursor%ISOPEN THEN
  OPEN c_emp_cursor;
END IF;
LOOP
  FETCH c_emp_cursor...
```

20

- You can fetch rows only when the cursor is open. Use the %ISOPEN cursor attribute to determine whether the cursor is open.
- Fetch rows in a loop. Use cursor attributes to determine when to exit the loop.
- Use the %ROWCOUNT cursor attribute to do the following:
 - Process an exact number of rows.
 - Fetch the rows in a loop and determine when to exit the loop.

Note: %ISOPEN returns the status of the cursor: TRUE if open and FALSE if not.

%ROWCOUNT and %NOTFOUND: Example

```
DECLARE
  CURSOR c_emp_cursor IS SELECT employee_id,
    last_name FROM employees;
  v_emp_record c_emp_cursor%ROWTYPE;
BEGIN
  OPEN c_emp_cursor;
  LOOP
    FETCH c_emp_cursor INTO v_emp_record;
    EXIT WHEN c_emp_cursor%ROWCOUNT > 10 OR
      c_emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE( v_emp_record.employee_id
      || ' ' || v_emp_record.last_name);
  END LOOP;
  CLOSE c_emp_cursor;
END ; /
```

PL/SQL procedure successfully completed.

174 Abel
166 Ande
130 Atkinson
105 Austin
204 Baer
116 Baida
167 Banda
172 Bates
192 Bell
151 Bernstein

21

The example in the slide shows how the %ROWCOUNT and %NOTFOUND attributes can be used for the exit conditions in a loop.

In the given code, the exit condition for the loop was either that the cursor displayed 10 rows or the cursor cannot fetch any rows according to the SELECT statement.

The %ROWCOUNT attribute keeps track of the number of rows retrieved through the cursor. The %NOTFOUND attribute checks for the end of the active set of rows retrieved by the cursor.

The output of the code displays 10 rows from the employees table.

Cursor FOR Loops Using Subqueries

There is no need to declare the cursor.

```
BEGIN
  FOR emp_record IN (SELECT employee_id,
    last_name
    FROM employees WHERE department_id =30)
  LOOP
    DBMS_OUTPUT.PUT_LINE( emp_record.employee_id
      ||' '||emp_record.last_name);
  END LOOP;
END;
/
```

PL/SQL procedure successfully completed.

```
114 Raphaely
115 Khoo
116 Baida
117 Tobias
118 Himuro
119 Colmenares
```

22

Note that there is no declarative section in this PL/SQL block. The difference between the cursor `FOR` loops by using subqueries and the cursor `FOR` loop lies in the cursor declaration. If you are writing cursor `FOR` loops by using subqueries, you need not declare the cursor in the declarative section. You have to provide the `SELECT` statement that determines the active set in the loop itself.

The example that was used to illustrate a cursor `FOR` loop is rewritten in the slide to illustrate a cursor `FOR` loop by using subqueries.

Note: You cannot reference explicit cursor attributes if you use a subquery in a cursor `FOR` loop because you cannot give the cursor an explicit name.

Agenda

- What are explicit cursors?
- Using explicit cursors
- Using cursors with parameters
- Locking rows and referencing the current row

Cursors with Parameters

- Pass parameter values to a cursor when the cursor is opened and the query is executed.
- Open an explicit cursor several times with a different active set each time.

Syntax:

```
CURSOR cursor_name
  [(parameter_name datatype, ...)]
IS
  select_statement;
```

```
OPEN  cursor_name (parameter_value, ..... ) ;
```

24

You can pass parameters to a cursor. This means that you can open and close an explicit cursor several times in a block, returning a different active set on each occasion. For each execution, the previous cursor is closed and reopened with a new set of parameters.

Each formal parameter in the cursor declaration must have a corresponding actual parameter in the `OPEN` statement. The parameter data types are the same as those for scalar variables, but you do not give them sizes. The parameter names are for reference in the query expression of the cursor.

In the syntax:

<i>cursor_name</i>	Is a PL/SQL identifier for the declared cursor
<i>parameter_name</i>	Is the name of a parameter
<i>datatype</i>	Is the scalar data type of the parameter
<i>select_statement</i>	Is a <code>SELECT</code> statement without the <code>INTO</code> clause

The parameter notation does not offer greater functionality; it simply allows you to specify input values easily and clearly. This is particularly useful when the same cursor is referenced repeatedly.

Cursors with Parameters

```
DECLARE
  CURSOR c_emp_cursor (deptno NUMBER)
  IS
    SELECT employee_id, last_name FROM employees WHERE department_id = deptno;
  v_empno employees.employee_id%TYPE;
  v_lname employees.last_name%TYPE;
BEGIN
  OPEN c_emp_cursor (30);
  LOOP
    FETCH c_emp_cursor INTO v_empno, v_lname;
    EXIT WHEN c_emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE( v_empno || ' ' ||v_lname);
  END LOOP;
  CLOSE c_emp_cursor;
  DBMS_OUTPUT.PUT_LINE('Opening the cursor the second time');
  OPEN c_emp_cursor (10);
  LOOP
    FETCH c_emp_cursor INTO v_empno, v_lname;
    EXIT WHEN c_emp_cursor%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE( v_empno || ' ' ||v_lname);
  END LOOP;
  CLOSE c_emp_cursor;
END;
```

PL/SQL procedure successfully completed.

```
114 Raphaely
115 Khoo
116 Baida
117 Tobias
118 Himuro
119 Colmenares
Opening the cursor second time
200 Whalen
```

25

The slide shows the usage of cursors with parameters. Each time you open a cursor, you can pass a parameter to the cursor to create a new active set of records.

In the example in the slide, you pass `department_id` as a parameter.

The first time, you open the cursor with `department_id` 30. The employee IDs and last names of all employees working in the department with department number 30 are retrieved.

You close the cursor. The second time, you open it with parameter 10. Now the employee ID and last name of the employee working in department number 10 is retrieved.

Agenda

- What are explicit cursors?
- Using explicit cursors
- Using cursors with parameters
- Locking rows and referencing the current row

FOR UPDATE Clause

Syntax:

```
SELECT    ...  
FROM      ...  
FOR UPDATE [OF column_reference] [NOWAIT | WAIT n];
```

- Use explicit locking to deny access to other sessions for the duration of a transaction.
- Lock the rows *before* an update or a delete.

27

If there are multiple sessions for a single database, there is the possibility that the rows of a particular table were updated after you opened your cursor. You see the updated data only when you reopen the cursor. Therefore, it is essential to have locks on the rows before you update or delete rows. In the absence of locks, the data would be updated inconsistently. You can lock rows in databases with the `FOR UPDATE` clause in the cursor query. Locks are used in databases to maintain data integrity when multiple users are trying to access data.

In the syntax:

<i>column_reference</i>	Is a column in the table against which the query is performed (A list of columns may also be used.)
NOWAIT	Returns an Oracle Server error if the rows are locked by another session
WAIT <i>n</i>	Waits for <i>n</i> seconds of time for the rows to be unlocked if they are locked by other sessions

The `FOR UPDATE` clause is the last clause in a `SELECT` statement, even after `ORDER BY` (if it exists). When you want to query multiple tables, you can use the `FOR UPDATE` clause to confine row locking to particular tables. `FOR UPDATE OF col_name(s)` locks rows only in tables that contain `col_name(s)`.

The `SELECT . . . FOR UPDATE` statement identifies the rows that are to be updated or deleted, and then locks each row in the result set. This is useful when you want to base an update on the existing values in a row. In that case, you must make sure that the row is not changed by another session before the update.

The optional `NOWAIT` keyword tells the Oracle Server not to wait if the requested rows have been locked by another user. Control is immediately returned to your program so that it can execute other steps before trying again to acquire the lock. If you omit the `NOWAIT` keyword, the Oracle Server waits until the rows are available.

Example:

```
DECLARE
  CURSOR c_emp_cursor IS
    SELECT employee_id, last_name, FROM employees
    WHERE department_id = 80 FOR UPDATE OF salary NOWAIT;
  . . .
```

If the Oracle Server cannot acquire the locks on the rows it needs in a `SELECT FOR UPDATE` operation, it waits indefinitely. Use `NOWAIT` to handle such situations. If the rows are locked by another session and you have specified `NOWAIT`, opening the cursor results in an error. You can try to open the cursor later. You can use `WAIT` instead of `NOWAIT`, specify the number of seconds to wait, and then check whether the rows are unlocked. If the rows are still locked after *n* seconds, an error is returned.

It is not mandatory for the `FOR UPDATE OF` clause to refer to a column, but you must specify which tables to lock if the statement is a join of multiple tables.

WHERE CURRENT OF Clause

- Use cursors to update or delete the current row.
- Include the `FOR UPDATE` clause in the cursor query to first lock the rows.
- Use the `WHERE CURRENT OF` clause to reference the current row from an explicit cursor.

Syntax:

```
WHERE CURRENT OF cursor ;
```

```
UPDATE employees  
  SET    salary = ...  
  WHERE CURRENT OF c_emp_cursor;
```

29

The `WHERE CURRENT OF` clause is used in conjunction with the `FOR UPDATE` clause to refer to the current row in an explicit cursor. The `WHERE CURRENT OF` clause is used to perform an operation such as `UPDATE` or `DELETE` on the current row pointed by the cursor. The `FOR UPDATE` clause is specified in the cursor declaration.

You must include the `FOR UPDATE` clause in the cursor query so that the rows are locked on `OPEN`.

In the syntax:

cursor

Is the name of a declared cursor (The cursor must have been declared with the `FOR UPDATE` clause.)

WHERE CURRENT OF Clause: Example

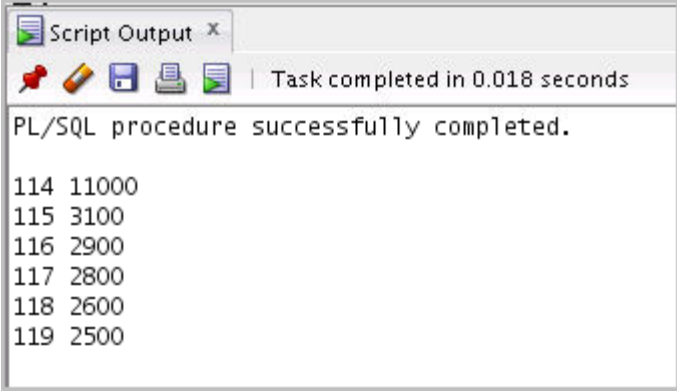
```
DECLARE
  CURSOR c_emp_cursor IS
    SELECT employee_id, salary FROM employees
    WHERE department_id =30 FOR UPDATE;
BEGIN
  FOR emp_record IN c_emp_cursor
  LOOP
    DBMS_OUTPUT.PUT_LINE( emp_record.employee_id
    || ' ' || emp_record.salary);
    UPDATE employees
    SET salary = 5000
    WHERE CURRENT OF c_emp_cursor;
  END LOOP;
END;
/
SELECT employee_id, salary FROM employees
WHERE department_id =30;
```

30

The code in the slide shows the usage of the `FOR UPDATE` clause and the `WHERE CURRENT OF` clause in a PL/SQL block.

The cursor retrieves the `employee_id` and salary of employees who belong to the department with `department_id 30`.

The output of the execution of the code, which shows the value before the `UPDATE` on the `employees` table, is as follows:



Script Output x

Task completed in 0.018 seconds

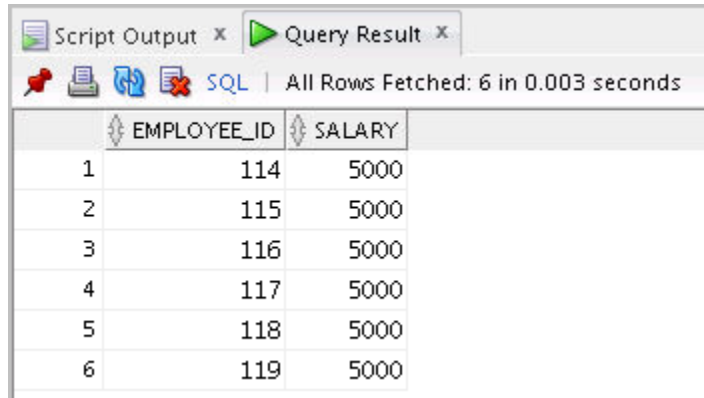
PL/SQL procedure successfully completed.

114	11000
115	3100
116	2900
117	2800
118	2600
119	2500

After executing the PL/SQL block, you can see the effect of the `UPDATE` by executing the following `SELECT` statement:

```
SELECT employee_id, salary FROM employees
WHERE department_id =30;
```

The following is the output of the query, where all the salary values are set to 5000.



The screenshot shows a SQL Developer window with a 'Query Result' tab. The status bar indicates 'All Rows Fetched: 6 in 0.003 seconds'. The query results are displayed in a table with two columns: 'EMPLOYEE_ID' and 'SALARY'. There are 6 rows of data, all with a salary of 5000.

	EMPLOYEE_ID	SALARY
1	114	5000
2	115	5000
3	116	5000
4	117	5000
5	118	5000
6	119	5000

Quiz



Explicit cursor functions enable a programmer to manually control explicit cursors in the PL/SQL block.

- a. True
- b. False



0

32

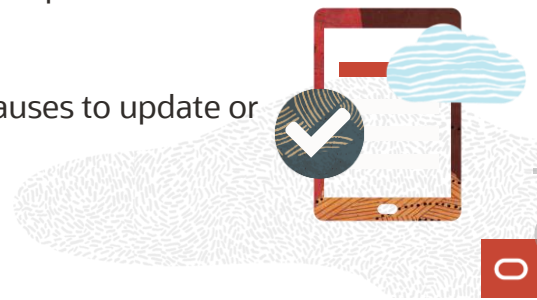
Answer: a

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Distinguish cursor types:
 - Implicit cursors are used for all DML statements and single-row queries.
 - Explicit cursors are used for queries of zero, one, or more rows.
- Create and handle explicit cursors
- Use simple loops and cursor `FOR` loops to handle multiple rows in the cursors
- Evaluate cursor status by using cursor attributes
- Use the `FOR UPDATE` and `WHERE CURRENT OF` clauses to update or delete the current fetched row



33

The Oracle Server uses work areas to execute SQL statements and to store processing information. You can use a PL/SQL construct called a *cursor* to name a work area and to access its stored information. There are two kinds of cursors: implicit and explicit. PL/SQL implicitly declares a cursor for all SQL data manipulation statements, including queries that return only one row. For queries that return multiple rows, you must explicitly declare a cursor to process the rows individually. These are called explicit cursors.

Every explicit cursor and cursor variable has four attributes: `%FOUND`, `%ISOPEN`, `%NOTFOUND`, and `%ROWCOUNT`. When appended to the cursor variable name, these attributes return useful information about the execution of a SQL statement. You can use cursor attributes in procedural statements but not in SQL statements.

Use simple loops or cursor `FOR` loops to operate on each of the multiple rows fetched by the cursor. If you are using simple loops, you have to open, fetch, and close the cursor; however, cursor `FOR` loops do this implicitly. If you are updating or deleting rows, lock the rows by using a `FOR UPDATE` clause. This ensures that the data that you are using is not updated by another session after you open the cursor. Use a `WHERE CURRENT OF` clause in conjunction with the `FOR UPDATE` clause to reference the current row fetched by the cursor.

Practice 8: Overview

This practice covers the following topics:

- Declaring and using explicit cursors to query the rows of a table
- Using a cursor `FOR` loop
- Applying cursor attributes to test the cursor status
- Declaring and using cursors with parameters
- Using the `FOR UPDATE` and `WHERE CURRENT OF` clauses



34

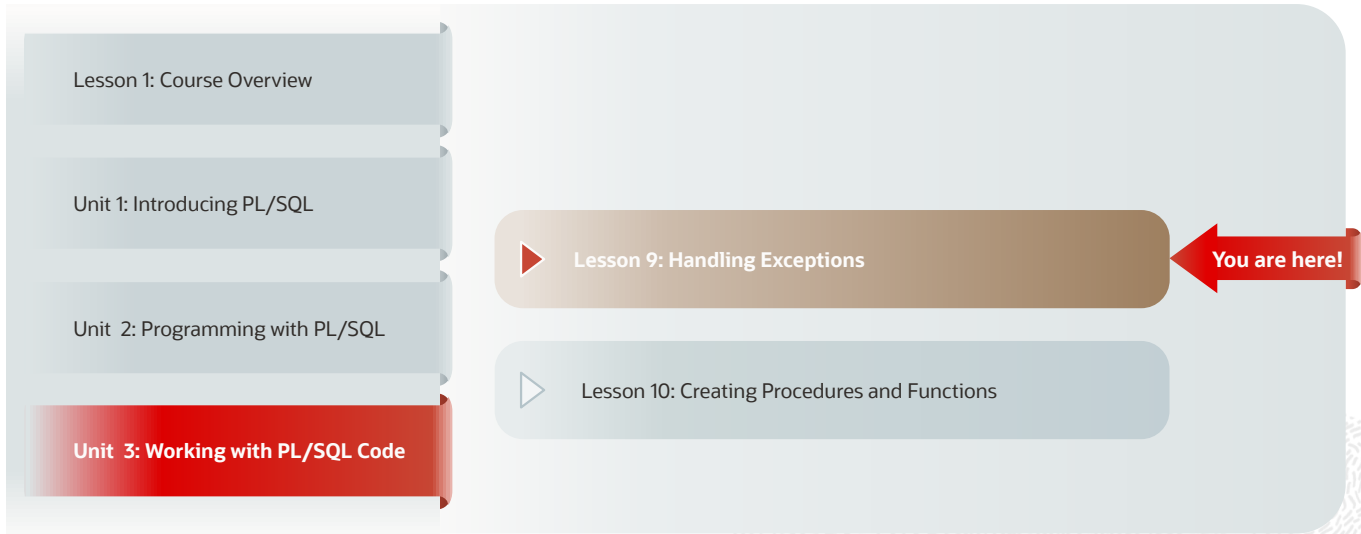
In this practice, you apply your knowledge of cursors to process a number of rows from a table and to populate another table with the results by using a cursor `FOR` loop. You also write a cursor with parameters.

Handling Exceptions

For Instructor Use Only.

This document should not be distributed.

Course Road Map



2

In Unit 3, you are introduced to exception handling in PL/SQL blocks and programming PL/SQL program units.

0
For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to do the following:

- Define PL/SQL exceptions
- Recognize unhandled exceptions
- List and use different types of PL/SQL exception handlers
- Trap unanticipated errors
- Describe the effect of exception propagation in nested blocks
- Customize PL/SQL exception messages



0

3

You learned to write PL/SQL blocks with a declarative section and an executable section. All the SQL and PL/SQL code that must be executed is written in the executable block.

So far, it has been assumed that the code works satisfactorily if you take care of compile-time errors. However, the code may cause some unanticipated errors at run time. In this lesson, you learn how to deal with such errors in a PL/SQL block.

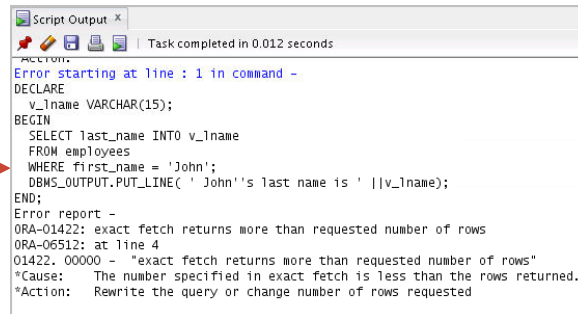
For Instructor Use Only.
This document should not be distributed.

Agenda

- Understanding PL/SQL exceptions
- Trapping exceptions

What Is an Exception?

```
DECLARE
  v_lname VARCHAR2(15);
BEGIN
  SELECT last_name INTO v_lname
  FROM employees
  WHERE first_name='John';
  DBMS_OUTPUT.PUT_LINE ('John''s last name is : ' ||v_lname);
END;
```

A screenshot of a 'Script Output' window showing an error. The window title is 'Script Output x' and it says 'Task completed in 0.012 seconds'. The error message is: 'Error starting at line : 1 in command - DECLARE v_lname VARCHAR(15); BEGIN SELECT last_name INTO v_lname FROM employees WHERE first_name = 'John'; DBMS_OUTPUT.PUT_LINE(' John''s last name is ' ||v_lname); END; Error report - ORA-01422: exact fetch returns more than requested number of rows ORA-06512: at line 4 01422. 00000 - "exact fetch returns more than requested number of rows" *Cause: The number specified in exact fetch is less than the rows returned. *Action: Rewrite the query or change number of rows requested'. A red arrow points from the 'END;' line of the code block above to the 'END;' line of the error report.

Script Output x | Task completed in 0.012 seconds

Error starting at line : 1 in command -
DECLARE
 v_lname VARCHAR(15);
BEGIN
 SELECT last_name INTO v_lname
 FROM employees
 WHERE first_name = 'John';
 DBMS_OUTPUT.PUT_LINE(' John''s last name is ' ||v_lname);
END;
Error report -
ORA-01422: exact fetch returns more than requested number of rows
ORA-06512: at line 4
01422. 00000 - "exact fetch returns more than requested number of rows"
*Cause: The number specified in exact fetch is less than the rows returned.
*Action: Rewrite the query or change number of rows requested

5

Consider the example in the slide. There are no syntax errors in the code, which implies that you must be successfully executing the PL/SQL block. The `SELECT` statement in the block retrieves the last name of the employee whose first name is “John.”

However, you see the following error report when you execute the code:

```
Error report -
ORA-01422: exact fetch returns more than requested number of rows
ORA-06512: at line 4
01422. 00000 - "exact fetch returns more than requested number of rows"
*Cause: The number specified in exact fetch is less than the rows returned.
*Action: Rewrite the query or change number of rows requested
```

The code does not work as expected. You expected the `SELECT` statement to retrieve only one row; however, it retrieves multiple rows. Such errors that occur at run time are called *exceptions*. When an exception occurs, the PL/SQL block is terminated. You can handle such exceptions in your PL/SQL block, for better application performance.

Handling an Exception: Example

```
DECLARE
    v_lname VARCHAR2(15);
BEGIN
    SELECT last_name INTO v_lname
    FROM employees
    WHERE first_name='John';
    DBMS_OUTPUT.PUT_LINE ('John''s last name is : ' || v_lname);
EXCEPTION
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE (' Your select statement retrieved
        multiple rows. Consider using a cursor. ');
END;
/
```

PL/SQL procedure successfully completed.
Your select statement retrieved multiple rows.
Consider using a cursor

6

You learned how to write PL/SQL blocks with a declarative section (beginning with the `DECLARE` keyword) and an executable section (beginning and ending with the `BEGIN` and `END` keywords, respectively).

For exception handling, you include another optional section called the *exception section*.

- This section begins with the `EXCEPTION` keyword.
- If present, this must be the last section in a PL/SQL block.

Example

In the example in the slide, the code from the previous slide is rewritten to handle the exception that occurred. The output of the code is shown in the slide as well.

By adding the `EXCEPTION` section of the code, the PL/SQL program does not terminate abruptly. When the exception occurs, control shifts to the exception section and all the statements in the exception section are executed. The PL/SQL block terminates with a normal, successful completion.

Understanding Exceptions with PL/SQL

- An exception is a PL/SQL error that is raised during program execution.
- An exception can be raised:
 - Implicitly by the Oracle Server
 - Explicitly by the program
- An exception can be handled:
 - By trapping it with a handler
 - By propagating it to the calling environment



0

An exception is an error in PL/SQL that is raised during the execution of a block. A block always terminates when PL/SQL raises an exception, but you can define an exception handler to perform final actions before the block ends.

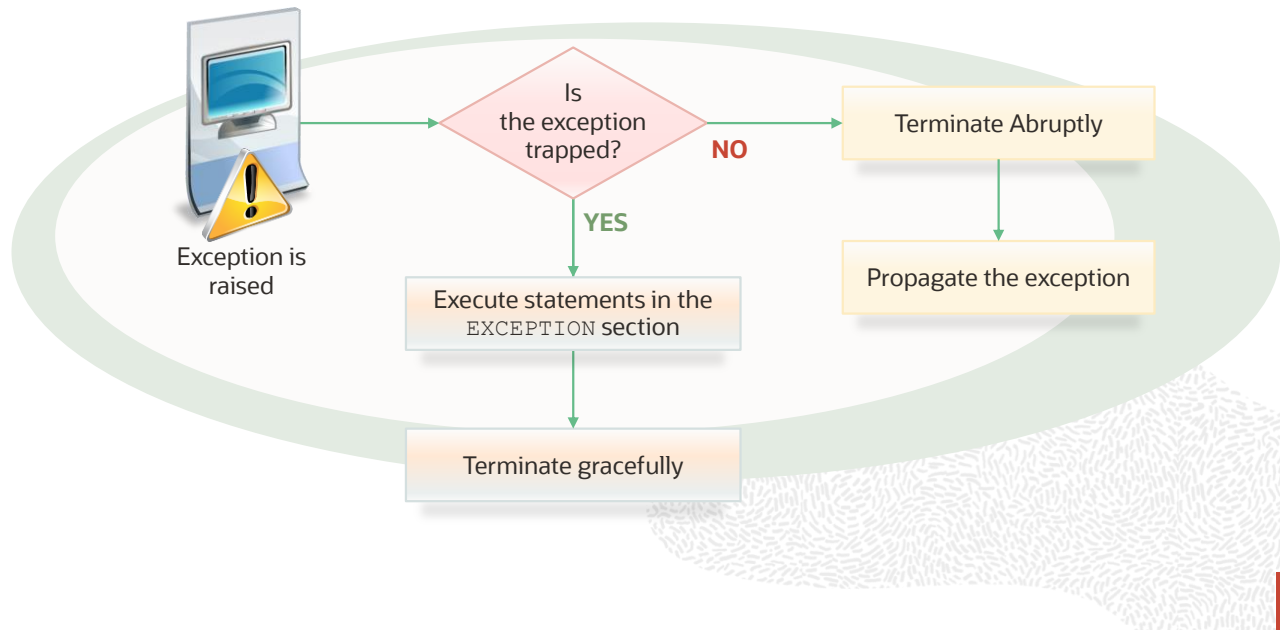
Two methods for raising an exception

- An Oracle error occurs and the associated exception is raised automatically.
For example, when you execute a `SELECT` statement in a PL/SQL block that does not retrieve any rows, an ORA-01403 error occurs. These are exceptions raised by the runtime system implicitly.
- Depending on the business functionality that your program implements, you may have to explicitly raise an exception. You raise an exception explicitly by issuing the `RAISE` statement in the block.

Two methods of handling exceptions

- To handle an exception, you have to anticipate the exception and write code for handling the exception in an `EXCEPTION` block.
- The exception propagates to the host environment when there is no exception handler defined in the PL/SQL block.

Handling Exceptions



8

Trapping an Exception

Include an `EXCEPTION` section in your PL/SQL program to trap exceptions. If an exception is raised in the executable section of a block, processing branches to the corresponding exception handler in the `EXCEPTION` section of the block. If an exception is raised in a block that has no exception handler for it, the exception propagates. That is, the exception reproduces itself in successive enclosing blocks until a block has a handler for it. If PL/SQL successfully handles the exception, the exception does not propagate to the enclosing block or to the calling environment. The PL/SQL block terminates successfully.

Propagating an Exception

If an exception is raised in the executable section of a block and there is no corresponding exception handler, the PL/SQL block terminates with failure and the exception is propagated to an enclosing block or to the calling environment. The calling environment can be any application (such as SQL*Plus that invokes the PL/SQL program), which determines the outcome of the exception.

Exception Types

- Internally defined
- Predefined

} Implicitly raised

- User-defined

Explicitly raised

9

There are three types of exceptions:

Exception	Description	Directions for Handling
Internally defined	ORA-n errors, which are implicitly raised by the runtime system	You can explicitly give names to these exceptions and write the corresponding <code>EXCEPTION</code> blocks.
Predefined	Exceptions declared globally in the package <code>STANDARD</code> . They have predefined names such as <code>INVALID_CURSOR</code> , <code>CASE_NOT_FOUND</code> .	You write an <code>EXCEPTION</code> block in the executable section with the name of the predefined exception.
User-defined error	Exceptions defined by user according to the application context. You have to declare these exceptions and their corresponding names in <code>DECLARE</code> section.	You write an <code>EXCEPTION</code> block based on the name that you used to declare the exception in the declarative section.

Agenda

- Understanding PL/SQL exceptions
- Trapping exceptions

Syntax to Trap Exceptions

```
EXCEPTION
  WHEN exception1 [OR exception2 . . .] THEN
    statement1;
    statement2;
    . . .
  [WHEN exception3 [OR exception4 . . .] THEN
    statement1;
    statement2;
    . . .]
  [WHEN OTHERS THEN
    statement1;
    statement2;
    . . .]
```

11

You can trap any error by including a corresponding handler within the exception-handling section of the PL/SQL block. Each handler consists of a `WHEN` clause, which specifies an exception name, followed by a sequence of statements to be executed when that exception is raised.

You can include any number of handlers within an `EXCEPTION` section to handle specific exceptions. However, you cannot have multiple handlers for a single exception.

The exception-trapping syntax includes the following elements:

<i>exception</i>	Is the standard name of a predefined exception or the name of a user-defined exception that is declared within the declarative section
<i>statement</i>	Is one or more PL/SQL or SQL statements
OTHERS	Is an optional exception-handling clause that traps any exceptions that have not been explicitly handled

The WHEN OTHERS Exception Handler

As stated previously, the exception-handling section traps only those exceptions that are specified.

To trap any exceptions that are not specified, you use the `OTHERS` exception handler. This option traps any exceptions that are not yet handled. For this reason, if the `OTHERS` handler is used, it must be the last exception handler that is defined.

Example:

```
WHEN NO_DATA_FOUND THEN
    statement1;
...
WHEN TOO_MANY_ROWS THEN
    statement1;
...
WHEN OTHERS THEN
    statement1;
```

Example

Consider the preceding example. If the `NO_DATA_FOUND` exception is raised by a program, the statements in the corresponding handler are executed. If the `TOO_MANY_ROWS` exception is raised, the statements in the corresponding handler are executed. However, if some other exception is raised, the statements in the `OTHERS` exception handler are executed.

The `OTHERS` handler traps all the exceptions that are not already trapped. Some Oracle tools have their own predefined exceptions that you can raise to cause events in the application. The `OTHERS` handler also traps these exceptions.

Guidelines for Trapping Exceptions

- The `EXCEPTION` keyword starts the exception-handling section.
- Several exception handlers are allowed.
- Only one handler is processed before leaving the block.
- `WHEN OTHERS` is the last clause.

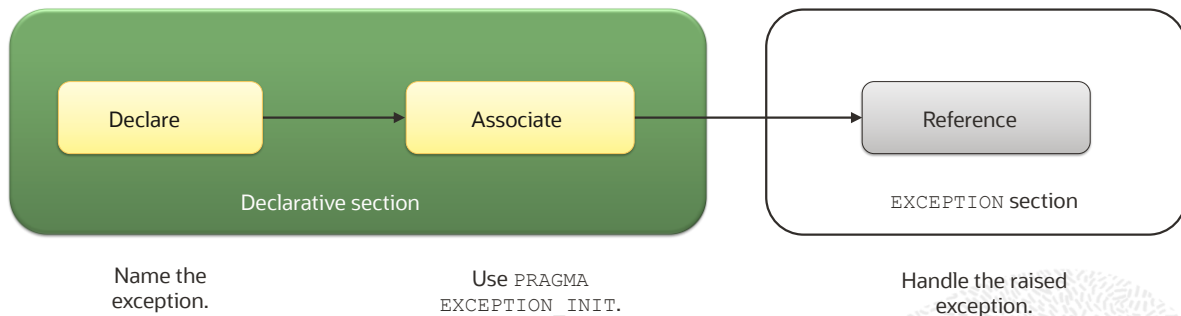


0

- Begin the exception-handling section of the block with the `EXCEPTION` keyword.
- Define several exception handlers, each with its own set of actions, for the block.
- When an exception occurs, PL/SQL processes only one handler before leaving the block.
- Place the `OTHERS` clause after all other exception-handling clauses.
- You can have only one `OTHERS` clause.
- Exceptions cannot appear in assignment or SQL statements.

For Instructor Use Only.
This document should not be distributed.

Trapping Internally Predefined Exceptions



14

Internally defined exceptions are ORA-n errors. You define `EXCEPTION` blocks for internally defined exceptions by using the `PRAGMA EXCEPTION_INIT` function.

You declare an exception name and associate it with an internally defined exception in the declarative section. `PRAGMA EXCEPTION_INIT` associates an exception name with an Oracle error number. This enables you to refer to any internal exception by name and to write a specific handler for it.

Note: `PRAGMA` (also called *pseudoinstructions*) is the keyword that signifies that the statement is a compiler directive, which is not processed when the PL/SQL block is executed. Rather, it directs the PL/SQL compiler to interpret all occurrences of the exception name within the block as the associated Oracle Server error number.

Internally Defined Exception Trapping: Example

To trap Oracle Server error 01400 ("cannot insert NULL"):

```
DECLARE
  e_insert_excep EXCEPTION;
  PRAGMA EXCEPTION_INIT(e_insert_excep, -01400);
BEGIN
  INSERT INTO departments
    (department_id, department_name) VALUES (280, NULL);
EXCEPTION
  WHEN e_insert_excep THEN
    DBMS_OUTPUT.PUT_LINE('INSERT OPERATION FAILED');
END;
```

The diagram illustrates the three steps in defining an exception block for internally defined exceptions:

1. Declare the name of the exception in the declarative section by using the following syntax:
`exception EXCEPTION;`
2. Associate the declared exception with the standard Oracle Server error number by using the `PRAGMA EXCEPTION_INIT` function. Use the following syntax:
`PRAGMA EXCEPTION_INIT(exception, error_number);`
3. Reference the declared exception within the corresponding exception-handling routine.

The example in the slide tries to insert a NULL value for the `department_name` column of the `departments` table. However, the operation is not successful because `department_name` is a NOT NULL column.

Script Output X
Task completed in 0.004 seconds
PL/SQL procedure successfully completed.
Insert operation failed

15

The example in the slide illustrates the three steps in defining an `EXCEPTION` block for internally defined exceptions:

1. Declare the name of the exception in the declarative section by using the following syntax:

```
exception EXCEPTION;
```

In the syntax, *exception* is the name of the exception.

2. Associate the declared exception with the standard Oracle Server error number by using the `PRAGMA EXCEPTION_INIT` function. Use the following syntax:

```
PRAGMA EXCEPTION_INIT(exception, error_number);
```

In the syntax, *exception* is the previously declared exception and *error_number* is a standard Oracle Server error number.

3. Reference the declared exception within the corresponding exception-handling routine.

Example

The example in the slide tries to insert a NULL value for the `department_name` column of the `departments` table. However, the operation is not successful because `department_name` is a NOT NULL column.

Trapping Predefined Exceptions

- Reference the predefined name in the `EXCEPTION` block.
- Some sample predefined exceptions are:

- `NO_DATA_FOUND`
- `TOO_MANY_ROWS`
- `INVALID_CURSOR`
- `ZERO_DIVIDE`
- `DUP_VAL_ON_INDEX`

```
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    statement1;
    ...
  WHEN TOO_MANY_ROWS THEN
    statement1;
    ...
  WHEN OTHERS THEN
    statement1;
```

16

Trap a predefined Oracle Server error by referencing its predefined name within the corresponding exception-handling routine.

For a complete list of predefined exceptions, see the *PL/SQL User's Guide and Reference*.

Note: PL/SQL declares predefined exceptions in the `STANDARD` package.

The following table shows some of the predefined exceptions:

Exception Name	Oracle Server Error Number	Description
ACCESS_INTO_NULL	ORA-06530	Attempted to assign values to the attributes of an uninitialized object
CASE_NOT_FOUND	ORA-06592	None of the choices in the <code>WHEN</code> clauses of a <code>CASE</code> statement were selected, and there is no <code>ELSE</code> clause.
COLLECTION_IS_NULL	ORA-06531	Attempted to apply collection methods other than <code>EXISTS</code> to an uninitialized nested table or <code>VARRAY</code>
CURSOR_ALREADY_OPEN	ORA-06511	Attempted to open an already open cursor
DUP_VAL_ON_INDEX	ORA-00001	Attempted to insert a duplicate value
INVALID_CURSOR	ORA-01001	Occurrence of an illegal cursor operation
INVALID_NUMBER	ORA-01722	Conversion of character string to number failed
LOGIN_DENIED	ORA-01017	Logging on to the Oracle Server with an invalid username or password
NO_DATA_FOUND	ORA +100	Single row <code>SELECT</code> returned no data.
NOT_LOGGED_ON	ORA-01012	The PL/SQL program issues a database call without being connected to the Oracle Server.
PROGRAM_ERROR	ORA-06501	PL/SQL has an internal problem.
ROWTYPE_MISMATCH	ORA-06504	The host cursor variable and the PL/SQL cursor variable involved in an assignment have incompatible return types.

Exception Name	Oracle Server Error Number	Description
STORAGE_ERROR	ORA-06500	PL/SQL ran out of memory, or memory is corrupted.
SELF_IS_NULL	ORA-30625	Function invoked with a NULL object instance
SUBSCRIPT_BEYOND_COUNT	ORA-06533	Referenced a nested table or VARRAY element by using an index number that is larger than the number of elements in the collection
SUBSCRIPT_OUTSIDE_LIMIT	ORA-06532	Referenced a nested table or VARRAY element by using an index number that is outside the legal range (for example, -1)
SYS_INVALID_ROWID	ORA-01410	The conversion of a character string into a universal ROWID failed because the character string does not represent a valid ROWID.
TIMEOUT_ON_RESOURCE	ORA-00051	Timeout occurred while the Oracle Server was waiting for a resource.
TOO_MANY_ROWS	ORA-01422	A single-row SELECT returned multiple rows.
VALUE_ERROR	ORA-06502	Arithmetic, conversion, truncation, or size-constraint error occurred.
ZERO_DIVIDE	ORA-01476	Attempted to divide by zero

Functions for Trapping Exceptions

- **SQLCODE**: Returns the numeric value for the error code
- **SQLERRM**: Returns the message associated with the error number

19

When an exception occurs, you can identify the associated error code or error message by using two functions. Based on the values of the code or the message, you can decide which subsequent actions to take.

SQLCODE returns the Oracle error number for internal exceptions. **SQLERRM** returns the message associated with the error number.

Function	Description
SQLCODE	Returns the numeric value for the error code (You can assign it to a <code>NUMBER</code> variable.)
SQLERRM	Returns character data containing the message associated with the error number

SQLCODE Values: Examples

SQLCODE Value	Description
0	No exception encountered
1	User-defined exception
+100	<code>NO_DATA_FOUND</code> exception
<i>negative number</i>	Another Oracle Server error number

Functions for Trapping Exceptions

```
DECLARE
    error_code      NUMBER;
    error_message   VARCHAR2(255);
BEGIN
    ...
EXCEPTION
    ...
    WHEN OTHERS THEN
        ROLLBACK;
        error_code := SQLCODE ;
        error_message := SQLERRM ;
        INSERT INTO errors (e_user, e_date, error_code,
            error_message) VALUES (USER,SYSDATE,error_code,
            error_message);
END;
/
```

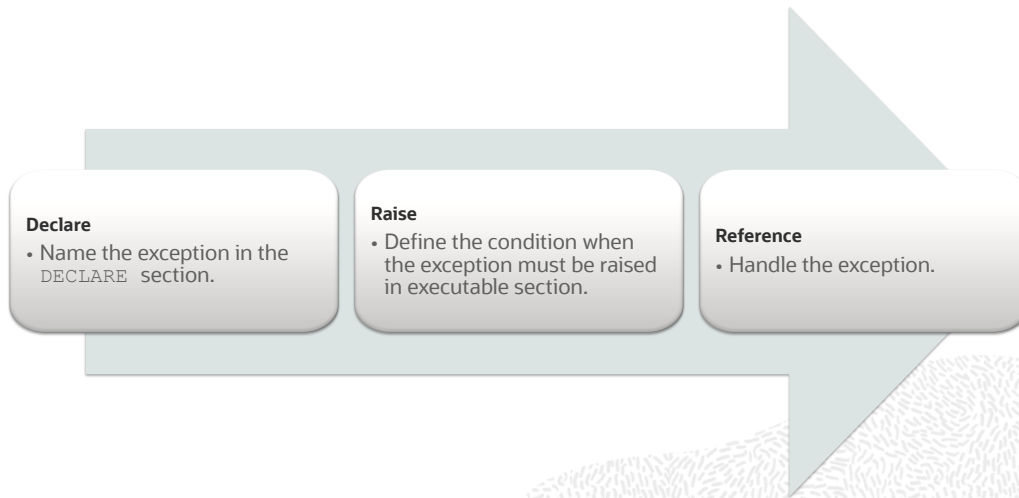
20

When an exception is trapped in the `WHEN OTHERS` exception handler, you can use a set of generic functions to identify those errors. The example in the slide illustrates the values of `SQLCODE` and `SQLERRM` assigned to variables, and then those variables being used in a SQL statement.

You cannot use `SQLCODE` or `SQLERRM` directly in a SQL statement. Instead, you must assign their values to local variables, and then use the variables in the SQL statement, as shown in the following example:

```
DECLARE
    err_num NUMBER;
    err_msg VARCHAR2(100);
BEGIN
    ...
EXCEPTION
    ...
    WHEN OTHERS THEN
        err_num := SQLCODE;
        err_msg := SUBSTR(SQLERRM, 1, 100);
        INSERT INTO errors VALUES (err_num, err_msg);
END;
/
```

Trapping User-Defined Exceptions



21

PL/SQL enables you to define your own exceptions depending on the requirements of your application.

For example, you may prompt a user to enter the age value in a valid range if the user entered an invalid age value. You must define an exception here, which checks on the range of value of age. If the age is not in the valid range, you may raise a user-defined exception.

PL/SQL exceptions must be:

- Declared in the declarative section of a PL/SQL block
- Raised explicitly with `RAISE` statements
- Handled in the `EXCEPTION` section

RAISE Statement

- Stops normal execution of a PL/SQL block or subprogram, and transfers control to an exception handler
- Explicitly raises predefined exceptions or user-defined exceptions
- Syntax:

```
RAISE exception_name ;
```

22

In the syntax, `exception_name` is the name of a predefined or a user-defined exception. `exception_name` is optional only in an exception handler, where the default is the current exception. Outside an exception handler, you must specify the name of the exception.

Trapping User-Defined Exceptions

```
DECLARE
  v_deptno NUMBER := 500;
  v_name VARCHAR2(20) := 'Testing';
  e_invalid_department EXCEPTION;
BEGIN
  UPDATE departments
  SET department_name = v_name
  WHERE department_id = v_deptno;
  IF SQL%NOTFOUND THEN
    RAISE e_invalid_department;
  END IF;
  COMMIT;
EXCEPTION
  WHEN e_invalid_department THEN
    DBMS_OUTPUT.PUT_LINE('No such department id.');
```

Diagram illustrating the trapping of a user-defined exception:

1. Declaration of the user-defined exception: `e_invalid_department EXCEPTION;`
2. Raising the exception: `RAISE e_invalid_department;`
3. Exception handling: `WHEN e_invalid_department THEN DBMS_OUTPUT.PUT_LINE('No such department id.');`

Script Output:

```
Task completed in 0.007 seconds
PL/SQL procedure successfully completed.
No such department id
```

23

0

You trap a user-defined exception by declaring it and raising it explicitly.

1. Declare the name of the user-defined exception within the declarative section.

Syntax:

```
exception EXCEPTION;
```

In the syntax, *exception* is the name of the exception.

2. Use the `RAISE` statement to raise the exception explicitly within the executable section.

Syntax:

```
RAISE exception;
```

In the syntax, *exception* is the previously declared exception.

3. Reference the declared exception within the corresponding exception-handling routine.

Example

The block shown in the slide updates the `department_name` of a department. The user supplies the department number and the new name. If the supplied department number does not exist, no rows are updated in the `departments` table. An exception is raised and a message is printed for the user that an invalid department number was entered.

Note: Use the `RAISE` statement by itself within an exception handler to raise the same exception again and propagate it back to the calling environment.

Propagating Exceptions in a Sub-Block

Sub-blocks can handle an exception or pass the exception to the enclosing block.

```
DECLARE
    . . .
    e_no_rows exception;
    e_integrity    exception;
    PRAGMA EXCEPTION_INIT (e_integrity, -2292);
BEGIN
    FOR c_record IN emp_cursor LOOP
        BEGIN
            SELECT ...
            UPDATE ...
            IF SQL%NOTFOUND THEN
                RAISE e_no_rows;
            END IF;
        END;
    END LOOP;
EXCEPTION
    WHEN e_integrity THEN ...
    WHEN e_no_rows THEN ...
END;
/
```

24

When a sub-block handles an exception, it terminates normally. Control resumes in the enclosing block immediately after the sub-block's `END` statement.

However, if PL/SQL raises an exception and the current block does not have a handler for that exception, the exception propagates to successive enclosing blocks until it finds a handler. If none of these blocks handles the exception, an unhandled exception is propagated to the host environment.

When the exception propagates to an enclosing block, the remaining executable actions in that sub-block are bypassed.

One advantage of this behavior is that you can enclose statements that require their own exclusive error handling in their own block, while leaving more general exception handling to the enclosing block.

Note in the example that the exceptions (`e_no_rows` and `e_integrity`) are declared in the outer block. In the inner block, when the `e_no_rows` exception is raised, PL/SQL looks for the exception handler in the sub-block. Because the exception is not handled in the sub-block, the exception propagates to the outer block, where PL/SQL finds the handler.

The RAISE_APPLICATION_ERROR Procedure

Syntax:

```
raise_application_error (error_number,  
                        message[, {TRUE | FALSE}]);
```

- You can use this procedure to issue user-defined error messages from stored subprograms.
- You can report errors to your application and avoid returning unhandled exceptions.

25

Use the `RAISE_APPLICATION_ERROR` procedure to communicate a predefined exception interactively by returning a nonstandard error code and an error message. With `RAISE_APPLICATION_ERROR`, you can report errors to your application and avoid returning unhandled exceptions.

In the syntax:

<i>error_number</i>	Is a user-specified number for the exception between –20,000 and –20,999
<i>message</i>	Is a user-specified message for the exception; is a character string up to 2,048 bytes long
TRUE FALSE	Is an optional Boolean parameter (If <code>TRUE</code> , the error is placed on the stack of previous errors. If <code>FALSE</code> , which is the default, the error replaces all previous errors.)

The RAISE_APPLICATION_ERROR Procedure

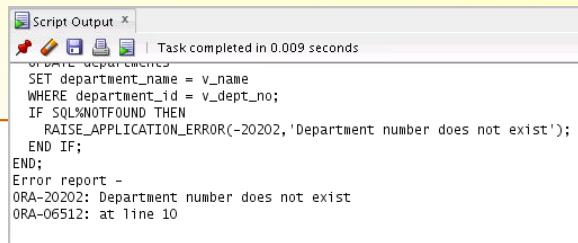
- Is used in two different places:
 - Executable section
 - Exception section
- Returns error conditions to the user in a manner that is consistent with other Oracle Server errors

26

The `RAISE_APPLICATION_ERROR` procedure can be used in either the executable section or the exception section of a PL/SQL program, or both. The returned error is consistent with how the Oracle Server produces a predefined, non-predefined, or user-defined error. The error number and message are displayed to the user.

The RAISE_APPLICATION_ERROR Procedure

```
DECLARE
  v_deptno NUMBER := 500;
  v_name VARCHAR2(20) := 'Testing';
  e_invalid_department EXCEPTION;
BEGIN
  UPDATE departments
  SET department_name = v_name
  WHERE department_id = v_deptno;
  IF SQL%NOTFOUND THEN
    RAISE_APPLICATION_ERROR(-20202, 'Department number does not exist');
  END IF;
END;
```



Script Output x

Task completed in 0.009 seconds

```
UPDATE departments
SET department_name = v_name
WHERE department_id = v_deptno;
IF SQL%NOTFOUND THEN
  RAISE_APPLICATION_ERROR(-20202, 'Department number does not exist');
END IF;
END;
Error report -
ORA-20202: Department number does not exist
ORA-06512: at line 10
```

27

The slide shows how the `RAISE_APPLICATION_ERROR` procedure can be used in the executable section of a PL/SQL program. When called, `RAISE_APPLICATION_ERROR` ends the subprogram and returns a user-defined error number and message to the application. The error number and message can be trapped like any Oracle error.

You can see in the output that the error displayed is the one defined in the PL/SQL block.

Quiz



You can trap any error by including a corresponding handler within the exception-handling section of the PL/SQL block.

- a. True
- b. False



0

28

Answer: a

You can trap any error by including a corresponding handler within the exception-handling section of the PL/SQL block. Each handler consists of a `WHEN` clause, which specifies an exception name, followed by a sequence of statements to be executed when that exception is raised. You can include any number of handlers within an `EXCEPTION` section to handle specific exceptions. However, you cannot have multiple handlers for a single exception.

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Define PL/SQL exceptions
- Add an `EXCEPTION` section to the PL/SQL block to deal with exceptions at run time
- Handle different types of exceptions:
 - Internally defined exceptions
 - Predefined exceptions
 - User-defined exceptions
- Propagate exceptions in nested blocks and call applications



29

In this lesson, you learned how to deal with different types of exceptions. In PL/SQL, a warning or error condition at run time is called an exception. Internally defined exceptions can be any standard Oracle Server errors. Predefined exceptions are error conditions that are defined by the Oracle Server. User-defined exceptions are exceptions that are specific to your application. The `PRAGMA EXCEPTION_INIT` function can be used to associate a declared exception name with an Oracle Server error.

You can define exceptions of your own in the declarative section of any PL/SQL block. For example, you can define an exception named `INSUFFICIENT_FUNDS` to flag overdrawn bank accounts.

When an error occurs, an exception is raised. Normal execution stops, and control is transferred to the exception-handling section of your PL/SQL block. Internal exceptions are raised implicitly (automatically) by the runtime system; however, user-defined exceptions must be raised explicitly. To handle raised exceptions, you write separate routines called exception handlers.

Practice 9: Overview

This practice covers the following topics:

- Creating and invoking user-defined exceptions
- Handling named Oracle Server exceptions

30

In these practices, you create exception handlers for a predefined exception and a standard Oracle Server exception.

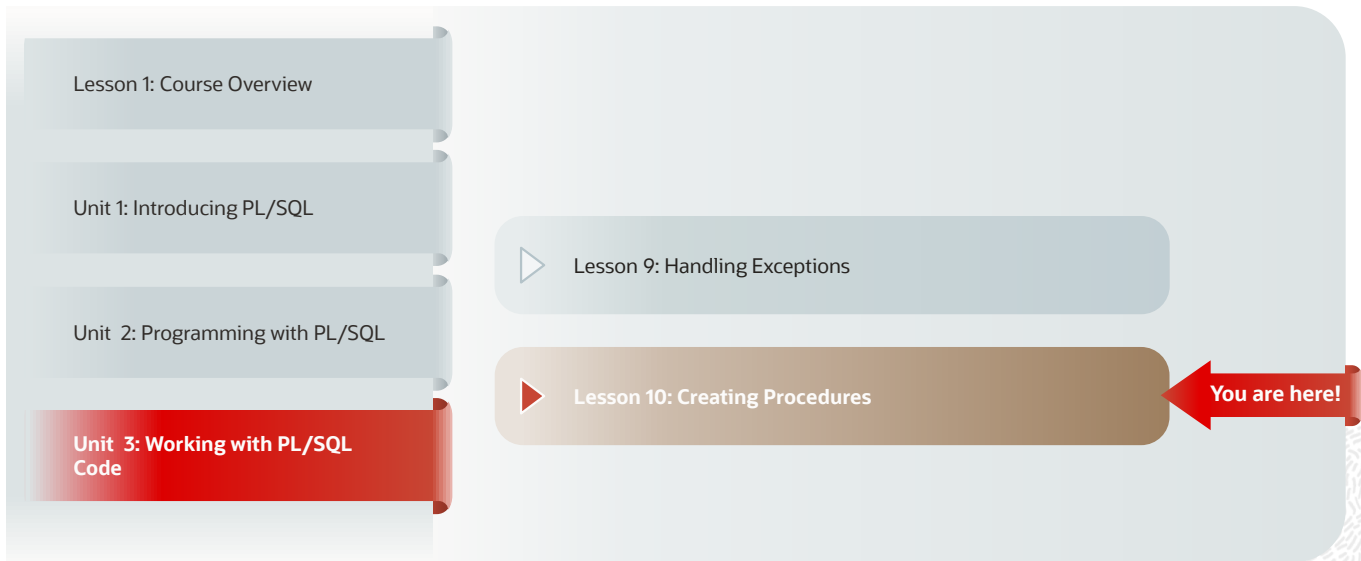


0

For Instructor Use Only.
This document should not be distributed.

Introducing Stored Procedures and Functions

Course Road Map



2

In Unit 3, you are introduced to the basic program units in PL/SQL such as stored procedures and functions.

0
For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to do the following:

- Differentiate between anonymous blocks and subprograms
- Create a simple procedure and invoke it from an anonymous block
- Create a simple function
- Create a simple function that accepts a parameter
- Differentiate between procedures and functions



0

3

You learned about anonymous blocks. This lesson introduces you to named blocks, which are also called *subprograms*. Procedures and functions are PL/SQL subprograms. In the lesson, you learn to differentiate between anonymous blocks and subprograms.

For Instructor Use Only.
This document should not be distributed.

Agenda

- PL/SQL subprograms
- Previewing procedures
- Previewing functions

What Are PL/SQL Subprograms?

- A PL/SQL subprogram is a named PL/SQL block that can be called with or without a set of parameters.
- A subprogram consists of a specification and a body.
- A subprogram can be a procedure or a function.
- Typically, you use a procedure to perform an action and a function to compute and return a value.



5

A PL/SQL subprogram is a named, executable PL/SQL block. It can exist as an independent object on the database. You generally write subprograms to perform an operation or to process data and return a value.

Subprogram Parts

- A subprogram consists of a specification (spec) and a body.
- To declare a subprogram, you must provide the specifications, which include the name of the subprogram and parameters.
- To define a subprogram, you must provide both the specification and the body.
- You can either declare a subprogram first and define it later in the same block or subprogram, or declare and define it at the same time.

Subprogram Types

- PL/SQL has two types of subprograms: procedures and functions.
- Typically, you use a procedure to perform an action and a function to compute and return a value.

A subprogram contains the following sections:

- **Declarative section:** Subprograms can have an optional declarative section. However, unlike anonymous blocks, the declarative section of a subprogram does not start with the `DECLARE` keyword. The optional declarative section follows the `IS` or `AS` keyword in the subprogram declaration.

- **Executable section:** This is the mandatory section of the subprogram, which contains the implementation of the business logic. Looking at the code in this section, you can easily determine the business functionality of the subprogram. This section begins and ends with the `BEGIN` and `END` keywords, respectively.
- **Exception section:** This is an optional section that is included to handle exceptions.

For Instructor Use Only.
This document should not be distributed.

Differences Between Anonymous Blocks and Subprograms

Anonymous Blocks	Subprograms
Unnamed PL/SQL blocks	Named PL/SQL blocks
Compiled every time	Compiled only once
Not stored in the database	Stored in the database
Cannot be invoked by other applications	Named and, therefore, can be invoked by other applications
Do not return values	If functions, must return values
Cannot take parameters	Can take parameters

7

The table in the slide not only shows the differences between anonymous blocks and subprograms, but also highlights the general benefits of subprograms.

Anonymous blocks are not persistent database objects. They are compiled every time they are to be executed. They are not stored in the database for reuse. If you want to reuse them, you must rerun the script that creates the anonymous block, which causes recompilation and execution.

Procedures and functions are compiled and stored in the database in a compiled form. They are recompiled only when they are modified. Because they are stored in the database, any application can make use of these subprograms based on appropriate permissions. The calling application can pass parameters to the procedures if the procedure is designed to accept parameters. Similarly, a calling application can retrieve a value if it invokes a function or a procedure.

Agenda

- Introducing procedures and functions
- Previewing procedures
- Previewing functions

Procedure: Syntax

```
CREATE [OR REPLACE] PROCEDURE procedure_name
  [(argument1 [mode1] datatype1,
    argument2 [mode2] datatype2,
    . . .)]
IS|AS
procedure_body;
```

9

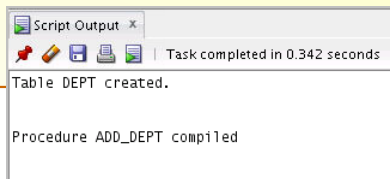
The slide shows the syntax for creating procedures. In the syntax:

<i>procedure_name</i>	Is the name of the procedure to be created
<i>argument</i>	Is the name given to the procedure parameter. Every argument is associated with a mode and data type. You can have any number of arguments separated by commas.
<i>mode</i>	Mode of argument: IN (default) OUT IN OUT
<i>datatype</i>	Is the data type of the associated parameter. The data type of parameters cannot have explicit size; instead, use %TYPE.
<i>Procedure_body</i>	Is the PL/SQL block that makes up the code

The argument list is optional in a procedure declaration. You learn about procedures in detail in the course titled *Oracle Database: Develop PL/SQL Program Units*.

Creating a Procedure

```
...  
CREATE TABLE dept AS SELECT * FROM departments;  
CREATE PROCEDURE add_dept IS  
  v_dept_id dept.department_id%TYPE;  
  v_dept_name dept.department_name%TYPE;  
BEGIN  
  v_dept_id:=280;  
  v_dept_name:='ST-Curriculum';  
  INSERT INTO dept(department_id,department_name)  
  VALUES(v_dept_id,v_dept_name);  
  DBMS_OUTPUT.PUT_LINE(' Inserted ' || SQL%ROWCOUNT || ' row ');  
END;
```



10

In the code example in the slide, the `add_dept` procedure inserts a new department with department ID 280 and department name ST-Curriculum.

In addition, the example shows the following:

- The declarative section of a procedure starts immediately after the procedure declaration, and does not begin with the `DECLARE` keyword.
- The procedure declares two variables: `dept_id` and `dept_name`.
- The procedure uses the implicit cursor attribute or the `SQL%ROWCOUNT` SQL attribute to verify that the row was successfully inserted. A value of 1 should be returned in this case.

When you compile this script, you see the Script output as shown in the slide. The output shows that a table `DEPT` has been created and that the procedure `ADD_DEPT` is compiled. When a procedure is compiled, an object is created in the database, which can later be invoked. So here we did not execute the procedure yet.

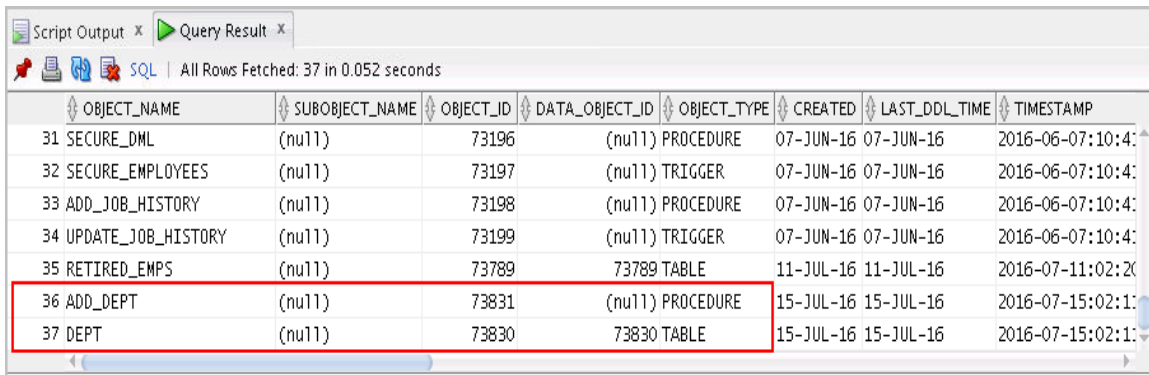
Note: See the following page for more notes on the example.

Procedure: Example

Note

- When you create any object, entries are made into the `user_objects` table. When the code in the slide is executed successfully, you can check the `user_objects` table for the new objects, by issuing the following command:

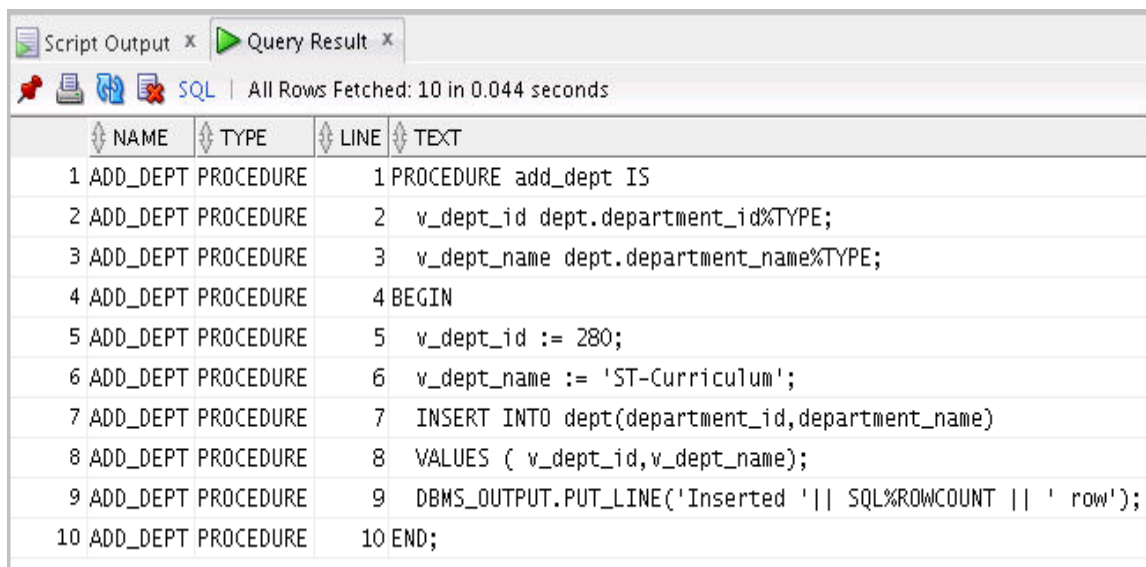
```
SELECT * FROM user_objects;
```



OBJECT_NAME	SUBOBJECT_NAME	OBJECT_ID	DATA_OBJECT_ID	OBJECT_TYPE	CREATED	LAST_DDL_TIME	TIMESTAMP
31 SECURE_DML	(null)	73196	(null)	PROCEDURE	07-JUN-16	07-JUN-16	2016-06-07:10:41
32 SECURE_EMPLOYEES	(null)	73197	(null)	TRIGGER	07-JUN-16	07-JUN-16	2016-06-07:10:41
33 ADD_JOB_HISTORY	(null)	73198	(null)	PROCEDURE	07-JUN-16	07-JUN-16	2016-06-07:10:41
34 UPDATE_JOB_HISTORY	(null)	73199	(null)	TRIGGER	07-JUN-16	07-JUN-16	2016-06-07:10:41
35 RETIRED_EMPS	(null)	73789	73789	TABLE	11-JUL-16	11-JUL-16	2016-07-11:02:20
36 ADD_DEPT	(null)	73831	(null)	PROCEDURE	15-JUL-16	15-JUL-16	2016-07-15:02:11
37 DEPT	(null)	73830	73830	TABLE	15-JUL-16	15-JUL-16	2016-07-15:02:11

- The source of the procedure is stored in the `user_source` table. You can check the source for the procedure, by issuing the following command:

```
SELECT * FROM user_source WHERE name='ADD_DEPT';
```

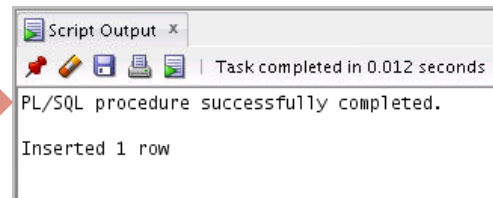


NAME	TYPE	LINE	TEXT
1 ADD_DEPT	PROCEDURE	1	PROCEDURE add_dept IS
2 ADD_DEPT	PROCEDURE	2	v_dept_id dept.department_id%TYPE;
3 ADD_DEPT	PROCEDURE	3	v_dept_name dept.department_name%TYPE;
4 ADD_DEPT	PROCEDURE	4	BEGIN
5 ADD_DEPT	PROCEDURE	5	v_dept_id := 280;
6 ADD_DEPT	PROCEDURE	6	v_dept_name := 'ST-Curriculum';
7 ADD_DEPT	PROCEDURE	7	INSERT INTO dept(department_id,department_name)
8 ADD_DEPT	PROCEDURE	8	VALUES (v_dept_id,v_dept_name);
9 ADD_DEPT	PROCEDURE	9	DBMS_OUTPUT.PUT_LINE('Inserted ' SQL%ROWCOUNT ' row');
10 ADD_DEPT	PROCEDURE	10	END;

Invoking a Procedure

Invoke the procedure in a PL/SQL block

```
...  
BEGIN  
  add_dept;  
END;  
/
```



Run the following query to check the result

```
SELECT department_id,  
       department_name  
FROM dept  
WHERE department_id = 280;
```

DEPARTMENT_ID		DEPARTMENT_NAME	
1		280	ST-Curriculum

12

The slide shows how to invoke a procedure from an anonymous block. You must include the call to the procedure in the executable section of the anonymous block. Similarly, you can invoke the procedure from any application, such as a Forms application or a Java application. The `SELECT` statement in the code checks to see whether the row was successfully inserted.

You can also invoke a procedure with the SQL statement `CALL <procedure_name>`.

Agenda

- Introducing procedures and functions
- Previewing procedures
- Previewing functions

Function: Syntax

```
CREATE [OR REPLACE] FUNCTION function_name
  [(argument1 [mode1] datatype1,
    argument2 [mode2] datatype2,
    . . .)]
RETURN datatype
IS|AS
function_body;
```

14

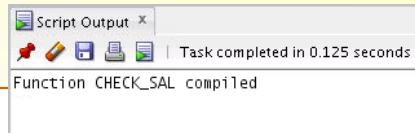
The slide shows the syntax for creating a function. In the syntax:

<i>function_name</i>	Is the name of the function to be created
<i>argument</i>	Is the name given to the function parameter (Every argument is associated with a mode and data type. You can have any number of arguments separated by a comma. You pass the argument when you invoke the function.)
<i>mode</i>	Is the type of parameter (Only <code>IN</code> parameters should be declared.)
<i>datatype</i>	Is the data type of the associated parameter
RETURN <i>datatype</i>	Is the data type of the value returned by the function
<i>function_body</i>	Is the PL/SQL block that makes up the function code

The argument list is optional in a function declaration. The difference between a procedure and a function is that a function must return a value to the calling program. Therefore, the syntax contains *return_type*, which specifies the data type of the value that the function returns. A procedure may return a value via an `OUT` or `IN OUT` parameter.

Creating a Function

```
CREATE FUNCTION check_sal RETURN Boolean IS
v_dept_id employees.department_id%TYPE;
v_empno   employees.employee_id%TYPE;
v_sal     employees.salary%TYPE;
v_avg_sal employees.salary%TYPE;
BEGIN
v_empno:=205;
SELECT salary,department_id INTO v_sal,v_dept_id FROM employees
WHERE employee_id= v_empno;
SELECT avg(salary) INTO v_avg_sal FROM employees WHERE department_id=v_dept_id;
IF v_sal > v_avg_sal THEN
RETURN TRUE;
ELSE
RETURN FALSE;
END IF;
EXCEPTION
WHEN NO_DATA_FOUND THEN
RETURN NULL;
END;
```



15

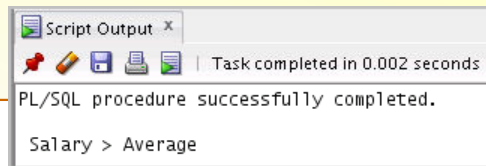
Function: Example

The `check_sal` function is written to determine whether the salary of a particular employee is greater than or less than the average salary of all employees working in the same department. The function returns `TRUE` if the salary of the employee is greater than the average salary of the employees in the department; if not, it returns `FALSE`. The function returns `NULL` if a `NO_DATA_FOUND` exception is thrown.

Note that the function checks for the employee with employee ID 205. The function is hard-coded to check only for this employee ID. If you want to check for any other employees, you must modify the function itself. You can solve this problem by declaring the function such that it accepts an argument. You can then pass the employee ID as parameter.

Invoking a Function

```
BEGIN
  IF (check_sal IS NULL) THEN
    DBMS_OUTPUT.PUT_LINE('The function returned
      NULL due to exception');
  ELSIF (check_sal) THEN
    DBMS_OUTPUT.PUT_LINE('Salary > average');
  ELSE
    DBMS_OUTPUT.PUT_LINE('Salary < average');
  END IF;
END;
/
```

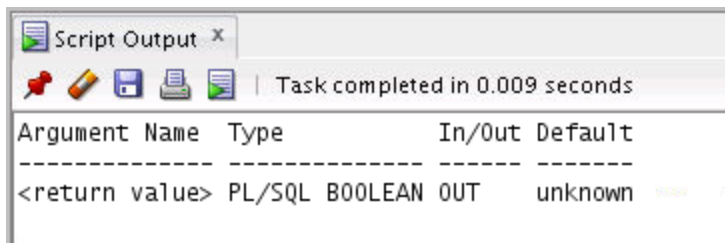


16

You include the call to the function in the executable section of the anonymous block. The function is invoked as part of a statement. Remember that the `check_sal` function returns `Boolean` or `NULL`. Thus the call to the function is included as the conditional expression for the `IF` block.

Note: You can use the `DESCRIBE` command to check the arguments and return type of the function, as in the following example:

```
DESCRIBE check_sal;
```



Argument Name	Type	In/Out	Default
<return value>	PL/SQL BOOLEAN	OUT	unknown

Passing a Parameter to the Function

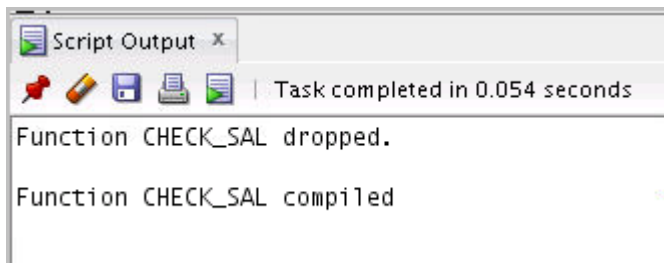
```
DROP FUNCTION check_sal;
CREATE FUNCTION check_sal(p_empno employees.employee_id%TYPE) RETURN Boolean IS
  v_dept_id employees.department_id%TYPE;
  v_sal      employees.salary%TYPE;
  v_avg_sal  employees.salary%TYPE;
BEGIN
  SELECT salary,department_id INTO v_sal,v_dept_id FROM employees
    WHERE employee_id=p_empno;
  SELECT avg(salary) INTO v_avg_sal FROM employees
    WHERE department_id=v_dept_id;
  IF v_sal > v_avg_sal THEN
    RETURN TRUE;
  ELSE
    RETURN FALSE;
  END IF;
EXCEPTION
  ...
```

17

Remember that the function was hard-coded to check the salary of the employee with employee ID 205. The code shown in the slide removes that constraint because it is rewritten to accept employee number as a parameter. You can now pass different employee numbers and check for the employee's salary.

You learn more about functions in the *Oracle Database: Develop PL/SQL Program Units* course.

The output of the code example in the slide is as follows:

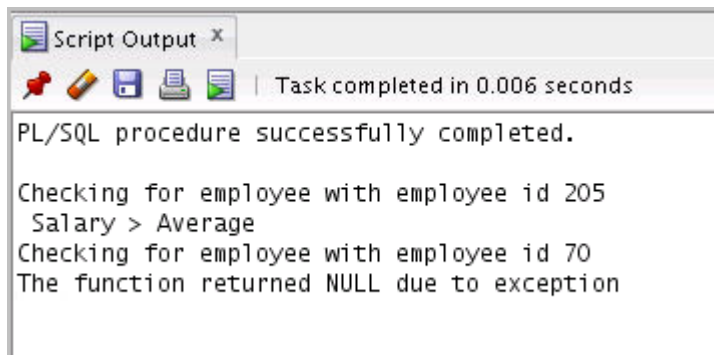


Invoking the Function with a Parameter

```
BEGIN
DBMS_OUTPUT.PUT_LINE('Checking for employee with id 205');
IF (check_sal(205) IS NULL) THEN
DBMS_OUTPUT.PUT_LINE('The function returned
  NULL due to exception');
ELSIF (check_sal(205)) THEN
DBMS_OUTPUT.PUT_LINE('Salary > average');
ELSE
DBMS_OUTPUT.PUT_LINE('Salary < average');
END IF;
DBMS_OUTPUT.PUT_LINE('Checking for employee with id 70');
IF (check_sal(70) IS NULL) THEN
DBMS_OUTPUT.PUT_LINE('The function returned
  NULL due to exception');
ELSIF (check_sal(70)) THEN
...
END IF;
END;
/
```

18

The code in the slide invokes the function twice by passing parameters. The output of the code is as follows:



Quiz



Subprograms:

- a. Are named PL/SQL blocks, and can be invoked by other applications
- b. Are compiled only once
- c. Are stored in the database
- d. Do not have to return values if they are functions
- e. Can take parameters



0

19

Answer: a, b, c, e

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Create a simple procedure
- Invoke the procedure from an anonymous block
- Create a simple function
- Create a simple function that accepts parameters
- Invoke the function from an anonymous block



20

You can use anonymous blocks to design any functionality in PL/SQL. However, the major constraint with anonymous blocks is that they are not stored and therefore, cannot be reused.

Instead of creating anonymous blocks, you can create PL/SQL subprograms. Procedures and functions are called subprograms, which are named PL/SQL blocks. Subprograms express reusable logic by virtue of parameterization. The structure of a procedure or function is similar to the structure of an anonymous block. These subprograms are stored in the database and are therefore, reusable.

Practice 10: Overview

This practice covers the following topics:

- Converting an existing anonymous block to a procedure
- Modifying the procedure to accept a parameter
- Writing an anonymous block to invoke the procedure



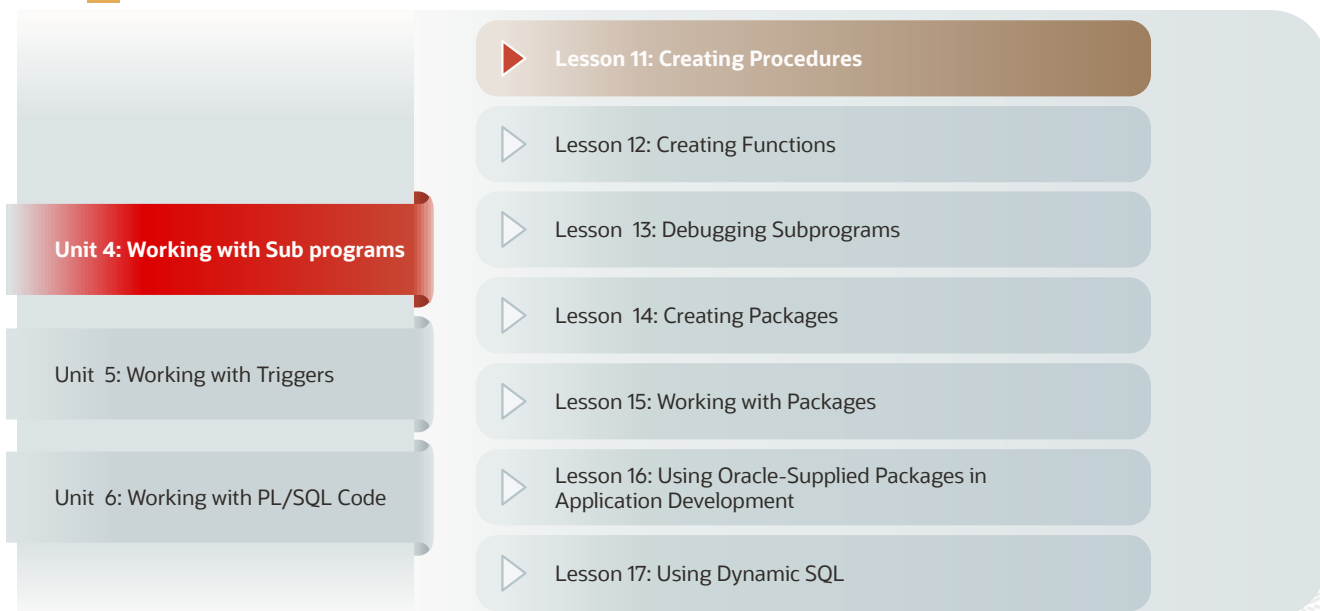
0

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Creating Procedures

Course Road Map



2

In the fourth unit, we have seven lessons – Creating Stored Procedures, Creating Functions, Debugging Subprograms, Creating Packages, Working with Packages, Using Oracle Supplied Packages in Application Development, and Using Dynamic SQL. You will learn to write PL/SQL subprograms, packages, and use them in Application Development.

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to:

- Identify the benefits of modularized and layered subprogram design
- Create and call procedures
- Use formal and actual parameters
- Use positional, named, or mixed notation for passing parameters
- Identify the available parameter-passing modes
- Handle exceptions in procedures
- Remove a procedure and display its information



0

3

In this lesson, you learn to create, execute, and remove procedures with or without parameters. Procedures are the foundation of modular programming in PL/SQL. To make procedures more flexible, it is important that varying data is either calculated or passed into a procedure by using input parameters. Calculated results can be returned to the caller of a procedure by using OUT parameters.

To make your programs robust, you should manage exception conditions by using the exception-handling features of PL/SQL.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

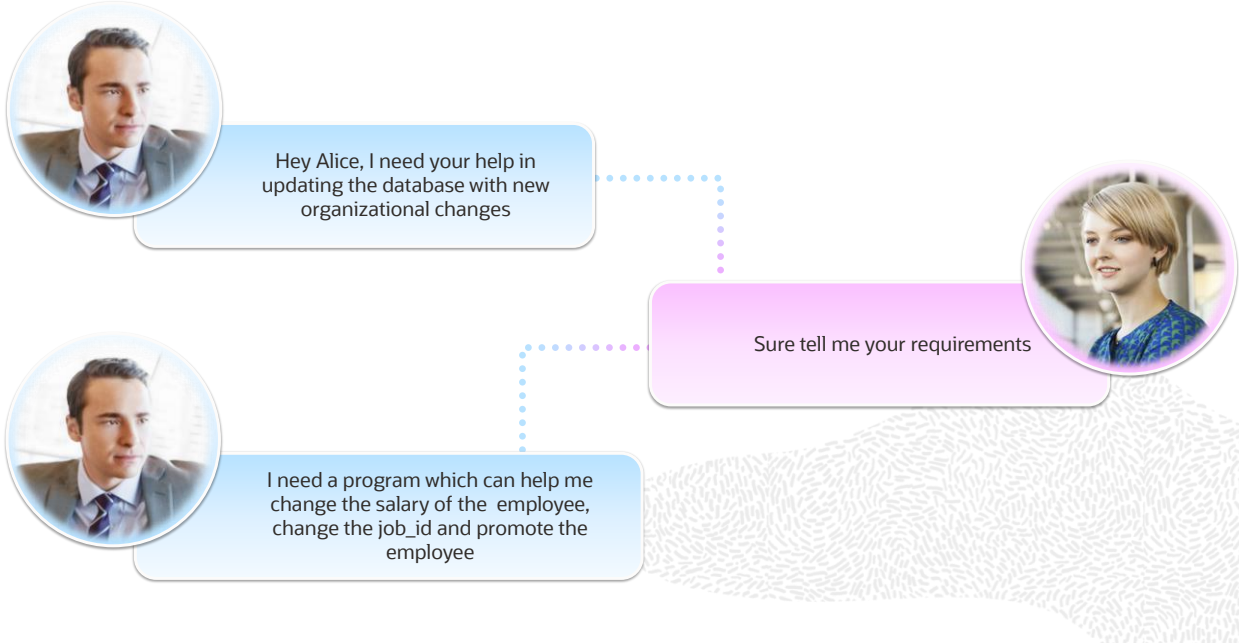
- Modularizing code with PL/SQL
- Working with procedures:
 - Creating and calling procedures
 - Identifying the available parameter-passing modes
 - Using formal and actual parameters
 - Using positional, named, or mixed notation
- Handling exceptions in procedures, removing a procedure, and displaying the procedure's information



For Instructor Use Only.

This document should not be distributed.

Modularized Program Design



5

Here the HR manager comes up with a requirement for which Alice has to write a code. The HR manager wants to put the organizational changes in the database. The organizational changes are:

- **Salary Hike** – Employees get a salary hike but practically every employee wouldn't get a salary hike and even when they get the amount of hike might not be uniform.
- **Promotion** – Employees got a promotion but practically every employee wouldn't get a promotion.

However it is clear that these two functions on the database are independent of each other, because an employee who gets a hike may not get a promotion and vice versa may also be true.

Therefore it would be efficient if Alice implement them as independent modules. When implemented as two independent functions they can be used together also if there's a requirement, a situation where an employee gets both salary hike and promotion.

PL/SQL provides for this kind of modular design where you write a subprogram to implement an independent functionality and when required you can merge the functionality of more than one modules together.

Modularizing Code with PL/SQL

- PL/SQL is a block-structured language. The PL/SQL code block helps modularize code by using:
 - Anonymous blocks
 - Procedures and functions
 - Packages
 - Database triggers

6

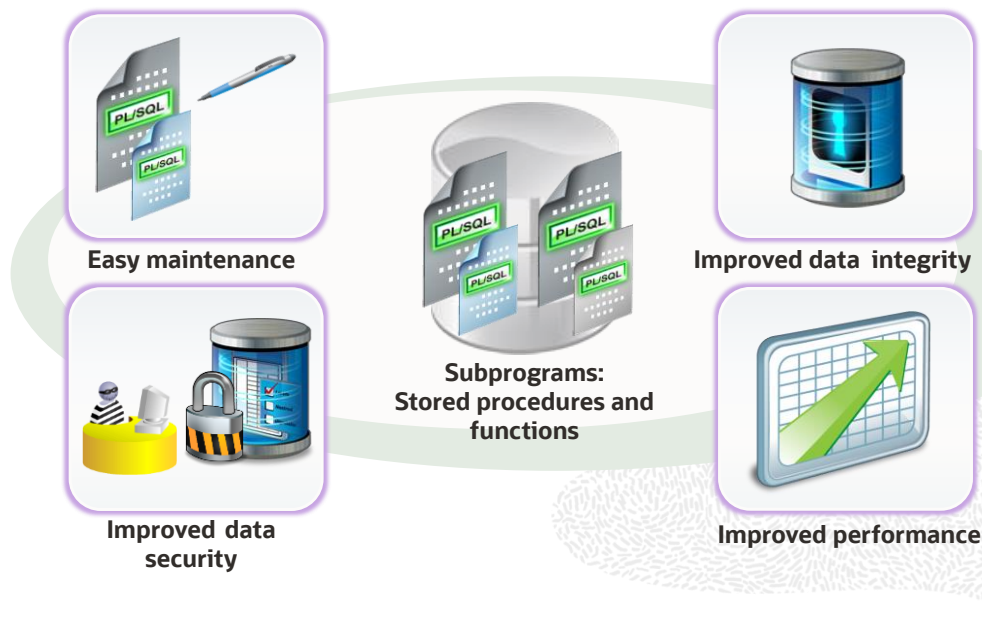
Modularization converts large blocks of code into smaller groups of code called modules. When you create modules, you should ensure that each module is representing a logically independent function of the application. The modules can be reused by the same program or shared with other programs. It is easier to maintain and debug code that comprises of smaller modules than it is to maintain code in a single large program. These modules are implemented as subprograms in PL/SQL.

You create a subprogram with a declarative section, executable section, and an exception section. A PL/SQL block can't exist without an executable section, but it can exist without a declarative section and exception section. Apart from anonymous blocks, all other PL/SQL blocks mentioned in the slide can be compiled and stored in the database as independent objects. As you store them in the database, you can independently invoke them and use them in other PL/SQL blocks, thus reducing the effort of rewriting the same code.

When you reuse a tried and tested PL/SQL subprogram in your application, it brings in certain amount of assured performance into the application.

Note: Knowing how to develop anonymous blocks is a prerequisite for the rest of this course.

Benefits of Modularization



7

Procedures and functions have many benefits due to modularizing of code:

- **Easy maintenance** - When you make changes and recompile the code of the subprogram stored in the database, the changes would be in effect in all the following calls to the subprogram. A change in one place is sufficient, you need not make changes in every module which uses the subprogram.
- **Data integrity** is maintained through subprograms. Each subprogram usually represents a logical function. A subprogram is executed completely or doesn't execute at all which implies a logical function executes completely or it doesn't execute at all, thus leaving your database in a consistent state.
- **Improved performance** is assured when you reuse a tried and tested code. The compiled subprograms in the database therefore ensure better performance in your application.
- **Improved data security** is assured because PL/SQL subprograms reside on the server, where the implementation isn't exposed to the client. The internal details, the structure of the database are hidden from the end user. The client can use the functionality of the sub program but can't modify the internal structure of the database.

What Are PL/SQL Subprograms?

- A PL/SQL subprogram is a named PL/SQL block.
- It enables modularization in the application.
- A subprogram consists of a specification and a body.
- Subprograms can be grouped into PL/SQL packages.



0

8

You write a subprogram to implement a logical function of the application. For example, you can write a subprogram to reflect the promotion of an employee.

Syntactically a subprogram has the following components:

- A subprogram consists of a specification (spec) and a body.
- The specification is the name of the subprogram and the description of parameters if it is defined to accept any.

To define a subprogram, you must provide both the specifications and the body. You can either declare a subprogram first or define it later.

- **Subprogram Types:** PL/SQL has three types of subprograms: procedures, functions, and triggers.
- **Procedures** are used to perform an action on the database.
- **Functions** are used to perform an operation and return a value.
- **Triggers** are used to execute actions in response to some event in the database. Procedures and functions can be invoked in the PL/SQL block to perform an action whereas triggers are defined to respond to an event in the database.

Lesson Agenda

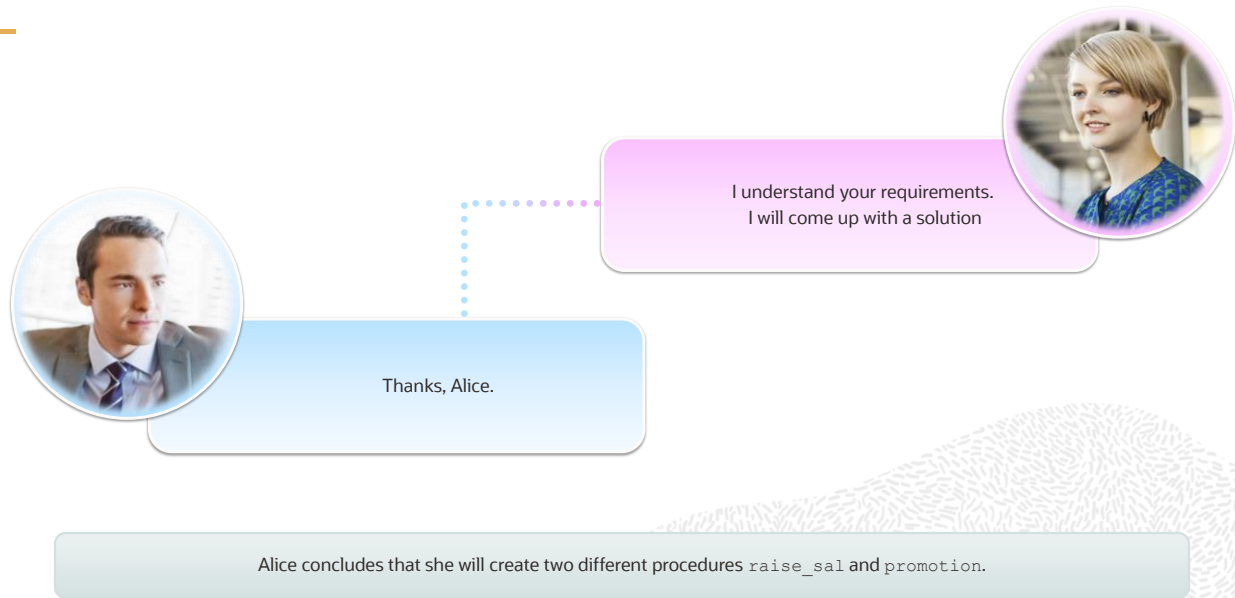
- Modularizing code with PL/SQL
- Working with procedures:
 - Creating and calling procedures
 - Identifying the available parameter-passing modes
 - Using formal and actual parameters
 - Using positional, named, or mixed notation
- Handling exceptions in procedures, removing a procedure, and displaying the procedures' information



For Instructor Use Only.

This document should not be distributed.

Procedures



10

Alice started the task of creating two procedures – `raise_sal` and `promotion`. Let us first understand how to write procedures and then discuss how can we use them to solve this problem.

What Are Procedures?

- A type of subprogram that performs an action
- Can be stored in the database as a schema object
- Promote reusability and maintainability



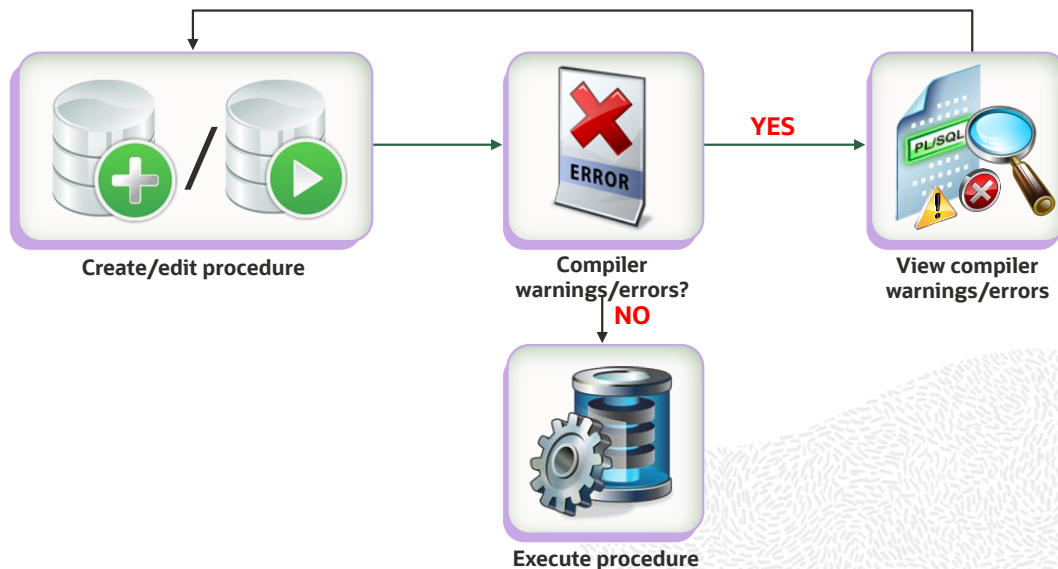
11

A procedure is a named PL/SQL block that can accept parameters (sometimes referred to as arguments). Generally, you use a procedure to perform an action. It has a header, a declaration section, an executable section, and an optional exception-handling section. A procedure is invoked by using the procedure name in the execution section of another PL/SQL block.

A procedure is compiled and stored in the database as a schema object.

Procedures promote reusability and maintainability. When validated, they can be used in any number of applications.

Creating Procedures: Overview



12

To develop a procedure by using a tool such as SQL Developer, perform the following steps:

1. Create the procedure using SQL Developer or SQL Plus.
2. Compile the procedure. The procedure is created in the database on compilation. The `CREATE PROCEDURE` statement creates and stores the source code and the compiled *m-code* in the database. The compilation process can be GUI based as in SQL Developer or as in case of command line interface such as SQL Plus, you can compile the procedure like any other command.
3. If compilation errors exist, then the *m-code* is not generated and you must edit the source code to make corrections. You cannot invoke a procedure that contains compilation errors. You can view the compilation errors in Compiler log of the SQL Developer, SQL*Plus when you compile the code.
4. After successful compilation, execute the procedure to perform the desired action. You can run the procedure using SQL Developer GUI or use the `EXECUTE` command in SQL*Plus.

Creating Procedures

- Use the `CREATE` clause to create a stand-alone procedure that is stored in the Oracle database.
- Use the `OR REPLACE` option to overwrite an existing procedure.

```
CREATE [OR REPLACE] PROCEDURE procedure_name
[(parameter1 [mode] datatype1,
 parameter2 [mode] datatype2, ...)]
IS|AS
[local_variable_declarations; ...]
BEGIN
-- actions;
END [procedure_name];
```

PL/SQL block

13

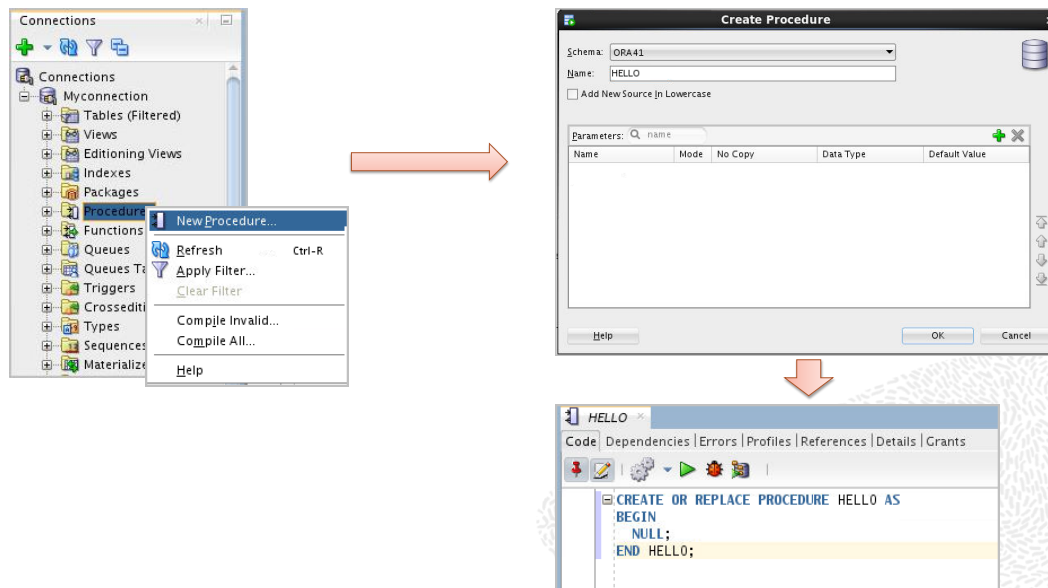
You can use the `CREATE PROCEDURE` SQL statement to create stand-alone procedures that are stored in an Oracle database. A procedure performs a specific action. You specify the name of the procedure, its parameters, its local variables, and the `BEGIN-END` block that contains its code and handles any exceptions.

- Procedures start with `BEGIN`, optionally preceded by the declaration of local variables and end with either `END` or `END procedure_name`.
- The `REPLACE` option indicates that if the procedure exists, it is dropped and replaced with the new version created by the statement. The `REPLACE` option doesn't drop any of the privileges associated with the procedure.

Other Syntactic Elements

- *parameter1* represents the name of a parameter.
- The *mode* option defines how a parameter is used: `IN` (default), `OUT`, or `IN OUT`.
- *datatype1* specifies the parameter data type, without any precision.

Creating Procedures Using SQL Developer



14

To create a procedure by using SQL Developer, perform the following steps:

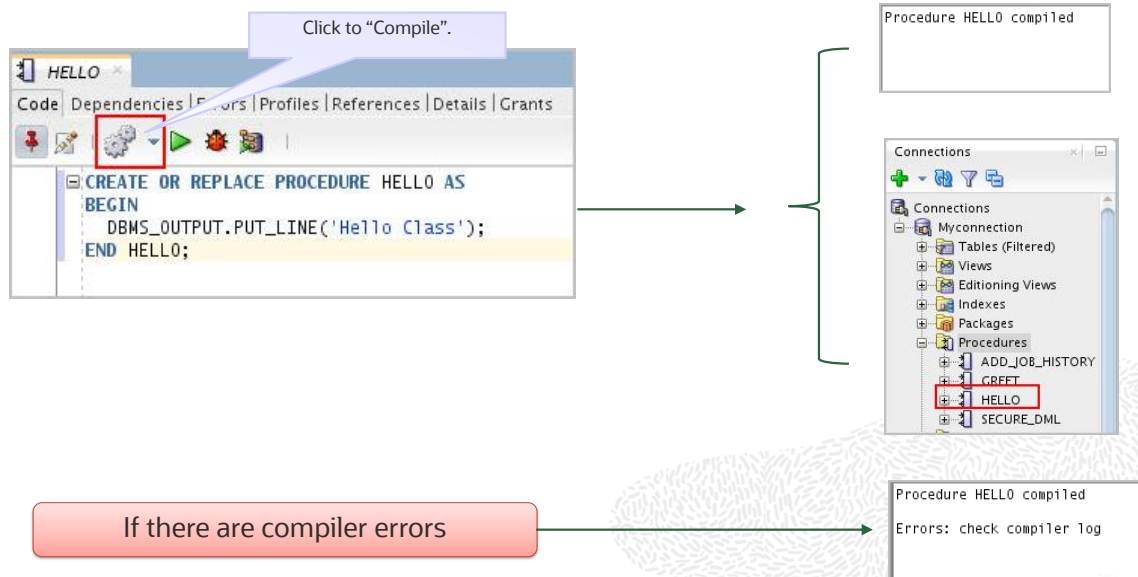
1. Right-click the **Procedures** in the **Connections** tab.
2. Select **New Procedure** from the shortcut menu. The **Create PL/SQL Procedure** dialog box is displayed.
3. Specify the information for the new procedure.
4. Click OK to create the subprogram and have it displayed in the Editor window, where you can enter the details.

Note: You can also write the code of the procedure in a SQL worksheet and execute it like any other SQL statement.

The components of the **Create PL/SQL Procedure** dialog box are as follows:

- **Schema:** The database schema in which to create the PL/SQL subprogram
- **Name:** The name of the subprogram that must be unique within a schema
- **Add New Source in Lowercase:** If this option is selected, new text appears in lowercase regardless of the case in which you enter it. This option affects only the appearance of the code, because PL/SQL isn't case-sensitive in its execution.
- **Parameters tab:** To add a parameter, click the Add (+) icon. For each parameter in the procedure to be created, specify the parameter name, data type, mode, and optionally the default Value. Use the Remove (X) icon and the arrow icons to delete and to move a parameter up or down in the list, respectively.

Compiling Procedures



15

You can compile procedures by clicking the compile button. It provides you with two options: **Compile** and **Compile for Debug**. You choose the appropriate one and run compilation.

If the compilation goes through successfully, you see the response 'Procedure HELLO Compiled' in the **Messages** tab and a Procedure object is created in **Connections** tab.

If there are compiler errors you see the response 'Procedure HELLO Compiled' Errors: check compiler log in the **Messages** tab. You can check the **Compiler Log** for description on errors.

You can fix the errors and recompile the code.

Calling Procedures

- You can call procedures from:
 - Anonymous blocks
 - Another named PL/SQL block
 - Command prompt using `EXECUTE`

```
--invoking procedure from anonymous block
BEGIN
  hello;
END;
/
```

```
--invoking procedure in another procedure
CREATE OR REPLACE PROCEDURE greet IS
BEGIN
  hello;
END greet;
/
```

```
--invoking procedure with EXECUTE
EXECUTE greet;
```

16

You can invoke procedures by using:

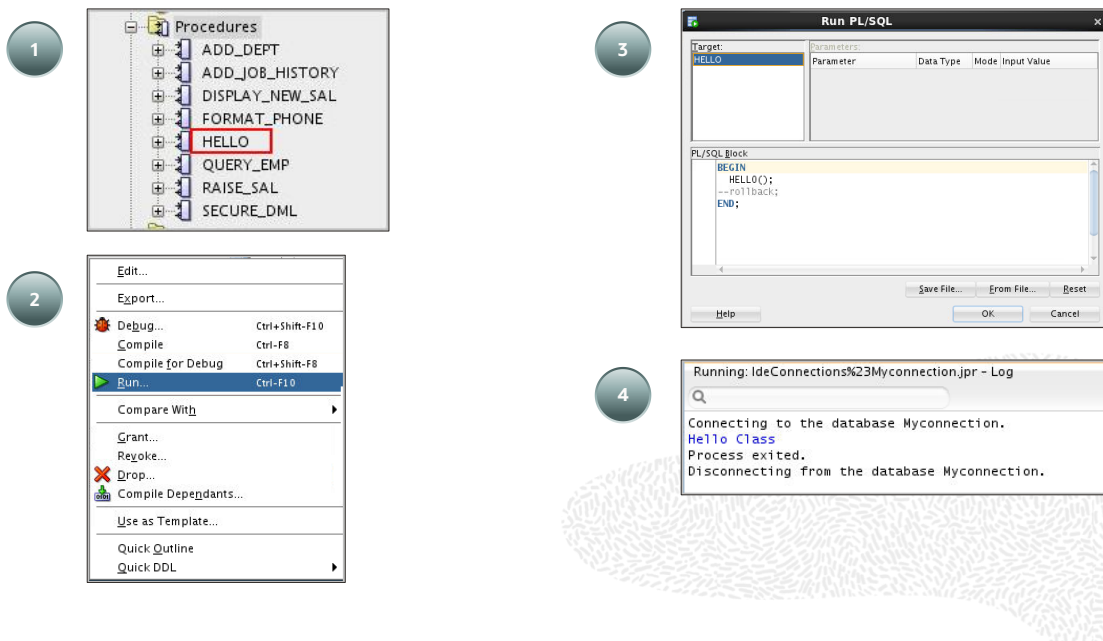
- Anonymous blocks
- Another procedure or PL/SQL subprogram
- `EXECUTE` command with procedure name at the command prompt

Examples in the slide have been illustrated using all the three options.

The example in the slide shows you how to invoke a procedure `hello` from anonymous PL/SQL block. You can also see that the procedure `hello` is invoked from the procedure `greet`.

The third code box shows invoking the procedure from the command prompt using the `EXECUTE` command.

Calling Procedures Using SQL Developer



17

In the slide example, the `hello` procedure is called to raise the current salary of employee 176 (\$ 8,600) by 10 percent as follows:

1. Right-click the procedure name in the **Procedures** node, and then click **Run**. The **Run PL/SQL** dialog box is displayed.
2. You see that the procedure `hello` is enclosed in an anonymous PL/SQL block for execution.
3. Click OK. SQL Developer runs the procedure and displays "Hello Class".

Results		
Results:		
EMPLOYEE_ID	SALARY	
1	176	9460

Procedures

```
CREATE OR REPLACE PROCEDURE raise_sal AS
v_raise_percent NUMBER := 10;
BEGIN
UPDATE employees
SET
salary = salary * (1+1/v_raise_percent);
END salary_hike;
```

Alice created the procedure `raise_sal`, but she had questions for the HR manager before creating the `promotion` procedure



There is a problem here Alice!

I have created this procedure where you can increment the salary of all the employees by 10%. You can change the percentage to some other value too



18

Alice here created a procedure `raise_sal` which would update the salary of all the employees by 10%, where you can change the percentage of hike by initializing the variable with a different value. However things don't work like this practically. The increment an employee gets depend on factors like employee's performance, the job he/she hold and other factors and remember every employee may not get an increment.

Our HR manager expressed the same to Alice. Now Alice has to modify this procedure to accept input such as `employee_id` and raise percentage as input and perform the expected operation on the database.

We will further look into the concept of parameters. Parameters enable you to provide a different input each time you execute the procedure. Alice can use parameters to pass input to the increment procedure and perform the required operations.

What Are Parameters and Parameter Modes?

- Means of communication between the calling environment and the Procedure
- Each parameter is associated with a mode:
 - An `IN` parameter mode (the default) provides values for a subprogram to process.
 - An `OUT` parameter mode returns a value to the caller.
 - An `IN OUT` parameter mode supplies an input value, which may be returned (output) as a modified value.

19

Parameters are used to transfer data values among the calling environment and the procedure (or subprogram). Parameters are declared in the subprogram header, after the name and before the declaration section for local variables.

Parameters are defined with one of the three parameter-passing modes: `IN`, `OUT`, or `IN OUT`.

- An `IN` parameter passes a constant value from the calling environment into the procedure.
- An `OUT` parameter passes a value from the procedure to the calling environment.
- An `IN OUT` parameter passes a value from the calling environment to the procedure and a possibly different value from the procedure back to the calling environment using the same parameter.

Formal and Actual Parameters

- Formal parameters are place holders for data in a subprogram specification.
- Actual parameters are literal values, variables or expressions substituted in the parameter list while invoking the subprogram.

```
-- Procedure definition Formal parameters
CREATE PROCEDURE raise_sal(p_id NUMBER) IS
BEGIN
  . . .
END raise_sal;
```

```
--Procedure calling, Actual parameters (arguments)
execute raise_sal(100)
```



20

Formal parameters are declared in the parameter list of a subprogram specification. In the procedure definition of `raise_sal`, the variable `p_id` identifier represent the formal parameter.

The actual parameters can be literal values, variables, and expressions that are provided in the parameter list while calling a procedure. In the procedure call, the `raise_sal` procedure is invoked with a value of 100, which is the actual parameter.

Actual parameters:

- Are associated with formal parameters during the subprogram call
- Can also be expressions, as in the following example:

```
raise_sal(v_emp_id, raise+100);
```

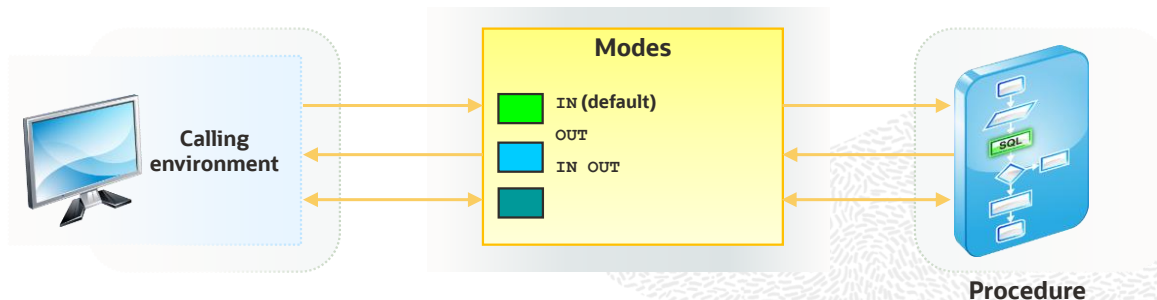
where `v_emp_id` and `raise` are variables with some initialized values

The formal and actual parameters should be of compatible data types. If necessary, before assigning the value, PL/SQL converts the data type of the actual parameter value to that of the formal parameter.

Procedural Parameter Modes

- Parameter modes are specified in the formal parameter declaration, after the parameter name and before its data type.
- The `IN` mode is the default if no mode is specified.

```
CREATE PROCEDURE proc_name(param_name [mode] datatype,...)  
...
```



21

The parameter mode `IN` is the default passing mode—that is, if no mode is specified with a parameter declaration, the parameter is considered to be an `IN` parameter. The parameter modes `OUT` and `IN OUT` must be explicitly specified in their parameter declarations.

The *datatype* is specified without a size specification. It can be specified:

- As an explicit data type
- Using the `%TYPE` definition
- Using the `%ROWTYPE` definition

Note: One or more formal parameters can be declared, each separated by a comma.

Comparing the Parameter Modes

IN	OUT	IN OUT
It is the default mode.	It must be specified.	It must be specified.
Value is passed into subprogram.	Value is returned to the calling environment.	Value is passed into sub-program; value is returned to calling environment.
Formal parameter acts as a constant.	It is uninitialized variable.	It is initialized variable.
Actual parameter can be a literal, expression, constant, or initialized variable.	It must be a variable.	It must be a variable.
It can be assigned a default value.	It cannot be assigned a default value.	It cannot be assigned a default value.

22

The **IN** parameter mode is the default mode if no mode is specified in the declaration. The **OUT** and **IN OUT** parameter modes must be explicitly specified with the parameter declaration.

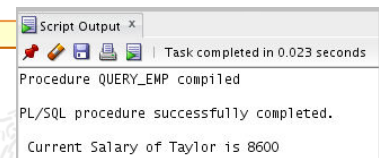
A formal parameter of **IN** mode cannot be assigned a value and cannot be modified in the body of the procedure. By default, the **IN** parameter is passed by reference. An **IN** parameter can be assigned a default value in the formal parameter declaration, in which case the caller need not provide a value for the parameter if the default applies.

An **OUT** or **IN OUT** parameter must be assigned a value before returning to the calling environment. The **OUT** and **IN OUT** parameters cannot be assigned default values.

Using the IN Parameter Mode: Example

```
CREATE OR REPLACE PROCEDURE query_emp
(p_id IN employees.employee_id%TYPE) AS
  v_name employees.last_name%TYPE;
  v_salary employees.salary%TYPE;
BEGIN
  SELECT last_name, salary INTO v_name,v_salary
  FROM employees
  WHERE employee_id = p_id;
  DBMS_OUTPUT.PUT_LINE(' Current Salary of '||v_name || ' is '|| v_salary);
END query_emp;
```

```
EXECUTE query_emp(176)
```



23

The slide shows an example for using IN mode of procedure parameters. IN mode implies input to the procedure.

In the example, the procedure accepts an employee ID as input parameter. This input parameter is used to retrieve data from the employees table inside the procedure and display the current value of the employee salary.

Here in the slide we invoke the procedure by using the EXECUTE command.

You can also invoke a procedure from another procedure by making a direct call inside an executable section of the calling block. Invoke the procedure with actual parameters in the PL/SQL block as shown below:

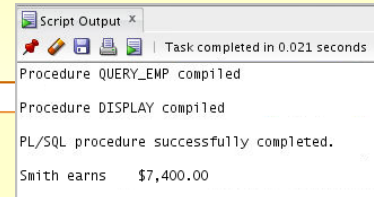
```
...
BEGIN
    query_emp(176);
END;
```

Note: IN parameters are passed as read-only values from the calling environment into the procedure. Attempts to change the value of an IN parameter result in a compile-time error.

Using the OUT Parameter Mode: Example

```
CREATE OR REPLACE PROCEDURE query_emp
(p_id      IN  employees.employee_id%TYPE,
p_name     OUT employees.last_name%TYPE,
p_salary  OUT employees.salary%TYPE) IS
BEGIN
  SELECT  last_name, salary INTO p_name, p_salary
  FROM    employees
  WHERE   employee_id = p_id;
END query_emp;
```

```
CREATE OR REPLACE PROCEDURE display AS
  v_emp_name employees.last_name%TYPE;
  v_emp_sal  employees.salary%TYPE;
BEGIN
  query_emp(171, v_emp_name, v_emp_sal);
  DBMS_OUTPUT.PUT_LINE(v_emp_name||' earns '|| to_char(v_emp_sal, '$999,999.00'));
END display;
EXECUTE display;
```



24

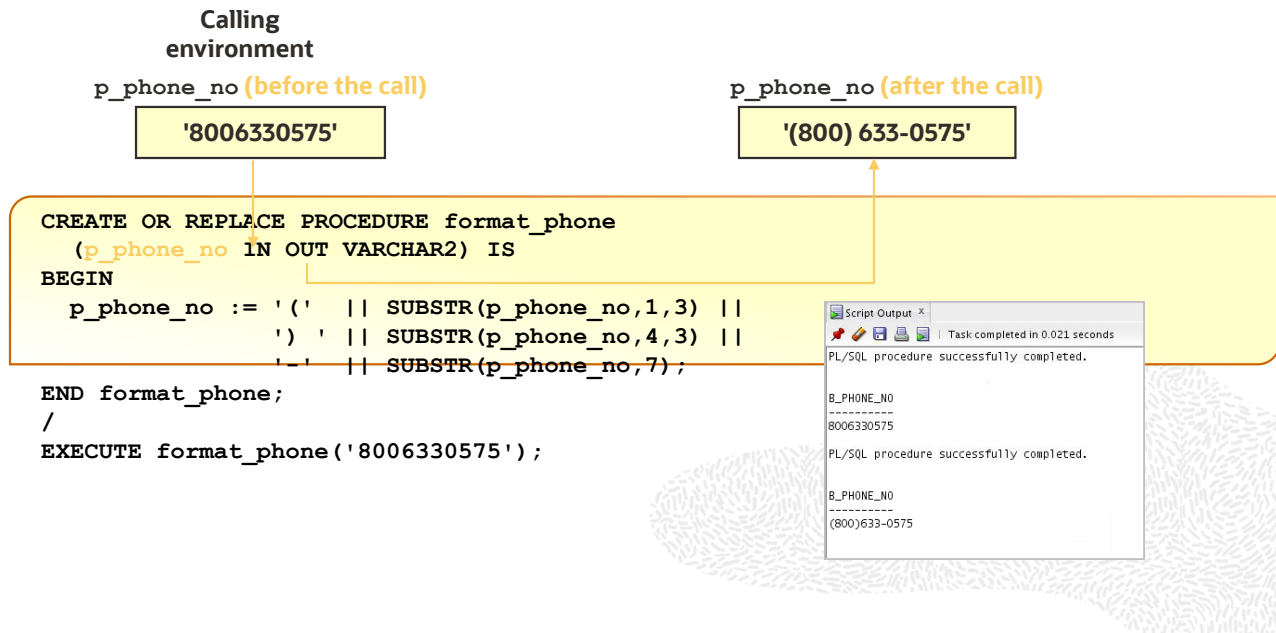
The slide shows the usage of OUT parameters in a procedure.

The first piece of code is the definition of a procedure `query_emp` which has one parameter set to IN mode and two parameters set to OUT mode. The parameter set to IN mode is the value of the `employee_id` and the parameters set to OUT mode are the values of `last_name` and `salary` of the employee. We use this procedure in another procedure `display` defined in the second code box.

The `query_emp` procedure is invoked in the `display` procedure by passing only the input parameter. There are place holders for the output parameter, declared as variables in the `display` procedure. The output parameters of the `query_emp` procedure are retrieved into the `display` procedure and used in the output message.

Note: Make sure that the data type for the actual parameter variables used to retrieve values from the OUT parameters has a size sufficient to hold the data values being returned.

Using the IN OUT Parameter Mode: Example



25

Using an IN OUT parameter, you can pass a value into a procedure that can be updated and returned as output. The actual parameter value supplied from the calling environment can return either the original unchanged value or a new value that is set within the procedure.

The slide example creates a procedure with an IN OUT parameter to accept a character string containing digits for a phone number. The procedure returns the phone number formatted with parentheses around the first three characters and a hyphen after the sixth digit; for example, the phone string 8006330575 is returned as (800) 633-0575.

The following code uses the b_phone_no host variable of SQL*Plus to provide the input value passed to the FORMAT_PHONE procedure. The procedure is executed and returns an updated string in the b_phone_no host variable. The output of the following code is displayed in the slide:

```
VARIABLE b_phone_no VARCHAR2(15)
EXECUTE :b_phone_no := '8006330575'
PRINT b_phone_no
EXECUTE format_phone (:b_phone_no)
PRINT b_phone_no
```


Passing Parameters to Procedures

- When calling a subprogram, you can write the actual parameters using the following notations:
 - **Positional:** Lists the actual parameters in the same order as the formal parameters
 - **Named:** Lists the actual parameters in arbitrary order and uses the association operator (`=>`) to associate a named formal parameter with its actual parameter
 - **Mixed:** Lists some of the actual parameters as positional and some as named

26

When calling a subprogram, you can write the actual parameters by using the following notations:

- **Positional:** You list the actual parameter values in the same order in which the formal parameters are declared. This notation is compact, but if you specify the parameters (especially literals) in the wrong order, the procedure execution may give wrong results or it might result in an error.
- **Named:** You list the actual values in an arbitrary order and use the association operator to associate each actual parameter with its formal parameter by name. The PL/SQL **association operator** is an “equal” sign followed by an “is greater than” sign, without spaces: `=>`. The order of the parameters is not significant. This notation is more verbose, but makes your code easier to read and maintain. You can sometimes avoid changing your code if the procedure’s parameter list changes, for example, if the parameters are reordered or a new optional parameter is added.
- **Mixed:** You list the first parameter values by their position and the remainder by using the special syntax of the named method. You can use this notation to call procedures that have some required parameters, followed by some optional parameters.

Passing Actual Parameters: Creating the `raise_sal` Procedure

```
CREATE OR REPLACE PROCEDURE raise_sal
(p_id IN employees.employee_id%TYPE,
p_percent IN NUMBER) AS
BEGIN
  UPDATE employees
  SET salary = salary * (1 + p_percent/100)
  WHERE employee_id = p_id;
END raise_sal;
```



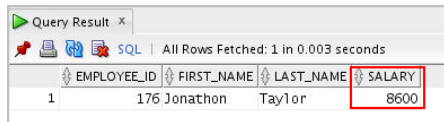
27

You can see in the slide that Alice has modified the `raise_sal` procedure to accept input parameters. The procedure accepts the `employee_id` and the percentage of salary raise that has to be given to the specific employee.

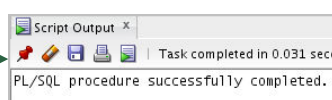
Based on the values of the input parameters an `UPDATE` command is executed in the executable section of the `raise_sal` procedure.

Passing Actual Parameters: Examples

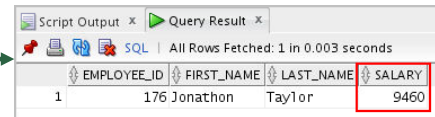
```
-- Passing parameters using the positional notation.  
SELECT * FROM EMPLOYEES WHERE employee_id = 176;  
EXECUTE raise_sal ('176', 10)  
SELECT * FROM EMPLOYEES WHERE employee_id = 176;
```



EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
1	176 Jonathon	Taylor	8600



PL/SQL procedure successfully completed.



EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
1	176 Jonathon	Taylor	9460

```
-- Passing parameters using the named notation.  
EXECUTE raise_sal( p_percent =>10,p_id => 176)
```



Thanks Alice. This will help. Please find a solution to update the promotion data also in the database

28

Passing actual parameters by position is shown in the first call to execute `raise_sal` in the first code example in the slide. The first actual parameter supplies the value 176 for the `p_id` formal parameter. The second actual parameter value of 10 is assigned by position to the `p_percent` formal parameter.

Passing parameters using the named notation is shown in the second code example in the slide. The `p_percent` actual parameter, which is declared as the second formal parameter, is referenced by name in the call, where it is associated with the actual value of 10. The `p_id` parameter is associated with the value 176. The order of the actual parameters is irrelevant if all parameter values are specified with the names.

Note: You must provide a value for each parameter unless the formal parameter is assigned a default value. We discuss specifying default values for formal parameters in the following slides.

Now the HR manager can either execute multiple `EXECUTE` commands for updating the salaries of different employees or ask Alice to write a PL/SQL block to for all the UPDATES by providing the employee ids and their corresponding salary raise percentage.

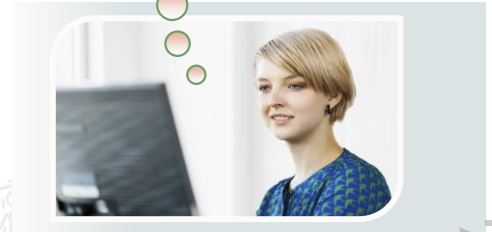
Using the DEFAULT Option for the Parameters

- Defines default values for parameters
- Provides flexibility by combining the positional and named parameter-passing syntax

```
CREATE OR REPLACE PROCEDURE raise_sal
(p_id IN employees.employee_id%TYPE,
 p_percent IN NUMBER DEFAULT 10) AS
BEGIN
  UPDATE employees
  SET salary = salary * (1 + p_percent/100)
  WHERE employee_id = p_id;
END raise_sal;
```

```
EXECUTE raise_sal(176) or
EXECUTE raise_sal(176,10)
EXECUTE raise_sal (p_id => 176)
```

An enhancement !



29

This is an enhancement provided by Alice where the HR manager can apply a default value for raise percentage. Wherever there are exceptions to the default value the HR manager can explicitly enter the value while invoking the procedure.

You can assign a default value to an `IN` parameter through one of the following options:

- Through an assignment operator (`:=`)
- Through `DEFAULT` option, as shown for the `p_percent` parameter in the slide

When default values are assigned to formal parameters, you can call the procedure without supplying an actual parameter value for the parameter. Thus, you can pass different numbers of actual parameters to a subprogram, either by accepting or by overriding the default values as required.

Note: You cannot assign default values to the `OUT` and `IN OUT` parameters.

The second code box in the slide shows three ways of invoking the `raise_sal` procedure:

- The first example assigns the default value for the `p_percent` parameter.
- The second example illustrates assigning values to both the parameters
- The last example uses the named notation to assign value to the parameter `p_id`

Lesson Agenda

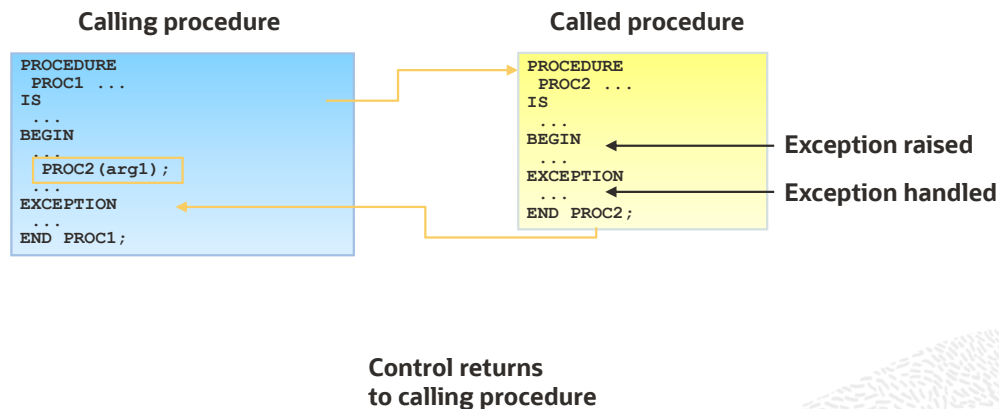
- Modularizing code with PL/SQL
- Working with procedures:
 - Creating and calling procedures
 - Identifying the available parameter-passing modes
 - Using formal and actual parameters
 - Using positional, named, or mixed notation
- Handling exceptions in procedures, removing a procedure, a procedure's information



For Instructor Use Only.

This document should not be distributed.

Handled Exceptions



31

When you develop procedures that are called from other procedures, you should be aware of the effects that handled and unhandled exceptions have on the transaction and the calling procedure.

When an exception is raised in a called procedure, the control immediately goes to the exception section of that block. An exception is considered handled if the exception section provides a handler for the exception raised.

When an exception occurs and is handled, the following code flow takes place:

1. The exception is raised.
2. Control is transferred to the exception handler.
3. The block is terminated.
4. The calling program/block continues to execute unaffected.

If a transaction was started in the calling block, then the transaction is unaffected. A DML operation performed within the procedure before the exception is rolled back.

Note: You can explicitly end a transaction by executing a `COMMIT` or `ROLLBACK` operation in the exception section.

Handled Exceptions: Example

```
CREATE PROCEDURE add_department(  
  p_name VARCHAR2, p_mgr NUMBER, p_loc NUMBER) IS  
BEGIN  
  INSERT INTO DEPARTMENTS (department_id,  
    department_name, manager_id, location_id)  
  VALUES (DEPARTMENTS_SEQ.NEXTVAL, p_name, p_mgr, p_loc);  
  DBMS_OUTPUT.PUT_LINE('Added Dept: ' || p_name);  
EXCEPTION  
  WHEN OTHERS THEN  
    DBMS_OUTPUT.PUT_LINE('Err: adding dept: ' || p_name);  
END;
```

```
CREATE PROCEDURE create_departments IS  
BEGIN  
  add_department('Media', 100, 1800);  
  → add_department('Editing', 99, 1800);  
  add_department('Advertising', 101, 1800);  
END;
```



32

0

The two procedures in the slide are the following:

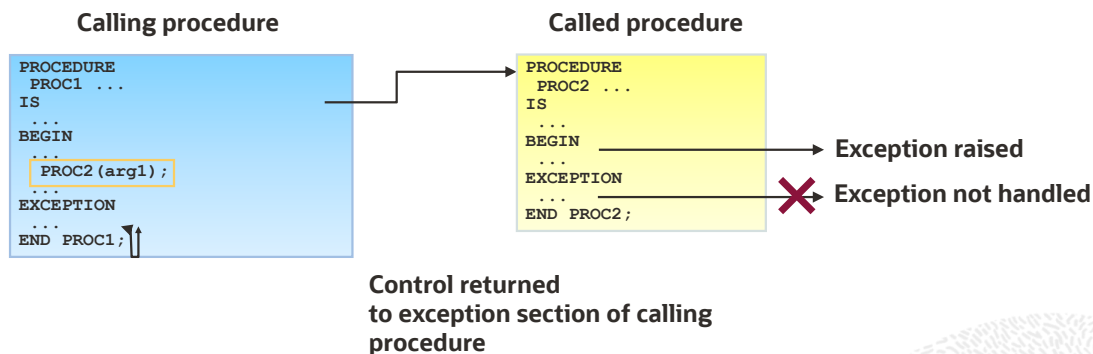
- The `add_department` procedure creates a new department record by allocating a new department number from an Oracle sequence, and sets the `department_name`, `manager_id`, and `location_id` column values using the `p_name`, `p_mgr`, and `p_loc` parameters, respectively.
- The `create_departments` procedure creates more than one department by using calls to the `add_department` procedure.

The `add_department` procedure catches all raised exceptions in its own handler. When `create_departments` is executed, the following output is generated:

```
PL/SQL procedure successfully completed.  
  
Added Dept: Media  
Err: adding dept: Editing  
Added Dept: Advertising
```

The `Editing` department with a `manager_id` of 99 is not inserted because a foreign key integrity constraint violation on `manager_id` ensures that no manager has an ID of 99. Because the exception was handled in the `add_department` procedure, the `create_department` procedure continues to execute. A query on the `DEPARTMENTS` table where the `location_id` is 1800 shows that `Media` and `Advertising` are added but the `Editing` record is not.

Exceptions Not Handled



33

As discussed earlier, when an exception is raised in a called procedure, control immediately goes to the exception section of that block. If the exception section does not provide a handler for the raised exception, then it is not handled. The following code flow occurs:

1. The exception is raised.
2. The block terminates because no exception handler exists; any DML operations performed within the procedure are rolled back.
3. The exception propagates to the exception section of the calling procedure—that is, control is returned to the exception section of the calling block, if one exists.

If an exception is not handled, then all the DML statements in the calling procedure and the called procedure are rolled back along with any changes to any host variables.

Exceptions Not Handled: Example

```
SET SERVEROUTPUT ON
CREATE PROCEDURE add_department_noex(
  p_name VARCHAR2, p_mgr NUMBER, p_loc NUMBER) IS
BEGIN
  INSERT INTO DEPARTMENTS (department_id,
    department_name, manager_id, location_id)
  VALUES (DEPARTMENTS_SEQ.NEXTVAL, p_name, p_mgr, p_loc);
  DBMS_OUTPUT.PUT_LINE('Added Dept: ' || p_name);
END;
```

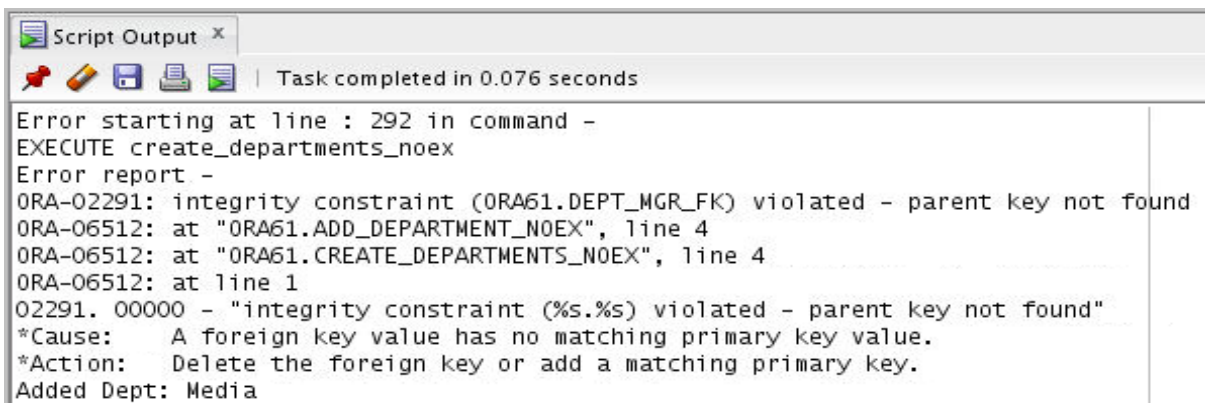
```
CREATE PROCEDURE create_departments_noex IS
BEGIN
  add_department_noex('Media', 100, 1800);
  add_department_noex('Editing', 99, 1800);
  add_department_noex('Advertising', 101, 1800);
END;
/
EXECUTE create_departments_noex
```

XX

34

0

The code example in the slide shows `add_department_noex`, which does not have an exception section. In this case, the exception occurs when the `Editing` department is added. Because of the lack of exception handling in either of the subprograms, no new department records are added into the `DEPARTMENTS` table. Executing the `create_departments_noex` procedure produces a result that is similar to the following:



The screenshot shows the 'Script Output' window in SQL Developer. The title bar says 'Script Output x'. Below the title bar, there are icons for a red pushpin, a yellow pencil, a blue folder, a green document, and a blue document with a magnifying glass. To the right of these icons, it says 'Task completed in 0.076 seconds'. The main area of the window contains the following text:

```
Error starting at line : 292 in command -
EXECUTE create_departments_noex
Error report -
ORA-02291: integrity constraint (ORA61.DEPT_MGR_FK) violated - parent key not found
ORA-06512: at "ORA61.ADD_DEPARTMENT_NOEX", line 4
ORA-06512: at "ORA61.CREATE_DEPARTMENTS_NOEX", line 4
ORA-06512: at line 1
02291. 00000 - "integrity constraint (%s.%s) violated - parent key not found"
*Cause:      A foreign key value has no matching primary key value.
*Action:     Delete the foreign key or add a matching primary key.
Added Dept: Media
```

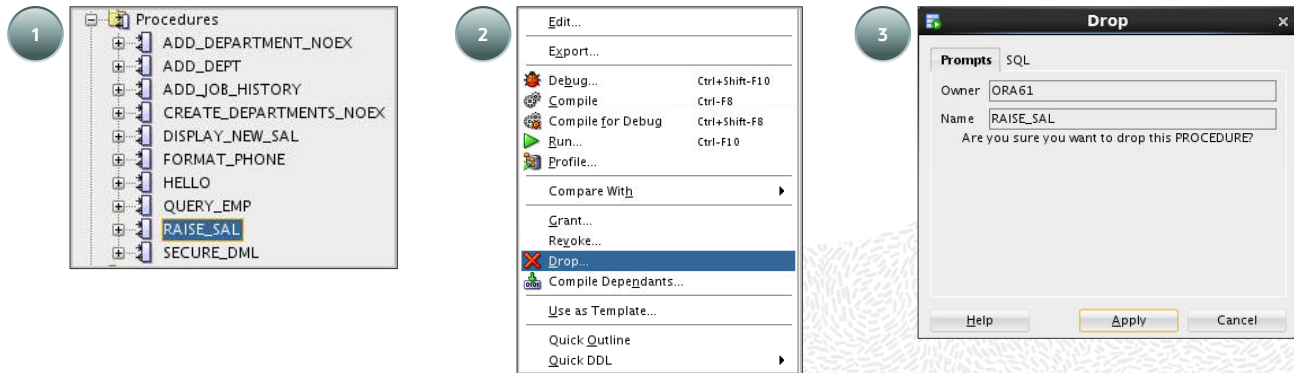
Although the results show that the `Media` department was added, its operation is rolled back because the exception was not handled in either of the subprograms invoked.

Removing Procedures: Using the DROP SQL Statement or SQL Developer

- Using the DROP statement:

```
DROP PROCEDURE raise_sal;
```

- Using SQL Developer:



35

When a stored procedure is no longer required, you can use the `DROP PROCEDURE` SQL statement followed by the procedure's name to remove it as follows:

```
DROP PROCEDURE procedure_name
```

You can also use SQL Developer to drop a stored procedure as follows:

- Right-click the procedure name in the **Procedures** node, and then click **Drop**. The **Drop** dialog box is displayed.
- Click **Apply** to drop the procedure.

Note

- Whether successful or not, executing a data definition language (DDL) command such as `DROP PROCEDURE` commits any pending transactions that cannot be rolled back.
- You might have to refresh the **Procedures** node before you can see the results of the drop operation. To refresh the **Procedures** node, right-click the procedure name in the **Procedures** node, and then click **Refresh**.

Viewing Procedure Information Using the Data Dictionary Views

```
DESCRIBE user_source
```

```
SELECT text
FROM   user_source
WHERE  name = 'RAISE_SAL' AND type = 'PROCEDURE'
ORDER BY line;
```

Name	Null	Type
NAME		VARCHAR2(128)
TYPE		VARCHAR2(12)
LINE		NUMBER
TEXT		VARCHAR2(4000)
ORIGIN_CON_ID		NUMBER

TEXT
1 PROCEDURE raise_sal
2 (p_id IN employees.employee_id%TYPE,
3 p_percent IN NUMBER) AS
4 BEGIN
5 UPDATE employees
6 SET salary = salary * (1 + p_percent/100)
7 WHERE employee_id = p_id;
8 END raise_sal;

36

The source code for PL/SQL subprograms is stored in the data dictionary tables. The source code is accessible to PL/SQL procedures that are successfully or unsuccessfully compiled. To view the PL/SQL source code stored in the data dictionary, execute a `SELECT` statement on the following tables:

- The `USER_SOURCE` table to display PL/SQL code that you own
- The `ALL_SOURCE` table to display PL/SQL code to which you have been granted the `EXECUTE` right by the owner of that subprogram code

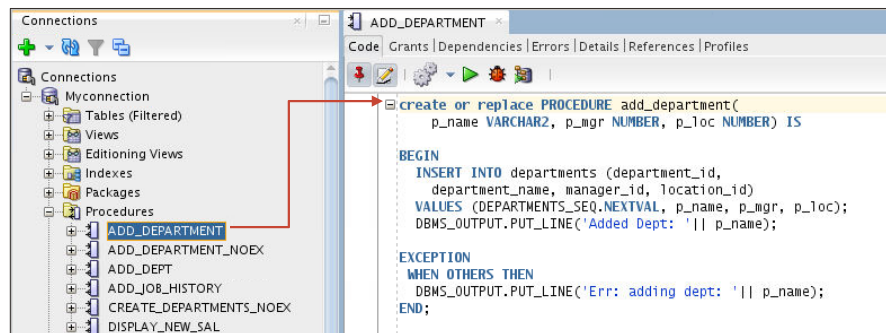
The query example shows all the columns provided by the `USER_SOURCE` table:

- The `NAME` column holds the name of the subprogram in uppercase text.
- The `TYPE` column holds the subprogram type, such as `PROCEDURE` or `FUNCTION`.
- The `LINE` column stores the line number for each source code line.
- The `TEXT` column holds PL/SQL source code.
- The `ORIGIN_CON_ID` holds the ID of the container where the data originates. The value of the container id can range from 0-n based on database configuration.

The `ALL_SOURCE` table provides an `OWNER` column in addition to the preceding columns.

Note: You cannot display the source code for Oracle PL/SQL built-in packages, or PL/SQL whose source code has been wrapped by using a `WRAP` utility. The `WRAP` utility converts the PL/SQL source code into a form that cannot be deciphered by humans.

Viewing Procedures Information Using SQL Developer



37

To view a procedure's code in SQL Developer, perform the following steps:

1. Click the **Procedures** node in the **Connections** tab.
2. Click the procedure's name.
3. The procedure's code is displayed in the **Code** tab as shown in the slide.

Quiz



Which of the following is an invalid procedure call for a procedure?

```
ADD(a IN NUMBER DEFAULT 10, b IN NUMBER DEFAULT 20, result OUT  
NUMBER)
```

- a. ADD
- b. ADD (ex)
- c. ADD (10,20,ex)
- d. ADD (b=>45, a=>35, ex)

38

Answer: a

Options b and c and show procedure calls with the parameter specified through `OUT` mode but option a does have the parameter with the `OUT` mode as defined in the procedure specification.

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Identify the benefits of modularized and layered subprogram design
- Create and call procedures
- Use formal and actual parameters
- Use positional, named, or mixed notation for passing parameters
- Identify the available parameter-passing modes
- Handle exceptions in procedures
- Remove a procedure and display its information



Practice 11-1 Overview: Creating a New Database Connection

This practice covers the following topics:

- Creating a new Database Connection to:
 - The user ora61 which will be named Myconnection



40

Practice 11: Overview

In this practice, you create, compile, and invoke procedures that issue DML and query commands. You also learn how to handle exceptions in procedures.

If you encounter compilation errors when you execute procedures, you can use the **Compiler-Log** tab in SQL Developer.

Note: It is recommended to use SQL Developer for this practice.

Important

All practices in this course and the practice solutions assume that you create objects such as procedures, functions, and so on using the SQL Worksheet area in SQL Developer. When you create an object in the SQL Worksheet area, you need to refresh the object node in order for the new object to be displayed in the Navigator tree. To compile the newly created object, you can right-click the object name in the Navigator tree, and then select **Compile** from the shortcut menu. For example, after you enter the code to create a procedure in the SQL Worksheet area, click the **Run Script** icon (or **press F5**) to run the code. This creates and compiles the procedure.

Alternatively, you can create objects such as procedures using the **PROCEDURES** node in the **Navigator tree**, and then compile the procedure. Creating objects using the **Navigator tree** automatically displays the newly created object.

Practice 11-2 Overview: Creating, Compiling, and Calling Procedures

This practice covers the following topics:

- Creating stored procedures to:
 - Insert new rows into a table using the supplied parameter values
 - Update data in a table for rows that match the supplied parameter values
 - Delete rows from a table that match the supplied parameter values
 - Query a table and retrieve data based on supplied parameter values
- Handling exceptions in procedures
- Compiling and invoking procedures



41

Practice 11: Overview

In this practice, you create, compile, and invoke procedures that issue DML and query commands. You also learn how to handle exceptions in procedures.

If you encounter compilation errors when you execute procedures, you can use the **Compiler-Log** tab in SQL Developer.

Note: It is recommended to use SQL Developer for this practice.

Important

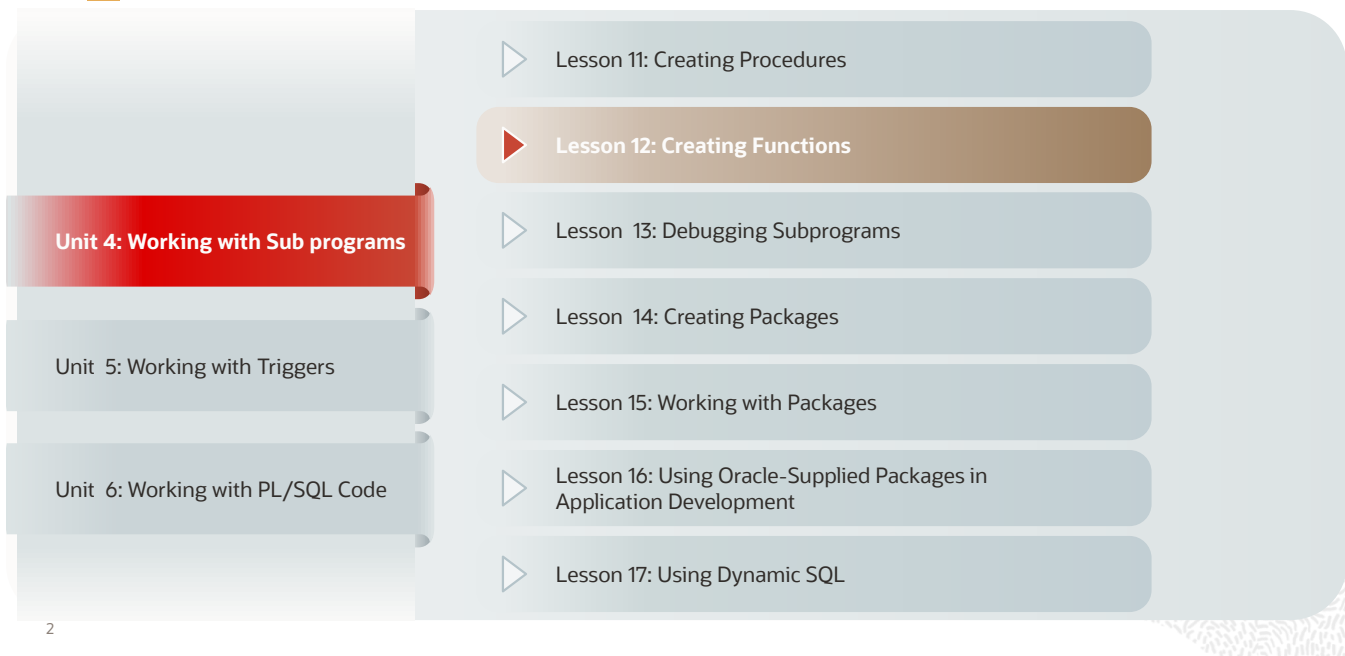
All practices in this course and the practice solutions assume that you create objects such as procedures, functions, and so on using the SQL Worksheet area in SQL Developer. When you create an object in the SQL Worksheet area, you need to refresh the object node in order for the new object to be displayed in the Navigator tree. To compile the newly created object, you can right-click the object name in the Navigator tree, and then select **Compile** from the shortcut menu. For example, after you enter the code to create a procedure in the SQL Worksheet area, click the **Run Script** icon (or **press F5**) to run the code. This creates and compiles the procedure.

Alternatively, you can create objects such as procedures using the **PROCEDURES** node in the **Navigator tree**, and then compile the procedure. Creating objects using the **Navigator tree** automatically displays the newly created object.

For Instructor Use Only.
This document should not be distributed.

Creating Functions

Course Road Map



2

In the fourth unit, we have seven lessons – Creating Stored Procedures, Creating Functions, Debugging Subprograms, Creating Packages, Working with Packages, Using Oracle Supplied Packages in Application Development, and Using Dynamic SQL. You will learn to write PL/SQL subprograms, packages, and use them in Application Development.

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to:

- Differentiate between a procedure and a function
- Describe the uses of functions
- Create stored functions
- Invoke a function
- Remove a function
- Explain the basic functionality of the SQL Developer debugger



3

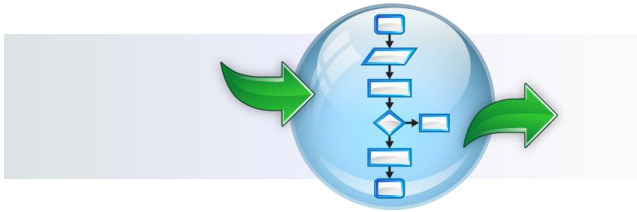
In this lesson, you learn how to create, invoke, and maintain functions.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Creating and invoking functions
- Functions in SQL expressions
- Passing parameters to functions
- Removing stored functions

Functions



- Named PL/SQL block like procedure
- Accepts input as parameters
- Processes the input and returns a value
- Stored as a database object
- Enables modularization in the application
- Can be invoked as part of an expression

5

You know about procedures: they are named PL/SQL blocks stored on the database. Functions are similar to procedures in most of its features except for the fact that they return a single value.

Functions are named PL/SQL blocks that are used to implement modularization in the application. You should define functions to represent a logical functionality of the application. Procedures do the same job except for the fact that functions return a value to the calling environment. In case of procedures, you can see the output of the procedure by accessing the `OUT` parameters of the procedure and procedures may or may not have an output, but functions will always return a value.

Each function has a `RETURN` statement in its PL/SQL block. There can be more than one return statements in a PL/SQL block; however, a function will execute only one of these return statements based on the function logic.

Creating Functions syntax

The PL/SQL block must have at least one `RETURN` statement.

```
CREATE [OR REPLACE] FUNCTION function_name
[(parameter1 [mode1] datatype1, . . .)]
RETURN datatype IS|AS
[local_variable_declarations;
. . .]
BEGIN
-- actions;
RETURN expression;
END [function_name];
```

6

Syntax for Creating Functions

A function is a PL/SQL block that returns a value. A `RETURN` statement must be provided to return a value with a data type that is consistent with the function declaration.

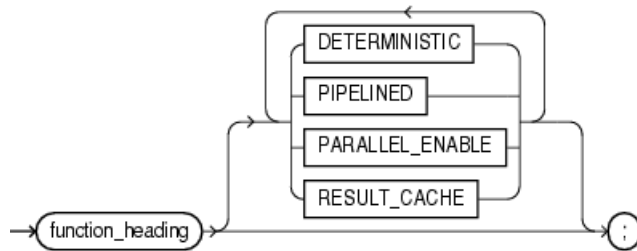
You create new functions with the `CREATE FUNCTION` statement, which may declare a list of parameters, must return one value, and must define the actions to be performed by the standard PL/SQL block.

The type of value returned from the function can be a scalar data type or a composite data type.

You should consider the following points about the `CREATE FUNCTION` statement:

- The `REPLACE` option indicates that if the function exists, it is dropped and replaced with the new version that is created by the statement.
- The `RETURN` data type must not include a size specification.
- The PL/SQL block starts with a `BEGIN` after the declaration of any local variables and ends with an `END`, optionally followed by the *function_name*.
- There must be at least one `RETURN expression` statement.
- You can't reference host or bind variables in the PL/SQL block of a stored function.

The function syntax shown in the slide is good enough to create a usable function; however, a function declaration may have following additional options to improve performance in an operational application. These options are not required to implement the basic functionality of a function.



DETERMINISTIC option	A deterministic option in the function declaration implies that each time you give certain values as parameters, the function would return the same value. The value returned by the function is determined by the values of the input parameters. The function do not contain any nondeterministic entities . This is helpful for caching the results and improving the performance
PARALLEL_ENABLE option	Enables the function for parallel execution, which implies that there are no dependencies on the execution of other functions or modules. Each parallel session using the function will have private copies of the function's local variables.
PIPELINED option	A pipelined table function returns a row to its invoker immediately after processing that row and continues to process other rows. It is used only with table functions.
RESULT_CACHE option	Stores function results in the PL/SQL function result cache (appears only in declaration)
RESULT_CACHE clause	Stores function results in the PL/SQL function result cache (appears only in definition)

Tax Calculation



I could update the database with the promotions and salary raise. Thanks Alice. Now I need your help in calculating the tax applicable for each employee.



I will mail you the formula. It's a bit complicated. The code you write should help me in arriving at a value based on input. It should be good to use in mathematical formulas. I may have to calculate taxes in many places.

Sure, tell me the formula for calculation.



Got it



8

The HR manager in this scenario wants to calculate the tax applicable for an employee. He wants to use this in some mathematical expression. For instance to calculate the payable salary for an employee, the tax per month must be deducted from the salary value in the `EMPLOYEES` table for that employee.

$$\text{net salary} = \text{salary} - \text{tax applicable}$$

The HR manager requires a program construct that can substitute the value of tax applicable in the mathematical expression.

Functions are useful in such scenarios. You can write a function for tax and pass the `employee_id` as the input parameter. The function should pick up the value of the salary from the database, calculate the applicable tax, and return the value of the tax into the mathematical expression. The expression would in turn calculate the net salary.

The Difference Between Procedures and Functions

Procedures	Functions
Execute as a PL/SQL statement	Invoke as part of an expression
Do not contain <code>RETURN</code> clause in the header	Must contain a <code>RETURN</code> clause in the header
Can pass values (if any) using output parameters	Must return a single value
Can contain a <code>RETURN</code> statement without a value	Must contain at least one <code>RETURN</code> statement
Cannot invoke from SQL expressions	Can be invoked from a SQL expression

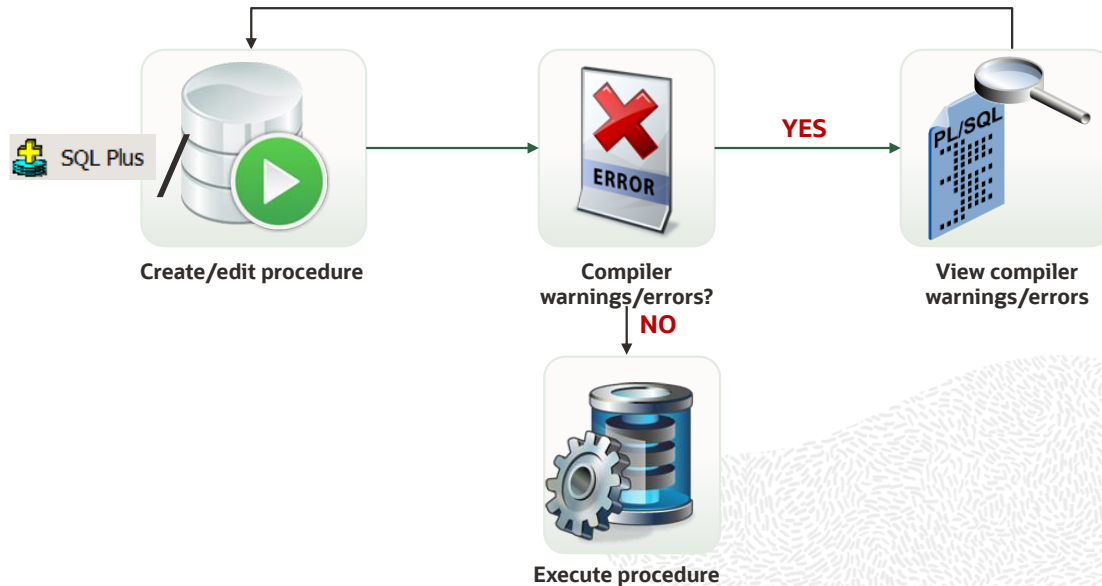
9

You create a procedure to store a series of actions for later execution. A procedure can contain zero or more parameters that can be transferred to and from the calling environment, but a procedure doesn't have to return a value. A procedure can call a function to assist with its actions.

Note: A procedure containing a single `OUT` parameter would be better rewritten as a function returning the value.

You create a function when you want to compute a value that must be returned to the calling environment. A function can contain zero or more parameters that are transferred from the calling environment. Functions typically return only a single value, and the value is returned through a `RETURN` statement. Functions used in SQL statements should not use `OUT` or `IN OUT` mode parameters. Although a function using output parameters can be used in a PL/SQL procedure or block, it can't be used in SQL statements.

Creating Functions: Overview



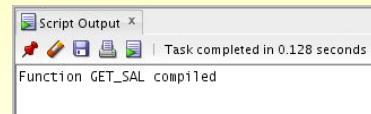
10

To develop a function by using a tool such as SQL Developer, perform the following steps:

1. You can create the function using SQL Developer or SQL Plus.
2. Compile the function. The function is created in the database on compilation. The `CREATE FUNCTION` statement creates and stores the source code and the compiled *m-code* in the database. The compilation process can be GUI based as in SQL Developer or as in the case of command-line interface such as SQL Plus, you can compile the function like any other command.
3. If compilation errors exist, then the *m-code* isn't generated and you must edit the source code to make corrections. You can't invoke a function that contains compilation errors. You can view the compilation errors in the **Compiler Log** of SQL Developer, SQL*Plus when you compile the code.
4. After successful compilation, execute the procedure to perform the desired action. You can run the procedure using SQL Developer GUI or use the `EXECUTE` command in SQL*Plus.

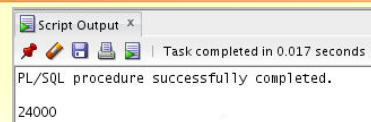
Invoking a Stored Function: Example

```
CREATE OR REPLACE FUNCTION get_sal
(p_id employees.employee_id%TYPE) RETURN NUMBER IS
v_sal employees.salary%TYPE := 0;
BEGIN
    SELECT salary
    INTO   v_sal
    FROM   employees
    WHERE  employee_id = p_id;
    RETURN v_sal;
END get_sal;
/
```



```
-- Invoke the function as an expression or as
-- a parameter value.
```

```
EXECUTE dbms_output.put_line(get_sal(100))
```



11

In this slide, you create a `get_sal` function to retrieve the value of the salary of an employee. The code in the first code box creates the function. When the function is successfully compiled, you see the response 'Function GET_SAL compiled' as shown in the slide.

The function has one input parameter and returns a `NUMBER` value. It accepts the `employee_id` and retrieves the salary of the employee with the given `employee_id`.

The `get_sal` function follows a common programming practice of using a single `RETURN` statement that returns a value assigned to a local variable. If your function has an exception section, then it may also contain a `RETURN` statement.

You can incorporate a function call as part of a PL/SQL expression because the function will return a value to the calling environment. You can incorporate multiple function calls in a single PL/SQL expression resulting in a simpler and maintainable code.

The second code box uses the SQL*Plus `EXECUTE` command to call the `DBMS_OUTPUT.PUT_LINE` procedure whose argument is the return value from the function `get_sal`. In this case, `get_sal` is invoked first to calculate the salary of the employee with ID 100. The salary value returned is supplied as the value of the `DBMS_OUTPUT.PUT_LINE` parameter, which displays the result (if you have executed a `SET SERVEROUTPUT ON`).

Note: A function must always return a value. The example doesn't return a value if a row is not found for a given `id`. Ideally, create an exception handler to return a value as well.

Using Different Methods for Executing Functions

-- As a PL/SQL expression, get the results using host variables

```
VARIABLE b_salary NUMBER  
EXECUTE :b_salary := get_sal(100)
```

PL/SQL procedure successfully completed.

B_SALARY
24000

-- As a PL/SQL expression, get the results using a local
-- variable

```
SET SERVEROUTPUT ON  
DECLARE  
    sal employees.salary%type;  
BEGIN  
    sal := get_sal(100);  
    DBMS_OUTPUT.PUT_LINE('The salary is: ' || sal);  
END;  
/
```

PL/SQL procedure successfully completed.

The salary is: 24000

12

Functions can be invoked in the following ways:

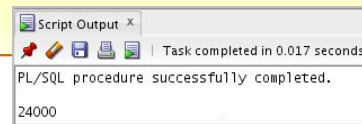
- **As part of PL/SQL expressions:** You can use host or local variables to hold the returned value from a function. The first example in the slide uses a host variable and the second example uses a local variable in an anonymous block.

Note: The benefits and restrictions that apply to functions when used in a SQL statement are discussed in the following slides.

Using Different Methods for Executing Functions

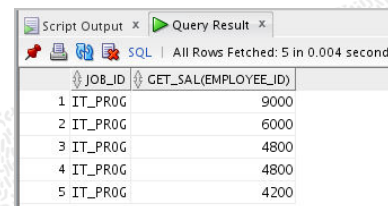
```
-- Use as a parameter to another subprogram
```

```
EXECUTE dbms_output.put_line(get_sal(100))
```



```
-- Use in a SQL statement (subject to restrictions)
```

```
SELECT job_id, get_sal(employee_id)
FROM employees
WHERE department_id = 60;
```



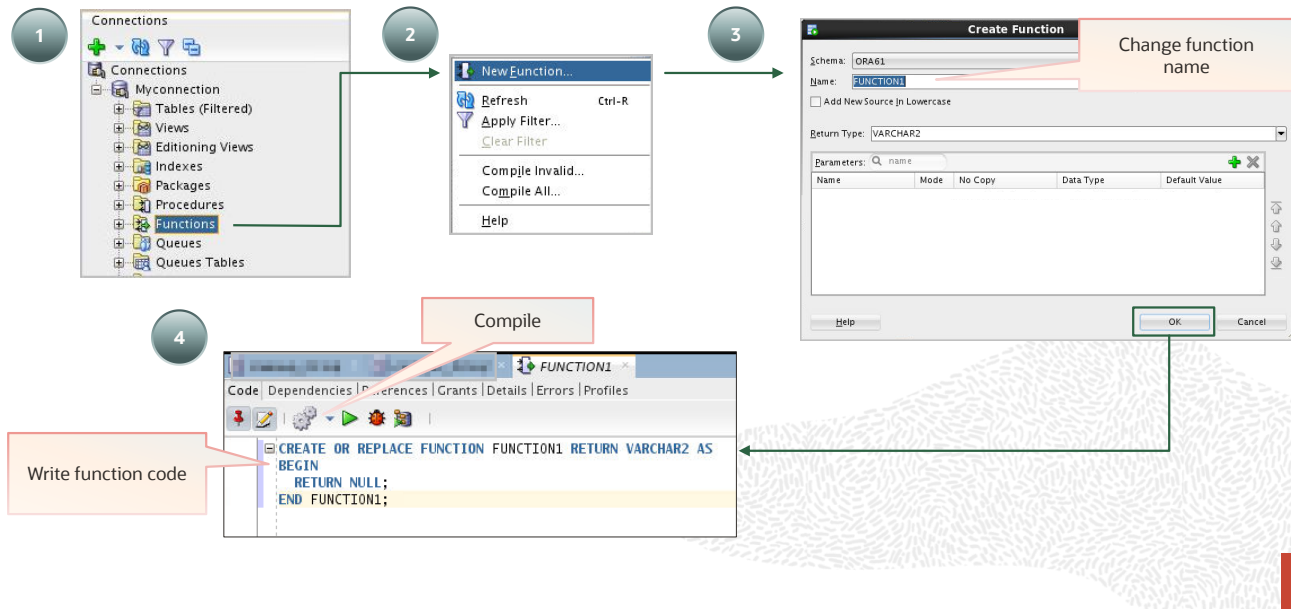
The screenshot shows a 'Query Result' window with a title bar 'Query Result x'. Below the title bar, it says 'All Rows Fetched: 5 in 0.004 seconds'. The main area displays a table with two columns: 'JOB_ID' and 'GET_SAL(EMPLOYEE_ID)'. The table contains five rows of data.

JOB_ID	GET_SAL(EMPLOYEE_ID)
1 IT_PROG	9000
2 IT_PROG	6000
3 IT_PROG	4800
4 IT_PROG	4800
5 IT_PROG	4200

13

- **As a parameter to another subprogram:** The first example in the slide demonstrates this usage. The `get_sal` function with all its arguments is nested in the parameter required by the `DBMS_OUTPUT.PUT_LINE` procedure. This comes from the concept of nesting functions as discussed in the course titled *Oracle Database: SQL Fundamentals I*.
- **As an expression in a SQL statement:** The second example in the slide shows how a function can be used as a single-row function in a SQL statement.

Creating and Compiling Functions Using SQL Developer



14

You can create a new function in SQL Developer by using the following steps:

1. Right-click the **Functions** node.
2. Select **New Function** from the shortcut menu. The **Create PL/SQL Function** dialog box is displayed.
3. Select the schema, function name, and the parameters list (using the **+** icon), and then click **OK**. The code editor for the function is displayed.
4. Enter the function's code.
5. To compile the function, click the **Compile** icon.

Note

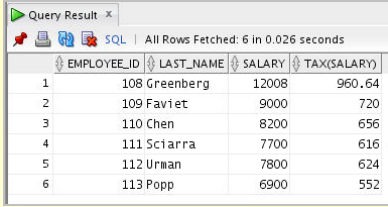
- To create a new function in SQL Developer, you can also enter the code in the SQL Worksheet, and then click the **Run Script** icon.
- For additional information about creating functions in SQL Developer, access the appropriate online Help topic titled "Create PL/SQL Subprogram Function or Procedure."

Lesson Agenda

- Creating and invoking functions
- Functions in SQL expressions
- Passing parameters to functions
- Removing stored functions

Using a Function in a SQL Expression: Example

```
CREATE OR REPLACE FUNCTION tax(p_id IN employees.employee_id%type)
RETURN NUMBER IS
    v_sal employees.salary%type;
BEGIN
    select salary into v_sal
    from employees
    where employee_id = p_id;
    RETURN (v_sal * 0.08);
END tax;
/
SELECT employee_id, last_name, salary, tax(employee_id)
FROM employees
WHERE department_id = 100;
```



Query Result: x | All Rows Fetched: 6 in 0.026 seconds

	EMPLOYEE_ID	LAST_NAME	SALARY	TAX(SALARY)
1	108	Greenberg	12008	960.64
2	109	Faviet	9000	720
3	110	Chen	8200	656
4	111	Sciarra	7700	616
5	112	Urman	7800	624
6	113	Popp	6900	552

16

The example in the slide shows how to create a `tax` function to calculate income tax. The function accepts an `employee_id` as input parameter and calculates the tax applicable for that employee based on the salary of that employee.

To execute the code shown in the slide example in SQL Developer, enter the code in the SQL Worksheet, and then click the **Run Script** icon. The `tax` function is invoked as an expression in the `SELECT` clause along with the employee ID, last name, and salary for employees in a department with ID 100. The return result from the `tax` function is displayed with the regular output from the query.

Calling User-Defined Functions in SQL Statements

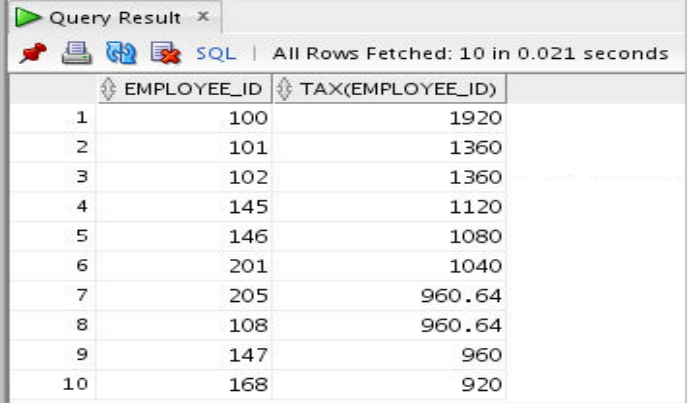
User-defined functions act like built-in single-row functions and can be used in:

- The `SELECT` list or clause of a query
- Conditional expressions of the `WHERE` and `HAVING` clauses
- The `ORDER BY` and `GROUP BY` clauses of a query
- The `VALUES` clause of the `INSERT` statement
- The `SET` clause of the `UPDATE` statement

17

A PL/SQL user-defined function can be called from any SQL expression where a built-in single-row function can be called as shown in the following example:

```
SELECT employee_id, tax(employee_id)
FROM   employees
WHERE  tax(employee_id) > (SELECT MAX(tax(employee_id))
                           FROM employees
                           WHERE department_id = 30)
ORDER BY tax(employee_id) DESC;
```



The screenshot shows a 'Query Result' window with the following data:

	EMPLOYEE_ID	TAX(EMPLOYEE_ID)
1	100	1920
2	101	1360
3	102	1360
4	145	1120
5	146	1080
6	201	1040
7	205	960.64
8	108	960.64
9	147	960
10	168	920

Note: You can use SQL functions with `CONNECT BY` and `START WITH` clauses in hierarchical SQL queries.

Restrictions When Calling Functions from SQL Expressions

- User-defined functions that are callable from SQL expressions must:
 - Be stored in the database
 - Accept only `IN` parameters with valid SQL data types and PL/SQL-specific data types
 - Return valid SQL data types and PL/SQL-specific data types
- When calling functions in SQL statements:
 - Be the owner of the function
 - Have the `EXECUTE` privilege
 - Enable the `PARALLEL_ENABLE` option in function specification

18

The user-defined PL/SQL functions that are callable from SQL expressions must meet the following requirements:

- The function must be stored in the database.
- The function parameters must be `IN` and of valid SQL data types and PL/SQL-specific data types.
- The functions must return data types that are valid SQL data types and PL/SQL-specific data types such as `BOOLEAN`, `RECORD`, or `TABLE`.

The following restrictions apply when calling a function in a SQL statement:

- Parameters must use positional notation or named notation.
- You must own or have the `EXECUTE` privilege on the function.
- You may need to enable the `PARALLEL_ENABLE` option in the function specification to allow a parallel execution of the SQL statement using the function. Each parallel session using the function will have private copies of the function's local variables.

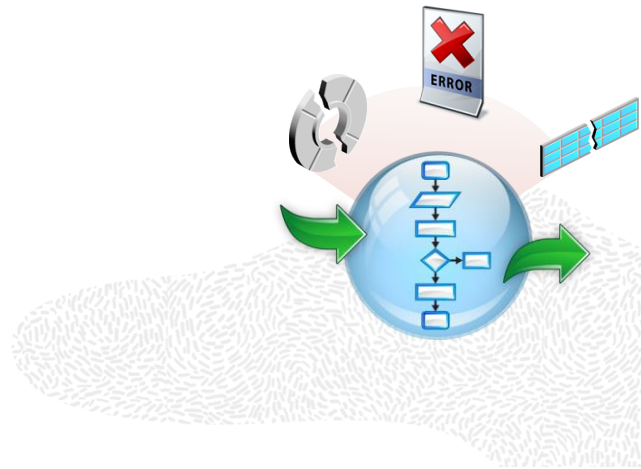
Other restrictions on a user-defined function include the following:

- You cannot invoke the function from the `CHECK` constraint clause of a `CREATE TABLE` or `ALTER TABLE` statement.
- You cannot use a function to define the default value of a column.

Side Effects of Function Execution

A subprogram may have side effects when it performs one of the following operations:

- Updates its own `OUT` or `IN OUT` parameter
- Modifies a global variable
- Modifies a public variable in a package
- Updates a database table
- Modifies the database



19

A function executes with or without input parameters and returns a value. A function is expected to return one value of scalar data type or composite data type. However, developers can also use `OUT` or `IN OUT` parameters, which also reflect the output of the function. Functions can update global variables, perform update and other DML operations on the database.

If the function definition has any of the operations mentioned in the slide, then those updates and modifications are side effects of the function execution besides the value returned.

These side effects might sometimes produce undesirable effects in the database. Execution of such functions may result in inconsistent data in the database or an error. Therefore, it is essential to follow some guidelines while writing code for functions.

Controlling Side Effects

```
CREATE OR REPLACE FUNCTION dml_call_sql(p_sal NUMBER)
RETURN NUMBER IS
BEGIN
  INSERT INTO employees(employee_id, last_name,
                        email, hire_date, job_id, salary)
  VALUES(1, 'Frost', 'jfrost@company.com',
          SYSDATE, 'SA_MAN', p_sal);
  RETURN (p_sal + 100);
END;
```

```
UPDATE employees
SET salary = dml_call_sql(2000)
WHERE employee_id = 170;
```

```
FUNCTION DML_CALL_SQL compiled
Error starting at line 127 in command:
UPDATE employees
SET salary = dml_call_sql(2000)
WHERE employee_id = 170
Error report:
SQL Error: ORA-04091: table ORA61.EMPLOYEES is mutating, trigger/function may not see it
ORA-06512: at "ORA61.DML_CALL_SQL", line 4
04091. 00000 - "table %s.%s is mutating, trigger/function may not see it"
Cause:      A trigger (or a user defined plsql function that is referenced in
            this statement) attempted to look at (or modify) a table that was
            in the middle of being modified by the statement which fired it.
Action:     Rewrite the trigger (or function) so it does not read that table.
```

20

The code example in the slide demonstrates that, when a function is called from an `UPDATE` or `DELETE` statement, the function cannot query or modify database tables modified by that statement. The `dml_sql_call` function is invoked in the `UPDATE` statement on the `EMPLOYEES` table and the function definition has an `INSERT` statement on the `EMPLOYEES` table.

You can see that `dml_call_sql` function in the slide contains an `INSERT` statement that inserts a new record into the `EMPLOYEES` table and returns the input salary value incremented by 100. This function is invoked in the `UPDATE` statement that modifies the salary of employee 170 to the amount returned from the function. The `UPDATE` statement fails with an error indicating that the table is mutating.

Therefore, a function invoked in an `UPDATE` statement on a table, which in turn has DML statements for the same table, would lead to an error.

Guidelines to Control Side Effects

Functions called from:

- A `SELECT` statement cannot contain DML statements
- An `UPDATE` or `DELETE` statement on a table `T` cannot query or contain DML on the same table `T`
- SQL statements cannot end transactions (that is, cannot execute `COMMIT` or `ROLLBACK` operations)

Note: Calls to subprograms that break these restrictions are not allowed in the function.

21

The slide discusses the guidelines to be followed to avoid undesirable side effects of a function execution.

Additional restrictions apply when a function is called in expressions of SQL statements:

- When a function is called from a `SELECT` statement or a parallel `UPDATE` or `DELETE` statement, the function cannot modify database tables.
- When a function is called from an `UPDATE` or `DELETE` statement, the function cannot query or modify database tables modified by that statement.
- When a function is called from a `SELECT`, `INSERT`, `UPDATE`, or `DELETE` statement, the function cannot execute directly or indirectly through another subprogram or SQL transaction control statements such as:
 - A `COMMIT` or `ROLLBACK` statement
 - A session control statement (such as `SET ROLE`)
 - A system control statement (such as `ALTER SYSTEM`)
 - Any DDL statements (such as `CREATE`) because they are followed by an automatic commit

Lesson Agenda

- Creating and Invoking functions
- Functions in SQL expressions
- Passing parameters to functions
- Removing stored functions



0

For Instructor Use Only.
This document should not be distributed.

Passing Parameters to Functions

Functions accept :

- Named parameters
- Positional parameters
- Mixed parameters



Let us see a variant of the tax function. This new function would accept the `employee_id` and nontaxable allowances as input parameters and calculate the tax applicable for the employee. The nontaxable allowances per month has to be deducted from the salary and tax has to be calculated on the remaining amount.

23

PL/SQL allows arguments in a subroutine call to be specified using positional, named, or mixed notation.

Named and Mixed Notation from SQL: Example

```
CREATE OR REPLACE FUNCTION tax_new(p_id IN employees.employee_id%type,  
p_allowances NUMBER default 500)  
RETURN NUMBER IS  
v_sal employees.salary%type;  
BEGIN  
    select salary into v_sal  
    from employees  
    where employee_id = p_id;  
    v_sal := v_sal-p_allowances;  
    RETURN (v_sal * 0.08);  
END tax_new;
```

```
EXECUTE DBMS_OUTPUT.PUT_LINE(tax_new(100));  
EXECUTE DBMS_OUTPUT.PUT_LINE(tax_new(100,1500));  
EXECUTE DBMS_OUTPUT.PUT_LINE(tax_new(p_allowances =>1500, p_id=>100));  
EXECUTE DBMS_OUTPUT.PUT_LINE(tax_new(p_id=>100));
```

24

The slide shows a tax calculation function which accepts two parameters as input. The input parameters `p_id` and `p_allowances` represent the `employee_id` and allowances payable to the employee, respectively.

The second code box shows various valid function calls for the function `tax_new`.

The first function call `tax_new(100)` uses positional parameter, where the first parameter is the `employee_id`. The `p_allowances` parameter is substituted with the default value of 500.

The second function call also uses positional parameters where the first value represents the `employee_id` and the second value represents the allowances for the employee.

The third function call uses named parameters where the values of the parameters are mentioned along with the names.

The fourth function call has the parameter `employee_id` with the named notation and the parameter `p_allowances` is provided through the default value.

Viewing Functions Using Data Dictionary Views

```
DESCRIBE USER_SOURCE
```

Name	Null	Type
NAME		VARCHAR2(128)
TYPE		VARCHAR2(12)
LINE		NUMBER
TEXT		VARCHAR2(4000)
ORIGIN_CON_ID		NUMBER

```
SELECT text
FROM   user_source
WHERE  type = 'FUNCTION'
ORDER  BY line;
```

25

The source code for PL/SQL subprograms is stored in the data dictionary tables. The source code is accessible to PL/SQL procedures that are successfully or unsuccessfully compiled. To view the PL/SQL source code stored in the data dictionary, execute a `SELECT` statement on the following tables:

- The `USER_SOURCE` table to display PL/SQL code that you own
- The `ALL_SOURCE` table to display PL/SQL code to which you have been granted the `EXECUTE` right by the owner of that subprogram code

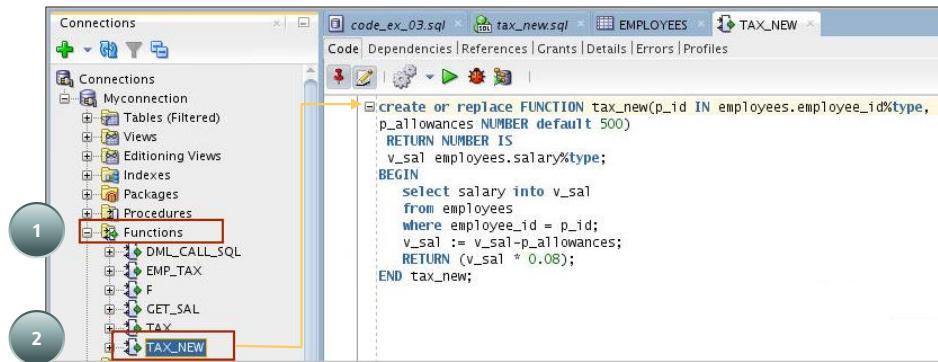
The query example shows all the columns provided by the `USER_SOURCE` table:

- The `NAME` column holds the name of the subprogram in uppercase text.
- The `TYPE` column holds the subprogram type, such as `PROCEDURE` or `FUNCTION`.
- The `LINE` column stores the line number for each source code line.
- The `TEXT` column holds PL/SQL source code.
- The `ORIGIN_CON_ID` holds the ID of the container where the data originates. The value of the container ID can range from 0–n based on database configuration.

The `ALL_SOURCE` table provides an `OWNER` column in addition to the preceding columns.

You can also use the `USER_OBJECTS` data dictionary view to display a list of your function names.

Viewing Functions Information Using SQL Developer



26

To view a function's code in SQL Developer, perform the following steps:

1. Click the **Functions** node in the **Connections** tab.
2. Click the function's name.
3. The function's code is displayed in the **Code** tab as shown in the slide.

Lesson Agenda

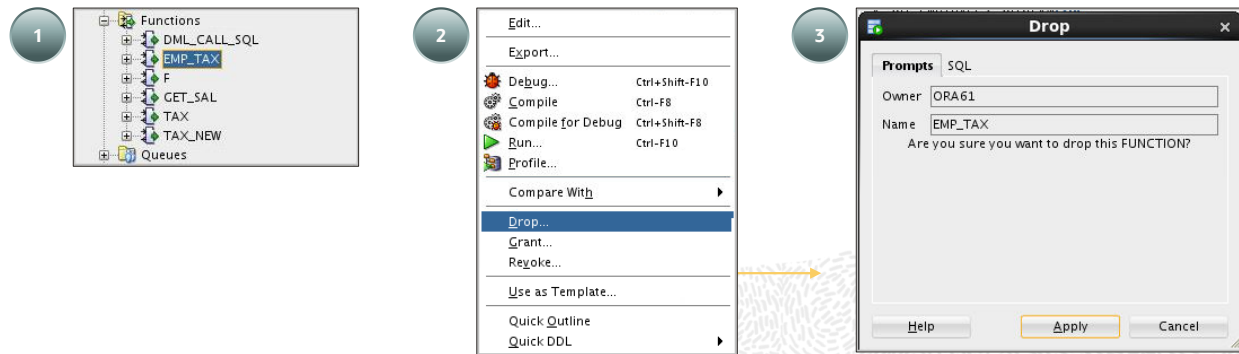
- Creating and Invoking functions
- Functions in SQL expressions
- Passing parameters to functions
- Removing stored functions

Removing Functions: Using the DROP SQL Statement or SQL Developer

- Using the DROP statement:

```
DROP FUNCTION function_name;
```

- Using SQL Developer:



28

Using the DROP statement

When a stored function is no longer required, you can use a SQL statement in SQL*Plus to drop it. To remove a stored function by using SQL*Plus, execute the `DROP FUNCTION SQL` command.

Using CREATE OR REPLACE Versus DROP and CREATE

The `REPLACE` clause in the `CREATE OR REPLACE` syntax is equivalent to dropping a function and re-creating it. When you use the `CREATE OR REPLACE` syntax, the privileges granted on this object to other users remain the same. When you `DROP` a function and then re-create it, all the privileges granted on this function are automatically revoked.

Using SQL Developer

To drop a function in SQL Developer, right-click the function name in the **Functions** node, and then select **Drop**. The **Drop** dialog box is displayed. To drop the function, click **Apply**.

Quiz



Which of the following statements are true about a PL/SQL stored function?

- a. Can be invoked as part of an expression
- b. Must contain a `RETURN` clause in the header
- c. Must return a single value
- d. Must contain at least one `RETURN` statement
- e. Does not contain a `RETURN` clause in the header

29

Answer: a, b, c, d

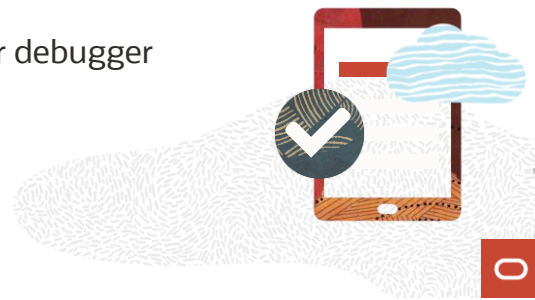
0

For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Differentiate between a procedure and a function
- Describe the uses of functions
- Create stored functions
- Invoke a function
- Remove a function
- Explain the basic functionality of the SQL Developer debugger



30

A function is a named PL/SQL block that must return a value. Generally, you create a function to compute and return a value, and you create a procedure to perform an action.

A function can be created or dropped.

A function is invoked as a part of an expression.

For Instructor Use Only.
This document should not be distributed.

Practice 12: Overview

This practice covers the following topics:

- Creating stored functions:
 - To query a database table and return specific values
 - To be used in a SQL statement
 - To insert a new row, with specified parameter values, into a database table
 - Using default parameter values
- Invoking a stored function from a SQL statement
- Invoking a stored function from a stored procedure



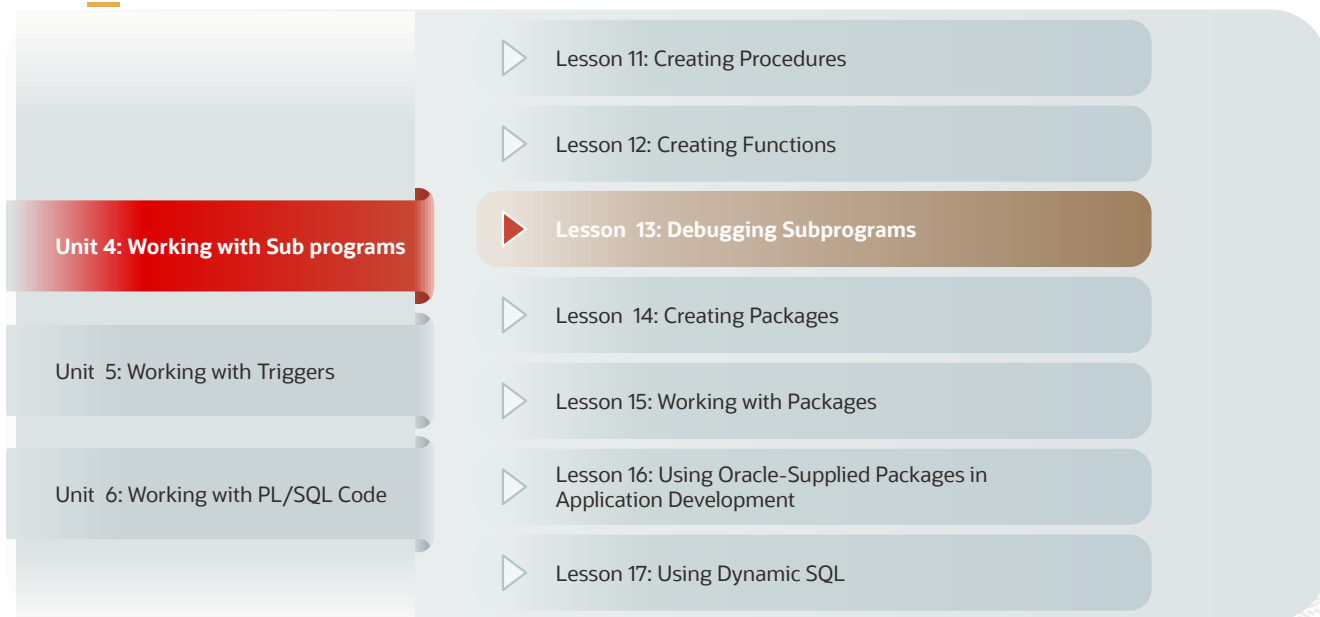
31

It is recommended to use SQL Developer for this practice.

For Instructor Use Only.
This document should not be distributed.

Debugging Subprograms

Course Road Map



2

In the fourth unit, we have seven lessons – Creating Stored Procedures, Creating Functions, Debugging Subprograms, Creating Packages, Working with Packages, Using Oracle Supplied Packages in Application Development, and Using Dynamic SQL. You will learn to write PL/SQL subprograms, packages, and use them in Application Development.

For Instructor Use Only.
This document should not be distributed.

Objectives

After completing this lesson, you should be able to:

- Describe the functionality of the SQL Developer debugger
- Debug Subprograms
- Use various features of SQL Developer debugger for debugging



For Instructor Use Only.
This document should not be distributed.

In this lesson, you learn how to create, invoke, and maintain functions.

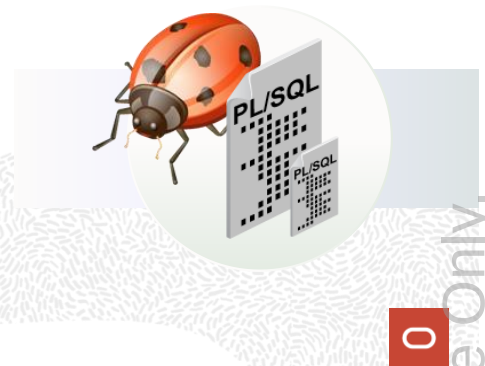
Lesson Agenda

- Before debugging PL/SQL subprograms
- Debugging a subprogram
- Using the debugging Log Tab toolbar
- Debugging a procedure

Before Debugging PL/SQL Subprograms

Before you start debugging PL/SQL Subprograms, you should have the following privileges:

- `DEBUG CONNECT SESSION` privilege
- `DEBUG ANY PROCEDURE` privilege
- `EXECUTE` privilege on `DBMS_DEBUG_JDWP`
- `EXECUTE` privilege on the object you want to debug



0

5

Debugging is the process of locating and removing any defects in the program code. To debug a PL/SQL Subprogram you (user of the schema through which you have logged in) should have certain privileges mentioned in the slide.

You can look at the privileges for the current user by executing the following query :

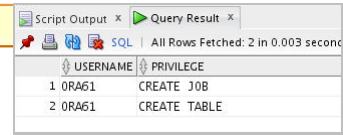
```
select * from user_sys_privs;
```

For Instructor Use Only.
This document should not be distributed.

Before Debugging PL/SQL Subprograms

You can look at the privileges of the current user by executing the following statement:

```
SELECT * from user_sys_privs;
```

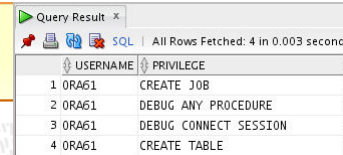


The screenshot shows a SQL Developer window with a 'Query Result' tab. It displays two rows of data from the 'user_sys_privs' table. The first row shows 'ORA61' with the 'CREATE JOB' privilege, and the second row shows 'ORA61' with the 'CREATE TABLE' privilege. The status bar indicates 'All Rows Fetched: 2 in 0.003 seconds'.

USERNAME	PRIVILEGE
1 ORA61	CREATE JOB
2 ORA61	CREATE TABLE

ORA61 does not have DEBUG CONNECT SESSION and DEBUG ANY PROCEDURE privileges. You can grant them by executing the following statements as a SYSDBA role user.

```
GRANT DEBUG ANY PROCEDURE TO ora61;  
GRANT DEBUG CONNECT SESSION TO ora61;  
-- Login as ora61  
select * from user_sys_privs;
```



The screenshot shows a SQL Developer window with a 'Query Result' tab. It displays four rows of data from the 'user_sys_privs' table. The first row shows 'ORA61' with the 'CREATE JOB' privilege, the second row shows 'ORA61' with the 'DEBUG ANY PROCEDURE' privilege, the third row shows 'ORA61' with the 'DEBUG CONNECT SESSION' privilege, and the fourth row shows 'ORA61' with the 'CREATE TABLE' privilege. The status bar indicates 'All Rows Fetched: 4 in 0.003 seconds'.

USERNAME	PRIVILEGE
1 ORA61	CREATE JOB
2 ORA61	DEBUG ANY PROCEDURE
3 ORA61	DEBUG CONNECT SESSION
4 ORA61	CREATE TABLE

6

The process of granting privileges is demonstrated in the slide. You query the existing privileges using the query in the first code box.

Note: To grant privileges to a user, you have to be a SYSDBA role user.

To grant the required privileges to ora61, log in as a SYSDBA role user and run the GRANT statements in the second code box.

To see the current set of privileges, log in as ora61 and then run the query:

```
select * from user_sys_privs;
```

Before Debugging PL/SQL Subprograms

You might encounter the following error while running the SQL debugger in Oracle 19c.



```
Debugging: IdeConnections%23Myconnection.jpr - Log
Connecting to the database Myconnection.
Executing PL/SQL: ALTER SESSION SET PLSQL_DEBUG=TRUE
Executing PL/SQL: CALL DBMS_DEBUG_JDWP.CONNECT_TCP( '127.0.0.1', '4000' )
ORA-24247: network access denied by access control list (ACL)
ORA-06512: at "SYS.DBMS_DEBUG_JDWP", line 68
ORA-06512: at line 1
Process exited.
Disconnecting from the database Myconnection.
```

The error implies that the current database session is trying to establish a connection with the debugger on the local host, but not able to do so.

7

While debugging in Oracle 19c, in spite of having the `DEBUG CONNECT SESSION` privilege, you might encounter the error mentioned in the slide. This error occurs when the database session tries to connect to the debugger on the `localhost`. Your current database session should have the `jdwp ACL` privilege to successfully connect to the debugger.

SQL developer uses Java Debug Wire Protocol (JDWP)–based debugger to debug an Oracle 19c PL/SQL subprogram. Debugger generally resides on the local host. Your database session should connect to the debugger on the local host to start the debugging process. You must be granted the `jdwp ACL` privilege to connect to the debugger on the local host.

Before Debugging PL/SQL Subprograms

You have to execute the following PL/SQL block to grant the `jdwp` ACL privilege as a `SYSDBA` role user.

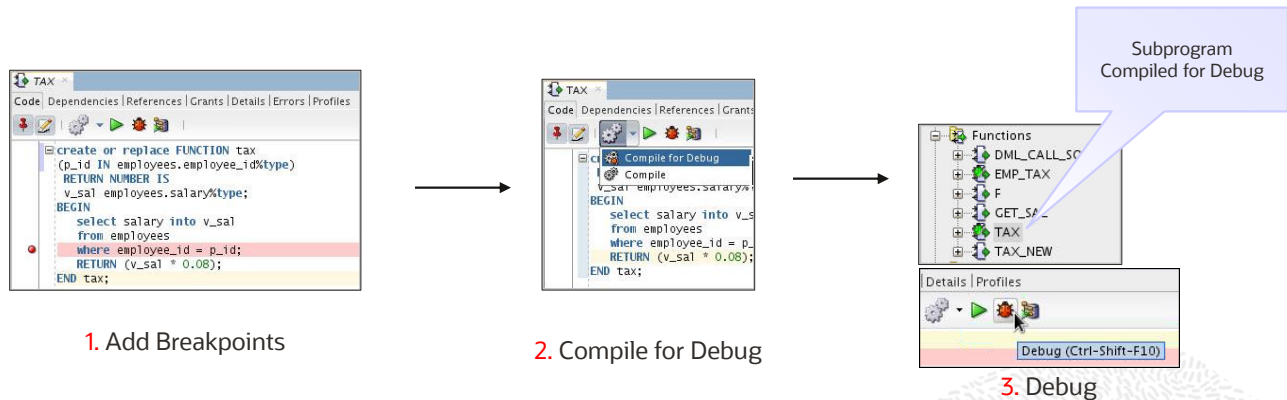
```
BEGIN
  DBMS_NETWORK_ACL_ADMIN.APPEND_HOST_ACE
  (
    host => '127.0.0.1',
    lower_port => null,
    upper_port => null,
    ace => xs$ace_type(privilege_list => xs$name_list('jdwp'),
    principal_name => 'ora61',
    principal_type => xs_acl.ptype_db)
  );
END;
```

Lesson Agenda

- Before debugging PL/SQL subprograms
- Debugging a subprogram
- Using the debugging Log Tab toolbar
- Debugging a procedure



Debugging a Subprogram: Overview



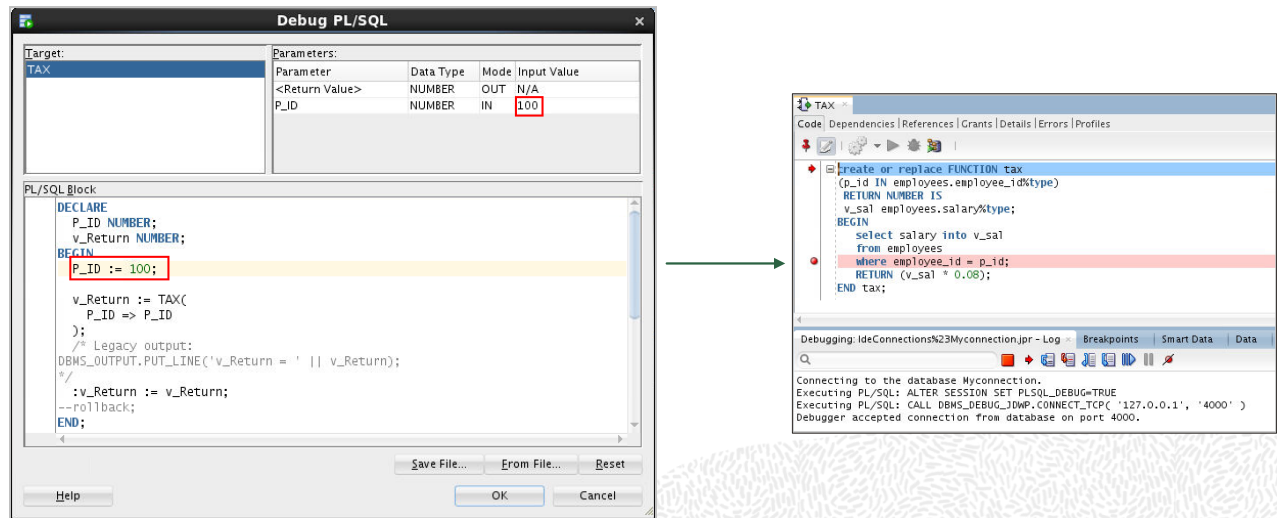
10

The process of debugging starts by adding breakpoints. Breakpoint is added at a statement where the developer wants to inspect various parameters of the code or values of different variables in the code. Identify such statements in your code and add breakpoints.

When the debugging process reaches the breakpoint, you can use Data and Smart Data tabs in SQL Developer to inspect the state of various variables and data objects during the program execution.

After adding breakpoints, compile the code through the Compile for Debug option.

Debugging a Subprogram: Overview



11

When you start debugging the subprogram, the debugger creates an anonymous PL/SQL block which invokes the function. The debugger also declares variables to accept the values returned by the function.

As you see in the slide the wizard has a PL/SQL block which invokes the function “tax” from an anonymous PL/SQL block.

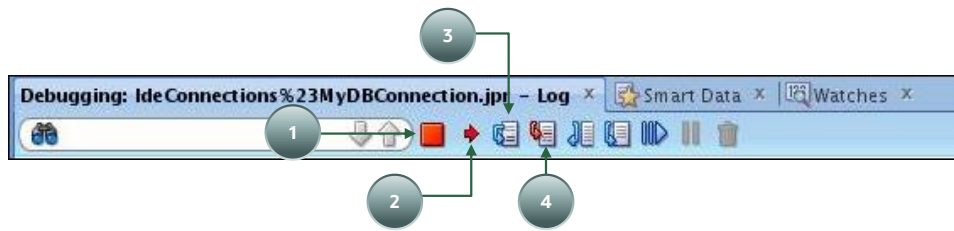
You can modify the value of the input variables accepted by the function in this wizard. In the slide the value of the variable `p_id` is set to 100.

The debug process starts once you click OK in the wizard.

Lesson Agenda

- Before debugging PL/SQL subprograms
- Debugging a subprogram
- Using the debugging Log Tab toolbar
- Debugging a procedure

The Debugging – Log Tab Toolbar



Icon	Description
1. Terminate	Halts and exits the execution
2. Find Execution Point	Goes to the next execution point
3. Step Over	Bypasses the execution of statements of a subprogram invoked from the current subprogram and executes the statement after the subprogram call
4. Step Into	Executes a single program statement at a time. If the execution point is located on a call to a subprogram, it steps into the first statement in that subprogram.

13

0

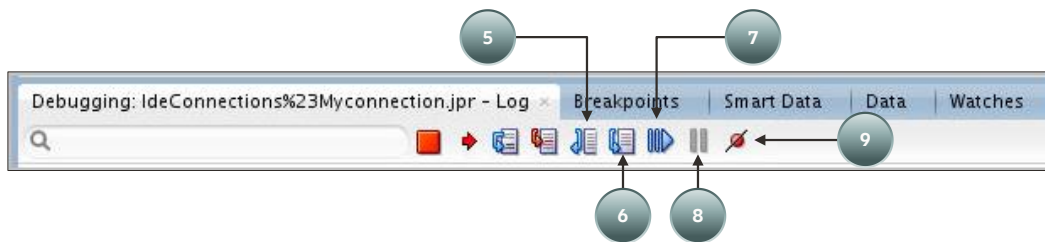
After the debugging process starts, the debug toolbar shown in the slide appears in the SQL developer.

The Debugging Log has all the informational messages and the following tabs:

1. **Terminate:** Halts and exits the execution. You cannot resume execution from this point; instead, to start running or debugging from the beginning of the subprogram, click the **Run** or **Debug** icon in the Source tab toolbar.
2. **Find Execution Point:** Goes to the execution point (the next line of source code to be executed by the debugger)
3. **Step Over:** When there is a subprogram call, **Step Over** skips entering the subprogram block. The next statement to be executed will be the statement following the subprogram call. So the debugger steps over debugging the subprogram body which is invoked from the subprogram currently being executed.
4. **Step Into:** Executes a single program statement at a time. If the execution point is located on a call to a subprogram, **Step Into** steps into that subprogram and places the execution point on the first statement of the invoked subprogram.

Note: The functionality of **Step Over** and **Step Into** would effectively be the same when there are no subprogram calls in the current subprogram.

The Debugging – Log Tab Toolbar

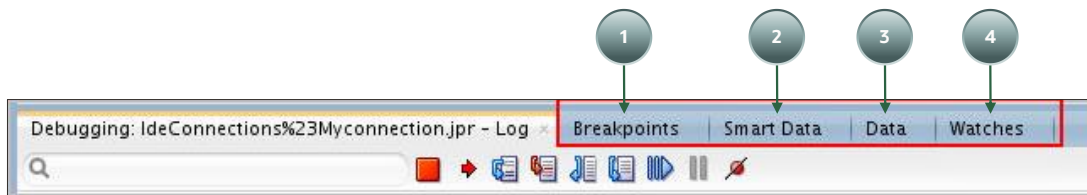


Icon	Description
5. Step Out	Leaves the current subprogram and goes to the next statement with a breakpoint
6. Step to End of Method	Goes to the last statement of the current subprogram
7. Resume	Continues execution
8. Pause	Halts execution but does not exit
9. Suspend all breakpoints	Turns off all the breakpoints in the current PL/SQL block

14

5. **Step Out:** Leaves the current subprogram and goes to the next executable statement
6. **Step to End of Method:** Goes to the last statement of the current subprogram
7. **Resume:** Continues execution
8. **Pause:** Halts execution but does not exit, thus allowing you to resume execution
9. **Suspend all breakpoints:** Renders all the breakpoints defined in the subprogram ineffective

Tracking Data and Execution



Tab	Description
1. Breakpoints	It displays breakpoints, both system defined and user defined.
2. Smart Data	Displays information about variables. You can specify these preferences by right-clicking in the Smart Data window and selecting Preferences.
3. Data	You can see the data in all the variables.
4. Watches	You can monitor all the watches set while debugging.

15

You can monitor how the data is changing while executing the subprogram through the tabs: Breakpoints, Data, Smart Data, and Watches.

Breakpoints: Breakpoints are defined by the developer while debugging. The execution of the subprogram stops at the breakpoint and allows the developer to inspect data or modify certain data if required. After performing the data inspection, developer can start the debugging process by clicking on Step Into or Step Over.

Smart Data: Smart Data tab displays information about the variables in the subprogram. SQL Developer allows to modify the data here enabling the developer to try all the possibilities.

Data: The Data tab has all the values of the variables used in the subprogram being debugged.

Watches: A watch enables you to monitor the changing values of variables as the subprogram executes. You can create a Watch expression in the Watches tab based on the values of variables used in the subprogram.

Lesson Agenda

- Before debugging PL/SQL subprograms
- Debugging a subprogram
- Using the debugging Log Tab toolbar
- Debugging a procedure



For Instructor Use Only.
This document should not be distributed.

Debugging a Procedure Example: Creating a New emp_list Procedure

```
1 CREATE OR REPLACE PROCEDURE emp_list(pmaxrows IN NUMBER) AS
2 CURSOR emp_cursor IS
3 SELECT d.department_name,
4        e.employee_id,
5        e.last_name,
6        e.salary,
7        e.commission_pct
8 FROM departments d,
9        employees e
10 WHERE d.department_id = e.department_id;
11 emp_record emp_cursor % rowtype;
12 type emp_tab_type IS TABLE OF emp_cursor % rowtype INDEX BY binary_integer;
13 emp_tab emp_tab_type;
14 i NUMBER := 1;
15 v_city VARCHAR2(30);
16 BEGIN
17
18     OPEN emp_cursor;
19     FETCH emp_cursor
20     INTO emp_record;
21     emp_tab(i) := emp_record;
22     WHILE (emp_cursor % FOUND)
23     AND (i <= pmaxrows)
24     LOOP
25         i := i + 1;
26         FETCH emp_cursor
27         INTO emp_record;
28         emp_tab(i) := emp_record;
29         v_city := get_location(emp_record.department_name);
30         DBMS_OUTPUT.PUT_LINE('Employee ' || emp_record.last_name || ' works in ' || v_city);
31     END LOOP;
32
33     CLOSE emp_cursor;
34     FOR j IN REVERSE 1 .. i
35     LOOP
36         DBMS_OUTPUT.PUT_LINE(emp_tab(j).last_name);
37     END LOOP;
38 END emp_list;
```

17

Now let us look at the debugging process of the procedure `emp_list`. The procedure retrieves the employee details such as `employee_id`, `last_name`, `salary`, and `commission_pct` from the `EMPLOYEES` table along with the department name in which the employee works in the `DEPARTMENT` table. The procedure also has a function call to retrieve the location of the employee based on the department in which the employee works. All these details are retrieved into a record called `emp_record`, these records are later stored in a table `emp_tab`. And finally displayed in a for loop.

Let us see the process of debugging this procedure.

This procedure has a cursor, function call, and for loop. A developer would like to set a breakpoint at these statements because they are some likely locations of error in the subprogram.

Note: Make sure that you are displaying the procedure code in edit mode. To edit the procedure code, click the Edit icon on the procedure's toolbar.

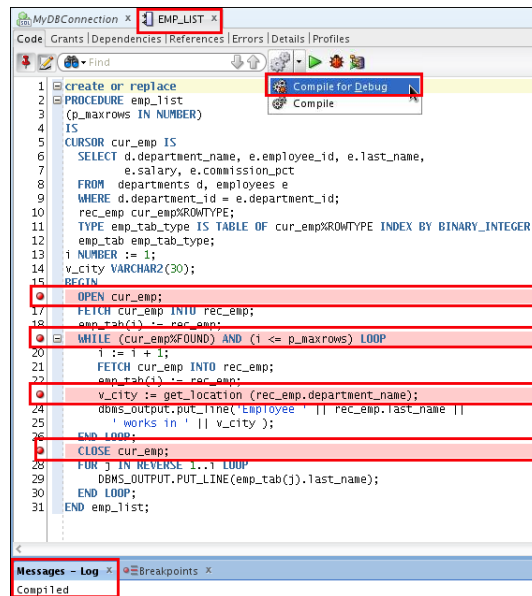
Debugging a Procedure Example: Creating a New get_location Function

```
1 CREATE OR REPLACE FUNCTION get_location(p_deptname IN VARCHAR2) RETURN VARCHAR2 AS
2   v_loc_id NUMBER;
3   v_city VARCHAR2(30);
4 BEGIN
5   SELECT d.location_id,
6         l.city
7   INTO v_loc_id,
8        v_city
9  FROM departments d,
10       locations l
11 WHERE UPPER(department_name) = UPPER(p_deptname)
12       AND d.location_id = l.location_id;
13 RETURN v_city;
14 END get_location;
```

18

This function returns the city in which an employee works. It is called from the `emp_list` procedure.

Setting Breakpoints and Compiling emp_list for Debug Mode



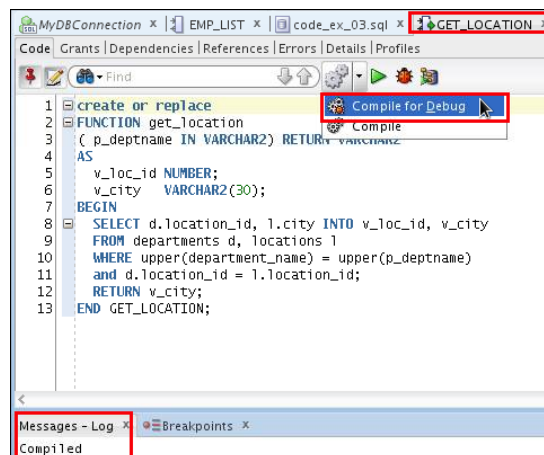
19

The slide shows adding breakpoints for the cursor, function call, and the loop statement. You generally add a break point by clicking the appropriate line number.

Before debugging, it is essential to **Compile for Debug**. This will add additional overhead which enables debugging process.

The slide shows the `emp_list` procedure displayed in edit mode, and four breakpoints added to various locations in the code. To compile the procedure for debugging, right-click the code, and then select **Compile for Debug** from the shortcut menu. The **Messages – Log** tab displays the message that the procedure was compiled.

Compiling the get_location Function for Debug Mode



20

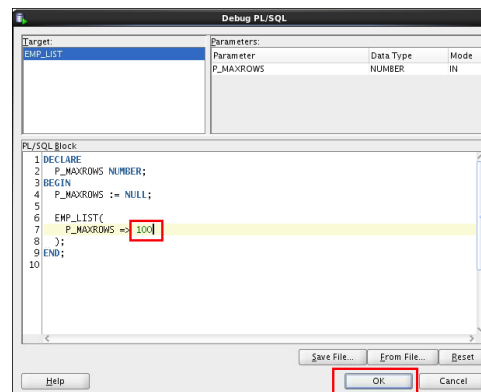
You should also compile the user-defined functions invoked from the subprogram you intend to debug. Therefore, you should debug the `get_location` function also.

In the slide, the `get_location` function is displayed in edit mode. To compile the function for debugging, right-click the code, and then select **Compile for Debug** from the shortcut menu. The **Messages – Log** tab displays the message that the function was compiled.

Debugging emp_list and Entering Values for the P_MAXROWS Parameter



Enter the procedure's
parameter value using the
anonymous block.



21

The next step in the debugging process is to debug the procedure using any of the several available methods mentioned earlier, such as clicking the **Debug** icon on the procedure's toolbar. When you start the debugging process an anonymous block is displayed, where you are prompted to enter the parameters for this procedure. emp_list has one parameter, P_MAXROWS, which specifies the number of records to return. Replace the second P_MAXROWS with a number such as 100, and then click OK.

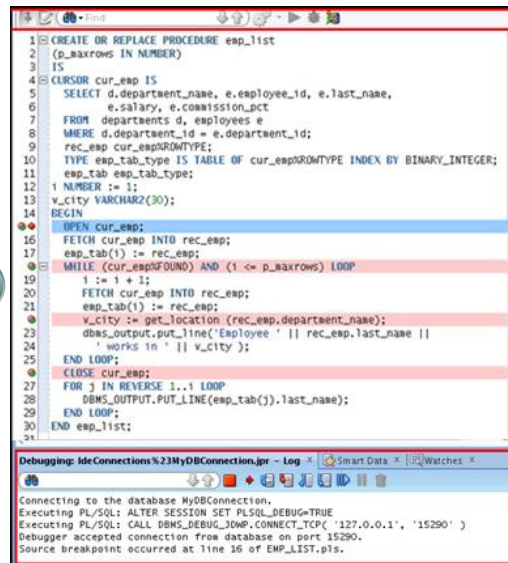
After you start the debugging process, you can use **Step Into** or **Step Over** to debug the subprogram step by step.

Note: Use **Step Over** to avoid entering the called function subprogram from the currently executing subprogram.

Debugging emp_list: Step Into (F7) the Code

Program control stops at first breakpoint.

1



```
1 CREATE OR REPLACE PROCEDURE emp_list
2 (p_maxrows IN NUMBER)
3 IS
4 CURSOR cur_emp IS
5 SELECT d.department_name, e.employee_id, e.last_name,
6        e.salary, e.commission_pct
7 FROM   departments d, employees e
8 WHERE  d.department_id = e.department_id;
9 rec_emp cur_emp%ROWTYPE;
10 TYPE emp_tab_type IS TABLE OF cur_emp%ROWTYPE INDEX BY BINARY_INTEGER;
11 emp_tab emp_tab_type;
12 i NUMBER := 1;
13 v_city VARCHAR2(30);
14 BEGIN
15     OPEN cur_emp;
16     FETCH cur_emp INTO rec_emp;
17     emp_tab(i) := rec_emp;
18     WHILE (cur_emp%FOUND) AND (i <= p_maxrows) LOOP
19         i := i + 1;
20         FETCH cur_emp INTO rec_emp;
21         emp_tab(i) := rec_emp;
22         v_city := get_location(rec_emp.department_name);
23         dbms_output.put_line('Employee ' || rec_emp.last_name ||
24                               ' works in ' || v_city);
25     END LOOP;
26     CLOSE cur_emp;
27     FOR j IN REVERSE 1..i LOOP
28         DBMS_OUTPUT.PUT_LINE(emp_tab(j).last_name);
29     END LOOP;
30 END emp_list;
```

Debugging: IdeConnections %23MyDBConnection.jpr - Log X Smart Data X SQL Watches X

Connecting to the database MyDBConnection.
Executing PL/SQL: ALTER SESSION SET PLSQL_DEBUG=TRUE
Executing PL/SQL: CALL DBMS_DEBUG_JDWP.CONNECT_TCP('127.0.0.1', '15290')
Debugger accepted connection from database on port 15290.
Source breakpoint occurred at line 16 of EMP_LIST.pks.

22

The slide demonstrates the debugging process. The slide shows that the program control is stopped at the first breakpoint in the code. The arrow next to the breakpoint indicated that this is the line of code that will be executed next. Note the various tabs at the bottom of the window.

At this breakpoint, you can inspect different values of the data and monitor the expression watches set by navigating through the **Smart Data**, **Data**, and **Watches** tabs.

Viewing the Data

The first screenshot shows the initial state of the `REC_EMP` record variable. All fields are NULL.

Name	Value	Type
REC_EMP		
COMMISSION_PCT	NULL	NUMBER(2,2)
DEPARTMENT_NAME	NULL	VARCHAR2(30)
EMPLOYEE_ID	NULL	NUMBER(6,0)
LAST_NAME	NULL	VARCHAR2(25)
SALARY	NULL	NUMBER(8,2)

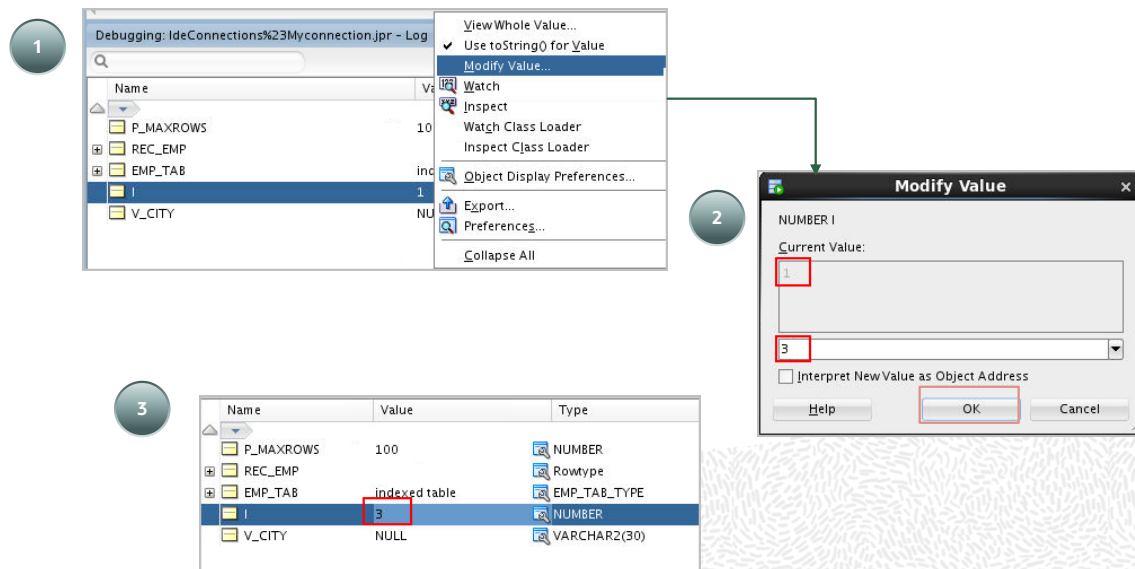
The second screenshot shows the state of `REC_EMP` after the first iteration of the loop. The values are populated from the `emp_tab` table.

Name	Value	Type
REC_EMP		
COMMISSION_PCT	NULL	NUMBER(2,2)
DEPARTMENT_NAME	'Administration'	VARCHAR2(30)
EMPLOYEE_ID	200	NUMBER(6,0)
LAST_NAME	'Whalen'	VARCHAR2(25)
SALARY	4400	NUMBER(8,2)

23

While you are debugging your code, you can use the **Data** tab to display and modify the variables. You can also set watches to monitor a subset of the variables displayed in the **Data** tab. To display or hide the **Data**, **Smart Data**, and **Watch** tabs, select **View > Debugger**, and then select the tabs that you want to display or hide.

Modifying the Variables While Debugging the Code

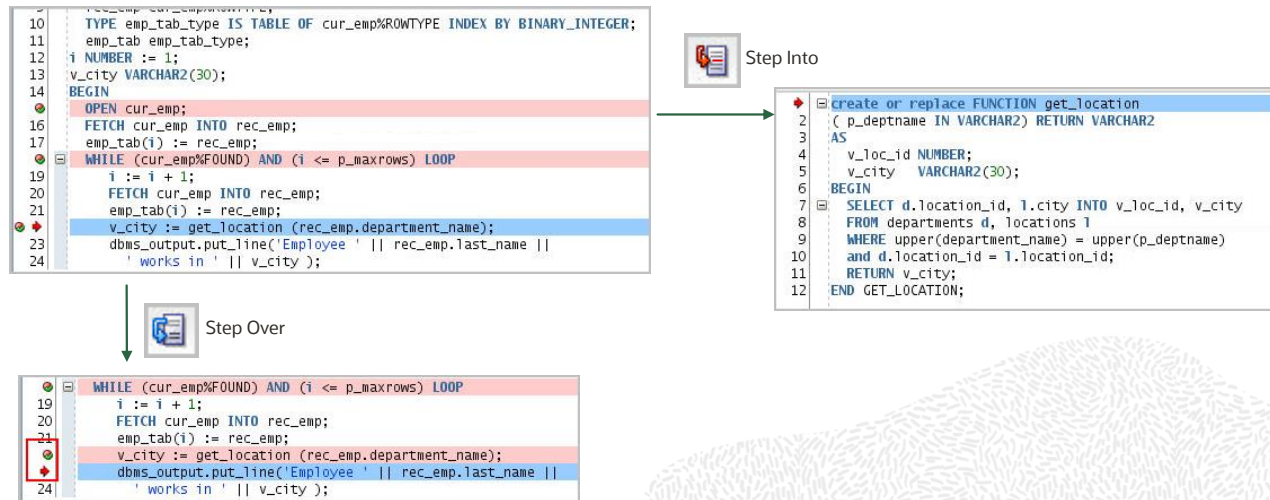


24

To modify the value of a variable in the **Data** tab, right-click the variable name, and then select Modify Value from the shortcut menu. The Modify Value window is displayed with the current value for the variable. You can enter a new value in the second text box, and then click **OK**.

You can see in the slide that the value of variable `i` is modified from 1 to 3.

Debugging emp_list : Step Over Versus Step Into

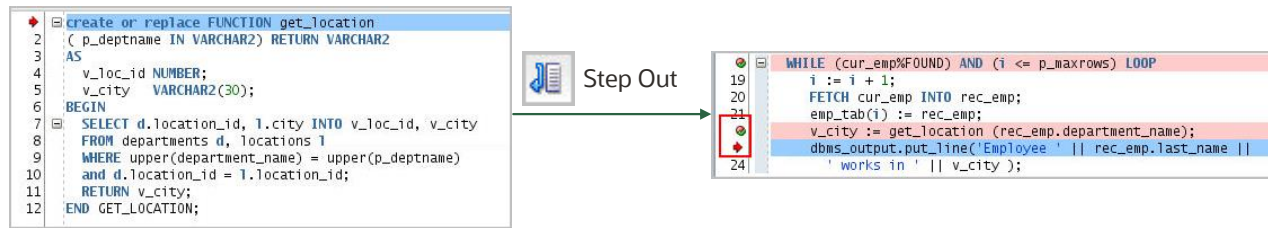


25

You can appreciate the difference between **Step Into** and **Step Over** process of debugger at code line number 21.

We have a function call `get_location` at line 21. At this point when you use Step Into, the debugging control shifts to the first line of the function definition of the `get_location` function. Instead of Step Into if you use Step Over, the debug control will remain in the current sub program. The control will be transferred to next statement (line 22) in the `emp_list` subprogram.

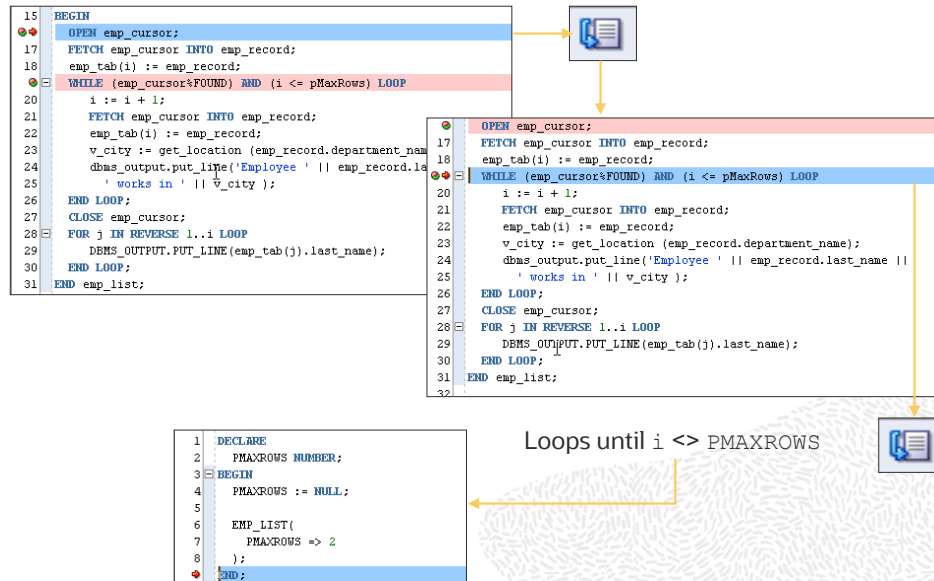
Debugging emp_list: Step Out of the Code (Shift + F7)



26

To look at how **Step Out** works, let us consider that currently we are debugging the first line of the `get_location` function. When we **Step Out** control here, debugging of the entire `get_location` function is skipped and the control moves to the statement after the function call in the `emp_list` procedure.

Debugging emp_list : Step to End of Method

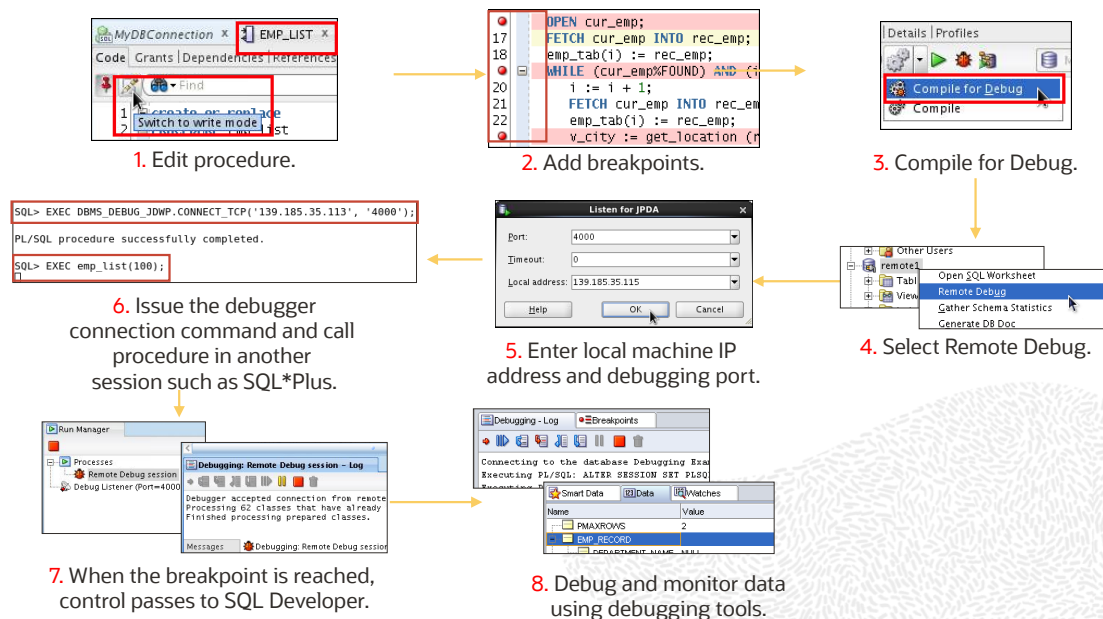


27

Step to End of Method goes to the last statement in the current subprogram or to the next breakpoint if there are any in the current subprogram.

In the example in the slide, the **Step to End** of Method of debugging tool is used. Because there is a second breakpoint, selecting **Step to End** of Method transfers control to that breakpoint. Selecting **Step to End** of Method again goes through the iterations of the while loop first, and then transfers the program control to the next executable statement in the anonymous block.

Debugging a Subprogram Remotely: Overview



28

You can use remote debugging to connect to any PL/SQL code in a database and debug it, regardless of where the database is located as long as you have access to the database and have created a connection to it. Remote debugging is when something else, other than you as a developer, kicks off a procedure that you can then debug. Remote debugging and local debugging have many steps in common. To debug remotely, perform the following steps:

1. Edit the subprogram that you would like to debug.
2. Add the breakpoints in the subprogram.
3. Click the **Compile for Debug** icon in the toolbar.
4. Right-click the connection for the remote database, and select **Remote Debug**.
5. In the **Debugger - Attach to JPDA** dialog box, enter the local machine IP address and port number, such as 4000, and then click **OK**.
6. Issue the debugger connection command using a different session such as SQL*Plus, and then call the procedure in that session.
7. When a breakpoint is reached, control is passed back to the original SQL Developer session and the debugger toolbar is displayed.
8. Debug the subprogram and monitor the data using the debugging tools discussed earlier.

Summary

In this lesson, you should have learned how to:

- Describe the basic functionality of the SQL Developer debugger
- Run the debugging process
- Use various features that can be used while debugging



29

A function is a named PL/SQL block that must return a value. Generally, you create a function to compute and return a value, and you create a procedure to perform an action.

A function can be created or dropped.

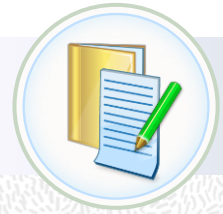
A function is invoked as a part of an expression.

For Instructor Use Only.
This document should not be distributed.

Practice 13 Overview: Introduction to the SQL Developer Debugger

This practice covers the following topics:

- Creating a procedure and a function
- Inserting breakpoints in the procedure
- Compiling the procedure and function for debug mode
- Debugging the procedure and stepping into the code
- Displaying and modifying the subprograms' variables

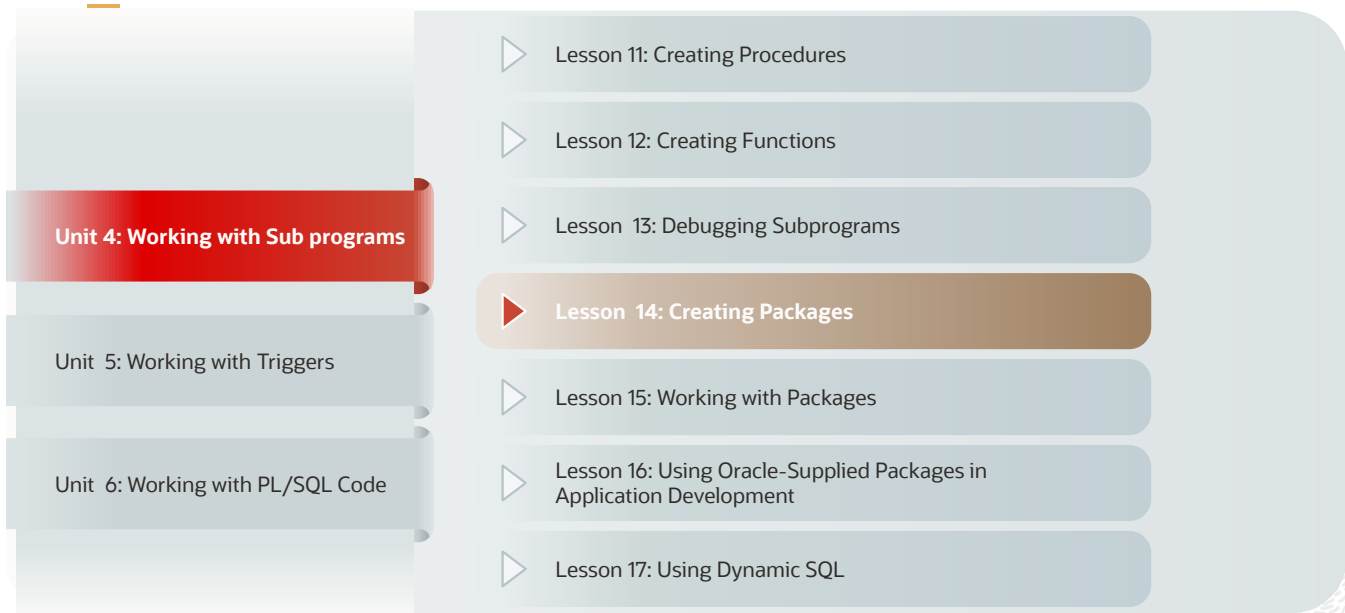


30

You must use SQL Developer for this practice. This practice introduces you to the basic functionality of the SQL Developer debugger.

Creating Packages

Course Road Map



2

In the fourth unit, we have seven lessons – Creating Stored Procedures, Creating Functions, Debugging Subprograms, Creating Packages, Working with Packages, Using Oracle Supplied Packages in Application Development, and Using Dynamic SQL. You will learn to write PL/SQL subprograms, packages, and use them in Application Development.

Objectives

After completing this lesson, you should be able to:

- Describe packages and list their components
- Create a package to group together related variables, cursors, constants, exceptions, procedures, and functions
- Designate a package construct as either public or private
- Invoke a package construct
- Describe the use of a bodiless package



3

In this lesson, you learn what a package is and what its components are. You also learn how to create and use packages.

For Instructor Use Only.
This document should not be distributed.

Lesson Agenda

- Identifying the benefits and the components of packages
- Working with packages:
 - Creating the package specification and body
 - Invoking the package subprograms
 - Displaying the package information
 - Removing a package



DBMS_OUTPUT.PUT_LINE

What is DBMS_OUTPUT?

- DBMS_OUTPUT is an Oracle-supplied package.

What is PUT_LINE?

- A procedure in the DBMS_OUTPUT package that enables displaying messages to output buffer.

DBMS_OUTPUT also have other procedures such as PUT_LINE.

5

You've been using `DBMS_OUTPUT.PUT_LINE` in your code in functions and procedures.

What does `DBMS_OUTPUT.PUT_LINE` mean?

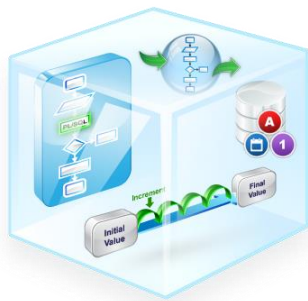
Oracle supplies many PL/SQL packages with the database to extend its functionality. You can use these packages when creating your applications. `DBMS_OUTPUT` is one such Oracle-supplied package and `PUT_LINE` is a procedure in that package. By using this procedure, you avoid writing code for displaying information onto the output buffer. Oracle provides several such packages which can be used in applications by developers.

Apart from the `PUT_LINE` procedure there are other procedures also defined in the `DBMS_OUTPUT` package. All the procedures in the `DBMS_OUTPUT` package are collectively used to display messages or read data from the end user or perform some other related functionality. Developers can use these procedures in their applications.

Oracle allows developers create their own packages with various procedures in it.

From the design perspective, you create a package to create a group of related procedures, functions and other program units. For instance, `DBMS_OUTPUT` has procedures that deal with displaying output. In your application, you can group multiple procedures performing a logical function into a package.

What Is a Package?



A package is a schema object that groups logically related PL/SQL types, variables, sequences, and subprograms.

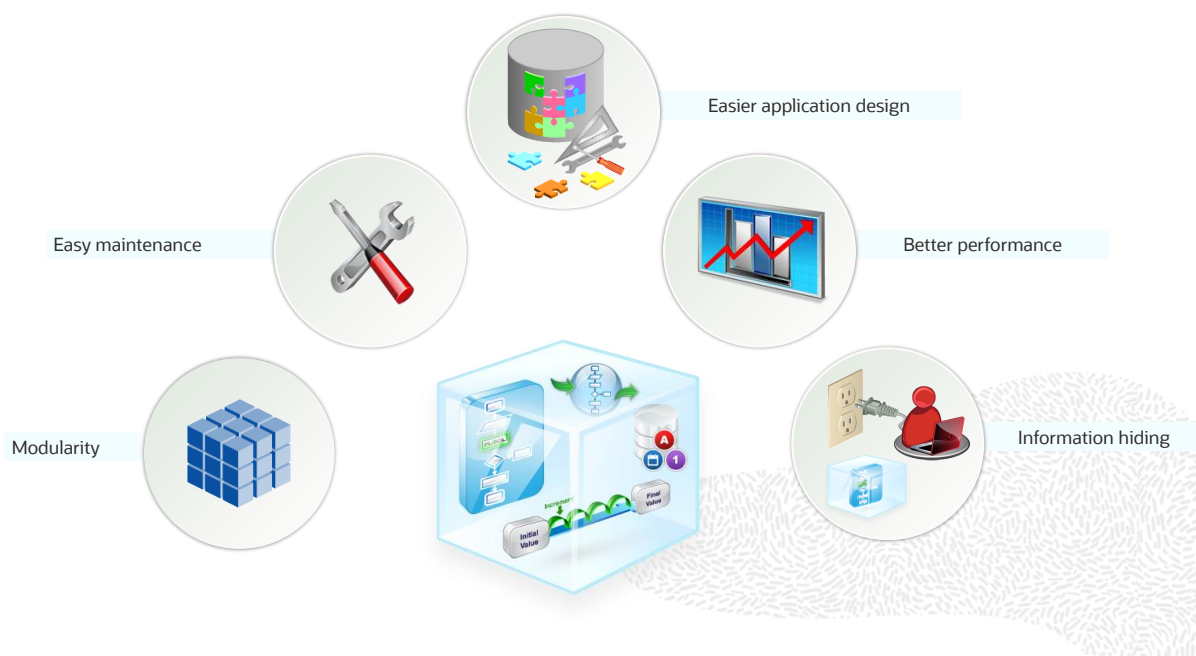
A package is compiled and stored in the database, where many applications can share its contents.

6

A package is a schema object that groups all logically related objects in an application. For instance, the `DBMS_OUTPUT` package, which is an Oracle Supplied Package, has all the procedures and functions related to handling output in PL/SQL.

A package may contain various members such as variables, constants, cursors, exceptions, sequences, and so on.

Advantages of Packages



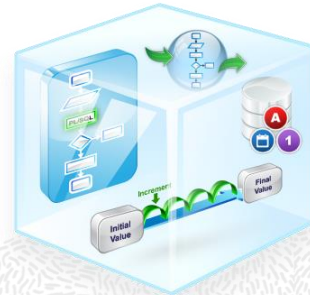
7

The following are the advantages of packages:

- **Modularity:** Packages enable modularity by allowing the developer to logically group various variables, sequences, procedures in the application. You encapsulate logically related programming structures in a named module.
- **Easier maintenance:** The specification and body of the package are independently maintained. You can modify the body of the package without modifying the specification of the package. This flexibility enables better maintenance of the application.
- **Easier application design:** When designing an application, all you need is the interface information in the package specifications. You can code and compile specifications without their bodies. Next, you can compile stand-alone subprograms that reference the packages. You need not fully define the package bodies until you are ready to complete the application.
- **Information hiding:** Packages let you share your interface information in the package specification, and hide the implementation details in the package body. This enables you to change the implementation details without affecting the application interface.
- **Better performance:** The first time you invoke a package subprogram, Oracle database loads the entire package into memory. Subsequent invocations of other subprograms don't result in additional I/O.

How Do You Create PL/SQL Packages?

- Packages usually have two parts:
 - A specification (spec)
 - A body
- The specification is the interface to the package.
- The body has the implementation.



8

If you are a developer creating a HR package, then the package may have a sequence for generating `employee_id` (which has a requirement of being unique in the company) for a new employee, a variable which provides the head count of employees in the company, a procedure to calculate the tax applicable to an employee and so on.

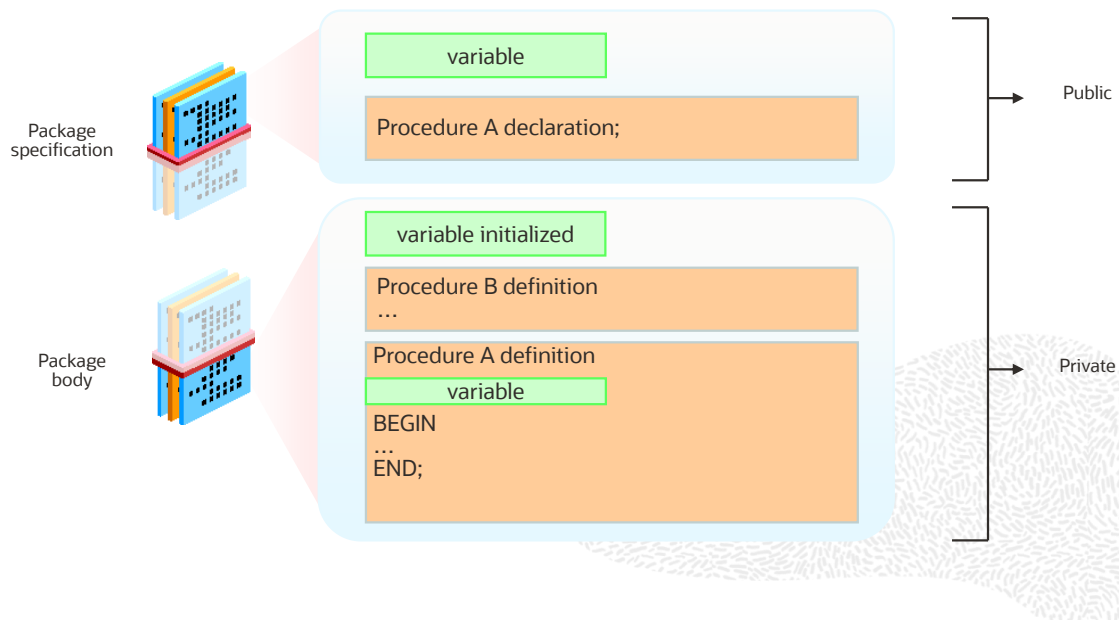
A package usually consists of two parts stored separately in the database:

- A specification
- A body

Package specification is declarative, where you declare the types, variables, constants, exceptions, declared cursors, and subprograms that can be referenced from outside the package. A package specification is an **application program interface (API)**: it has all the information that client programs need to invoke its subprograms. The package specification is about what is in the package for the client.

The package body is about implementation of the declarations in the package specification. You can also define local subprograms in the package body which aren't declared in the package specification. These local subprograms are hidden from client programs.

Components of a PL/SQL Package



9

You create a package in two parts:

- The **package specification** is the interface to your applications. It declares the public types, variables, constants, exceptions, cursors, and subprograms available for use. The package specification may also include `PRAGMAS`, which are directives to the compiler.
- The **package body** defines its own subprograms and must fully implement subprograms declared in the specification part. The package body may also define PL/SQL constructs, such as types, variables, constants, exceptions, and cursors.

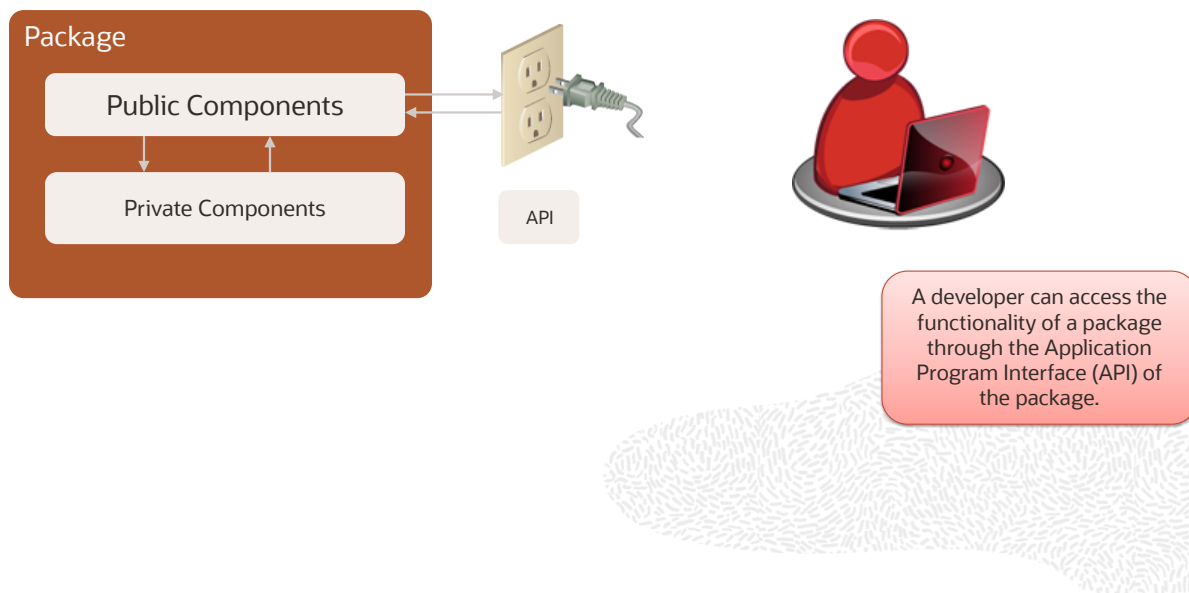
When you populate the package with variables, procedures, and other components, you can define whether the component can be accessed by the client or other external components by specifying them to be **public** or **private**.

When certain package member is declared as public, then those package members can be accessed by entities outside the package. All the members declared in package specification are public.

When certain package members are declared as private, then those package members can only be accessed by other package members. Private package members can't be accessed by components external to the package.

In the slide, you can see that the package specification has two members, a variable and procedure A. However in the package body, you see that there is definition of procedure A and procedure B. Procedure A is public and, therefore, can be accessed by external components, whereas procedure B is private and, therefore, can only be accessed by procedure A or other components within the package.

Application Program Interface



10

The **Application Programming Interface(API)** comprises of all the public components of the application.

Public components are declared in the package specification. The specification defines a public application programming interface (API) for users of package functionality. These public components can be referenced from any Oracle server environment that is external to the package.

Private components are placed in the package body and can be referenced only by other constructs within the same package body. Private components can reference the public components of a package.

Note: If a package specification doesn't contain subprogram declarations, there is no requirement for a package body.

A client can access the package functionality by invoking procedures or functions in the package. The implementation of the package procedures and functions is hidden from the client.

Lesson Agenda

- Identifying the benefits and the components of packages
- Working with packages:
 - Creating the package specification and body
 - Invoking the package subprograms
 - Displaying the package information
 - Removing a package



0

For Instructor Use Only.
This document should not be distributed.

Creating the Package Specification: Using the CREATE PACKAGE Statement

```
CREATE [OR REPLACE] PACKAGE package_name IS|AS
    public type and variable declarations
    subprogram specifications
END [package_name];
```

12

To create packages, you declare all public constructs within the package specification.

- Specify the `OR REPLACE` option if overwriting an existing package specification.
- Initialize a variable with a constant value or formula within the declaration, if required; otherwise, the variable is initialized implicitly to `NULL`.

The following are definitions of items in the package syntax:

- **package_name** specifies a name for the package that must be unique among objects within the owning schema. Including the package name after the `END` keyword is optional.
- **public type and variable declarations** declares public variables, constants, cursors, exceptions, user-defined types, and subtypes.
- **subprogram specification** specifies the public procedure or function declarations.

The package specification should contain procedure and function headings terminated by a semicolon, without the `IS` (or `AS`) keyword and its PL/SQL block. The implementation of a procedure or function that is declared in a package specification is done in the package body.

The Oracle database stores the specification and body of a package separately. This enables you to change the implementation of a program construct in the package body without invalidating other schema objects that call or reference the program construct.

All the constructs declared in a package specification are visible to users who are granted privileges on the package.

Creating Package Specification: Using SQL Developer

The image illustrates the four steps to create a package specification in SQL Developer:

- Step 1:** Right-click **Packages** in the **Connections** tab and select **New Package**.
- Step 2:** Provide the package name in the wizard.
- Step 3:** Add declarations to the specification and compile it.
- Step 4:** On successful compilation, package specification is created in the **Connections** tab and you can also see **Compiled** message in the **Messages** tab.

13

You can use SQL Developer to create the package specification as follows:

1. Right-click the **Packages** node in the **Connections** navigation tree and select **New Package** from the shortcut menu.
2. In the **Create PL/SQL Package** window, select the schema name, enter the name for the new package, and then click **OK**. A tab for the new package is displayed along with the shell for the new package.
3. Add code for the declarations in the package specification for the new package. Compile or save the new package. If there are errors in the package specification, the **Messages – Log** tab will display **Compiled (with errors)**. You’ve to rectify the errors and recompile the package specification.
4. If there are no errors in the specification, then the **Messages – Log** tab will display **Compiled** as shown in the slide. The newly created package is displayed under the **Packages** node in the **Connections** navigation tree.

Creating the Package Body: Using SQL Developer

The image shows a three-step process for creating a package body in SQL Developer:

- Step 1:** A screenshot of the 'Connections' pane showing the 'Packages' node expanded. A right-click context menu is open over 'PACKAGE1', with 'Create Body...' selected. A callout box says: 'Right-click the Package created and select Create Body.'
- Step 2:** A screenshot of the 'PACKAGE1 Body' editor. The 'Code' tab is active, showing the SQL template:

```
1 CREATE OR REPLACE
2 PACKAGE BODY PACKAGE1 AS
3
4 END PACKAGE1;
```

 A callout box says: 'Add procedure and function definitions to the package and compile it.'
- Step 3:** A screenshot of the 'Connections' pane showing 'PACKAGE1 Body' created under 'Packages'. A 'Messages - Log' window is open, showing 'Compiled'. A callout box says: 'On successful compilation, package body is created in the Connections tab and you can also see Compiled message in the Messages tab.'

14

You can use SQL Developer to create the package body by performing the following steps:

1. Right-click the package name for which you are creating a body in the **Packages** node in the **Connections** navigation tree. Select **Create Body** from the shortcut menu. A tab for the new package body is displayed along with the shell for the new package body.
2. Enter the code for the new package body. Compile or save the package body.
3. The **Messages – Log** tab displays whether compilation was successful or not.
4. The newly created package body is displayed under the **Packages** node in the **Connections** navigation tree.

Example of a Package Specification: `comm_pkg`

```
-- The package spec with a public variable and a
-- public procedure that are accessible from
-- outside the package.

CREATE OR REPLACE PACKAGE comm_pkg IS
    v_std_comm NUMBER := 0.10; --initialized to 0.10
    PROCEDURE reset_comm(p_new_comm NUMBER);
END comm_pkg;
/
```

- `v_std_comm` is a *public* global variable initialized to 0.10.
- `reset_comm` is a *public* procedure used to reset the standard commission based on some business rules. It is implemented in the package body.

15

The example in the slide creates a package called `comm_pkg` used to manage business processing rules for commission calculations.

The `v_std_comm` public (global) variable is declared to hold a maximum allowable percentage commission for the user session, and it is initialized to 0.10 (that is, 10%).

The `reset_comm` public procedure is declared to accept a new commission percentage that updates the standard commission percentage if the commission validation rules are accepted. The validation rules for resetting the commission aren't made public and don't appear in the package specification. The validation rules are managed by using a private function in the package body.

Creating the Package Body

```
CREATE [OR REPLACE] PACKAGE BODY package_name IS|AS
    private type and variable declarations
    subprogram bodies
    [BEGIN initialization statements]
END [package_name];
```

- The `OR REPLACE` option drops and re-creates the package body.
- Identifiers defined in the package body are *private* and not visible outside the package body.
- All *private* constructs must be declared before they are referenced.
- Public constructs are visible to the package body.

16

Create a package body to define and implement all public subprograms and supporting private constructs. While creating a package body, perform the following steps:

- Specify the `OR REPLACE` option to overwrite an existing package body.
- Define the subprograms in an appropriate order. The basic principle is that you must declare a variable or subprogram before it can be referenced by other components in the same package body. It is common to see all private variables and subprograms defined first and the public subprograms defined last in the package body.
- Complete the implementation for all procedures or functions declared in the package specification within the package body.

The following are definitions of items in the package body syntax:

- **package_name** specifies a name for the package that must be the same as its package specification. Using the package name after the `END` keyword is optional.
- **private type and variable declarations** declares private variables, constants, cursors, exceptions, user-defined types, and subtypes.
- **subprogram specification** specifies the full implementation of any private and/or public procedures or functions.
- **[BEGIN initialization statements]** is an optional block of initialization code that executes when the package is first referenced.

Example of a Package Body: comm_pkg

```
CREATE OR REPLACE PACKAGE BODY comm_pkg IS
  FUNCTION validate(p_comm NUMBER) RETURN BOOLEAN IS
    v_max_comm      employees.commission_pct%type;
  BEGIN
    SELECT MAX(commission_pct) INTO v_max_comm
    FROM   employees;
    RETURN (p_comm BETWEEN 0.0 AND v_max_comm);
  END validate;

  PROCEDURE reset_comm(p_new_comm NUMBER) IS
  BEGIN
    IF validate(p_new_comm) THEN
      v_std_comm := p_new_comm; -- reset public var
    ELSE RAISE_APPLICATION_ERROR(
      -20210, 'Bad Commission');
    END IF;
  END reset_comm;
END comm_pkg;
```

17

The slide shows the complete package body for `comm_pkg`, with a private function `validate` to check for a valid commission. The validation requires that the commission be positive and less than the highest commission among existing employees. The `reset_comm` procedure invokes the private validation function before changing the standard commission in `v_std_comm`. In the example, note the following:

- The `v_std_comm` variable referenced in the `reset_comm` procedure is a public variable. Variables declared in the package specification, such as `v_std_comm`, can be directly referenced without qualification.
- The `reset_comm` procedure implements the public definition in the specification.
- In the `comm_pkg` body, the `validate` function is private and is directly referenced from the `reset_comm` procedure without qualification.

Note: The `validate` function appears before the `reset_comm` procedure because the `reset_comm` procedure references the `validate` function. It's possible to create forward declarations for subprograms in the package body if their order of appearance needs to be changed. If a package specification declares only types, constants, variables, and exceptions without any subprogram specifications, then the package body is unnecessary. However, the body can be used to initialize items declared in the package specification.

Invoking the Package Subprograms: Examples

```
-- Invoke a function within the same packages:
CREATE OR REPLACE PACKAGE BODY comm_pkg IS ...
  PROCEDURE reset_comm(p_new_comm NUMBER) IS
  BEGIN
    IF validate(p_new_comm) THEN
      ...
    END IF;
  END reset_comm;
END comm_pkg;
```

```
-- Invoke a package procedure from SQL*Plus:
EXECUTE comm_pkg.reset_comm(0.15)
```

```
-- Invoke a package procedure in a different schema:
EXECUTE scott.comm_pkg.reset_comm(0.15)
```

18

After the package is stored in the database, you can invoke public or private subprograms within the same package, or public subprograms if external to the package. Fully qualify the subprogram with its package name when invoked externally from the package. Use the `package_name.subprogram` syntax.

Fully qualifying a subprogram when invoked within the same package is optional.

Example 1: Invokes the `validate` function from the `reset_comm` procedure within the same package. The package name prefix is not required; it is optional.

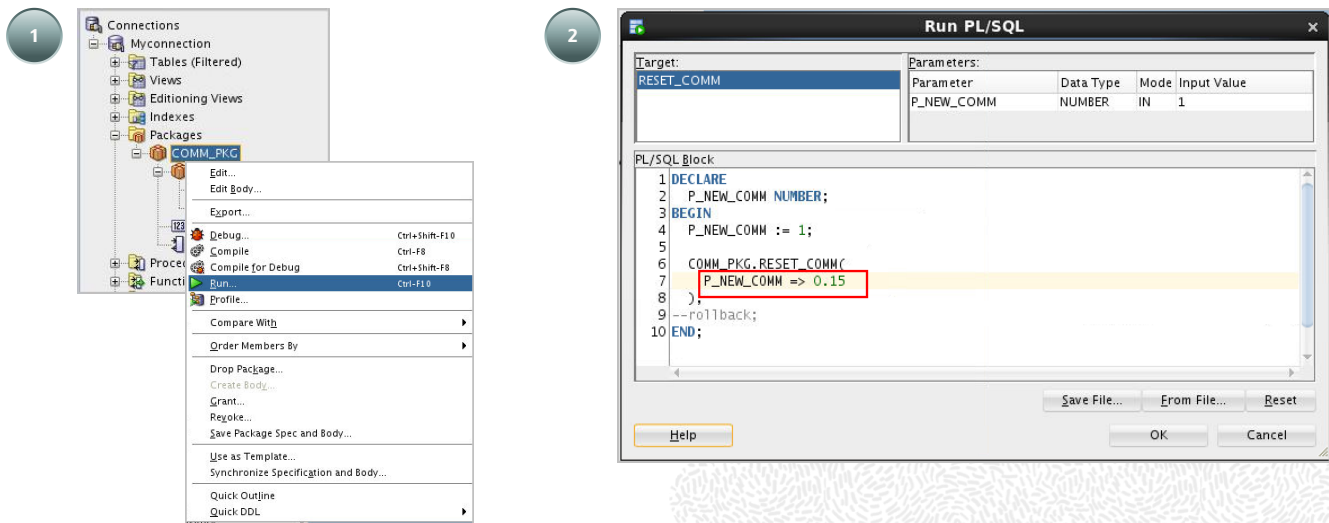
Example 2: Calls the `reset_comm` procedure from SQL*Plus (an environment external to the package) to reset the prevailing commission to 0.15 for the user session

Example 3: Calls the `reset_comm` procedure that is owned by a schema user called `SCOTT`. Using SQL*Plus, the qualified package procedure is prefixed with the schema name. This can be simplified by using a synonym that references the `schema.package_name`.

Assume that a database link named `NY` has been created for a remote database in which the `reset_comm` package procedure is created. To invoke the remote procedure, use:

```
EXECUTE comm_pkg.reset_comm(0.15)
```

Invoking Package Subprograms: Using SQL Developer



19

You can use SQL Developer to invoke a package's subprogram by performing the following steps:

1. Right-click the package's name in the **Packages** node in the Navigation tree and Select **Run** from the floating menu. The **Run PL/SQL** window is displayed. You can use the **Run PL/SQL** window to specify parameter values for running a PL/SQL function or procedure. (If you specify a package, select a function or procedure in the package.) Specify the following:
 - a. **Target:** Select the name of the function or procedure to run.
 - b. **Parameters:** This section lists each parameter for the specified target. The mode of each parameter can be **IN** (the value is passed in), **OUT** (the value is returned), or **IN/OUT** (the value is passed in, and the result of the function or procedure's action is stored in the parameter).
2. In the **PL/SQL Block** section, change the formal **IN** and **IN/OUT** parameter specifications in this block to actual values that you want to use for running the function or procedure. For example, to specify 0.15 as the value for an input parameter named P_NEW_COMM, change P_NEW_COMM => P_NEW_COMM to P_NEW_COMM => 0.15.
3. Click **OK**. SQL Developer runs the function or procedure.

Creating and Using Bodiless Packages

```
CREATE OR REPLACE PACKAGE global_consts IS
  c_mile_2_kilo      CONSTANT  NUMBER :=  1.6093;
  c_kilo_2_mile      CONSTANT  NUMBER :=  0.6214;
  c_yard_2_meter     CONSTANT  NUMBER :=  0.9144;
  c_meter_2_yard     CONSTANT  NUMBER :=  1.0936;
END global_consts;
```

```
SET SERVEROUTPUT ON
BEGIN
  DBMS_OUTPUT.PUT_LINE('20 miles = ' ||
    20 * global_consts.c_mile_2_kilo || ' km');
END;
```

```
SET SERVEROUTPUT ON
CREATE FUNCTION mtr2yrd(p_m NUMBER) RETURN NUMBER IS
BEGIN
  RETURN (p_m * global_consts.c_meter_2_yard);
END mtr2yrd;
/
EXECUTE DBMS_OUTPUT.PUT_LINE(mtr2yrd(1))
```

20

The variables and constants declared within stand-alone subprograms exist only for the duration that the subprogram executes. To provide data that would exist throughout the user session, create a package specification containing public (global) variables and constant declarations. In this case, a package body isn't required, such packages are known as a *bodiless packages*. As discussed earlier in this lesson, if a specification declares only types, constants, variables, and exceptions, then the package body is unnecessary.

Examples:

The first code box in the slide creates a bodiless package specification with several constants to be used for conversion rates. A package body isn't required to support this package specification. It is assumed that the `SET SERVEROUTPUT ON` statement was issued before executing the code examples in the slide.

The second code box references the `c_mile_2_kilo` constant in the `global_consts` package by prefixing the package name to the identifier of the constant.

The third example creates a stand-alone function `c_mtr2yrd` to convert meters to yards, and uses the constant conversion rate `c_meter_2_yard` declared in the `global_consts` package. The function is invoked in a `DBMS_OUTPUT.PUT_LINE` parameter.

Rule to be followed: When referencing a variable, cursor, constant, or exception from outside the package, you must qualify it with the name of the package.

Viewing Packages by Using the Data Dictionary

```
-- View the package specification.  
SELECT text  
FROM   user_source  
WHERE  name = 'COMM_PKG' AND type = 'PACKAGE'  
ORDER BY LINE;
```

TEXT
1 PACKAGE comm_pkg IS
2 v_std_comm NUMBER := 0.10; --initialized to 0.10
3 PROCEDURE reset_comm(p_new_comm NUMBER);
4 END comm_pkg;

```
-- View the package body.  
SELECT text  
FROM   user_source  
WHERE  name = 'COMM_PKG' AND type = 'PACKAGE BODY'  
ORDER BY LINE;
```

21

The source code for PL/SQL packages is also stored in the `USER_SOURCE` and `ALL_SOURCE` data dictionary views. The `USER_SOURCE` table is used to display PL/SQL code that you own. The `ALL_SOURCE` table is used to display PL/SQL code to which you've been granted the `EXECUTE` right by the owner of that subprogram code and provides an `OWNER` column in addition to the preceding columns.

When querying the package, use a condition in which the `TYPE` column is:

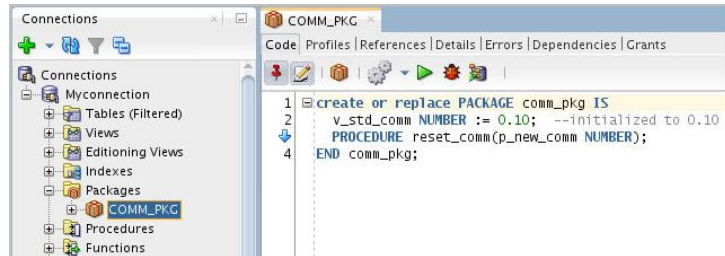
- Equal to 'PACKAGE' to display the source code for the package specification
- Equal to 'PACKAGE BODY' to display the source code for the package body

You can also view the package specification and body in SQL Developer using the package name in the Packages node.

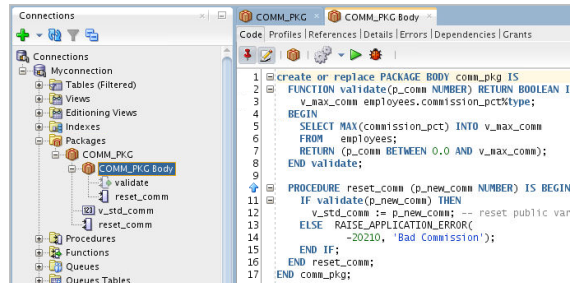
Note: You can't display the source code for Oracle PL/SQL built-in packages, or PL/SQL whose source code has been wrapped by using a `WRAP` utility or obfuscation. Obfuscating and wrapping PL/SQL source code is covered in the lesson titled "Working with Packages." Clicking the **Execute Statement** icon (or pressing F9) instead of the **Run Script** icon in the SQL Worksheet toolbar, sometimes displays a better formatted output in the Results tab as shown in the slide examples.

Viewing Packages by Using SQL Developer

- 1 To view the package spec, click the package name.



- 2 To view the package body, click the package body.



22

To view a package's spec in SQL Developer, perform the following steps:

1. Click the **Packages** node in the **Connections** tab.
2. Click the package's name.
3. The package's spec code is displayed in the **Code** tab as shown in the slide.

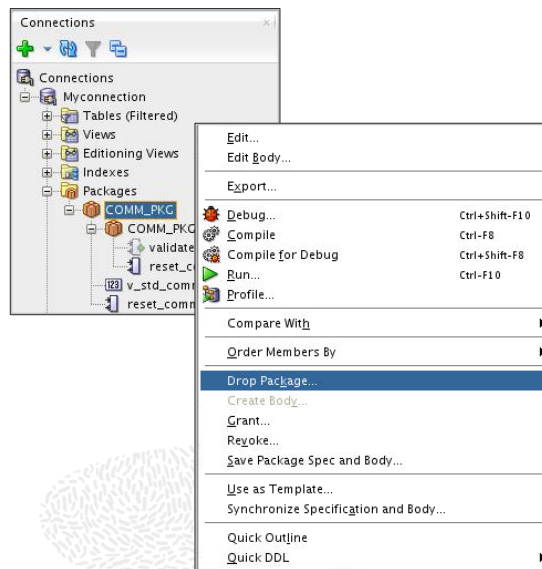
To view a package's body in SQL Developer, perform the following steps:

1. Click the **Packages** node in the **Connections** tab.
2. Click the package's body.
3. The package's body code is displayed in the **Code** tab as shown in the slide.

Removing Packages

Drop the package specification and body.

```
DROP PACKAGE package_name;
```



23

When a package is no longer required, you can use a SQL statement in SQL Developer to remove it. A package has two parts; therefore, you can remove the whole package, or you can remove only the package body and retain the package specification.

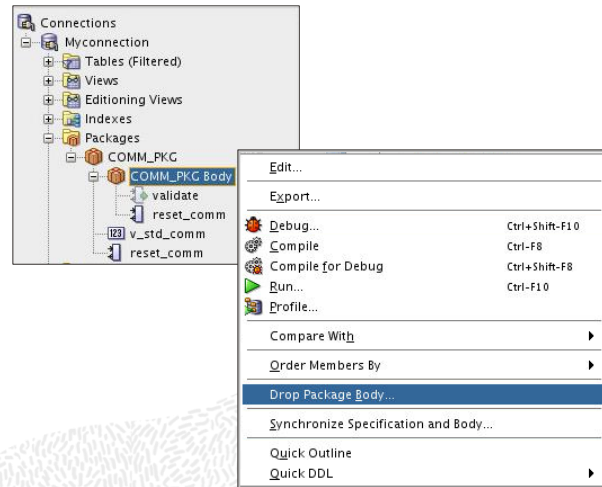
The slide shows removing both the package specification and package body. Through the command-line interface you can remove a package by executing `DROP PACKAGE package_name`.

In SQL Developer, you can drop a package by right-clicking the package name and selecting **Drop Package** from the menu.

Removing Package Bodies

Drop only the package body.

```
DROP PACKAGE BODY package_name;
```



24

The slide demonstrates how you can drop only the package body, instead of dropping the entire package.

You can drop the package body by executing the `DROP PACKAGE BODY package_name` command.

In SQL Developer, you drop package body by right-clicking the package body and selecting **Drop Package Body** from the menu.

Guidelines for Writing Packages

- Develop packages for general use.
- Define the package specification before the body.
- The package specification should contain only those constructs that you want to be public.
- Place items in the declaration part of the package body when you must maintain them throughout a session or across transactions.
- The package specification should contain as few constructs as possible.

25

Keep your packages as generic as possible, so that they can be reused in future applications. Also, avoid writing packages that duplicate features provided by the Oracle server.

Package specifications reflect the design of your application, so define them before defining the package bodies. The package specification should contain only those constructs that must be visible to the users of the package. Thus, other developers can't misuse the package by basing code on irrelevant details.

Place items in the declaration part of the package body when you must maintain them throughout a session or across transactions. For example, declare a variable called `NUMBER_EMPLOYED` as a private variable if each call to a procedure that uses the variable needs to be maintained. When declared as a global variable in the package specification, the value of that global variable is initialized in a session the first time a construct from the package is invoked.

Quiz



The package specification is the interface to your applications. It declares the public types, variables, constants, exceptions, cursors, and subprograms available for use. The package specification may also include PRAGMAs, which are directives to the compiler.

- a. True
- b. False

26

Answer: a



For Instructor Use Only.
This document should not be distributed.

Summary

In this lesson, you should have learned how to:

- Describe packages and list their components
- Create a package to group related variables, cursors, constants, exceptions, procedures, and functions
- Designate a package construct as either public or private
- Invoke a package construct
- Describe the use of a bodiless package



27

You group related procedures and functions in a package. Packages improve organization, management, security, and performance.

A package consists of a package specification and a package body. You can change a package body without affecting its package specification.

Packages enable you to hide source code from users. When you invoke a package for the first time, the entire package is loaded into memory. This reduces the disk access for subsequent calls.

Practice 14 Overview: Creating and Using Packages

This practice covers the following topics:

- Creating packages
- Invoking package program units



28

In this practice, you create package specifications and package bodies. You then invoke the constructs in the packages by using sample data.

Note: If you are using **SQL Developer**, your compile-time errors are displayed in the **Message Log** tab. If you are using **SQL*Plus** to create your stored code, use `SHOW ERRORS` to view compile errors.